Catapult HLS User View Scheduling Rules

Stuart Swan, Platform Architect, Siemens EDA

13 July 2026

Introduction

In a high-level synthesis (HLS) design methodology, much of the benefit is lost if design verification and debug must still be performed at the RTL level. In manual RTL design flows, RTL verification is typically one of the most costly and time-consuming phases of the overall flow. In HLS flows, RTL-level verification is even more challenging, because the RTL is machine-generated rather than hand-crafted.

This document presents a user-visible scheduling contract, modeling methodology, and collection of formal proofs that together support a flow in which full-system design verification and debug can be performed primarily on the pre-HLS SystemC model. The practical purpose of the contract is to give HLS users and verification engineers “no surprises”: interface scheduling is reduced to a small set of uniform rules for message-passing operations, signal I/O, and explicit synchronization, so that a common verification intent can be reused across pre-HLS and post-HLS models, while still allowing aggressive implementation optimizations such as loop pipelining, added FSM states, memory-access scheduling and reordering where permitted, direct-input late sampling, and bounded or elaborated channel buffering. The same bounded-capacity modeling also lets the pre-HLS SystemC model be made capacity-accurate and throughput-accurate with respect to the post-HLS RTL — for example, when modeled with a library such as Matchlib — so that throughput behavior and capacity effects can be assessed at the pre-HLS level rather than deferred to RTL simulation.

The formal guarantees are subject to the explicitly stated assumptions, observation-boundary choices, witness-selection conditions, and liveness/progress premises described in the formal appendices. The formal safety guarantee is an observable trace-compatibility result: every compliant RTL_B behavior carrying a globally witness-realizable annotation package (Definition J.GWR, an explicit implementation-side premise) has a matching Sys_ref behavior under the fixed stimulus, while transfer of a particular source-side simulation execution to RTL_B applies only to annotated RTL-consistent source prefixes or to source executions selected by the Appendix L snooping / witness-selection mechanism. The liveness/deadlock guarantee is separate and conditional. Corollary J.1 and the CF-0 certificate connect a selected RTL cutpoint to one reachable Sys_ref cutpoint through a common KObs record consisting of the canonical replay history and the complete attributed residual-offer set. Under the additional fairness, progress, drain, waiver, environment, and structural assumptions stated in the appendices, an RTL-matched prohibited wait structure therefore reconstructs to the corresponding source-side predicate, where the selected A5-L discharge can rule it out.

The methodology and proofs are organized around a latency-insensitive design style, while still fully supporting real-world techniques such as:

- both sequential and combinational logic
- pipelined designs
- shared memories between sequential processes (covered by the formal theorems under the J.Mem mapping-layer lowering condition; see Common Formal Definitions)
- reordering of RAM accesses to optimize hardware pipelines

- removal of pipeline registers for stable signal inputs to hardware pipelines
- latency-sensitive signal-level protocols and localized latency-sensitive islands, including cycle-accurate transactors, non-blocking arbiters, and one-way handshake protocols

These techniques capture the requirements gathered from hundreds of Catapult HLS customer tape-outs and are needed to meet the demanding quality-of-results (QOR) targets traditionally achieved with manual RTL design.

The methodology presented here builds on established verification practices such as SystemVerilog UVM, constrained-random stimulus generation, and functional and code coverage. The system under test is modeled and verified in SystemC, while RTL verification is focused on establishing that the synthesized or otherwise implemented RTL falls within the scope of the formal theorem instances. The proofs do not, by themselves, prove that any particular HLS tool invocation or generated RTL instance satisfies the required obligations. In particular, the RTL-side scheduling obligations, the per-channel E4 realization obligations, B-faithfulness — the faithfulness of each back-annotated capacity B(c) to the chosen explicit abstract boundary and to the RTL storage/credit represented at that boundary — and the other named side conditions must be established by the tool, by tool documentation, by generated certificates, by static checks, by RTL/interface checks, or by another validation method appropriate to the flow.

Although the focus of this document is HLS, the underlying modeling methodology and formal proof framework are not limited to HLS-generated RTL. The safety/liveness distinction introduced above is central to that generality: safety decomposes primarily into local process and channel-boundary obligations, while liveness has a more global dependency structure. The following paragraphs summarize these two decompositions.

For safety, the key benefit is compositionality in a strong local sense. Each process is verified independently against the scheduling and endpoint obligations at its own observable boundaries. The abstract channels connecting processes have fixed, methodology-defined semantics, such as rendezvous or bounded-FIFO capacity, and each channel's specification is fixed at the methodology/proof boundary. The producer and consumer processes need only satisfy their local endpoint scheduling and handshake obligations; the channel realization supplies the corresponding ChannelEnabled/ChannelDisabled behavior required by E4. When each process satisfies its R-rule conformance locally, and when the chosen Sys_ref / RTL_B comparison satisfies the applicable E4 channel semantics at the agreed abstract channel boundaries, the system-level trace-compatibility results follow from the formal composition theorems.

This safety decomposition is what makes the approach applicable beyond HLS. Because the trace-compatibility obligations are local to processes and to explicitly specified channel boundaries, the same proof structure applies to any RTL implementation whose individual processes and channel realizations satisfy the required implementation-side obligations. Those processes may be HLS-generated, hand-written, or produced by other generation methods. RTL verification is then focused on showing that each process and channel realization satisfies the same formal side conditions required of HLS-generated RTL, rather than constructing an ad hoc full-system safety proof over all process interactions.

The single most important deadlock-avoidance property is worth stating in plain terms, because a designer experiences it directly: HLS cannot by default reverse the order of the message-passing calls written in the source, so it cannot introduce new deadlocks via call-order reversal. This is the intuitive

benefit underlying the formal liveness result described below. That result additionally covers deadlock behavior that depends on timing rather than call order — most notably the overlapped execution of pipelined loops and finite-capacity (bounded-FIFO) effects — which is handled by the pipelined-loop discipline (automatic flush) and the machinery summarized below, not by the no-reverse rule alone.

Liveness and deadlock preservation decompose differently. The preserved property is absence of reachable closed internal wait-cycle cutpoints. The core theorem, Theorem K.1A, remains conditioned on the scheduler-robust source premise A5-L: no admissible Policy-L execution of the selected Sys_ref reaches a prohibited internal deadlock cutpoint. The primary user-facing discharge is Policy S. The strengthened serial-dominance theorem, Lemma K.2b, states that a Policy-L internal deadlock forces every complete maximal fair Policy-S execution of the same selected instance, fixed stimulus, and fixed witness to reach the general serial obstruction predicate Dead_KS. Consequently, one complete maximal fair Policy-S execution that avoids Dead_KS is sufficient; no Policy-S confluence theorem is needed. Dead_KS is defined on one extended wait graph that includes ordinary process-to-process wait edges and terminal edges to starvation terminals: exhausted environment inputs and normally completed producers. The optional static no-bypass condition K.EnvOrd, in its may-follow form and together with the completion-sufficiency condition K.Done, proves that no prohibited component can reach a starvation terminal, reducing the check to the ordinary internal K.1 predicate. Structural rank/credit certificates, direct Policy-L verification, and tool certificates remain alternate A5-L discharge routes. The result additionally requires the RTL_B scheduler/environment and channel realization to satisfy the applicable progress, fairness, and structural assumptions, including B2/B3/B5, K.Drain, Kε, J.KObsConf/CF-0, and A6-A11; the serial route further requires K.CV and NB-Align.

Thus, the methodology provides two related but distinct forms of scalability. Safety scales largely by local process and channel-boundary checking. Liveness/deadlock preservation scales by separating the proof into the scheduler-robust selected-Sys_ref premise A5-L, a recorded A5-L discharge route—primarily the Policy-S/A5-S route through serial dominance, but alternatively a structural, direct Policy-L, invariant, exploration, or tool-certificate route—and explicit RTL/environment/channel progress assumptions.

Together, these results support a verification flow in which much of the design reasoning can be performed at the pre-HLS SystemC level. For RTL produced by a compliant HLS flow, the required implementation-side obligations are expected to be discharged primarily by the tool and its associated flow evidence, such as tool documentation, generated certificates, static checks, or targeted RTL/interface checks. For hand-written RTL, modified generated RTL, or RTL produced by other generation methods, RTL verification or equivalent implementation checks must establish the same obligations to bring the implementation within the scope of the formal guarantees.

Because these rules are expressed as a user-visible, tool-neutral scheduling contract rather than as the internals of any particular HLS tool, a further goal of this document is to serve as a starting point for a standardization proposal — for example, within the Accellera Synthesis Working Group (SWG). A standardized scheduling contract would let design and verification teams write models and testbenches against a single, vendor-independent set of interface scheduling rules, rather than against the specifics of any one tool's implementation.

A document abstract is provided in Appendix M.

Document Goals

This document presents HLS tool scheduling rules and a modeling methodology. The goals are:

1. To provide easy-to-understand rules to HLS users.
2. To ensure precise and consistent rules in both SystemC and C++.
3. To offer rules that are effectively compatible with how Catapult currently operates.
4. To enable the best possible *quality of results* (QOR) in Catapult synthesis.
5. To cover all known user requirements and scenarios.
6. To serve as a suitable starting point for a standardization proposal (e.g., in Accellera SWG).

To illustrate the goals of this document more specifically, consider what an engineer writing a testbench for an HLS model needs to understand about how HLS tools operate. This engineer may be using SystemVerilog UVM or may be writing a testbench in C++/SystemC.

They likely are not an expert in any specific HLS tool (and may not want to be), but their testbench should be usable across both the pre-HLS model and the post-HLS model, subject to the latency-insensitivity, observation-boundary, and witness-selection conditions stated in this document. Thus, it is crucial for the DV engineer to have a precise understanding of how the HLS tool will transform the design while still enabling the resulting RTL behavior to be checked against the prescribed source reference model. This document describes what transformations the HLS tool is allowed to perform so that the pre-HLS and post-HLS models can be effectively verified with a common verification intent, and in the deterministic or annotated RTL-consistent cases with the same directly corresponding testbench execution. The overarching philosophy of the scheduling rules is to present “no surprises” to such a DV engineer, while still giving the HLS tool ample freedom to optimize the design.

The main body of this document is intended to be sufficiently precise and complete to support the Accellera standardization effort while still being understandable to HW architects. The formal appendices are intended to have a very high degree of precision and completeness sufficient to support AI-driven Lean mechanization of the formal theorems.

Background

HLS tools generate RTL from C++ models. Broadly speaking, this conversion takes a sequential C++ model and turns it into concurrent hardware that maintains the same behavior. HLS tools identify concurrent processes within the C++/SystemC model and then independently synthesize each process. Briefly, some of the techniques that HLS tools use to achieve good HW QOR when synthesizing each process include:

- Optimized scheduling based on the selected silicon target technology.
- Automatic HW pipeline construction according to the user’s specifications.
- Automatic HW resource sharing.
- Automatic scheduling of memory accesses.

The internal behavior of each process is specified by the control and dataflow behavior of the C++ code within the process. However, the external communication that each process has with other processes

and HW blocks is specified via IO operations that are coded within the model. To enable a reliable, scalable, and verifiable HLS flow that generates high quality hardware, the scheduling behavior of these IO operations needs to be precisely handled at all steps of the flow. This document specifies the rules that govern the scheduling behavior for these IO operations within HLS models.

These rules are specified with respect to an individual process, but the intent of the rules is to enable reliable and verifiable behavior of large sets of interacting processes operating as a system in real-world designs. (Appendix G provides the formal trace-compatibility guarantees between the prescribed source reference model and the post-HLS system, including the unrestricted Post → Ref direction and the restricted annotated RTL-consistent / witness-selected reverse use.)

By default, HLS tools can insert additional clock cycles (or latency) anywhere within a process -- for example, when pipelining a loop or to enable HW resource sharing. The overall approach used in this document is to make the entire design and testbench system *latency insensitive* to the maximum extent possible, while still fully enabling key HW optimizations. In addition, the overall approach enables the pre-HLS system simulation to be capacity-accurate and throughput-accurate with respect to the post-HLS RTL system.

In some cases, designs or testbenches may use protocols which are latency-sensitive. These situations can be handled by isolating the latency-sensitive portions to small, self-contained parts of the design or testbench, and then keeping the rest of the design and testbench latency insensitive. See Appendix D for more information.

In many cases the overall design will need to satisfy end-to-end latency requirements. For these designs it is still highly advantageous to use a latency-insensitive modeling approach and verify in the post-HLS model that the overall design latency requirements have been satisfied, since this is typically easy to do.

This document focuses on sequential HW processes. Combinational HW processes are supported but mostly not discussed since their synthesis is straightforward. All the examples and discussion in this document are for SystemC processes that are sensitive only to a single rising clock edge. (Appendix O discusses support for multiple clock domains.)

This document distinguishes between the following:

1. The *conceptual model* for the scheduling rules.
2. The simulation behavior of a model using the rules in C++ or SystemC.
3. The synthesis of a C++/SystemC model using the rules in an HLS tool such as Catapult.

The goal is to align each of these three cases as closely as possible, so that the user has easy-to-understand rules, while simulation and synthesis work without surprises. However, as we will see, there are practical considerations which may in certain cases cause small deviations from the conceptual model in either simulation or HLS.

A simple real-world example that illustrates the motivation for the scheduling rules is provided in Appendix A of this document.

The examples referred to in this document are available here:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/tree/master

The most up-to-date version of this document is available here:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/blob/master/matchlib_examples/doc/catapult_user_view_scheduling_rules.pdf

A summary of the scheduling rules is available here:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/blob/master/matchlib_examples/doc/brief_scheduling_rules.txt

An Analogy from RTL Synthesis

To better understand the specific purpose of this document, let's consider how RTL synthesis works. Say you have a sequential block that you are modeling in Verilog RTL, and it has an output port coded like:

```
Out1 <= #some_delay new_val;
```

In Verilog simulation, if some_delay is less than the clock period of the block, then it will probably not affect the overall cycle-level behavior of the system during simulation. However, if some_delay is more than the clock period, it probably will.

During RTL synthesis, all RTL synthesis tools will ignore all delays in the input model, in this case even if some_delay is greater than the clock period. Some RTL synthesis tools might give a warning or error for code like the above such as "Simulation and synthesis results are likely to mismatch because the delay in model is greater than clock period." Some RTL synthesis tools might outright reject a model containing such delays.

One might argue that RTL synthesis tools should always match the Verilog simulation behavior of the input model. But the overall approach works well because RTL is a good and simple *conceptual model* that users and tool vendors can align around. The slight differences between the *simulation model* and the *conceptual model* used by RTL synthesis tools can be easily managed.

We'll return to this example later in this document.

Next, let's clarify some terminology related to signal IO. Say you have a Verilog model like:

```
forever begin
  @(posedge clk);  // wait for 1st rising clock edge
  output1 <= input1 + 10;
  @(posedge clk);  // wait for 2nd rising clock edge
end
```

In this example, input1 is sampled when the process wakes on the first @(posedge clk) and evaluates the RHS. For the purposes of the scheduling rules, we anchor that Read(input1,·) to the most

recent synchronization point. The resulting Write(output1,·) is anchored to the next synchronization point (the cycle in which the written value is treated as stable for others to observe).
Cycle-level anchoring convention (used throughout this document). For cycle-level reasoning, each signal IO is assigned an anchor cycle relative to surrounding synchronization calls (e.g., wait() / SyncChannel / @(posedge clk)):

- Read(sig, val) is anchored to the closest preceding synchronization call, i.e. pred_S(Read).
- Write(sig, val) is anchored to the closest succeeding synchronization call, i.e. succ_S(Write).

This convention is only for cycle-level reasoning and does not depend on simulator update regions or delta-cycle ordering.

Catapult HLS Status Concerning Rules in this Document

A separate document provides a list of clarifications related to support within Catapult HLS for the rules described in this document. The items in the list are named *Cat#*, so that each item has a unique number.

This document annotates certain rules with *Cat#* to refer to the items in the separate document.

These Cat# annotations are not part of the formal proof itself. They identify tool-support and tool-conformance points that must be satisfied for a Catapult-generated RTL instance to fall within the scope of the formal guarantees. Where the appendices assume an RTL-side property such as E3 issue-order conformance, E4 channel-realization conformance, automatic-flush behavior for pipelined or otherwise overlapped control, an equivalent drain-only blocked-frontier discharge for ordinary non-overlapped control, finite explicit boundary storage/credit, or faithful back-annotation of B(c), the corresponding Cat# item or other tool-flow evidence is the mechanism by which that compliance obligation is expected to be discharged.

Terms Used in this Document

Latency-Insensitive: In digital hardware design*, latency-insensitive* refers to a system that operates correctly despite variable communication delays between components. This is achieved by using mechanisms to decouple computation from communication timing. Such designs improve scalability and reliability in complex systems with unpredictable or variable latencies. ARM's AXI4 and APB are examples of latency-insensitive protocols. ARM's AMBA 5 CHI credit-based NOC protocol is also an example of a latency-insensitive protocol.

Process: In Verilog, a process is an *always block* and its equivalent constructs. In SystemC, a process is an instance of an SC_THREAD or SC_METHOD.

Message-passing Interface: A *message-passing interface* reliably delivers messages (or transactions) from one process to another. This document uses this term to denote the type of communication found in Kahn Process Networks. See https://en.wikipedia.org/wiki/Kahn_process_networks
Message-passing read interfaces are always separate from message-passing write interfaces – there are no bidirectional message-passing interfaces. In this document, message-passing channels are assumed to be point-to-point: for each channel c, exactly one producer performs all Push_c operations and exactly one consumer performs all Pop_c operations.

Synchronization Interface: A *synchronization interface* synchronizes one process with another and/or with a global clock. For an example of a synchronization interface, see https://en.wikipedia.org/wiki/Barrier_(computer_science)

Signal IO: In digital HW design, signals are the fundamental communication mechanism. Signals enable communication between two HW blocks/processes, but communication with signals in real HW always incurs at least some delay because communication cannot be faster than the speed of light.
In HDLs and in SystemC, signal delays are modeled with the *delayed update* semantics.

blocking / non-blocking: In this document, a blocking message-passing operation is modeled as issuing a persistent request that remains asserted (and with stable payload, if applicable) until the operation commits (the message is sent or received). A non-blocking message-passing operation returns immediately with a status indicating whether the operation committed in that cycle.

One-way / two-way handshake protocols: A one-way handshake protocol only has one signal between a sender and receiver to synchronize communication. A two-way handshake protocol has two signals (one in each direction) between a sender and receiver to synchronize communication.

shall: This term indicates that a compliant tool or flow is required to follow the indicated rule.

may: This term indicates that a compliant tool or flow is allowed to follow the indicated rule but is not required to do so.

Classes of Operations Involved in Scheduling Rules

There are three classes of operations involved in the scheduling rules:

1. Calls to message-passing interfaces (which are all ac_channel methods, all SystemC Push/PushNB/Pop/PopNB calls except calls to SyncChannel)
2. Calls to synchronization interfaces (which are calls to ac_sync and Matchlib SyncChannel, also ac_wait and SystemC wait)
3. Signal IO (which are SystemC signal reads and writes, also C++ model *direct inputs*)

All the operations above are referred to as *IO operations*.

Basic Conceptual Model

The basic conceptual model encompasses processes that have no loop pipelining but may have preserved loops. If a process has a preserved loop, then the user may place a wait statement in the loop. Wait statements explicitly placed in the model are called explicit wait statements in this document and are classified as calls to a synchronization interface. However, for a loop to be treated as an HLS-pipelined loop under the formal guarantees in this document, the loop body shall not contain any wait/ac_wait statement or other explicit synchronization call used as an in-loop wait boundary. Such synchronization boundaries must be placed outside the pipelined loop, or the loop must be treated as non-pipelined/preserved or outside the theorem instance.

The basic conceptual model rules are:

1. Synchronization interface calls within a process always remain in the source code order.

2. Signal read operations occur at the closest preceding call to a synchronization interface. (Cat1)

3. Signal write operations occur at the closest succeeding call to a synchronization interface.

4. Message-passing operations may be shifted in time, issued in the same cycle, or overlapped by simulation or synthesis, subject to the following constraints:

- All message-passing operations before a call to a synchronization interface shall be issued before or in the same cycle in which the synchronization interface commits. (Here "the synchronization interface commits" refers to the commit of that synchronization call in the calling process's trace. For blocking message-passing operations (Push/Pop), the process cannot complete the synchronization call until any earlier issued blocking message requests in that interval have committed.)
- All message-passing operations after a call to a synchronization interface shall be issued after the cycle in which the synchronization interface call commits.
- Message-passing operations on the same message-passing interface shall be issued in source order; they shall not be reversed. This same-interface order requirement applies both within a single source interval and across overlapped iterations of a pipelined loop.
- Two message-passing operations on separate interfaces which appear in sequence in the model are source-ordered. They may be issued in the same source sequence, and the later operation may in some permitted modes issue before the earlier operation commits, including in the same clock cycle where allowed by the issue policy. They shall not be issued in the reverse sequence. (Cat2)

Here "parallel" issue means timing overlap or same-cycle request assertion; it does not mean that ordinary source-level message calls on distinct interfaces become unordered or incomparable. The source order used by the formal rules remains the dynamic program order of the executed source calls, after applying the explicit signal-anchoring rules for signal IO. Any true message-operation multi-frontier behavior used by the formal proof must be introduced by the selected source reference model through explicit Sys_B^elab staged/bundle/join/epoch-gate structure, rather than inferred merely from the fact that two ordinary calls use distinct interfaces.

Some explanation for the very last point: While pure message-passing systems with unbounded FIFOs are immune to reordering concerns, real-world hardware implementations have finite buffer sizes. Reversing the order of message-passing calls in a system with bounded FIFOs can introduce deadlocks that were not present in the original model. The last rule prevents HLS from introducing deadlocks via this specific mechanism: reversing the dynamic source order of message-passing calls that appear in sequence in the source. However, deadlock behavior can still be affected by transformations that change the timing of request/commit while preserving source order—most notably overlapped execution in pipelined loops, eager same-interval issue under Policy L, and finite-capacity effects.

Pipelined Loops

When a loop is pipelined in HLS, the body of the loop is split into pipeline stages. HLS may start the next iteration of the loop before the current iteration has completed, thus increasing throughput as compared to a non-pipelined implementation.

A loop selected for HLS pipelining shall not contain wait/ac_wait statements, SyncChannel/ac_sync calls, or any other explicit synchronization interface call inside the loop body. The pipelined-loop rules in this document therefore apply to wait-free pipelined loop bodies, with ordinary synchronization boundaries only outside the pipelined loop or around the pipelined region. If a design requires a synchronization call inside the loop body, the flow shall either reject that loop for pipelining under this contract, require the source to be rewritten so that the synchronization boundary is outside the pipelined loop, or exclude that pipelined-loop instance from the Appendices G-K formal guarantees unless a separate specialized theorem/certificate is supplied.
This restriction keeps R1/E1 and R3/E5 simple: the observable synchronization set S contains only real synchronization calls outside the pipelined loop body, and a pipelined loop does not introduce additional in-body synchronization events that would have to be ordered or used to split intervals under loop overlap.

To guarantee equivalent behavior between the pre-HLS and post-HLS systems, pipelined loops must be implemented with automatic flush to prevent deadlocks. See Appendix B for a formal presentation of pipelined loops.

When HLS pipelines a loop, multiple iterations of the loop are overlapped and execute at the same time. During loop pipelining, for all IO operations, HLS shall ensure that an access to a specific message-passing interface, signal, or synchronization interface shall not be reordered relative to same-interface accesses from other iterations.

During pipelining, signal reads and writes are physically embedded in specific pipeline stages, but the loop body itself contains no explicit wait/ac_wait or other in-body synchronization call under the formal contract above. The logical anchor of an ordinary signal Read/Write is therefore still determined by the surrounding real synchronization calls selected into S under R2/E2, not by an in-body wait. Considering the entire set of signals read or written by the process, if HLS pipelining would cause the order of signal IO to differ between the pre-HLS and post-HLS models, or if any two signal IO operations that occur on the same clock edge in the pre-HLS model would not occur on the same clock edge in the post-HLS model, then the HLS tool shall detect such a situation and require the user to explicitly indicate in the model source (e.g., via a pragma) that HLS pipelining can still be used. (Cat7) In other words, if a process communicates with other processes using only latency-insensitive protocols, then HLS pipelining by default shall not break any of the protocols.

Synchronization boundary for pipelined loops

If a pipelined loop is separated from later work by an explicit synchronization interface call, then while the process is pending at that synchronization call, any back-annotated message-passing capacity B(c) whose accepted data could otherwise be consumed only by work after that synchronization call shall be treated as unavailable to upstream producers until the synchronization call commits. Equivalently, the producer-facing availability corresponding to such capacity is temporarily withdrawn at the synchronization boundary. Work already accepted before the boundary may drain under the automatic-flush rules, but no new transfer may be accepted across that boundary into the next post-sync interval before the sync commits.

Formal modeling requirement

This withdrawal is not a hidden exception to E4 at an otherwise ordinary bounded-FIFO boundary. The prescribed source reference model shall represent this behavior in Sys_B^elab, using explicit sync-boundary / epoch-gate abstract channels, or equivalent explicit staged realization structure, so that the refusal to accept new post-sync work appears as ordinary ChannelDisabled behavior at an explicit abstract boundary. Thus the pre-HLS back-annotated model ramps down in the same sense as the RTL: already accepted work may drain, but capacity belonging only to suff_S(s) is not producer-facing available until s commits.

Direct Inputs

Ordinary SystemC signal reads occur at the closest preceding synchronization interface call (e.g., wait statement) in both the pre-HLS and post-HLS models. (Cat1). If the HLS tool adds states to the process, or if it pipelines the process, then this implies that the HLS tool must add registers for each such read operation such that the read occurs where specified in the pre-HLS model, and the value is stored until the point where it is consumed in the post-HLS model. The area cost of such registers may be high if there are a lot of signals, and in some cases, it may be unneeded area since the value of the signals may not actually need to be stored internally to the process.

The simplest case is if such external signals are held stable after the process comes out of reset. In this case, HLS may assume that it is free to physically sample the signal values as late as possible, with no need for register storage, while the logical Σ-visible Read event remains anchored as specified by E2. This case is handled with the following pragma on SystemC signals and ports (Cat6):

```
#pragma hls_direct_input
```

To ensure that there are no pre-HLS versus post-HLS simulation mismatches, the environment that drives the signal shall hold it stable within each relevant reset epoch: after all receiving processes that use this signal with this pragma have come out of a reset boundary, the signal value shall remain stable until the next reset boundary that affects those receiving processes, unless the signal is instead governed by an explicit hls_direct_input_sync update contract as described below. A *dynamic reset* starts a new stability epoch for the affected receiving logic and its associated direct-input signals. Thus it remains allowable to have *dynamic resets*, i.e. activation of block resets and associated resetting of direct input signals as part of the normal operation of the HW.

A related but somewhat more complex case is where input signals to the HW block may only be changed at “agreed upon” times, typically while the portion of the HW block that relies on them is temporarily idle. For example, a block may process 2D images. At the start of each new image, it may be desirable for the TB or external environment to update the control signals for how the block will process the next image. Typically HLS designs are pipelined, and the HW pipeline for the current iteration must be fully *ramped down* before the input signals can be updated to affect the next iteration. In the pipelined-loop case, this relies on the general synchronization-boundary rule for pipelined loops: while the process is pending at the designated sync, any back-annotated message-passing capacity whose accepted data would belong to the next post-sync interval is temporarily unavailable to upstream producers until the sync commits. In this case we can use the SystemC *SyncChannel* or C++ *ac_sync* primitives to precisely synchronize the DUT with the TB/environment to enable the input signals to be updated at the correct time. The `#pragma hls_direct_input_sync` directive shown below associates the sync operation with

the direct inputs that it controls. The precise synchronization scheme shown here ensures that there are no pre-HLS versus post-HLS simulation mismatches even though we are using direct inputs and also changing their values while the design is executing.

When a signal or port covered by #pragma hls_direct_input is also included in the scope of a later #pragma hls_direct_input_sync directive, the sync directive selects the effective update contract for that controlled signal set. Such signals remain direct inputs for purposes of late physical sampling, but they are not required to remain stable for the entire reset epoch; instead, they must satisfy the sync-controlled discipline described here: updates may occur only at the designated synchronization handshake, and the values must remain stable between permitted sync-controlled updates and reset boundaries.

```
  // This is example 61 in Catapult Matchlib examples
  sc_in<bool> SC_NAMED(clk);
  sc_in<bool> SC_NAMED(rst_bar);

  Connections::Out<uint32_t>  SC_NAMED(out1);
  Connections::In<uint32_t>   sample_in[num_samples];
  Connections::SyncIn         SC_NAMED(sync_in);
#pragma hls_direct_input
  sc_in<uint32_t> direct_inputs[num_direct_inputs];


  void main() {
    out1.Reset();
    sync_in.Reset();

#pragma hls_unroll yes
    for (int i=0; i < num_samples; i++) {
      sample_in[i].Reset();
    }

    wait();  // reset state

    while (1) {
#pragma hls_direct_input_sync all
     sync_in.sync_in();

#pragma hls_pipeline_init_interval 1
#pragma hls_stall_mode flush
     for (uint32_t x=0; x < direct_inputs[0]; x++) {
       for (uint32_t y=0; y < direct_inputs[1]; y++) {
        uint32_t sum = 0;
#pragma hls_unroll yes
        for (uint32_t s=0; s < num_samples; s++) {
           sum += sample_in[s].Pop() * direct_inputs[2 + s];
        }
        ac_int<32, false> ac_sum = sum;
        ac_int<32, false> sqrt = 0;
        ac_math::ac_sqrt(ac_sum, sqrt);  // internal loop unrolled in cat .tcl
file
        if (sqrt > direct_inputs[7])
          out1.Push(sqrt);
       }
     }
    }
  }
```

```
};
```

From the perspective of the testbench or the environment, the updating of the signals controlled by the `hls_direct_input_sync` directive shall only occur at a precise point. The TB shall first wait for the `rdy` signal for the sync to be asserted by the DUT, and then the TB shall update all the input signals it wishes to change while simultaneously driving the sync `vld` signal high for one cycle.

It is important to note that the only safe operation to use to synchronize the updating of direct inputs is the sync operation as shown above. Other operations such as Push/Pop or ac_channel operations should not be used for this. In terms of the Appendix G trace model, the Σ-visible Read(sig,val) events remain anchored to their synchronization points; any additional internal sampling made by the implementation between anchors is treated as an ε-step and must not change the recorded value label.

In the example above the DUT block that is being synthesized determines when to call sync, and thus it determines when the direct inputs will be updated. In some cases, it may be necessary for the environment around the DUT to determine when the direct inputs should be updated. In this case the same approach as shown above should be used; however a separate input from the environment to the DUT (either using a signal or a message-passing interface) should request that the DUT call sync as soon as feasible. This will ensure that the DUT has properly ramped down its pipeline and is ready to receive the newly updated direct inputs as per the overall synchronization scheme described above.

Additional Options for Scheduling Message-passing Interfaces

The following option may be added during HLS (Cat2):

```
STRICT_IO_SCHEDULING=relaxed
```

When this is specified, the HLS tool is allowed to reorder message-passing interface calls freely on distinct interfaces (still not moving calls across synchronization interface calls). This relaxed mode may violate the default no-reverse discipline and therefore may introduce deadlocks that are not present under the default rules. It is recommended that this relaxed option only be used when the user wants to see what order the HLS tool prefers to schedule message-passing interface calls (e.g., to achieve best QOR).

Once the user knows the preferred order, it is recommended that the user modify the pre-HLS source code to reflect the preferred order, and then return to the use of the default scheduling modes. This methodology ensures that HLS cannot introduce new deadlocks *via reversal of source order on distinct message interfaces* as described earlier. Deadlock behavior may still depend on bounded-capacity effects and on overlapped execution (e.g., loop pipelining); those cases are addressed separately in this document via the pipelined-loop discussion (automatic flush / WFG-inertness) and the later deadlock reasoning. The formal equivalence and deadlock-preservation results in Appendices G–K assume STRICT_IO_SCHEDULING is not set to relaxed.

Scheduling of Array Accesses

Arrays may appear in HLS models, and they may be preserved through synthesis and mapped to RAMs.

Pointers may also appear in HLS models, and pointer dereferences are resolved to array accesses during HLS.

There are two cases to consider for arrays for the purposes of the scheduling rules:

1. Array instantiation in the HW is internal to the process
- The array accesses are not visible outside the process, and thus their scheduling is also not visible externally.
- All the scheduling rules described elsewhere in this document remain unaffected in this case.

2. Array instantiation in the HW is external to the process
- In this case the user model shall indicate how array accesses are mapped onto IO operations that are external to the process.
- We call this the *array access mapping layer*. The *array access mapping layer* maps array accesses onto IO operations described above (signal IO, message-passing interface calls, and synchronization calls).
- The user model may indicate that it is allowable for HLS to transform array accesses, for example, to cache, merge, split, or reorder array accesses (e.g., to improve QOR). These transformed operations, if allowed, are an outcome of using the *array access mapping layer*.
- In all cases the scheduling rules described elsewhere in this document for the core IO operations (signal IO, message-passing calls, synchronization calls) remain unaffected.
- In all cases array accesses shall remain constrained by any explicit synchronization interface calls present in the process in the source model. Precisely speaking, all array reads or writes before a synchronization call shall commit before or in the same cycle at which the synchronization call commits, and all array reads or writes after a synchronization call shall commit in a cycle after the synchronization call commits. This rule is stated in terms of commit cycles (the cycle the mapped memory storage is accessed); the scheduler enforces it by choosing issue cycles consistent with the fixed access latency defined by the array access mapping layer.
- Note that if transformed operations occur and array accesses are visible externally in both the pre-HLS and post-HLS model, then comparison of pre-HLS and post-HLS behaviors may need to account for the transformed operations.

When a loop is pipelined, multiple iterations of the loop are overlapped and execute at the same time. During loop pipelining, an access to a memory interface (or array) may be moved over or in parallel with an access to the same memory if the HLS tool can prove the reordering is conflict-free. If the array/memory is external to the process, the *array access mapping layer* shall indicate that such reordering is allowable if such reordering is to occur during loop pipelining.

Definitions: Issue vs. Commit

Informally, we can define **commit** and **issue** as follows:

Definition: Commit (message channel) — as observed by an external clocked observer
Commit is the cycle in which an external clocked observer sees a transfer *complete*:
- A message commits in the cycle when the observer sees **both valid and ready high at the same time**.
- The committed payload is the data value seen in that same cycle.

Definition: Issue (message channel) — request assertion, with scheduler/witness attribution
Issue(q) is the cycle assigned to a source-level message operation instance q at the chosen message-channel boundary when the calling process begins requesting the corresponding transfer. Its observable boundary manifestation is the endpoint-owned request signal: for a Push, vld is asserted; for a Pop, rdy is asserted.

Unlike Commit, Issue(q) is not, in general, fully reconstructible from external observation alone. An external clocked observer can see the request level, but the attribution of that request to a particular dynamic operation instance q is supplied by the scheduler/witness and by the Issue/Commit linkage conditions in Appendix C. This distinction matters for continuously asserted requests, back-to-back operations, failed non-blocking polls, and witness-selected executions.

See Appendix C for formal definitions of **issue** and **commit**.

For RAM reads/writes as presented above, **commit** denotes the precise cycle the memory storage is accessed; since the access latency is fixed and known to the HLS scheduler, it can schedule the operation's issue cycle so that the commit cycle satisfies the required ordering relative to Sync. The "Scheduling of Array Accesses" constraints above are therefore written in commit-time.

Additional States Added by HLS Synthesis

By default, HLS synthesis tools may add additional states to processes (e.g. add latency to enable resource sharing), which may introduce latency differences in the interface behavior between the pre-HLS and post-HLS models. These additional states are never included in the set of *synchronization interface calls* as described above.

When the directive IMPLICIT_FSM=true is set on a process, the HLS synthesis tool shall ensure that the cycle-level behavior of the interfaces of the pre-HLS and post-HLS models shall be identical. With this option, the internal state machines of the pre-HLS and post-HLS models will be the same.

When the directive IO_MODE=FIXED is set on a process, the HLS synthesis tool shall ensure that the cycle-level behavior of the interfaces of the pre-HLS and post-HLS models shall be identical. With this option, it is still possible that the state machine internal to the process is different between the pre-HLS and post-HLS models (e.g. the post-HLS model may choose to use a pipelined multiplier where the pre-HLS model did not.)

Avoiding Pre-HLS and Post-HLS Simulation Mismatches

The scheduling rules described in this document are designed to be easy to understand, while providing good QOR via HLS and generally avoiding any mismatches between the pre-HLS and post-HLS simulation behaviors.

Non-blocking message-passing operations (PushNB/PopNB in SystemC, C++ ac_channel nb_read/nb_write) are a potential source of mismatches between pre-HLS and post-HLS simulations since their behavior is inherently dependent on the latency within the model, which often changes

during HLS. Because of this, non-blocking message-passing interfaces should only be used when no alternative approach is possible. For example, non-blocking message-passing interfaces are required to model time-based arbitration of multiple message streams which access a shared resource. A full discussion of recommended guidelines on the use and verification of non-blocking message-passing interfaces is provided in Appendix L. Note that the scheduling rules described previously in this document fully specify how HLS tools are required to schedule such operations.

Unidirectional message-passing between two processes should not be relied on to achieve synchronization between the two processes, since in general the message latency and storage capacity between the processes may be variable. Also, depending on buffering/pipelining in the realization, the time between a producer's committed Push and the consumer's committed Pop (and the transient in-flight occupancy) may vary. Two unidirectional message-passing channels in opposite directions can be relied upon for synchronization between two processes. (An example of this is the AXI4 ar and r, and aw and b, channels). Note that such synchronization is weaker than explicit synchronization like SyncChannel or signal IO that uses a two-way handshake.

SystemC signal IO operations are a potential source of pre-HLS versus post-HLS simulation mismatches since timing behaviors may change between the two models. The following section provides guidance and rules to help avoid potential mismatches due to signal IO.

When signal IO operations are synchronized with a wait statement, there generally should be a proper two-way handshake associated with the wait statement so that the signal IO is latency insensitive. (Note that this statement does not apply to the signal synchronization approaches described in the Direct Inputs section.)

For example, here's a simple two-way handshake protocol when writing the signal `out_dat`:

```
out_dat = value;
out_vld = 1;
do {
  wait();
} while (out_rdy != 1);
out_vld = 0;
```

And here's a two-way handshake example when reading signal `in_dat`:

```
in_rdy = 1;
do {
  wait();
} while (in_vld != 1);
value = in_dat;
in_rdy = 0;
```

Some signal-level protocols have different two-way handshaking approaches (e.g. ARM APB), but they are still latency insensitive.

If signal IO operations are associated with a wait statement and that wait statement does not have a proper two-way handshake, then the signal IO is likely to be latency-sensitive and may result in pre-HLS versus post-HLS simulation mismatches. In some systems a one-way signal handshake is sufficient for reliable system operation. See Appendix E for further discussion.

The scheduling rules state that signal IO operations occur at either SystemC wait statements or SyncChannel calls (sync_in and sync_out). In the remainder of this section, we will use *wait* statements to refer to both.

RULE 1: It is always best coding style to group signal write operations just before their corresponding wait statement, and signal read operations just after their corresponding wait statement. (Cat3). An example is below:

```
sc_in<int> i1;
sc_in<bool> go;
sc_out<int> o1;
void my_thread() {
  int new_val=0;
  while (1) {
    o1.write(new_val);
    do {
      wait();
    } while (!go.read());
    new_val = i1.read();
    new_val = some_function(new_val); // function has no internal IO
  }
}
```

By placing the signal IO operations as close as possible to their corresponding wait statement, the HW intent is very clear. And there is no benefit either in terms of simulation performance or HLS QOR if they are placed further away from their corresponding wait statement.

Let's look at another similar example, which now also uses a Matchlib Connections blocking Pop operation:

```
sc_in<int> i1;
sc_in<bool> go;
sc_out<int> o1;
Connections::In<int> pop1;
void my_thread() {
  int new_val=0;
  while (1) {
    o1.write(new_val);
    do {
      wait();
    } while (!go.read());
    int pop_val = pop1.Pop();
    new_val = i1.read();
    new_val = some_function(new_val + pop_val); // function has no internal
IO
  }
}
```

Under the anchoring rules, i1.read() remains associated with the same synchronization point regardless of the intervening Pop. However, in raw pre-HLS simulation, a blocking Pop may stall for one or more cycles, so i1 can change before the i1.read() executes—creating behavior that differs from the anchored interpretation (and potentially from post-HLS scheduling). The fix is simply to

place i1.read() immediately after its intended wait() anchor; this removes the mismatch risk without affecting QOR or simulation performance.

For a proof-backed flow, RULE 1 conformance is a source well-formedness obligation, not merely a recommended diagnostic. If a blocking message-passing operation separates a signal read or write operation from its corresponding synchronization interface call, then the flow shall either reject the model for purposes of applying the formal guarantees, require the source to be rewritten so the signal IO is adjacent to its intended anchor, or discharge an explicit source-level well-formedness argument showing that the signal IO has a unique unconditional real synchronization anchor on every dynamic path and cannot observe a value inconsistent with that anchor. HLS tools may implement this requirement using errors, warnings, lint checks, or other diagnostics, but an unresolved warning is not sufficient to claim the RULE 1 / RULE 2-based guarantees.

Another scenario in which RULE 1 applies is shown below:

```
sc_in<bool> go;
sc_out<int> o1;
void my_thread() {

  while (1) {
    wait();              // WAIT 1
    o1.write(some_value);
    if (go.read()) {
      some_value = some_function();
    }
    else {
      wait();            // WAIT 2
    }
    some_other_function();
    wait();              // WAIT 3
  }
}
```

The signal read of `go` is clearly and uniquely associated with WAIT 1. However, the signal write of `o1` associates with WAIT 2 if `go` is false and WAIT 3 if it is not. This is a violation of RULE 1 and should be flagged as an error during HLS. The fix, as before, is to move the signal IO operation as close as possible to its intended wait statement so that the association is unconditional.

Next, let's consider *rolled* (or *preserved*) loops that perform signal IO within the loop body. Consider the following example:

```
sc_in<int> i1;
sc_out<int> o1;
void my_thread() {
    wait(); // reset state
    while (1) {
      wait(); // start of while loop
      #pragma hls_unroll no
      for (int i=0; i < 10; i++) {
        o1.write(i1.read() * i);
      }
  }
}
```

Note that the `i1.read()` operation is located inside the `for` loop, so presumably the user's intent is that it should be read as the loop iterates. *If that is not the user's intent, they simply should move the i1.read() operation before the loop start.*

In the post-HLS simulation, each iteration of the loop will consume at least one clock cycle, and a new value for i1 will be read (and a new value for o1 written) on each iteration. Again, this is the user's intent as per the code. In the pre-HLS simulation, the for loop body will execute in zero time, and only the last write to o1 will have any effect. The solution to avoid this mismatch is to manually place a `wait()` statement within the for loop body so that the signal IO synchronization is explicit in the pre-HLS simulation.

RULE 2: If you have signal IO operations within *rolled* (or *preserved)* loops, manually place a wait statement within the body of the loop to avoid pre-HLS versus post-HLS simulation mismatches, and while doing so also follow **RULE 1**.
Scope of RULE 2 for pipelining. RULE 2 applies only to rolled or preserved loops that are not selected for pipelining under this contract. A loop body selected for pipelining shall not contain wait/ac_wait statements, SyncChannel/ac_sync calls, or any other explicit synchronization interface call. Therefore, if ordinary per-iteration signal IO inside a loop requires a per-iteration wait anchor, that loop is outside the pipelined-loop contract unless the source is rewritten: move the signal IO outside the pipelined loop, model the interaction through message-passing, use hls_direct_input / hls_direct_input_sync under their stated contracts, or forgo pipelining for that loop. Without an in-body synchronization anchor, repeated per-iteration signal reads or writes inside one surrounding source interval do not become distinct per-iteration Σ-visible Read/Write events merely because HLS later pipelines the loop; their logical anchors remain the surrounding synchronization calls selected into S under R2/E2.

For a proof-backed flow, a rolled loop with signal IO operations but no wait statement in the loop body shall be treated as a source well-formedness failure unless an explicit source-level argument shows that each observable signal Read/Write still has a unique real synchronization anchor on every dynamic path. HLS tools may implement this requirement using errors, warnings, lint checks, or other diagnostics. However, if a diagnostic is emitted as a warning rather than an error, the warning must be resolved, waived with an explicit well-formedness justification, or treated as excluding that model from the formal guarantees. (Cat4)

If a pre-HLS model has established RULE 1 and RULE 2 conformance, then all signal IO in the post-HLS model shall occur only at clock cycles that correspond to explicit wait statements or explicit synchronization statements. For direct-input signals/ports (e.g., #pragma hls_direct_input and #pragma hls_direct_input_sync), this requirement refers to the logical (Σ-visible) anchor cycle of the Read/Write: the event is still considered to occur at its synchronization point even if HLS physically samples a stable signal later; such late-sampling micro-steps are ε-steps and are not Σ-visible. User designs that do not adhere to both RULE 1 and RULE 2 are ill-formed and are outside the formal guarantees unless the nonconformance has been eliminated or explicitly discharged by a valid source-level well-formedness argument.

Returning to the Analogy from RTL Synthesis

At the beginning of this document, we presented the example of a Verilog sequential block with an output coded like:

```
Out1 <= #some_delay new_val;
```

Recall that in Verilog simulation, if some_delay is less than the clock period of the block, then it will probably not affect the overall cycle-level behavior of the system during simulation. However, if some_delay is more than the clock period, it probably will.

During RTL synthesis, all RTL synthesis tools will ignore all delays in the input model, in this case even if some_delay is greater than the clock period. Some RTL synthesis tools might give a warning for the code above like “Simulation and synthesis results are likely to mismatch because delay in model is greater than clock period.”

HLS tools or associated lint/qualification flows that adhere closely to the conceptual model presented in this document should provide a concrete mechanism for detecting or discharging violations of RULE 1 and RULE 2 as described in the section above. This is analogous to the error or warning that the RTL synthesis tool would provide in the example directly above.

However, the formal guarantees in this document do not depend on any particular diagnostic mechanism. If an HLS synthesis tool does not report a RULE 1 / RULE 2 issue, the design is not thereby made well-formed. The proof-backed flow must still establish that the source model satisfies the RULE 1 / RULE 2 well-formedness obligation before applying the trace-compatibility guarantees.

Summary

At the beginning of this document, we said that the intent was to present “no surprises” to a DV engineer who wants a common verification methodology for the pre-HLS and post-HLS models. The key aspects of the document which support this are:

- Three groups of IO operations are defined (message-passing, signal IO, and synchronization calls) and each is treated uniformly. These IO operations are easy for verification engineers to understand because they are already using them in their testbenches.
- The document specifically avoids complex constructs such as *protocol regions* used in some HLS tools.
- The document preserves the ability of the pre-HLS SystemC model to be *throughput accurate* by using a library such as Matchlib.

- Synchronization calls can affect the scheduling (anchoring) of signal IO operations, and synchronization calls can affect the scheduling of message-passing calls. In the conceptual anchoring semantics of this document, message-passing calls do not affect the anchor cycle of signal IO operations (and signal IO does not affect the legality/order of committed message transfers) except through explicit synchronization. Practically, in raw pre-HLS SystemC simulation a blocking message operation may introduce additional cycles of waiting, so signal IO statements that are lexically separated from their intended anchor can observe different values; RULE 1 addresses this by recommending placement of signal IO adjacent to its anchor.

- HLS cannot by default reverse the order of message-passing calls, so it cannot introduce new deadlocks *via call-order reversal*. For pipelined loops (overlapped iterations) and other transformations that change the timing of request/commit under bounded capacities, deadlock preservation depends on the additional pipelined-loop discipline described in this document (automatic flush / WFG-inertness), not on the no-reverse rule alone.

- HLS pipelining is largely a *don't care* from the perspective of the verification engineer. If the design and the testbench are insensitive to changes in latency, and if external array accesses are not reordered or rearranged during loop pipelining, then the possible use of HLS pipelining will not affect verification, provided the pipelines flush automatically. Even if the design or testbench are sensitive to changes in latency, or if they are sensitive to reordering or rearranging of external memory accesses due to the use of HLS loop pipelining, then the behavior of the DUT will only change in expected (rather than unexpected) ways, provided the applicable observation-boundary and witness-selection conditions are satisfied. (Appendix G provides formal trace-compatibility guarantees rather than an unrestricted bidirectional equivalence theorem.)

- For sequential processes, signal IO operations in the post-HLS model always occur exactly at synchronization points (e.g., wait statements) that are explicit in the pre-HLS source. For direct-input signals/ports, "occur at synchronization points" is meant in the logical/anchoring sense: the Σ-visible Read/Write is timestamped at the synchronization point even if the implementation samples the stable signal later; those internal late samples are ε-steps. For combinational processes, observable signal Read/Write events are instead interpreted at the cycle's ε-quiescent combinational fixed point, as specified by R2C.
  The formal proof architecture separates three concerns. Operational semantics retain Issue, Policy L/S, fairness, and DrainDiscipline; the safety/replay layer proves observable compatibility and canonical-history agreement; and the KObs cutpoint layer adds only the current attributed residual offers needed to reconstruct channel state, frontiers, WFG/WFG+, and the selected deadlock predicate. Certificates record the instance, replay history, request inventory, and proof evidence, while an independent checker reconstructs the derived graph and deadlock facts.

Appendix A – Factory Analogy

The scheduling rules described in this document apply to pre-HLS and post-HLS HW models.
Although the rules may seem abstract and perhaps even arbitrary, they are shaped by the need to model systems in the real world that are required to have predictable behavior.

To understand the motivation behind the rules, it might be helpful to draw a simple real-world analogy and its correspondence to the rules described earlier.

Consider a factory that produces various types of wooden furniture:

The factory consists of people (processes) stationed at workbenches with various tools.

Each person is given written instructions about the specific tasks they are to perform (C++ code within a process).

People are instructed to send or receive objects (messages) to or from other people in the factory.

Sending or receiving objects may be blocking or non-blocking from the perspective of a person.

There is a clock with a second hand on the factory wall that everyone can see. (HW clock).

People have colored flags they can raise or lower to communicate with other people (signals).

If someone raises or lowers a flag, this is only seen by others the next time the clock second hand is at the top of the clock. (propagation delay of signals between sequential processes)

A person can choose to pipeline the tasks that they were assigned by hiring subordinates (pipeline stages) and having each one do a subtask. In general, this will improve the throughput for the tasks that the person was assigned.

It is possible for two people to explicitly synchronize their work by communicating via a synchronization protocol such as a barrier (synchronization).

It is possible to restart the work of some or all the people by raising a reset flag (HW resets).

Assume:

1. That the time that people take to complete their various tasks is in general variable, and similarly that the time that objects (messages) take to pass between people is in general variable.
2. That each person in general wants to complete their tasks as quickly and efficiently as possible.
3. That the factory needs to be able to make multiple types of furniture at the same time. For example, it might make chairs that need to be different colors or have different styles.

To reliably produce output, each person in the factory will need to adhere to rules like those described in this document.

Here's a sketch of a specific way the above example relates to the scheduling rules:

Person 1 sends chair seats and chair backs to Person 2. This is done with blocking operations, and there is no storage capacity between the people when the objects are sent. (This means that a Push operation cannot complete until the corresponding Pop operation is performed.) Assume that the fastest an object can be sent between people is 1 minute.

The written instructions that person 1 is given are:

```
while (1) {
 // internal processing code for seats and backs ...
 seats.Push(seat_object);
 backs.Push(back_object);
}
```

The written instructions that person 2 is given are:

```
while (1) {
 seat_object = seats.Pop();
```

```
        back_object = backs.Pop();
        // internal processing code for seats and backs ...
       }
```

If both person 1 and person 2 choose to perform their tasks sequentially as written, then objects will be passed over time as:

```
            Person 1              Person 2
Minute 1:   seats.Push(seat1);
Minute 2:   backs.Push(back1);    seat1 = seats.Pop();
Minute 3:   seats.Push(seat2);    back1 = backs.Pop();
Minute 4:   backs.Push(back2);    seat2 = seats.Pop();
Minute 5:                         back2 = backs.Pop();
```

If both person 1 and person 2 choose to perform their IO operations in parallel, then objects will be passed over time as:

```
           Person 1                                  Person 2
Minute 1:  seats.Push(seat1); backs.Push(back1);
Minute 2:  seats.Push(seat2); backs.Push(back2);    seat1 = seats.Pop(); back1 = backs.Pop();
Minute 3:                                           seat2 = seats.Pop(); back2 = backs.Pop();
```

If person 1 chooses to perform their IO operations in parallel, and person 2 chooses to perform their IO operations sequentially as written, then objects will be passed over time as:

```
           Person 1                                  Person 2
Minute 1:  seats.Push(seat1); backs.Push(back1);
Minute 2:                                           seat1 = seats.Pop();
Minute 3:  seats.Push(seat2); backs.Push(back2);    back1 = backs.Pop();
Minute 4:                                           seat2 = seats.Pop();
Minute 5:                                           back2 = backs.Pop();
```

If person 1 chooses to perform their IO operations sequentially as written, but person 2 chooses to perform their IO operations sequentially in the reverse order as it was written, then objects will be passed over time as:

```
            Person 1              Person 2
Minute 1:   seats.Push(seat1);
Minute 2:   backs.Push(back1);    back1 = backs.Pop();
Minute 3:                         seat1 = seats.Pop();
```

In this case the person 1 seats.Push(seat1) operation at minute 1 will not complete at the start of minute 2. This means that the person 1 backs.Push(back1) operation will never start, and thus the Person 2 back1 backs.Pop() operation will never complete. So, the system will be in deadlock.

This example directly corresponds to the ordering rules related to message-passing operations in the *basic conceptual model* within this document, and in particular the specific rule that disallows reversing the order of message-passing operations.

Appendix B – Formal Presentation of Pipelined Loops

Modeling convention for pipelined-loop overlap.

Pipelined-loop body no-wait condition. The formal pipelined-loop model in this appendix applies only to HLS-pipelined loop bodies that contain no wait/ac_wait statement, SyncChannel/ac_sync call, or other explicit synchronization interface call inside the loop body. All ordinary synchronization calls selected into S must be outside the pipelined loop body or at explicit boundaries around the pipelined region. This is a well-formedness condition for applying the Appendices G-K theorem stack to a pipelined-loop instance; an in-loop synchronization call requires source rewriting, non-pipelined/preserved-loop treatment, or a separate proof/certificate outside this appendix.
The post-HLS pipelined-loop implementation may overlap iterations and may internally accept/stage up to the finite bound B(c_in) of input values for younger overlapped iterations before older iterations' outputs are produced. This internal accept/stage behavior is part of the back-annotated channel realization counted in B(c_in) between the Σ-visible endpoints of c_in (producer Push_{c_in} and consumer Pop_{c_in}). Here B(c_in) is an effective abstract-boundary capacity/credit bound, not a claim that the RTL contains a single physical FIFO named c_in; the proof obligation is that the chosen Sys_ref boundary exposes E4-consistent ChannelEnabled/ChannelDisabled behavior, with hidden physical storage represented as ε-state unless it is explicitly elaborated into Sys_B^elab channels. These internal accept/stage movements are modeled as ε-steps. **Σ records only the committed boundary transfers at the Σ-visible endpoints.** In particular, the consumer-side boundary Pop_{c_in} is the loop-body Pop_{c_in} operation from the pre-HLS source when that Pop is included in the chosen Σ / Σ_DUT projection. The externally relevant effect of pipelining is therefore captured entirely by B(c_in) (via boundary backpressure/acceptance behavior and overlap timing at the Σ-visible boundary), without introducing any new auxiliary or duplicate Σ-visible boundary Pops: the Σ-visible consumer-side Pop_{c_in} commits are still the same loop-body Pop_{c_in} operations from the pre-HLS source, though they may commit at different times (and thus at different prefixes) than in an unpipelined execution. Once the pipeline reaches a quiescent blocked state under the automatic flush discipline below, it does not initiate new blocking request assertions for younger overlapped iterations.

B-faithfulness obligation

The use of B(c_in) in this modeling convention assumes that the selected abstract boundary and the reported capacity faithfully summarize the corresponding RTL realization: all storage/credit that can accept, hold, or backpressure messages between the chosen producer and consumer endpoints is either counted in B(c_in) or represented as an explicit Sys_B^elab boundary, and movements inside that realization are ε-steps that do not create, delete, duplicate, reorder, or relabel Σ-visible boundary transfers. This is a tool/RTL-flow compliance obligation, not a fact derived by Appendix B.

Multi-input Pop points and coupler policies.

When a pipelined loop consumes from multiple input channels (e.g., c1,c2), back-annotation still assigns a separate finite bound B(ci) and cycle-boundary occupancy occ_ci(t) to each outer Σ-visible boundary channel ci. However, in the multi-input / coupler cases of Appendix Q the formal source-side proof target is no longer merely the outer-boundary Sys_B; it is the elaborated reference model Sys_B^elab of Appendix J, Remark 2. In Sys_B^elab, B(·), occ_·(t), E4, BoundaryReq, ChannelEnabled/ChannelDisabled, and the Appendix K blocking/WFG machinery are interpreted per explicit abstract channel, including any

introduced staged/bundle/join channels. Any conjunctive/join dependency (e.g., “join not met”) shall therefore be represented only via such explicit staged/bundle/join boundary channels in Sys_B^elab (not as cross-channel disablement at the outer Σ-visible endpoints), so that for Appendix K the blocking point is that explicit boundary and the relevant Front_P(σ) members there are jointly channel-disabled (“ALL disabled ⇒ stall”). For the detailed elaboration patterns and examples, see Appendix Q.

Practical note (outer Σ_DUT projection and proof-internal Σ)

The loop-body Pop_{c_in} in the pre-HLS source (i.e., the consumer-side Pop_{c_in} operation, conceptually after the in-flight capacity B(c_in)) may be exposed in the outer DUT/testbench-visible alphabet Σ_DUT if desired, but is typically omitted since it is latency-insensitive, and it is often practically difficult to expose/collect in the post-HLS RTL. The liveness/deadlock reasoning still accounts for this blocking point without requiring it to be Σ_DUT-visible: Appendix K defines WFG edges using the ChannelEnabled/ChannelDisabled status of the corresponding pending boundary operation instance q with kind(q)=Pop_{c_in} (via E4/occ_c/B(c)). In ordinary non-elaborated Sys_B cases, this does not require introducing any additional proof-internal abstract channel. In elaborated cases Sys_ref = Sys_B^elab, however, any introduced staged/bundle/join channel needed to represent a blocking point is an explicit proof-internal Σ boundary for Appendix K, even if its committed Push/Pop events are projected away from the outer Σ_DUT trace by Π_elab.

Drain-only blocked-frontier discipline and automatic flush.

Potential deadlocks are avoided by having the pipeline automatically flush.
In this document, “automatic flush” does not mean a global pipeline clear or a mandatory drain-to-empty phase. It is the pipelined or otherwise overlapped implementation of the more general drain-only blocked-frontier discipline used by Appendix K. Once the minimal pending blocking frontier is fully channel-disabled after ε-settling, the process is blocked at that frontier; no new later source work may issue past that frontier; already asserted blocking requests remain asserted until commit; and already-issued downstream work may continue to drain only insofar as it does not create new WFG-relevant blocking causes. For ordinary non-overlapped control, the same Appendix K discipline may instead be discharged structurally, as described below.

Terminology (pending blocking frontier / Front_P(σ))

In an ε-quiescent cycle, consider the set of pending source-level blocking message-operation frontier items x ∈ Ω_msg,P in the current interval between adjacent synchronization calls, where each such item has an owning dynamic message-call instance q = MsgOf_P(x) whose Issue(q) has occurred and whose Commit(q) has not yet occurred. By the Issue/Commit linkage (Appendix C) together with BoundaryReq’s definition at the Σ-visible boundary (as given in “Common Formal Definitions for Appendices I-K”), each such pending blocking frontier item x has its endpoint-owned boundary request asserted through q (Push: vld, Pop: rdy) until q commits; purely combinational channel-side structure (including couplers) may affect only the complementary mirror signal (Push: rdy, Pop: vld) and thus whether x commits in that cycle after ε-settling. Define Front_P(σ) to be the set of such pending blocking message-operation frontier items that are minimal with respect to the source-induced order ≤_P within that interval. Under the default ordinary source-order convention, source-level message calls that appear in sequence in one process are ordered, so an ordinary non-elaborated source interval has at most one minimal pending blocking message-operation frontier item. If Front_P(σ) has multiple message-operation members, that multiplicity must come from explicit Sys_B^elab proof-internal structure such as staged/bundle/join/epoch-gate abstract channels; it is not inferred merely from the

fact that ordinary source calls use distinct interfaces. We refer to Front_P(σ) as the pending blocking frontier (or "minimal pending blocking frontier"). This Front_P(σ) notion is distinct from NextFront_P (Definition R.6; reconstructed from KObs by Lemma R.21b and Corollary J.1), which denotes the next source-level frontier-item set over Ω_P and may include synchronization-call items, signal-anchored action items, and message-operation items. A message-operation item in NextFront_P may be pending before it contributes any committed Σ-event.

A pipelined loop supplies the drain-only blocked-frontier discipline by automatically flushing if, whenever the pending blocking frontier Front_P(σ) is non-empty and every frontier item x ∈ Front_P(σ) cannot make progress because it is not channel-enabled (evaluated after combinational ready/valid/backpressure has ε-settled for the cycle), then:

1. **Block point.** The process is blocked on message passing at that pending blocking frontier Front_P(σ), as in the unpipelined source.
2. **No new later work.** While blocked at that frontier (i.e., while Front_P(σ) remains non-empty and all of its members are channel-disabled), the implementation does not issue any new blocking Pop/Push message-operation frontier item y ∈ Ω_msg,P that is later in source order than some x ∈ Front_P(σ), including any younger overlapped iteration, and therefore does not begin asserting BoundaryReq(y,·) for such later work. Here issue is read through the owning message instance q_y = MsgOf_P(y).
3. **No withdrawal.** While blocked, the implementation does not withdraw any already-asserted blocking Pop/Push request (no "try-and-withdraw"); outstanding blocking requests remain asserted until their owning message instances commit.
4. **Clarification (Frontier-minimality vs. other asserted requests).** The Wait-For Graph in Appendix K is intentionally defined from the minimal pending blocking frontier Front_P(σ), i.e., the current blocking causes. Accordingly, an already-asserted blocking request whose frontier item is not in Front_P(σ) (for example, a request from a younger overlapped iteration that was issued before the blocked condition, or a downstream request draining work already past the frontier) may persist as protocol state under the "no withdrawal" rule, but it is not treated as a wait-for source while an earlier pending blocking frontier item remains frontier-minimal. Such a request contributes WFG blocking only if/when its frontier item later becomes a member of Front_P(σ).
5. **Drain-only downstream progress**. Work that has already passed the blocked frontier point(s) may continue to drain, and any already-asserted resulting output-Push requests may commit when channel-enabled, provided no work advances past the blocked frontier point(s). (Formally: "no work advances past" is meant in the ≤_P / issue sense—while Front_P(σ) is fully disabled, P does not issue any new blocking message-operation frontier item y, with owning instance q_y = MsgOf_P(y), such that, for some x ∈ Front_P(σ), x ≤_P y and x ≠ y; downstream drain may only complete/commit operations whose BoundaryReq was already asserted prior to reaching this blocked condition.) (Equivalently: this is the "ALL disabled ⇒ stall" policy—P stalls only when none of the frontier items in Front_P(σ) are channel-enabled; downstream work may drain while Front_P(σ) is fully disabled.)
6. **Finite explicit boundary storage/credit**. Every selected abstract boundary that can hold or credit already accepted work has a finite declared capacity B(c), or is represented by an explicit finite Sys_B^elab boundary chain whose component capacities are finite and B-faithful. Together with "no new later work," this makes the set of drainable work finite and non-replenishing.
7. **Synchronization-boundary withdrawal of future-epoch acceptance.**
   Let s be an explicit synchronization interface call that separates the current source interval from later work. While the process is pending at s, the implementation shall not present producer-facing availability for any back-annotated capacity intended only for suff_S(s); equivalently, any

asserted upstream producer request targeting such capacity shall remain channel-disabled until s commits. This does not prevent drain of work already accepted before reaching s. **Formal elaboration requirement**. For purposes of Appendices J–K, this withdrawal shall be represented in Sys_ref by the elaborated source model Sys_B^elab, not by adding a hidden extra enablement predicate to an otherwise E4-enabled outer channel. Sys_B^elab shall introduce explicit sync-boundary / epoch-gate abstract channels, or equivalent explicit staged realization structure, so that capacity usable only by suff_S(s) is separated from capacity available to the current interval. While s is pending, the gate/boundary for future-epoch acceptance is channel-disabled in the ordinary E4 sense at that explicit abstract boundary. After s commits, the corresponding post-sync capacity may again become producer-facing available according to the ordinary ChannelEnabled rules for the elaborated channels.

General drain-only blocked-frontier obligation used by Appendix K

Appendix K applies the drain-only blocked-frontier discipline to every source interval in which Policy L can leave source-later same-interval blocking work issued while an earlier minimal blocking frontier remains uncommitted. This includes pipelined-loop overlap and other explicitly modeled eager/overlapped issue admitted by the selected Sys_ref transition system; it is not a license to treat ordinary source-sequenced operations as incomparable, and it is not limited to physically pipelined control.
For pipelined or otherwise overlapped control, Appendix B automatic flush is the implementation-side discharge. For ordinary non-overlapped control, the obligation may be discharged by proving that, after the process reaches a channel-disabled blocked point, its control cannot launch any new source-later dynamic operation until the blocker commits. In both cases, Appendix C supplies the non-withdrawal discipline, and E4/B-faithfulness plus explicit Sys_B^elab accounting supply finite boundary storage or credit. The resulting drain set may shrink through already-issued commits but cannot be replenished across the blocked frontier.

Example (II=1, latency=4, automatic flush): a loop that performs one blocking Pop_{c_in} and one blocking Push_{c_out} per iteration in the pre-HLS source may, after pipelining in the post-HLS RTL, internally accept/stage up to four input values into pipeline staging (pipeline overlap) before the first output Push_{c_out} commits, subject to the back-annotated bound B(c_in) (E4). Values may then reside for several cycles in internal pipeline storage; such internal staging/hand-off is ε-state within the back-annotated realization and does not create any new auxiliary or duplicate Σ-visible Push_c/Pop_c events beyond the boundary commits at the Σ-visible endpoints (it may change only the timing of those boundary commits under overlapped execution). The drain-only blocking behavior is as defined above under “automatic flush”; in particular, if the blocked operation is in a late pipeline stage (e.g., the final Push), there may be little or no downstream work to drain.

Appendix C – Formal Definitions of Issue and Commit

Scope / shared definitions

The definitions below are with respect to the Σ-visible endpoints of the chosen source reference model Sys_ref (Appendix J, Remark 2)—i.e., the ready/valid interfaces at which Σ records message-transfer events in the raw rendezvous case Sys, the ordinary bounded-FIFO case Sys_B, or the elaborated back-annotated case Sys_B^elab. All shared notions about (i) Σ vs. ε steps, (ii) BoundaryReq / ChannelEnabled / ChannelDisabled, (iii) non-blocking polls as ε on failure, and (iv) the B(c) / occ_c(t) (cycle-boundary,

non-fall-through) interpretation of E4 are as defined in “Common Formal Definitions for Appendices I-K” and are to be interpreted over the explicit abstract channels of Sys_ref. Appendix C defines only Commit, Issue, and the Issue/Commit linkage conditions that make Issue(q) well-defined.

Definition: Commit (message channel) — as observed by an external clocked observer
Commit is the cycle in which an external clocked observer sees a transfer complete at a Σ-visible boundary endpoint: A message commits in the cycle when the observer sees both valid and ready high at the same time. The committed payload is the data value seen in that same cycle.

Definition: Issue (message channel) — as an auxiliary (scheduler/witness) timestamp
Issue(q) is an auxiliary timestamp used to state scheduling constraints (R4/E3/E5). Intuitively, it is the first cycle in which the calling process begins requesting the transfer corresponding to the source-level message operation instance q at the chosen Σ-visible boundary endpoint.

Same-cycle rendezvous clarification. For a rendezvous channel B(c)=0, if a matched Push_c operation instance and Pop_c operation instance first assert their endpoint-owned requests in the same cycle t, and the ready/valid rendezvous is selected/observed in that cycle, then both operation instances have Issue = t and both committed transfer events occur in cycle t. There is no implicit extra cycle between same-cycle matching endpoint requests and the corresponding rendezvous commit. If one endpoint request was already asserted from an earlier blocking operation, then its Issue may be earlier than t, but the matched Push_c/Pop_c commit is still a same-cycle commit in cycle t.

Issue/Commit linkage (well-formedness conditions)

Fix a Σ-visible endpoint direction, and let req(t) denote the endpoint-owned boundary request signal in cycle t (valid for Push, ready for Pop), i.e., the process-driven request level captured by BoundaryReq at that boundary (per “Common Formal Definitions for Appendices I-K”). For each source-level message operation instance q on that endpoint direction:

- Boundary-request soundness (blocking requests are non-withdrawable). req(Issue(q)) = 1. If q is a blocking Push/Pop, then once issued it remains the unique outstanding request on that endpoint direction until it commits: req(t)=1 in every cycle t from Issue(q) through the commit cycle of q (inclusive), with stable payload while asserted for Push. (This is the “no deassert-before-commit” discipline used throughout the document.)
- Stable attribution for a single outstanding source-level request. If req(t)=1 and no transfer commits on that endpoint direction in cycle t, then any continued assertion req(t+1)=1 is attributed to the same outstanding source-level message-operation instance q only so long as the requesting source-level operation remains that same outstanding instance. For blocking Push/Pop, this is the mandatory non-withdrawable case above, so attribution cannot change before commit. For non-blocking PushNB/PopNB, continuous assertion of the endpoint request signal, by itself, does not identify later executed non-blocking call instances as the same q; a new executed source-level non-blocking call creates a new dynamic instance $q' \in Q\_exec,P$ for the owning process P, even if req does not deassert between cycles.
- Back-to-back issuance under continuously asserted req. If q_o commits in cycle t and the next source-level operation q_1 on the same endpoint direction is issued back-to-back, then $Issue(q_1)=t+1$ (even if the request signal does not deassert between commits), provided q_o and q_1 lie within the same source interval between adjacent ordinary synchronization calls selected into S.
- Sync-boundary discipline. If $q_o \in pref\_S(s)$ and $q_1 \in suff\_S(s)$ for an ordinary synchronization call s selected into S, then $Issue(q_1)$ is strictly after s’s commit in the same trace (i.e., $Issue(q_1) >$

clk_pre(s) in τ_pre and Issue(q_1) > clk_post(s) in τ_post). Any boundary request assertion that persists while the process is still pending at s is attributed to the outstanding operation(s) in pref_S(s) and does not constitute issuance of any operation in suff_S(s). Under the pipelined-loop body no-wait condition, there are no in-body wait events that create additional sync-boundary cuts inside an HLS-pipelined loop.

For non-blocking message passing (PushNB/PopNB), a failed poll produces no Σ-event and is modeled as an ε-step, while a successful poll contributes the corresponding committed Push_c(v) / Pop_c(v) event; see "Non-blocking message-passing" in "Common Formal Definitions for Appendices I-K" and Appendix I.2. Each executed non-blocking call is a distinct dynamic source-level message-call instance q ∈ Q_exec,P for the owning process P. Thus, if repeated PopNB/PushNB polls occur in later loop iterations or at later source-level call instances, they are distinct q's even if the endpoint request remains continuously asserted and no intervening commit occurs. Stable attribution therefore applies to a single outstanding source-level request instance; it does not collapse distinct non-blocking dynamic call instances across iterations into one q. Successful non-blocking calls that commit induce message-operation frontier items x ∈ Ω_msg,P when their committed transfer is relevant to Ω_P, with owning message instances q = MsgOf_P(x) in Q_Ω,P. Failed non-blocking polls remain only in Q_exec,P and contribute no Ω_P or Ω_msg,P item.

Appendix D – Modeling Latency-Sensitive Protocols

In some cases, designs or testbenches may use protocols or local control policies which are latency-sensitive. These situations are handled by isolating the latency-sensitive portions into small, self-contained parts, and then keeping the rest of the design and testbench latency insensitive.
A latency-sensitive portion is any local block of logic whose functional or externally visible behavior depends on cycle-level timing that is not represented by the ordinary latency-insensitive Σ interface. This includes ordinary signal-level protocols with fixed cycle timing, global interrupt/request ordering logic, non-blocking arbitration whose winner depends on request arrival timing, and non-blocking retry/timeout logic whose later behavior depends on the number or cadence of failed PushNB/PopNB polls before success.

For such logic, the intended methodology is not to treat the hidden timing dependence as part of the ordinary latency-insensitive source-core proof. Instead, the latency-sensitive portion should be isolated behind an explicit boundary. On one side of the boundary, the transactor or local protocol block may be modeled cycle-accurately, snooped, or otherwise verified with its detailed timing behavior. On the other side of the boundary, the rest of the system should see a latency-insensitive interface, such as message-passing transactions, synchronization events, or a properly handshaken signal protocol.

Consider a case where a signal-level protocol is latency-sensitive. The protocol will have specific timing behavior that it must meet. To handle this, we create a transactor that has two sides: the first side interacts with the signals and handles the detailed timing requirements, and the second side sends and receives messages with the rest of the system. The message-passing side is latency insensitive. The same pattern applies to cadence-sensitive non-blocking polling logic: if retry counters, timeout counters, or arbitration decisions depend on the number or cadence of failed non-blocking polls, that polling loop is part of the latency-sensitive local protocol block. Its detailed failed-poll cadence should be contained inside the isolated block or aligned by the Appendix L snooping mechanism; it should not leak as an unmodeled hidden timing dependence into the ordinary Appendix I/J latency-insensitive proof.

To properly model the detailed timing behavior, the latency-sensitive transactor or local protocol block is modeled at the cycle-accurate level in SystemC, or is otherwise covered by an explicit snooping/alignment argument. There is an example of such a transactor model in the following document and example:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/blob/master/matchlib_examples/examples/53_transactor_modeling/transactor_modeling.pdf

For purposes of the formal appendices, once the latency-sensitive portion has been isolated, the ordinary Appendix I/J proof is applied at the chosen observable boundary of that isolated block. The failed-poll cadence internal to the isolated block remains local timing behavior; the rest of the system observes only the latency-insensitive or explicitly modeled boundary behavior.

Appendix E – One-Way Handshake Protocols

In some systems a one-way signal handshake is sufficient for reliable system operation. When using a one-way handshake, the implicit assumption is that the "missing" handshake isn't required since it is always true.

For example, in time-domain signal processing hardware designs, signal processing HW blocks may input new samples at a fixed rate. The HW blocks are always ready to receive new samples on each clock, but they need to know if the samples are valid. These types of designs can be modeled with the one-way handshake dat/vld protocol demonstrated in this example:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/tree/master/matchlib_examples/examples/32_dat_vld

Appendix F – Scheduling Rules: Modeling Guidelines Summary

1) Prefer to use Connections::In/Out + SyncChannel over signal IO and wait().

2) Prefer to use Pop()/Push() over PopNB()/PushNB().
3) When pipelining a loop, prefer to use `hls_stall_mode flush`.

When using signal IO:

4) If modeling a cycle-accurate process, use `disable_spawn` and follow the example 53_transactor_modeling style and do not use Push/Pop in the process.

5) If modeling a *direct input*, use `hls_direct_input` and possibly `hls_direct_input_sync`, and only change the signal at allowed times.

6) If combining signal IO with Connections::In/Out in the same process, use proper signal IO handshake that does not rely on In/Out ports for process synchronization. Place signal IO operations very close to their wait() statements.

7) Unless you are modeling a cycle-accurate process, you should expect that latency will change between the pre-HLS and post-HLS models.

Appendix G – Trace Compatibility Rules

This document informally states that a primary goal is to present "no surprises" to a DV engineer using the same testbench to verify the pre-HLS and post-HLS models.

Somewhat more formally, we can show how the scheduling rules within this document establish a set of trace-compatibility rules between the pre-HLS and post-HLS models that provide strong guarantees about their behaviors.

The *basic conceptual model* establishes the base behavior for a single process:

1) Synchronization interface calls always remain in source code order.
2) Signal reads occur at closest preceding synchronization interface call.
3) Signal writes occur at closest succeeding synchronization interface call.
4) All message-passing operations before a call to a synchronization interface shall be issued before or in the same cycle in which the synchronization interface commits. (For blocking message-passing operations (Push/Pop), this implies the corresponding commits occur no later than the synchronization call's commit cycle.)
5) All message-passing operations after a call to a synchronization interface shall be issued after the cycle in which the synchronization interface call commits.
6) Two message-passing operations on separate interfaces which appear in sequence in the model are source-ordered. They may issue with timing overlap or in the same clock cycle where the selected issue policy allows, but they shall not be issued in the reverse sequence. Operations on the same message-passing interface are issued in source order (they are never reversed).
7) Synchronization calls can affect the scheduling of signal IO operations, and synchronization calls can affect the scheduling of message-passing calls, but message-passing calls cannot affect the scheduling of signal IO operations and vice-versa.
8) The STRICT_IO_SCHEDULING=relaxed option is not used.
9) We distinguish the rendezvous source model Sys (B(c)=0 for all channels) from the buffered source interpretation Sys_B, which uses bounded-FIFO channel capacities B(c) matching the post-HLS RTL (Appendices I–K).

Conceptually, systems are composed of many such processes which follow the basic conceptual model, and which communicate using only synchronization interface calls, latency-insensitive signal-level protocols, and committed message-passing transfers. If a design uses non-blocking message-passing calls (PushNB/PopNB), then unsuccessful polls are treated as internal ε-steps (no Σ-event), and each successful completion is represented in Σ as the corresponding committed Push_c/Pop_c transfer (Σ records transfers, not failed polls). If the resulting latency-sensitive behavior is externally visible at the DUT boundary, the snooping wrapper of Appendix L may be used to align the selected admissible pre-HLS execution with the RTL. Here we call this system of processes the *conceptual system*.

Practically, the modeling approach described above is too restrictive for real-world systems with optimized HW, so the scheduling rules in this document allow for key HW optimizations. But this is done

in a carefully controlled manner, such that the HW optimizations do not break equivalent behavior with the *conceptual system*.

Here is a summary of the HW optimizations that the scheduling rules allow, and a brief description of how we maintain equivalent behavior with the *conceptual system*:

1. Pipelined loops: In the post-HLS RTL, the pipeline may overlap iterations to improve throughput. See Appendix B for a formal presentation of pipelined loops.

2. Pipelined loops: HLS by default will not break any latency-insensitive signal IO protocols when pipelining loops.

3. Direct inputs: Signals that do not change after a process comes out of reset can be read as late as possible using *hls_direct_input*, saving register area.

4. Direct inputs: When *hls_direct_input_sync* is used, signals read by a process are updated at the precise point at which the HW pipeline is guaranteed to be ramped down, so the signal values do not need to be saved within pipeline registers, saving register area.

5. Memory accesses: HLS can reorder memory accesses within a process only if it can prove that the reordering is conflict-free.

6. Shared memories: Memories shared between processes are modeled as simple C arrays but always include explicit synchronization between processes in both the pre-HLS and post-HLS models. For the formal results of Appendices I–K, the value-carrying accesses must additionally satisfy the J.Mem lowering condition (Common Formal Definitions).

7. Non-blocking message-passing interfaces: They are modeled at the level of committed transfers (successful completion), with unsuccessful polls treated as ε-steps and thus not part of Σ. If latency differences in these committed-transfer events are externally visible at the DUT boundary and must be aligned for verification, equivalent behavior between pre-HLS and post-HLS systems can be maintained by snooping the post-HLS system and using this to delay pre-HLS message arrival (Appendix L).

8. Latency-sensitive global signal IO: If latency differences are externally visible to the DUT, equivalent behavior between pre-HLS and post-HLS systems can be maintained by snooping the post-HLS system and using this to delay pre-HLS signals.

9. Latency-sensitive local signal IO: Isolate local latency-sensitive protocols to small, dedicated transactors, and use latency-insensitive signal IO or message-passing to communicate with the rest of the system.

10. One-way signal handshake protocols: Only use in cases where backpressure is not possible and embed assertions into the model to check that this is always true.

11. Combinational processes: If the pre-HLS model contains combinational processes, their observable signal Read/Write behavior is interpreted at each cycle's ε-quiescent combinational fixed point. Under the well-formedness requirement that the combinational network is

deterministic and reaches a unique fixed point, the RTL/tool-flow conformance obligation is to preserve the fixed-point signal relation, so combinational signal IO can be included in Σ without using synchronization anchors.

Taken as a whole, these rules support a verification methodology in which full-system verification is performed primarily on the pre-HLS system model, and post-HLS RTL verification can be reduced to targeted checks that the formal side conditions are met. The precise guarantee is the execution-level guarantee of Appendix J: every RTL_B observable behavior under the fixed stimulus, carrying a globally witness-realizable annotation package (Definition J.GWR), has a matching Sys_ref behavior (Post → Ref), while the reverse direction applies only to annotated RTL-consistent source-side prefixes or to source-side executions selected by the Appendix L snooping / witness-selection mechanism. Thus a source-side pass justifies the corresponding RTL_B behavior only within that annotated RTL-consistent / witness-selected subset. For liveness/deadlock guarantees (Appendices I–K), the results additionally assume weak fairness and channel-progress obligations for the scheduler/environment (Appendix N); the cycle-by-cycle R/E rules alone do not imply those progress properties.

Formalization of the Trace Compatibility Rules in Appendix G

The following notation is used in this section.

Symbol Meaning

P	**A process (thread or method) in the design**
Σ	The alphabet of observable IO actions
τ_P	**A (finite or infinite) per-cycle trace of observable actions executed by process P:** for each global clock cycle t, τ_P[t] is the (possibly empty) set of Σ-actions committed by P at t. Actions committed in the same cycle are treated as simultaneous (unordered); only cross-cycle order (and the explicit E1–E5 constraints) is semantically significant.
S	The subset of Σ that are **synchronization calls** (explicit wait, SyncChannel, START_P/FINISH_P indicating process start/finish)
R	The subset of Σ that are **signal reads**
W	The subset of Σ that are **signal writes**
M	The subset of Σ that are **committed message-passing transfers** (i.e., committed Push_c / Pop_c events on message-passing channels, including successful non-blocking transfers). Unsuccessful non-blocking polls contribute no Σ-event and are modeled as ε-steps, and therefore are not members of M.
Q_exec	The execution-level set of dynamic source-level message-call instances executed by a process at the Σ-visible endpoints (blocking Push/Pop, and non-blocking PushNB/PopNB calls). Elements of Q_exec may or may not commit. In particular, failed non-blocking polls are elements of Q_exec, have no committed Σ-event, and are modeled as ε-steps. The frontier-item universe Ω_P, defined below, is separate from Q_exec.

P	A process (thread or method) in the design
Ω_P	The local dynamic frontier-item universe for process P in the selected execution / witness. It contains synchronization-call items, signal-action items, and message-operation frontier items; it is distinct from Q_exec,P, the universe of dynamic source-level message-call instances.
Ω_msg,P	The message-operation slice of Ω_P: frontier items x for which MsgOf_P(x) is defined.
MsgOf_P(x)	Partial projection from a message-operation frontier item x ∈ Ω_msg,P to its owning dynamic source-level message-call instance q ∈ Q_exec,P. In Sys_B^elab, MsgOf_P need not be injective: one source-level q may own several elaborated frontier items.
BoundaryOf_P(x)	The explicit abstract channel boundary at which frontier item x is evaluated for BoundaryReq, ChannelEnabled, ChannelDisabled, Front_P, and Appendix K WFG reasoning. In ordinary unelaborated cases this boundary is unique for the owning source-level operation; in Sys_B^elab it is declared by the elaboration package.

A *system trace* is the tuple
τ = (τ_P)_P, one component per process.

For any action a ∈ Σ, let clk_pre(a) / clk_post(a) be the clock cycle at which a is committed.
For any issued message-call instance q ∈ Q_exec, issue_pre(q) / issue_post(q) denotes the auxiliary Issue timestamp for q in the pre-HLS / post-HLS trace, respectively, as defined in Appendix C and required to satisfy the associated Issue/Commit linkage (well-formedness) conditions with respect to the Σ-visible boundary request signal on q's endpoint direction.
If q commits, it contributes a unique committed transfer m(q) ∈ M with commit cycle clk_pre(m(q)) / clk_post(m(q)). If q does not commit (e.g., an unsuccessful non-blocking call), then m(q) is undefined / absent and no Σ-event is added to M.
(Q_exec-matching / witness convention.) Because unsuccessful non-blocking polls are ε-steps and produce no Σ-event, Q_exec is not in general reconstructible from the Σ-projection alone. Whenever E3, E5, or any later proof compares executed message-call instances across Sys_ref and RTL_B, the comparison is made relative to the chosen execution witness. That witness supplies a partial matching μ_Q between dynamic source-level message-call instances in the compared traces, preserving process identity, endpoint/interface identity, dynamic source-order position, and source-level call-instance identity where applicable. If q commits, then μ_Q(q) commits the corresponding matched transfer m(μ_Q(q)) with the same Σ-label/value under the event-matching convention. If q is an unsuccessful non-blocking poll, then q has no m(q) and no Σ-label; its presence and its match are supplied only by the witness-compatibility condition WC, by the sufficient NB-CF condition when applicable, or by the Appendix L snooping / witness-selection construction. Thus E3/E5 quantify over Q_exec only relative to this chosen witness; they do not require failed non-blocking polls to be inferable from Σ alone.

(No deassert-before-commit assumption.) For blocking message requests q ∈ Q_exec, once q is issued, its Σ-visible boundary request is not withdrawable: it remains asserted in every cycle from issue_pre(q) (resp. issue_post(q)) through q's commit cycle (if it commits), and for Push the payload remains stable while the request is outstanding.

Dynamic source-order convention

The order relation used below is over dynamic source-level operation/frontier items in the compared execution / witness, not over all syntactic call sites in the source text.

For a process P, let Q_exec,P denote the execution-level set of dynamic source-level message-call instances of P at the chosen abstract boundary. Q_exec,P includes blocking Push/Pop calls, successful non-blocking PushNB/PopNB calls, and failed non-blocking PushNB/PopNB polls.

Separately, let Ω_P denote the witness-specific dynamic universe of source-level frontier items that are actually reached under the fixed stimulus / selected witness and that are relevant to observable/frontier/deadlock reasoning. Items in Ω_P include synchronization-call instances, anchored signal Read/Write action instances, and message-operation frontier items at the chosen abstract boundaries. A message-operation frontier item is either (i) a blocking Push/Pop item that has been reached, whether pending or committed, or (ii) a successful non-blocking PushNB/PopNB item that contributes a committed Push_c(v) / Pop_c(v) event. A failed non-blocking poll q is an element of Q_exec,P, has no committed Σ-event m(q), and is not an element of Ω_P merely because it executed.

Because Q_exec,P is a universe of dynamic message-call instances while Ω_P is a universe of source-level frontier items, their message-operation overlap is not a literal set intersection. Let MsgOf_P(x) be the partial projection from a frontier item $x \in \Omega\_P$ to its owning dynamic message-call instance, defined exactly for message-operation frontier items and undefined for pure synchronization or signal-anchor items. Define the message-frontier slice

$\Omega\_msg,P := \{ x \in \Omega\_P \mid \exists\, q \in Q\_exec,P : MsgOf\_P(x) = q \}$.

Define the corresponding message-instance slice induced by Ω_P as

$Q\_\Omega,P := \{ q \in Q\_exec,P \mid \exists\, x \in \Omega\_msg,P : MsgOf\_P(x) = q \}$.

Let Σ_P denote the corresponding observable labels/events contributed by items of Ω_P when they are Σ-visible. For a message-call instance $q \in Q\_exec,P$, the committed Σ-event $m(q) \in M$ exists only if q commits; if q is a failed non-blocking poll, m(q) is absent and q contributes no Σ-event. For a message-operation frontier item $x \in \Omega\_msg,P$ with MsgOf_P(x)=q, item x contributes the committed Σ-event m(q) only if q commits. A pending blocking message-operation frontier item x may therefore be in Ω_P and Ω_msg,P, and its owning message instance q may be in Q_Ω,P, even though m(q) is not yet present.

Operations on untaken branches and unexecuted loop iterations are not members of Ω_P, Ω_msg,P, Q_Ω,P, or Q_exec,P. Distinct loop iterations, unrolled instances, and repeated calls are represented by distinct dynamic instances.

Default source order for ordinary message calls. For dynamic items x and y of process P, write $x <\_src\ y$, or equivalently $x <\_P\ y$ when the process must be explicit, to mean that P's source control flow and the R-rules force x to precede y in that dynamic execution / witness. For ordinary user-written source-level message-call instances in a sequential process, this is the dynamic program order along the executed control path. Thus, if two executed message-call instances $q1,q2 \in Q\_exec,P$ appear in sequence in the same process, exactly one of $q1 <\_src\ q2$ or $q2 <\_src\ q1$ holds according to that dynamic program order, including when q1 and q2 use distinct message-passing interfaces, occur in the same synchronization

interval, arise from repeated loop iterations, or arise from unrolled instances whose source-level expansion is ordered by the selected source elaboration. Same-cycle issue, same-cycle commit, or issue of a later operation before an earlier operation commits is a timing/issue-policy freedom; it does not make ordinary source message-call instances incomparable. The no-reverse rule R4/E3 is therefore stated over this ordered dynamic source relation.

Limited sources of incomparability. The relation ≤_P used for frontier reasoning is the reflexive closure of the restriction of this source-induced order to Ω_P, extended only by explicitly modeled partial-order structure. Ordinary distinct-interface source calls are not incomparable merely because they are on different interfaces. A message-operation multi-frontier may arise only from explicit Sys_B^elab proof-internal structure such as staged/bundle/join/epoch-gate abstract channels whose elaboration package declares the relevant frontier items, evaluated boundaries, partial order, and projection/attribution facts. Such elaborated frontier items are part of the selected Sys_ref proof model; they are not an implicit weakening of source order among the ordinary user-written message calls.

For sequential-process signal IO, <_src / <_P is interpreted through the R2 anchor semantics rather than through the raw lexical statement location of the signal read or write. Thus a signal Read is ordered, for formal frontier and trace-compatibility purposes, at its pred_S anchor, and a signal Write is ordered, for formal frontier and trace-compatibility purposes, at its succ_S anchor. A lexically earlier signal Write whose succ_S anchor is a later synchronization call does not, merely by lexical placement, precede or block intervening message-operation instances in the same source interval; conversely, a lexically later signal Read whose pred_S anchor is an earlier synchronization call does not, merely by lexical placement, wait behind intervening message-operation instances. Signal IO may constrain message IO only through the explicit synchronization anchors and the stated R/E rules, not by raw lexical adjacency inside an interval.

Conceptual Model for a Single Process

For every process P the pre-HLS trace τ_P must satisfy the following *partial-order constraints* over these dynamic operation instances:

R1 (Program order of S).

∀ s1, s2 ∈ S : s1 <_src s2 ⇒ clk_pre(s1) < clk_pre(s2)

R2 (Location of R/W for sequential processes).

For a sequential process P, ordinary signal Read/Write events are anchored to synchronization events as follows:

∀ r ∈ R_P : clk_pre(r) = clk_pre(pred_S(r))
∀ w ∈ W_P : clk_pre(w) = clk_pre(succ_S(w))

where pred_S(r) (resp. succ_S(w)) is the closest dynamic synchronization-call instance that precedes (resp. follows) the signal Read/Write on the dynamic source path of the current execution / witness, after applying the anchoring convention above. This anchoring determines the signal event's formal position for ≤_P / <_P, Done_P, Remain_P, and NextFront_P reasoning. In particular, for a sequential-process signal Write w, the fact that the write statement appears lexically before a message operation q

in the same synchronization interval does not imply w <_P q when w is anchored to the succeeding synchronization call; the write's formal observable event is placed at succ_S(w). Likewise, a signal Read anchored to pred_S(r) is not delayed by intervening message operations merely because the read statement appears lexically after them.

Every observable signal Read/Write of a sequential process must have a real bounding synchronization anchor on the dynamic path on which it occurs. A process-start synchronization event START_P may serve as the initial predecessor anchor when the modeling convention explicitly includes START_P in S. A terminating process event FINISH_P may serve as the terminal successor anchor when the process actually terminates and FINISH_P occurs. If no real predecessor anchor exists for a Read, or no real successor anchor exists for a Write on the dynamic path being modeled, the sequential-process signal IO is ill-formed for this formal model or lies outside the finite observable prefix being compared. No observable Read/Write event is assigned clock time ∞.

R2C (Combinational signal IO at the ε-fixed point).

A combinational process P has no process-owned synchronization events and no message-passing events in this formal clause; it may, however, contribute observable signal Read(sig,val) and Write(sig,val) events to Σ through explicitly selected R2C observation components. These events are not anchored by pred_S/succ_S. In every clock cycle t, the global state first reaches the cycle's ε-quiescent combinational fixed point after all sequential clock-boundary updates and all internal combinational propagation for that cycle have settled. Let μ_t denote this unique fixed-point valuation of the signal network.

R2C unit-generation convention

Fix an explicitly modeled combinational process or combinational
network component C_comb and its selected observable slice Obs(C_comb), consisting of the process/signal/direction labels whose fixed-point values are included in that component's R2C observation. The fixed-point valuation μ_t exists every cycle, but a Σ-visible R2C fixed-point signal-observation unit for C_comb is emitted only at canonical observation occurrences:
(i) the first cycle in the compared execution in which C_comb's observable slice is defined and observed; and
(ii) any later cycle in which the vector of fixed-point values on Obs(C_comb) differs from the vector in the most recent emitted R2C unit for C_comb, or in which an explicitly modeled observer/sampler requires a fresh observation unit.
Cycles in which C_comb's observable fixed-point slice is identical to the most recent emitted R2C unit, and in which no explicit observer/sampler requires a fresh unit, are R2C stutter cycles for C_comb. They do not emit a new Σ-visible R2C unit. Delta-cycle re-evaluation, intermediate combinational values, redundant re-drive before the fixed point, and such R2C stutter cycles are ε/stutter for this component and are not separately Σ-visible.
If a combinational process P reads sig in an emitted R2C unit of C_comb, the corresponding label is Read(sig, μ_t(sig)) for the cycle t at which that unit is emitted. If P drives sig in an emitted R2C unit of C_comb, the corresponding label is Write(sig, μ_t(sig)), where μ_t(sig) is the final resolved fixed-point value of sig at the chosen observation boundary. All Read/Write labels inside one emitted R2C unit are interpreted at the same ε-quiescent fixed point and may not be split.
Combinational-process signal IO is well formed only when the relevant combinational network is deterministic and reaches a unique ε-fixed point in each cycle being compared, and when each observable R2C component has a well-defined selected observable slice Obs(C_comb). For a multi-input combinational component whose inputs may be updated by independent latency-insensitive sources,

the component is Σ-observable under the stutter-canonicalized R2C rule only if the chosen observation boundary provides a single explicit anchoring domain for the slice, or if an additional component-specific well-formedness proof shows that all admissible interleavings induce the same emitted R2C unit sequence. Otherwise the component's internal combinational propagation must remain ε-hidden, or the design must expose the relevant behavior through explicit synchronization, message passing, a cycle-accurate transactor, or a cycle-locked R2C observation boundary. Pure combinational oscillations, ambiguous multiple-driver resolution, hidden stateful behavior, or intentional combinational-loop behavior outside a unique fixed-point interpretation are outside this R2C model unless represented by explicit state or synchronization elsewhere in the formal model.

R3 (Isolation around S).

Let s be a dynamic synchronization-call instance of process P. Let pref_S(s) be the set of executed dynamic message-call instances $q \in Q_{exec,P}$ in the immediately preceding source interval of P whose source-induced order places q before or at the boundary controlled by s. Let suff_S(s) be the set of executed dynamic message-call instances $q \in Q_{exec,P}$ in the immediately following source interval of P whose source-induced order places q after s. Then
$\forall q \in pref\_S(s) : issue\_pre(q) \le clk\_pre(s)$ and $\forall q \in suff\_S(s) : issue\_pre(q) > clk\_pre(s)$.
(For blocking message requests, if the synchronization call commits in the process trace, then any earlier blocking message requests in that dynamic source interval must have committed in some cycle ≤ that sync-commit cycle, by blocking-call control-flow semantics. Failed non-blocking polls in $Q_{exec,P}$ satisfy the same issue-side synchronization-boundary rule, but they contribute no Σ-event and do not become Ω_P frontier items merely because they executed.)

R4 (Safe message issue ordering).

For every pair $q1,q2 \in Q_{exec,P}$ within the same process and within the compared dynamic execution / witness,
$q1 <_{src} q2 \Rightarrow issue\_pre(q1) \le issue\_pre(q2)$.
(For same-channel operations, this states that dynamic calls on a single interface are issued in source-induced program order. For distinct channels, it forbids issuing dynamic message-call instances in the reverse of their source-induced order. This rule does not impose ordering between syntactic operations on untaken branches or between dynamic instances that are not ordered by the source-induced partial order of the selected witness. Failed non-blocking polls are included in this issue-order universe when they are executed, but remain ε-steps with no Σ-event.)

Prefix-closed reading of issue order. R4 and E3 are to be read not only as inequalities between already-defined issue timestamps, but also as closure conditions on issue itself. Within a compared dynamic execution / witness, if $q1 <_{src} q2$ and q2 has issued by cutpoint σ, then q1 has also issued by σ, and $Issue(q1) \le Issue(q2)$ on the same side of the comparison ($Issue\_pre$ for R4/source-side prefixes, $Issue\_post$ for E3/implementation-side prefixes). Equivalently, in each source interval, the set of issued message-operation instances is downward closed under the source-induced order used by R4/E3. This rule is what excludes the case where a source-later message operation has issued while a required source-earlier message operation in the same interval remains merely reached but unissued.

R5 (Channel capacity semantics; Sys vs. Sys_B).

Appendix G defines the raw rendezvous source model Sys, in which every channel has effective capacity 0. Concretely: for every channel c and any clock cycle t, the number of unmatched committed Push_c actions is zero and the number of unmatched committed Pop_c actions is zero; equivalently, no Push_c

shall commit unless a matching Pop_c also commits at t, and no Pop_c shall commit unless a matching Push_c also commits at t.

In Appendices I–K we additionally use the back-annotated source interpretation Sys_B: the same source processes as Sys, but interpreted under the case-split channel semantics of E4 with capacities B(c) matching RTL_B. Thus:
• B(c)=0 is interpreted as the rendezvous case of E4: matching Push_c(v) and Pop_c(v) endpoint events commit in the same cycle, and no unmatched committed endpoint event may remain after any cycle boundary.
• B(c)>0 is interpreted as the cycle-boundary, non-fall-through bounded-FIFO case of E4, using the occupancy predicate occ_c(t) and the FIFO availability predicates occ_c(t)<B(c) for Push_c and occ_c(t)>0 for Pop_c.

The source conceptual semantics for a process is thus a labeled partial order (Σ, ≤_P) generated by R1–R4 plus the applicable E4 channel semantics: raw rendezvous for Sys, or the case-split back-annotated channel semantics of Sys_B / Sys_ref.

Operational interpretation of Sys_ref and source issue policies

The R-rules above are constraints on a selected source-reference transition system, not merely constraints on a finished list of labels. For a fixed design, fixed stimulus, and selected reference-model choice, Sys_ref has the following operational reading.
A Sys_ref state consists of: (1) one local source state per process, including control location, local variables, call-stack/loop state, the current synchronization interval, and the reached/issued/committed status of dynamic message-call instances; (2) one state for each explicit abstract channel of the selected reference model, including B(c), occ_c(t), FIFO order when B(c)>0, rendezvous endpoint requests when B(c)=0, and any explicit Sys_B^elab staged/bundle/join/epoch-gate channel state; (3) signal and synchronization state at the selected observable anchors; and (4) ε-microstate for internal staging, direct-input sampling between anchors, and other proof-hidden activity. In Sys_B^elab, introduced abstract channels are explicit proof-internal boundaries: their commits may be Σ-events for Appendices J-K even when projected away from the outer DUT/testbench-visible alphabet Σ_DUT by Π_elab.
The transition relation contains the following kinds of steps. Pure source-evaluation ε-steps advance ordinary C++/SystemC computation until an IO boundary is reached and allocate dynamic operation instances in dynamic source order. A message-issue step for q may occur only after q has been reached, the endpoint request can be attributed to q, the R3 synchronization-boundary constraints are respected, and the issued set is prefix-closed under <_src. Issuing a blocking Push or Pop asserts the endpoint-owned persistent request for q and records Issue(q). A channel-commit step for a bounded FIFO boundary B(c)>0 may occur when BoundaryReq(q,σ) and ChannelEnabled(q,σ) hold at the selected abstract boundary; it emits the corresponding Push_c(v) or Pop_c(v) event at that proof boundary, updates the E4 channel state, and releases the owning blocking call at the process-facing boundary. For a rendezvous boundary B(c)=0, commit is a single atomic two-endpoint transfer step: the matching producer and consumer endpoint requests are both true in the same ε-quiescent cycle, and the paired Push_c(v) / Pop_c(v) observable unit commits together with one shared payload value. The rendezvous commit is not modeled as two independent single-endpoint commits that can be separated or ordered apart. Synchronization and signal-anchor steps emit wait/Sync and anchored Read/Write events according to R1/R2/R2C. ε-settling and drain steps update only proof-hidden or explicitly elaborated internal state and must respect the Kε / B-faithfulness obligations when used by Appendices J-K.

Policy L (liberal, witness-relative issue). Policy L is the most permissive source-side issue policy used by the safety witness and by the K.2b dominance/extraction bridge. It may issue a reached source-level message operation whenever the transition-system preconditions above, R3/R4/R5, E4, E5, the selected non-blocking outcome witness, and the source-order prefix-closure requirement are satisfied. Policy L may leave a source-later blocking request issued while a source-earlier same-interval blocking request of the same process remains uncommitted. This is how the reference model can represent loop overlap, RTL-matched eager issue, and other allowed same-cycle/overlapped request timing. Policy L is not a claim that every raw pre-HLS simulator mode can generate every such witness state.
Policy S (conservative serial-blocking issue). Policy S uses the same Sys_ref state space, channels, values, witness choices, and transition rules as Policy L, but adds one process-side restriction: within a source interval between adjacent ordinary synchronization calls selected into S, a later source-order blocking Push/Pop operation may not issue until every earlier source-order blocking Push/Pop operation in that same interval has committed in a strictly earlier clock cycle. In Sys_B^elab, the relevant commit for this restriction is the process-facing commit that completes, accepts, or releases the owning source-level blocking operation at its selected boundary; internal staged/bundle/join/epoch-gate transfers do not by themselves satisfy the serial condition unless the elaboration package identifies that transfer as the process-facing commit for the source operation. Policy S serializes only process-side issue of source-level blocking calls. It does not serialize independent peer processes, E4 channel-side drainage, FIFO movement, or ε-steps that are otherwise admissible.
Relation to practical pre-HLS simulation modes. A raw serial source simulation mode implements the Policy-S discipline for ordinary unelaborated blocking calls. A Matchlib-style eager or skidbuffered source simulation mode may execute source calls in dynamic program order while issuing requests without consuming a clock cycle until data or space is unavailable; that mode is Policy-L-compatible only to the extent that its request attribution, channel states, and non-blocking outcomes satisfy the transition-system conditions above. Neither mode treats ordinary distinct-interface source calls as unordered. The difference between serial and eager simulation modes is a difference in issue timing and buffering, not a difference in <_src.

Trace Compatibility Relation

Let τ_pre and τ_post be the Σ-traces (finite or infinite) of the same process before and after HLS, where each Σ-event is labeled exactly as in “Common Formal Definitions for Appendices I–K” (e.g., Push_c(v), Pop_c(v), Read(sig,val), Write(sig,val), and synchronization events).

Notation note. We write τ_pre ≈ τ_post as a named, ordered trace-compatibility predicate. The symbol ≈ is local to this document and is not used here to assert a mathematical equivalence relation; in particular, reflexivity, symmetry, and transitivity are not being claimed. Its arguments are ordered: the left trace supplies the pre-HLS/source reference witness, and the right trace supplies the post-HLS implementation trace constrained by E1–E5. In later appendices the same convention applies to τ_ref ≈ τ_post, with τ_ref as the prescribed Sys_ref witness.

Convention (S): all references to S, pred_S/succ_S, pref_S/suff_S, and <_src in E1–E5 use the dynamic source-induced order defined in R1–R4 for the compared execution / witness. Thus τ_pre denotes the Σ-trace of the pre-HLS source over the dynamic operation instances reached under the fixed stimulus / selected witness.

Σ-event matching convention. We use a canonical per-endpoint occurrence matching to make “the same Σ-events” precise. For each channel c, match committed Push_c and Pop_c events by their ordinal occurrence on that channel (e.g., the k-th committed Push on c in τ_pre matches the k-th committed Push on c in τ_post; likewise for Pop_c). For each such matched pair, the endpoint (c) and the payload/value label must agree. For the ordinal index k to be well-defined under the per-cycle “set/bucket” trace representation used in these appendices, we assume that for any fixed channel endpoint c, at most one committed Push_c and at most one committed Pop_c may occur in any single clock cycle.

For each observable signal sig of a sequential process, match Read(sig,·) and Write(sig,·) by their ordinal occurrence on that signal within the process trace, again requiring the value labels to agree.

For sequential processes, we apply the following canonicalization for observable signals: within each anchor interval between consecutive synchronization events in the order S, we record at most one Σ-visible Read(sig,val) and at most one Σ-visible Write(sig,val) for each (process, sig), with the anchor cycle given by E2. Any additional physical sampling of sig by the implementation, or redundant re-driving of sig within the same anchor interval, is treated as an internal ε-step and must not change the recorded value label. If a design intentionally relies on intra-interval changes of sig, or on distinguishing multiple reads/writes of sig within the same anchor interval, then sig must be modeled using an explicit synchronized interface rather than as an ordinary sequential-process observable signal.

For combinational processes, the canonical observable unit is the emitted R2C fixed-point signal-observation unit for the relevant combinational process/network component C_comb: the set of Read(sig,val) and Write(sig,val) labels for the selected observable slice Obs(C_comb), recorded at one ε-quiescent combinational fixed point under the R2C unit-generation convention. Intermediate delta-cycle reads/writes, repeated re-evaluations before the fixed point, and stutter cycles in which Obs(C_comb) is unchanged and no explicit observer/sampler requires a fresh unit are ε/stutter and are not separately Σ-visible.

R2C unit matching/index convention

For each combinational process or explicitly modeled combinational network component C_comb, project each trace to the subsequence of emitted R2C fixed-point signal-observation units for C_comb. The k-th emitted such unit in Sys_ref is matched with the k-th emitted such unit in RTL_B. This local emitted-occurrence ordinal, together with the component identity C_comb, is the correspondence rule for R2C fixed-point units; it is not the global observable-cycle bucket number of the whole-system trace. Thus repeated identical fixed-point observations across stutter cycles do not create additional units, while repeated equal-valued units created by explicit observer/sampler occurrences are distinguished by their local emitted-occurrence ordinal for C_comb.

Matched pre-HLS and post-HLS fixed-point units must agree on the included process/signal/direction labels and fixed-point values. The same-cycle agreement required here is local to the matched emitted R2C fixed-point unit: all Read/Write labels inside that unit are interpreted at one ε-quiescent fixed point and may not be split. This local atomicity requirement is one of the explicit same-cycle semantic constraints and does not require unrelated Sys_ref and RTL_B events to share the same global bucket decomposition. If a design requires every cycle’s combinational fixed point to be externally observable even when the fixed-point slice is unchanged, then that component must be declared cycle-locked; a cycle-locked R2C component is matched by its per-cycle emitted occurrences and therefore falls outside the bucket-stutter freedom claimed for ordinary latency-insensitive R2C components.

For synchronization events, match by their per-process occurrence index in the source order (R1; preserved on the post-HLS side by E1 below). Independent Σ-events on different endpoints (including distinct message-passing interfaces) may commute and/or be grouped differently within a clock cycle as permitted by the R-rules and E3. Here "independent" is meant in the Σ-equivalence sense: the pair is not additionally ordered by any explicit Σ-visible ordering/timestamp constraint in E1–E5 (notably E3/E5); it is not a statement about (in)comparability under the Appendix G causal-dependency relation (→). Accordingly, τ_pre and τ_post are not required to be identical as a single total order, nor to agree on same-cycle "grouping" of distinct-interface Push/Pop operations, beyond the explicit ordering/timestamp constraints in E1–E5 below.

We say $\tau\_pre \approx \tau\_post$ iff the two traces match on Σ in this sense, and all the following hold:

E1 (Per-process synchronization-order preservation).

For every process P,
$\forall\, s_1, s_2 \in S\_P : s_1 <\_P s_2 \Rightarrow clk\_post(s_1) < clk\_post(s_2)$.

Equivalently, synchronization events are matched and ordered by their per-process source occurrence index. E1 does not preserve incidental global clock ordering between synchronization events of different processes unless that ordering is induced by an explicit inter-process synchronization relation already represented in Σ.

E2 (Signal-visibility preservation).

Partition signal Read/Write events by process kind:
• R_seq and W_seq are the ordinary signal Read/Write events of sequential processes.
• R_comb and W_comb are the signal Read/Write events of combinational processes, interpreted by R2C.

For sequential-process signal IO:
$\forall\, r \in R\_seq : clk\_post(r) = clk\_post(pred\_S(r))$
$\forall\, w \in W\_seq : clk\_post(w) = clk\_post(succ\_S(w))$

Equivalently, ordinary signal Reads/Writes of sequential processes are logically timestamped at their synchronization anchors, as in R2. The same anchoring convention is used on the pre-HLS side when forming the matched trace.

For combinational-process signal IO, pred_S/succ_S anchoring does not apply. Instead, each R2C fixed-point signal-observation unit is matched by the component-local occurrence ordinal defined above: for each combinational process or explicitly modeled combinational network component C_comb, the k-th R2C fixed-point unit of C_comb in Sys_ref matches the k-th R2C fixed-point unit of C_comb in RTL_B. The matched units must contain the same corresponding combinational Read/Write events and fixed-point values at their respective ε-quiescent combinational fixed points, as defined by R2C. Any delta-cycle re-evaluation, intermediate combinational value, or redundant re-drive before the fixed point is an ε-step and is not separately Σ-visible. This condition does not require the matched R2C units to occur in the same global observable-cycle bucket number, nor to be grouped with the same unrelated events, in the two executions.

E3 (Safe message issue ordering). (Post-HLS preservation of R4).

For every pair of executed message-call instances $q1,q2 \in Q\_exec,P$ of the same process P:
(i) Same-interface source order: if q1 and q2 access the same message-passing interface and $q1 <\_src\ q2$, then $issue\_post(q1) \le issue\_post(q2)$. This preserves source order at cycle granularity; any stricter serialization required by the concrete interface is supplied by the channel/interface semantics, not by E3 itself.
(ii) Distinct-interface no-reverse: if q1 and q2 access distinct message-passing interfaces and appear in sequence in the source ($q1 <\_src\ q2$), then $issue\_post(q1) \le issue\_post(q2)$ (i.e., they shall not be issued in the reverse sequence).
(iii) Prefix-closed issue obligation: for every cycle-aligned cutpoint σ of the post-HLS execution / witness, if $q1 <\_src\ q2$ and q2 has issued by σ, then q1 has also issued by σ and $Issue(q1) \le Issue(q2)$. This applies across same-interface and distinct-interface source order. It is an implementation-side obligation on issued sets, not merely an inequality over pairs whose timestamps are already defined.
(Note: HLS internal pipeline staging, dequeue, and hand-off inside the back-annotated process/channel realization are not, by themselves, Σ-visible boundary issue events in Q_exec,P nor committed-transfer events in M; therefore such internal movement does not constitute a counterexample to (ii) or (iii).)

E4 (Per-channel channel semantics: rendezvous and bounded FIFO).

For every observable channel c with capacity B(c), the projection of τ_post to events on c is legal for that channel's capacity model. The channel model splits into two cases.

• Rendezvous case, B(c)=0. Channel c is a capacity-zero rendezvous channel. A committed transfer on c consists of a matching same-cycle pair $Push\_c(v)$ and $Pop\_c(v)$ with the same payload value v. No unmatched committed Push_c or Pop_c may exist after any cycle boundary. Equivalently, the k-th committed $Push\_c(v)$ and the k-th committed $Pop\_c(v)$ occur in the same clock cycle and carry the same value. A Push_c endpoint can commit in cycle t only when the matching Pop_c endpoint boundary request is also asserted in cycle t, and symmetrically a Pop_c endpoint can commit in cycle t only when the matching Push_c endpoint boundary request is also asserted in cycle t.

• Buffered case, B(c)>0. Channel c is a bounded FIFO with cycle-boundary, non-fall-through semantics. The committed $Pop\_c(v)$ events return exactly the FIFO-ordered sequence of values v previously committed by $Push\_c(v)$, with no drops, duplications, or reordering. For every cycle-aligned prefix π_t of the c-projection—i.e., π_t contains exactly the events on c committed in cycles < t—letting $push(\pi\_t)$ and $pop(\pi\_t)$ be the number of Push_c and Pop_c events in π_t, we have

$$0 \le push(\pi\_t) - pop(\pi\_t) \le B(c).$$

Equivalently, the abstract occupancy after cycle t−1 is $occ\_c(t) = push(\pi\_t) - pop(\pi\_t)$. Availability for commits in cycle t is evaluated against $occ\_c(t)$ at the start of cycle t: Pop_c may commit in cycle t only if $occ\_c(t) > 0$, Push_c may commit in cycle t only if $occ\_c(t) < B(c)$, and simultaneous Push_c and Pop_c commits in the same cycle are permitted iff $0 < occ\_c(t) < B(c)$.

Thus the inequality-based FIFO availability predicates are used only for B(c)>0. The B(c)=0 case is not interpreted by substituting B(c)=0 into the bounded-FIFO inequalities; it is interpreted by the rendezvous rule above.

Note: $occ\_c(t)$ is the logical occupancy of channel c at the chosen Σ boundary, induced solely by the Σ-visible boundary-commit history (Push_c/Pop_c) and the annotated capacity B(c) in the buffered case B(c)>0. For B(c)=0, the corresponding channel fact is the same-cycle rendezvous request/commit predicate rather than a nonzero FIFO occupancy predicate.

E5 (Messages cannot cross their immediate synchronization boundary).

For every synchronization call s (in the source order used by R3 / pref_S / suff_S over Q_exec,P):
$\forall q \in pref\_S(s) : issue\_post(q) \leq clk\_post(s)$
$\forall q \in suff\_S(s) : issue\_post(q) > clk\_post(s)$
(For blocking message requests, if the synchronization call commits in the process trace, then any earlier blocking message requests in that source interval must have committed in some cycle ≤ that sync-commit cycle, by blocking-call control-flow semantics. Under the Issue/Commit linkage above, any boundary request assertion that persists while the process is still pending at s reflects only already-issued outstanding operation(s) in pref_S(s); no operation $q \in suff\_S(s)$ may have $Issue(q) \leq clk\_post(s)$, and any back-to-back issuance on that endpoint direction may occur only in the first cycle strictly after clk_post(s). For non-blocking calls, a later executed source-level PushNB/PopNB in suff_S(s) is a distinct q and cannot be identified with an earlier pref_S(s) call solely because the endpoint request signal remained asserted.)

See *Appendix I – Per Process Trace Compatibility Proof* for formal proof of the following one-way witness statement:

For any single source-level process P that obeys the conceptual R rules and B rules, when interpreted as part of the prescribed source reference model Sys_ref (see Appendix J, Remark 2), every finite observable trace/prefix of the post-HLS RTL_B process has a matching source-level trace/prefix in Sys_ref that satisfies E1–E5 (Post → Ref). Appendix I does not claim the unrestricted converse for all source-side traces. When a reverse correspondence is needed, it is restricted to annotated RTL-consistent source-side prefixes as stated later in Appendix J / Theorem J.1. The raw rendezvous case is recovered when Sys_ref = Sys_B and all effective capacities are zero.

Allowed Transformations and Why They Preserve ≈

Transformation permitted by the rules	**Formal justification**
Loop pipelining (overlaps iterations)	Introduces internal pipeline stages (modeled as ε-steps) and overlaps iterations. R1–R5 (and hence E1–E5) are applied to the explicit S set. Loop pipelining may commute independent Σ-actions across iterations (e.g., across distinct channels)—where "independent" here means "not additionally ordered by any explicit Σ-visible constraint in E1–E5 between the pair (notably E3/E5)," rather than "incomparable under Appendix G's causal-dependency relation (→)"—but it must still preserve: (i) per-channel FIFO legality (E4), (ii) the required issue discipline (E3) for source-ordered pairs, and (iii) the sync-boundary constraints (E1/E5) with respect to the (explicit + implicit) S-actions.
Overlap of internal dequeue/staging vs. later pipeline work (ε)	Permitted because the internal staging/acceptance activity is ε-internal within the back-annotated channel realization (as defined in "Appendix B - Pipelined Loops") and does not constitute a Σ-visible boundary message-issue reorder. Σ-visible boundary message issues must still satisfy E3, and

Transformation permitted by the rules	Formal justification
	per-channel committed-transfer legality is ensured by E4; automatic flush / Appendix K addresses deadlock preservation for overlap timing.
#pragma hls_direct_input (late sampling of static signals)	This pragma imposes a designer/environment stability contract: the signal's value must remain stable after reset (or after its last permitted update) while the receiving process may consume it. The logical signal Read actions r remain anchored exactly as in E2 (clk_post(r) = clk_post(pred_S(r))). HLS may implement this by sampling the signal later (instead of inserting internal storage), but because the signal is stable the sampled value equals the value at pred_S(r); therefore E2 and ≈ are preserved.
#pragma hls_direct_input_sync (controlled updates)	This pragma imposes an explicit timing contract on the environment: the controlled direct-input signals may change only on the cycle of the designated synchronization call s* (i.e., during the sync handshake). With that contract, the logical anchoring required by E2 is preserved: for the receiving process, the relevant Reads are anchored at the closest preceding synchronization call pred_S(r) = s*, and any environment update Write w_update is aligned with synchronization (succ_S(w_update) = s*). HLS may still implement late sampling internally, but the value observed is the same as the value at the E2 anchor point, so ≈ is preserved without modifying E2. **Instrumentation note (logical signal timestamping).** In the Σ-traces used for ≈ checking, sequential-process signal Read/Write events are timestamped at their E2 anchor points (e.g., clk_post(pred_S(r)) for Reads), not at any later physical RTL sample cycle that HLS may introduce. Any internal late-sampling micro-steps are treated as ε-steps. Combinational-process signal Read/Write labels are handled by R2C rather than by pred_S/succ_S anchoring or by global observable-cycle bucket number. A monitor records a combinational fixed-point value only after the simulator/RTL has reached an ε-quiescent combinational fixed point, and that value contributes to Σ only when the relevant R2C observation component emits an R2C fixed-point signal-observation unit under the R2C unit-generation convention. Intermediate delta-cycle reads/writes, redundant re-evaluations, transient combinational values before the fixed point, and R2C stutter cycles whose selected observable slice is unchanged are ε/stutter and are not separately Σ-visible. Therefore, equivalence checking must instrument/record the appropriate logical Read/Write event: the anchored event

Transformation permitted by the rules	Formal justification
	for sequential-process signal IO, or the emitted R2C fixed-point signal-observation unit for combinational-process signal IO. A wrapper/transactor may be used when needed to expose only those logical events rather than naive physical sample, delta-cycle, or per-cycle stutter events.
Memory-access reordering proven conflict-free	Let addr(a) be the symbolic address of access a. If the tool proves addr(a_1) ≠ addr(a_2) for the overlapped window, then the commutation of a_1 and a_2 is observationally silent; no external signal/message depends on the internal order.
Implicit FSM states / added latency	Extra states introduce *silent* ε-steps between observable actions; the partial order on Σ is unchanged, so E1–E5 are unaffected.
Shared-memory arrays with explicit synchronization	When a memory is accessed by multiple processes, the designer explicitly coordinates access with synchronization operations modeled as S-actions, and the externally relevant memory behavior is defined by the array access mapping layer. E1 preserves the order of the synchronization actions that bracket or authorize the shared-memory accesses. The array access mapping layer then defines the mapped memory operations, their commit cycles, and the required serialization/coherence rule for the shared storage, and the pre-HLS and post-HLS traces must agree under that declared memory semantics. E2 is not used here except for ordinary signal IO. E4 is relevant only if the mapping layer represents memory commands/responses as message-passing channels; otherwise memory legality is discharged by the mapping layer's memory semantics, not by FIFO channel semantics. Under these conditions, the shared-memory transformation preserves the observable behavior required by ≈. Scope (J.Mem): the formal theorems of Appendices I–K discharge this entry only when the value-carrying memory traffic is lowered per the J.Mem shared-memory lowering condition (Common Formal Definitions); the non-lowered "declared memory semantics" branch states an agreement obligation that Theorem J.1 as proved does not discharge.
Non-blocking message-passing (PushNB/PopNB) verified by post-HLS snooping	If the latency differences are externally visible to the DUT, a verification wrapper delays the pre-HLS side so that the committed transfer events (successful completions) occur in the same order/timing as observed on the post-HLS channel. This wrapper is itself purely synchronising (S), so it cannot violate R1–R5. Once the wrapper is assumed, τ_pre and τ_post coincide on the committed message-transfer history recorded in Σ (i.e., the Push_c/Pop_c commit events), so E3–E5 are preserved. Unsuccessful non-blocking polls remain ε-steps and are not required to match. This is not a

Transformation permitted by the rules	Formal justification
	transformation that inherently preserves ≈, but rather a verification technique to enforce equivalence for inherently latency-sensitive operations.
Latency-sensitive *global* signal IO matched by snooping	If the latency differences are externally visible to the DUT, the same "mirror-and-delay" wrapper used for NB channels is applied to any globally-visible signal whose timing matters. Because the wrapper inserts only S-actions, the partial-order constraints are unchanged. This is not a transformation that inherently preserves ≈, but rather a verification technique to enforce equivalence for inherently latency-sensitive operations.
Latency-sensitive *local* signal IO isolated in cycle-accurate transactors	Local timing-exact protocols are confined to dedicated transactor processes that expose only latency-insensitive M or S operations to the rest of the design. Since the transactor boundary is now the observable interface, R1–R5 apply *outside* the timing-sensitive region, so system-level ≈ still holds.
One-way signal-handshake protocols with run-time assertions	For single-direction vld or rdy signals, an assertion proves that back-pressure can *never* occur. That assertion establishes a refinement in which the missing handshake is equivalent to a permanent logical '1', so the protocol is observationally identical to the corresponding two-way handshake with the missing side tied true. E2 then applies only to the logical timestamping of the signal Read/Write events, not to the protocol-refinement step itself.
Combinational processes	Combinational processes have no process-owned S actions and no message-passing M actions in this formal clause, but they may have observable signal Read/Write events in Σ through explicitly selected R2C observation components. Their signal IO is interpreted at ε-quiescent combinational fixed points as specified by R2C: intermediate delta-cycle propagation is ε, and only emitted fixed-point Read/Write labels in the selected observable slice are Σ-visible. Repeated cycles in which the selected fixed-point slice is unchanged are R2C stutter for that component unless an explicitly modeled observer/sampler requires a fresh emitted unit. Under the well-formedness requirement that the combinational network is deterministic, reaches a unique fixed point in each compared cycle, and has a well-defined R2C unit-generation rule for the selected component, the RTL/tool-flow conformance obligation is to preserve the emitted fixed-point signal-observation unit sequence at the chosen observation boundary. Thus the required correspondence is not equality of simulator delta-cycle behavior, nor equality of global

Transformation permitted by the rules	Formal justification
	cycle-bucket numbers, nor an automatic consequence of ordinary combinational synthesis; it is equality of matched emitted R2C fixed-point units by component identity and local emitted-occurrence ordinal. Designs with oscillation, ambiguous multiple-driver resolution, hidden stateful behavior, intentional combinational-loop behavior outside a unique fixed-point interpretation, or multi-input observation slices whose interleavings can change the emitted unit sequence are outside R2C unless represented by explicit state, synchronization, a cycle-accurate transactor, a cycle-locked R2C observation boundary, or another formally modeled boundary.

For the purposes of the formal analysis, non-blocking message-passing is modeled at the level of committed transfers: a successful PushNB/PopNB contributes the same committed Push_c/Pop_c event to Σ as a blocking Push/Pop, while unsuccessful polls are ε-steps.

If a design contains latency-sensitive global signals or non-blocking transfers whose timing/ordering is externally visible at the DUT boundary and must be reproduced exactly in both simulations, we take as an axiom that a purely synchronizing snooping wrapper can be applied to supply the same latency choices to the pre-HLS simulation as those observed in the post-HLS RTL (Appendix L). Conceptually, this wrapper is a witness-selection device: it does not enlarge the set of admissible pre-HLS behaviors, but restricts the pre-HLS run to one admissible execution whose Σ-events align with the observed RTL execution. Under this wrapper, the committed Σ-traces of the two simulations agree, and the trace-compatibility rules E1–E5 apply to the wrapped runs.

Appendix L provides detailed guidance to enable this perfect alignment to be achieved in practice.

Proof Structure

The proofs proceed in three stages:

First, in Appendix I we establish the per-process witness correspondence needed by the later system-level proof: under the imported local execution and witness-realization side conditions, every post-HLS RTL_B process trace (equivalently, every reachable finite observable prefix) has a matching source-level trace in the prescribed source reference model Sys_ref (Post → Ref), satisfying E1–E5. Here Sys_ref is the source-side proof target defined later in Appendix J, Remark 2: in the ordinary bounded-FIFO case, Sys_ref = Sys_B; in the elaborated back-annotated cases, Sys_ref = Sys_B^elab. The converse direction (Ref → Post) is not claimed for all Sys_ref traces in general; when it is needed, it is restricted to annotated RTL-consistent prefixes or to source executions selected by Appendix L's snooping / witness-selection mechanism. The finite-prefix construction uses the explicit finite selected-unit realization and realization-correctness obligations later packaged by J.GWR, together with the local operational semantics and witness compatibility. Weak fairness and channel-progress premises are required only when an infinite fair execution is itself asserted. These side conditions are not a claim that the entire system is deadlock-free, and they are not the liveness-preservation conclusion of Appendix K.

Second, in Appendix J we compose these per-process results into a system-level observable trace-compatibility theorem. This stage relies on channel-ordering, occupancy, cutpoint-abstraction, and required-order lemmas, but it remains a trace-compatibility argument. It does not assume or prove global source liveness, and it does not by itself prove that RTL_B is free of internal message-passing deadlock.

Finally, Appendix K addresses a separate liveness-preservation obligation for internal message-passing deadlock. Appendix K does not discharge the Appendix I/J weak-fairness, finite-depth, channel-progress, realization-correctness, K.Drain, Kε, J.KObsConf, or A6–A11 side conditions; those remain standing assumptions or tool/source/environment obligations, summarized in Appendix N. Nor does Appendix K derive A5-L, A5-S, or any alternate A5-L discharge from the local R-rules. The core Theorem K.1A assumes the scheduler-robust premise A5-L: no admissible Policy-L execution of the selected Sys_ref reaches a prohibited internal deadlock cutpoint. Under that premise and the imported correspondence, progress, fairness, structural, abstraction, and realization assumptions, any matched RTL_B internal deadlock reconstructs to a prohibited Policy-L Sys_ref cutpoint, contradicting A5-L. The primary user-facing discharge is A5-S: under Lemma K.2b, Corollary K.2c, and their serial-route side conditions, one complete maximal fair Policy-S execution that avoids Dead_KS is sufficient. K.EnvOrd and K.Done may reduce that check to the ordinary Dead_K predicate, while structural rank/credit evidence, direct Policy-L verification, invariant or exploration proofs, and tool certificates remain alternate A5-L discharge routes. In the ordinary case Sys_ref = Sys_B; in the elaborated back-annotated cases Sys_ref = Sys_B^elab. When Sys_ref = Sys_B and B(c)=0 for all channels, this specializes to the raw rendezvous model.
Thus the architecture is non-circular for a different reason than “discharging” the earlier side conditions: Appendices I and J use local execution/progress side conditions to establish conditional trace compatibility, while Appendix K separately preserves internal message-passing deadlock freedom under A5-L, discharged through A5-S or another recorded K.A5Cert route, together with the standing Appendix N assumptions. The liveness-preservation theorem is therefore conditional on those assumptions and the selected discharge evidence; it is not a derivation of them.

Causal Dependency Between Processes

To prove system-level trace compatibility, we must distinguish between incidental timing and functionally significant ordering.

Formal Definition of Causal Dependency

To formalize the notion of functionally significant dependency and resolve ambiguity, we define a causal-dependency relation, denoted by →, on the set of observable IO actions (Σ) across all processes in the system. The relation is causal/information-flow rather than purely temporal: events related by → may commit in the same clock cycle, and the direction of → indicates which event may be depended on by the other.
Important: → is a causal-dependency graph, not by itself the source order used for the Appendix J induction. In particular, because the methodology permits distinct-interface message operations to be issued with timing overlap or in the same cycle while still preserving dynamic source order, and because rendezvous transfers are same-cycle atomic Push/Pop transactions, the unquotiented dependency graph → need not be acyclic in every legal execution. A cycle in → can represent a legal same-cycle mutually enabling dependency among atomic transactions; it is not, by itself, an observable mismatch.

When Appendix J needs a well-founded induction order, it uses the separate required-order relation $\prec_J$ over observable units. That relation is generated only by the explicit ordering, timestamp, FIFO, atomicity, and synchronization constraints that must be preserved by the trace-compatibility predicate, not by the full causal-dependency graph → and not by raw per-process source order on distinct-interface message commits.

Well-formedness use of causal-dependency. The well-formedness rule below uses → as a design-intent and source-admissibility relation: if correctness depends on the relative timing, value, availability, or cross-process observation of two events, that dependence must be represented by an explicit modeled causal mechanism such as message passing, synchronization, signal handshakes/anchors, declared memory serialization, or an Appendix L snooped/witness-selected boundary. This use of → does not add proof-preserved observable ordering beyond $\prec_J$, FIFO legality, rendezvous/synchronization atomicity, signal-anchor constraints, synchronization-boundary constraints, and declared memory-serialization constraints.

The causal-dependency relation → is the smallest directed relation satisfying the conditions below:

1. **Intra-Process Order**: If actions a and b occur within the same process P and a precedes b in the program's execution trace, then a -> b. This directly reflects the sequential nature of the code within a single thread. **Clarification (non-temporal).** Here "precedes" refers to the order of IO action *instances* in the process's trace/program order (i.e., the order induced by the process's execution), **not** the relative order of their **commit cycles**; distinct-interface message actions may be **issued** in order while **committing** in either order depending on backpressure.

2. **Inter-Process Communication**: The relation is established by the direct exchange of information or synchronization between processes.

    - **Message-passing (committed transfer):** If a is the commit of a send on channel c in process P1 (a committed Push_c, including a successful PushNB), and b is the commit of the corresponding receive on c in process P2 (a committed Pop_c, including a successful PopNB), for the same logical message instance, then a → b. Here "corresponding receive" is defined by the channel's FIFO semantics: b is the Pop_c commit that consumes the element produced by a (matching is by FIFO position / transfer instance, not by payload-value equality; duplicate payload values do not introduce ambiguity). Equivalently, if we enumerate committed Pushes on c as Push_c^1, Push_c^2, … in commit order and committed Pops as Pop_c^1, Pop_c^2, … in commit order, then Push_c^k → Pop_c^k. For buffered channels with B(c)>0, E4 also imposes the required FIFO legality used by Appendix J's required-order relation $\prec_J$. For rendezvous channels with B(c)=0, the matching Push_c and Pop_c commits occur in the same clock cycle and are treated in Appendix J as one explicitly atomic observable unit. The directed dependency a → b records unidirectional information flow — the Pop observes the value supplied by the Push — but it does not make the two rendezvous endpoints separately orderable for the Appendix J induction, and it does not imply an earlier commit cycle. No reverse edge b → a is implied unless there is an explicit reverse-direction communication, such as another channel or a signal handshake. Unsuccessful non-blocking polls are ε-steps and do not participate in →.

- **Signal Handshake**: If a is a signal write action w(sig) in P1 and b is a signal read action r(sig) in P2 that reads the value written by a as part of an explicit handshake protocol, then a -> b. A complete two-way handshake (e.g., vld/rdy) creates a causal chain, such as w(vld)@P1 -> r(vld)@P2 -> w(rdy)@P2 -> r(rdy)@P1.

- **Explicit Synchronization (two-party SyncChannel / ac_sync**): Let *sout* be the commit of a SyncChannel sync_out (or equivalent ac_sync call) in process P1, and let *sin* be the commit of the matching sync_in in process P2 for the same synchronization instance. Then this synchronization acts as a two-process barrier:
  - Every observable action in P1 that occurs before *sout* (in P1's trace/program order) satisfies a→b for every observable action b in P2 that occurs after *sin*.
  - Symmetrically, every observable action in P2 that occurs before *sin* satisfies a→b for every observable action b in P1 that occurs after *sout*.
  - (The two sync commits *sout* and *sin* are treated as simultaneous; we do not impose *sout*→*sin* or *sin*→*sout*.)

3. **Transitivity**: The relation is transitive. If a -> b and b -> c, then a -> c.

Using this relation, we now provide a formal definition for causal dependency between synchronization events, which are the fundamental ordering anchors in the scheduling model.

Definition: Causal Dependency

A synchronization event s2 in process P2 is **causally dependent** on a synchronization event s1 in process P1 if and only if s1 -> s2.
If neither s1 -> s2 nor s2 -> s1 holds true, the events s1 and s2 are considered **causally independent** (or concurrent). Any change in the observed temporal ordering of causally independent events between the pre-HLS and post-HLS simulations is considered an incidental, functionally insignificant variation in latency. A **well-formed design**, under this methodology, **shall not rely** on a specific ordering of causally independent events for functional correctness.

To be clear, the shall-not-rely requirement is a design rule. To adhere to this design rule, designs must express functionally significant cross-process dependencies through explicit modeled mechanisms such as message passing, SyncChannel/ac_sync operations, signal handshakes/anchors, declared memory-serialization constraints in the array-access mapping layer, or Appendix L snooping/witness-selection boundaries. Designs that violate this design rule are outside the formal guarantees. This rule concerns design well-formedness; it does not convert every causal-dependency edge into a strict proof-order edge.

System-Level Theorem (B1-Specialized Appendix G Statement)

Appendix G gives the high-level system theorem in the same standing-assumption regime used by Appendices J–K: B1 is assumed for RTL_B under the fixed stimulus, so the practical "unique RTL_B trace" reading is available throughout the main theorem stack. The authoritative execution-set theorem, including the exact execution-set quantification and the conditional use of B4/B5 when ε-normalization or uniqueness up to finite ε-stutter is invoked, is Theorem J.1. Therefore, if this Appendix G statement and Theorem J.1 are read together, Theorem J.1 controls; Appendix G is the high-level summary of the same B1-standing theorem regime.

Fix an initial state and a fixed external stimulus/environment behavior for both the prescribed source reference model Sys_ref and RTL_B, with Sys_ref as defined in Appendix J, Remark 2. (The raw rendezvous model Sys is the special case obtained when Sys_ref = Sys_B and all effective capacities are zero.)

Interpret "fixed stimulus" per Appendix I.2, "Fixed-stimulus interpretation (reactive environments)." Let Exec_post denote the set of finite execution prefixes of RTL_B that are reachable under this stimulus and that are prefixes of at least one infinite execution under the same stimulus satisfying the stated fairness/progress premises; let Exec_ref denote the corresponding set of finite execution prefixes of Sys_ref. Let Reach_post / Reach_ref denote the corresponding finite reachability sets with no fair-extension qualifier (Definition J.Reach, Appendix J); Exec_post ⊆ Reach_post and Exec_ref ⊆ Reach_ref.

Assume the Appendix I / Appendix L witness machinery is available as follows—namely: the per-process Post → Ref witness construction from Appendix I, including the witness-compatibility premise WC for ε-modeled non-blocking polls (and any other ε-only internal choices that can affect the next Σ-visible control flow/frontier) as stated in Appendix I.2 and enforceable via Appendix L's snooping / witness-selection technique; with NB-CF as a sufficient condition that makes the witness trivial—together with the capacities/occupancies/enablement semantics of Sys_ref, finite capacities on every explicit abstract channel of Sys_ref, channel progress invariants P_push / P_pop (B3), weak fairness (B2), and the standing RTL_B determinism assumption B1 under the fixed stimulus. Use B4/B5 additionally when ε-normalization or uniqueness up to finite ε-stutter is invoked.

Then the B1-specialized system-level result is:

(Post → Ref existence) For every τ_post ∈ Reach_post equipped with a globally witness-realizable annotation package (Definition J.GWR, Appendix J), there exists τ_ref ∈ Reach_ref such that τ_ref,system ≈ τ_post,system under E1–E5.

(Ref → Post existence, annotated RTL-consistent prefixes only) For every τ_ref ∈ Exec_ref equipped with a selected RTL_B prefix τ_post^* ∈ Exec_post and a compatible witness/annotation package satisfying the annotated RTL-consistency requirement of Theorem J.1 (RTLConsistentRefPrefix), there exists τ_post ∈ Exec_post such that τ_ref,system ≈ τ_post,system.

Moreover, because B1 holds in this Appendix G specialization, the matching Σ-label sequence is unique (up to insertion/removal of finite ε-steps permitted by B4/B5). That uniqueness is only projection uniqueness; it does not by itself determine μ_Q, failed-poll instances, issue annotations, request attribution, E4 boundary histories, or the corresponding source-side witness prefix/state in Sys_ref.

Proof sketch. Appendix J first proves a pairwise composition lemma: any chosen compatible pair of system traces whose per-process projections satisfy ≈ lifts to system-level ≈. Appendix J then instantiates that lemma at the execution-set level using Appendix I's witness construction together with the Exec_post / Exec_ref scope conditions. Thus the main result is explicitly about sets of executions under fixed stimulus, not merely about a single pre-chosen trace pair. The Ref → Post direction is not a symmetric arbitrary-source construction; it applies only to annotated RTL-consistent source prefixes, and under this Appendix G specialization B1 supplies only the unique RTL_B Σ-trace against which the projection component of that consistency is judged. The compatible annotation package remains a separate witness obligation and is not inferred from bare Σ-label matching.

Practical Implication ("No Surprises" Guarantee)

If a verification environment exercises only the alphabet Σ (signals, message ports, synchronization calls) and does not test internal latency, then—under the assumptions of Theorem J.1 in Appendix J (fixed stimulus, the stated fairness/progress premises, and the Appendix I / Appendix L witness conditions where needed)—the prescribed source reference model Sys_ref and the post-HLS RTL agree at the chosen observable boundary in the following qualified execution-level sense:
• for every τ_post ∈ Reach_post carrying a J.GWR witness package, there exists a matching τ_ref ∈ Reach_ref; and
• in the reverse direction, only those source-side τ_ref ∈ Exec_ref that are annotated RTL-consistent in the sense of Theorem J.1—i.e., equipped with a selected RTL_B prefix and a compatible witness/annotation package satisfying RTLConsistentRefPrefix, not merely a bare Σ-projection match—are guaranteed to have a matching τ_post ∈ Exec_post.

Latency-sensitive testbench caveat

A testbench that branches on raw cycle counts, uses timeout watchdogs, checks performance/latency bounds, or otherwise changes its stimulus or pass/fail behavior based on timing not represented in Σ is not covered by the same-stimulus trace-compatibility guarantee unless that timing dependence is itself modeled through explicit Σ-visible causal mechanisms such as synchronization events, handshakes, or message-passing transactions. Such checks may still be useful, but they should be treated as latency-sensitive testbench logic, isolated as described in Appendix D, or performed as post-HLS / RTL-level performance checks rather than as properties transferred automatically from an arbitrary pre-HLS simulation run.

Accordingly, the precise "no surprises" reading is one-sided for arbitrary RTL behavior and restricted on the source side: no Σ-observable behavior can appear in RTL_B under the fixed stimulus without a matching behavior in Sys_ref; but a source-side pass claim transfers to RTL_B only for the annotated RTL-consistent subset of source-side behaviors covered by Theorem J.1 (or after the Appendix L snooping / witness-selection restriction has been applied where needed).

Thus, for a Σ-only testbench under the same fixed stimulus, any mismatch found on RTL_B corresponds to a matching mismatch on Sys_ref; however, a source-side pass justifies RTL_B only through the theorem's restricted reverse direction. Here Sys_ref means Sys_B in the ordinary bounded-FIFO case and Sys_B^elab in the elaborated back-annotated cases of Appendix B / Appendix Q. See Appendix J / Theorem J.1 for the full execution-level statement.

Appendix G states the trace-compatibility rules and a B1-specialized high-level implication. Appendix I provides the per-process Post → Ref witness construction, and Appendix J makes the main system theorem explicit at the execution-set level for the prescribed source reference model. Taken together, these appendices establish the stated witness-based execution-level correspondence at the chosen observable interfaces under fixed stimulus, the stated fairness/progress premises, and the Appendix I / Appendix L witness conditions where needed. In particular, the unrestricted guarantee is Post → Ref; the reverse direction ranges only over annotated RTL-consistent source-side behaviors covered by Theorem J.1.

Appendix H – Possible Criticisms

This section highlights several potential challenges in the verification approach of Appendix G.

1. Designer-Managed Shared-Memory Synchronization

Appendix G requires that any memory shared between processes use explicit synchronization primitives inserted by the designer. The HLS tool does not perform static verification of those primitives. In practice, we mitigate this risk by encapsulating shared memories within parameterized library classes (for example, see examples 12_ping_pong_mem and 15_native_ram_fifo) that are pre-verified for correct memory access coordination in both the pre-HLS and post-HLS models. Typically such classes will be parameterized for aspects such as element type and memory size. Formally, the Appendix I–K theorems cover shared memories only under the J.Mem lowering condition; such library classes should present their value-carrying memory traffic at boundaries satisfying J.Mem, or carry their own separate correctness argument outside the Appendix I–K theorems.

2. Latency-Sensitive Signal Alignment ("Snooping")

See Appendix L for detailed discussion on snooping.

3. By Default, Matchlib Connections Model a Skidbuffer inside In<> Ports

(Note that this overall document and specifically Appendix G are not intended to be strictly tied to Matchlib. Instead, this document is intended to be applicable to any pre-HLS model written in SystemC using signals, message-passing channels, and synchronization calls.)

Matchlib Connections supports throughput accurate modeling in the pre-HLS simulation. Currently the throughput accurate modeling mechanism relies on the Pre() and the Post() methods using a 1-place buffer to store transactions that are in flight on Connections::In<> ports. This means that by default In<> ports function like a skidbuffer. Connections::Out<> ports do not introduce any additional storage capacity into the pre-HLS simulation.

By default, Catapult adds skidbuffers on input ports into the post-HLS design, so the formal trace-compatibility relationship described in this document still holds. In other words, the default pre-HLS and post-HLS designs both still model rendezvous semantics, since both explicitly include a structural instantiation of a skidbuffer on input ports. In effect, we define the rendezvous boundary *before* the skidbuffer.

It is possible to remove some or all the skidbuffers during Catapult HLS. Because the abstract rendezvous boundary is defined before the skidbuffer, this choice is orthogonal to the trace-compatibility rules (R1–R5/E1–E5) as long as skidbuffer-internal behavior is not included in Σ. However, to keep the pre-HLS simulation's port micro-architecture aligned with the RTL configuration, the following compile flag must then be used in this case:

```
-DFORCE_AUTO_PORT=Connections::SYN_PORT
```

This will remove all the skidbuffers from the pre-HLS model. In this case, if some of the skidbuffers are still inserted during HLS, the capacity effects of those specific skidbuffers should be added into Sys_B so that the formal trace-compatibility relationship still holds.

In this case the recommended methodology is to use both approaches:

- Use the default throughput accurate mode in Matchlib for most analysis and debug, and for pre-HLS performance verification.

- To keep the pre-HLS simulation's *port micro-architecture* strictly aligned with the RTL configuration, then use -DFORCE_AUTO_PORT=Connections::SYN_PORT, and rerun final verification tests.

Common Formal Definitions for Appendices I-K

Throughout Appendices I–K, the formal comparison is between RTL_B and Sys_ref (Appendix J, Remark 2), i.e., the same source processes interpreted under the channel semantics of E4 with capacities B(c) matching the RTL. The E4 channel semantics are case-split: B(c)=0 is interpreted as a capacity-zero rendezvous channel, and B(c)>0 is interpreted as a cycle-boundary, non-fall-through bounded FIFO. The rendezvous case is therefore recovered as the B(c)=0 case of the channel model, not by applying the B(c)>0 FIFO-availability inequalities with B(c)=0.

Symbol / Rule	**Definition — concise but complete**
BoundaryReq(x, σ) (shorthand BoundaryReq(q, σ) only in ordinary unique-boundary case)	For any dynamic source-level message-operation frontier item x ∈ Ω_msg,P on an explicit abstract channel c = BoundaryOf_P(x) of Sys_ref, with owning source-level message instance q = MsgOf_P(x) and kind(q) ∈ {Push_c, Pop_c}, BoundaryReq(x,σ) means: at that chosen abstract boundary and ε-quiescent cutpoint σ, q's endpoint-owned boundary request for item x is asserted. Operationally (ready/valid): · for kind(q)=Push_c: the producer endpoint at BoundaryOf_P(x) asserts vld_c, with stable payload as required; and · for kind(q)=Pop_c: the consumer endpoint at BoundaryOf_P(x) asserts rdy_c. BoundaryReq is therefore a predicate about the endpoint's boundary-visible request assertion for a specific dynamic operation/frontier item, not about an already-committed Σ-event label. In the ordinary unelaborated case, where q has a unique evaluated boundary and frontier item, BoundaryReq(q,σ) is accepted as shorthand for BoundaryReq(x,σ). In Sys_B^elab, the frontier-item form is authoritative because one source-level call q may own a chain of proof-internal frontier items at multiple explicit boundaries. Purely combinational realization structure on the channel side may affect only the complementary mirror signal at that boundary (Push: rdy; Pop: vld) and thus the commit condition after ε-settling, but it does not change the endpoint request assertion captured by BoundaryReq(x,σ). When a cycle-index notation is

Symbol / Rule	Definition — concise but complete
	needed, BoundaryReq(x,t) is shorthand for BoundaryReq(x,σ_t), where σ_t is the ε-quiescent cutpoint for cycle t.
ChannelEnabled(x, σ) (shorthand ChannelEnabled(q, σ) only in ordinary unique-boundary case)	ChannelEnabled(x,σ) means: BoundaryReq(x,σ) holds, and in the ε-quiescent ready/valid fixed point at σ, the channel-side commit-enabling condition for the owning operation q = MsgOf_P(x) holds at the chosen boundary. Concretely, for bounded-FIFO channels B(c)>0, the channel-side condition is exactly the E4 availability predicate at that boundary: · for kind(q)=Push_c: occ_c(σ) < B(c), i.e. space is available; and · for kind(q)=Pop_c: occ_c(σ) > 0, i.e. data is available. For rendezvous channels B(c)=0, the channel-side condition is the matching complementary endpoint participation in the same ε-quiescent cycle. If the owning operation represented by x commits, it contributes the corresponding committed Σ-event m(q) ∈ M; before it commits, m(q) is absent, but BoundaryReq(x,σ) and ChannelEnabled(x,σ) are still well-defined for the pending frontier item. In the ordinary unique-boundary case, ChannelEnabled(q,σ) is shorthand for ChannelEnabled(x,σ); in Sys_B^elab, the frontier-item/boundary-specific form ChannelEnabled(x,σ) is authoritative.
ChannelDisabled(x, σ) (shorthand ChannelDisabled(q, σ) only in ordinary unique-boundary case)	ChannelDisabled(x,σ) means: BoundaryReq(x,σ) holds but ChannelEnabled(x,σ) does not. Intuitively: the endpoint is requesting the transfer for dynamic operation instance q, but the transfer is blocked by channel-side conditions, such as empty/full status or the rendezvous partner not simultaneously requesting, as evaluated at the ε-quiescent handshake fixed point used throughout the document's enablement tests. In the ordinary unique-boundary case, ChannelDisabled(q,σ) is shorthand; in Sys_B^elab, the frontier-item/boundary-specific form ChannelDisabled(x,σ) is authoritative.
Σ	Set of observable actions for the equivalence check (committed IO events) at the chosen observable boundary (often the DUT IO boundary for functional equivalence, but optionally expanded—e.g., for Appendix K deadlock reasoning): • channel commit events Push_c(v) and Pop_c(v) on channels designated as observable, labeled with the transferred message value (or abstract message identity) v • synchronization events wait(), Sync, START_P, FINISH_P • single-cycle signal actions Write(sig, val) and Read(sig, val) on signals designated as observable, labeled with the value written/read. Internal-only channels/signals may be excluded from Σ (treated as ε-microstate) when they are not part of the chosen observable boundary. If such channels/signals are later brought into the observable boundary (e.g., by snooping for debug or analysis), then they become Σ-events and must match under ≈. **Deadlock-analysis requirement (Appendix K):** for Appendix K, Σ is instantiated to include every message-passing channel whose endpoint transfers at the chosen Sys_ref boundary can contribute a dependency edge in the Appendix K

Symbol / Rule	Definition — concise but complete
	Wait-For Graph, either because the channel is on the declared observable boundary or because it is snooped. The WFG itself is defined over pending blocking message-operation frontier items $x \in \Omega_{msg,P}$ for the relevant owning process P, with owning dynamic message-call instance $q = MsgOf_P(x)$, not over already-committed Σ-event labels and not over failed non-blocking polls in $Q_{exec,P}$. Thus, when Appendix K refers to a blocked Push or Pop at a boundary, the relevant frontier object is x, and its owning pending source-level operation instance is q with $kind(q) \in \{Push\_c, Pop\_c\}$; the edge is determined by BoundaryReq(x,σ) together with ChannelEnabled(x,σ)/ChannelDisabled(x,σ) at the ε-quiescent cutpoint σ. Internal micro-interfaces that lie within the back-annotated realization counted in B(c) (e.g., post-HLS RTL pipeline stage implementation) are not treated as separate channels for this requirement: they may be exposed/snooped for debug, but Appendix K's WFG edges are defined using the ChannelEnabled/ChannelDisabled status of the corresponding boundary operation instance q via E4/occ_c/B(c), so such internal points can participate logically in the deadlock argument without being explicitly Σ-observable. **Non-blocking message-passing (PushNB/PopNB):** A non-blocking call that returns "not completed" produces no Σ-event and is modeled as an internal ε-step. When a non-blocking call succeeds (returns "completed"), that success is represented in Σ as the corresponding committed Push_c(v) or Pop_c(v) event on that channel, labeled with the same transferred value/message v (i.e., Σ records transfers and their payloads, not failed polls). A non-blocking call is considered issued in the cycle in which it executes (its Issue cycle). A call that returns "completed" commits in that same cycle and therefore yields a committed transfer $m(q) \in M$; a call that returns "not completed" produces no committed transfer event (m(q) undefined / absent) and may be treated as an ε-step with respect to Σ-visible message-transfer events. Each executed non-blocking call is still a distinct dynamic source-level message-call instance $q \in Q_{exec,P}$ for the owning process P. Since failed non-blocking q's have no Σ-label, their cross-trace matching is witness-relative: it is supplied by the chosen μ_Q / WC witness, by NB-CF when failed polls cannot affect the next Σ-visible control flow or frontier, or by Appendix L's snooping / witness-selection mechanism. Continuous assertion of an endpoint request does not by itself identify multiple executed non-blocking calls as one q, and the Σ trace alone is not required to determine how failed non-blocking q's are matched. A successful non-blocking call may also be a member of $Q_{\Omega,P}$ when it contributes a committed Push_c(v) / Pop_c(v) event; a failed non-blocking poll remains in $Q_{exec,P} \setminus \Omega_P$.
START_P	First observable action emitted by process P after reset. Marks the moment P begins executing user code. A new START_P is emitted each time P exits reset.

Symbol / Rule	Definition — concise but complete
FINISH_P	Last observable action of process P in the current reset epoch. If P executes an unbounded loop within that reset epoch, FINISH_P never occurs for that epoch, and the epoch-local trace is treated as infinite until a later reset boundary, if any. If a later reset affecting P occurs and P exits reset again, a new dynamic START_P begins a fresh reset epoch; any later termination is represented by a distinct dynamic FINISH_P occurrence.
ε-step	An *internal*, unobservable RTL state transition (pipeline advance, FSM micro-state, etc.).
B(c)	Back-annotated effective capacity of channel c at the chosen explicit abstract Sys_ref boundary used by E4 and Appendix K. In the ordinary non-elaborated case this is the chosen Σ-observable Sys_B boundary; in elaborated cases it is the corresponding explicit abstract channel boundary of Sys_B^elab, which may be proof-internal and later projected away from the outer Σ_DUT trace. Finite, B(c) ≥ 0. It reflects the total bounded "composite buffer" storage/credit in the RTL realization represented by that explicit abstract channel boundary, including FIFO storage, skid/elastic buffering, HLS-inserted pipeline staging that can hold in-flight messages, and any explicit staged/bundle/join channel capacity introduced by Sys_B^elab. • B(c)=0 ⇒ rendezvous (capacity-zero) channel. • B(c)>0 ⇒ bounded FIFO (of that effective capacity). **B-faithfulness.** For the formal results to apply to a concrete RTL_B instance, each back-annotated B(c) must be faithful to the chosen explicit abstract boundary. This means: • Boundary completeness: every RTL storage element, credit, skid/elastic buffer, pipeline stage, or elaborated staged/bundle/join component that can accept, hold, release, or backpressure messages across the abstract boundary c is either included in the effective capacity B(c) for that boundary or represented as a separate explicit Sys_B^elab boundary with its own capacity and E4 semantics. • No hidden capacity or hidden enablement: the RTL realization does not permit a producer-facing accept or consumer-facing delivery at boundary c except in cycles allowed by E4 using the corresponding occ_c(t) and B(c), and any additional enablement restriction that affects boundary behavior is represented by an explicit Sys_B^elab boundary rather than by an unmodeled predicate. • ε-opacity of internal movement: movement of an already accepted message within the RTL realization counted by B(c) is internal ε-state. Such movement does not create additional Σ-visible Push_c/Pop_c events and does not create, delete, duplicate, reorder, or relabel the Σ-visible boundary transfers. • Evidence obligation: B-faithfulness is a tool/RTL-flow compliance obligation. It may be discharged by a trusted back-annotation algorithm, generated certificate, structural RTL check, interface monitor, formal equivalence/checking flow, or other validation method appropriate to the implementation flow. It is not derived by E4, Proposition J.0, or Theorem J.1.

Symbol / Rule	Definition — concise but complete
	Point-to-point endpoint assumption (single-producer/single-consumer): For every message-passing channel c (including both buffered channels with B(c)>0 and rendezvous channels with B(c)=0), there is exactly one producer process that may execute Push_c on c, and exactly one consumer process that may execute Pop_c on c. Hence the complementary endpoint relevant to any blocked Push_c/Pop_c on c is unique. **Modeling note (shared resources / arbitration):** If an implementation conceptually has multiple producers and/or multiple consumers for a logical resource, that sharing must be modeled explicitly (e.g., an arbiter process plus per-client point-to-point channels), rather than as a single multi-endpoint "channel." This keeps WFG wait dependencies well-defined and unique per edge. **Single-transfer-per-cycle endpoint constraint (scalar channel interface):** For every message-passing channel c, in any clock cycle t, at most one Push_c may commit and at most one Pop_c may commit. Equivalently, the set of commits on a fixed channel in a single cycle contains ≤ 1 Push and ≤ 1 Pop (it may contain one of each in the same cycle). For rendezvous channels (B(c)=0), a committed transfer occurs only as a matching same-cycle Push_c(v) and Pop_c(v) pair. For buffered channels (B(c)>0), a same-cycle Push_c and Pop_c do not imply fall-through; the channel semantics are cycle-boundary, non-fall-through as stated in E4. **Reset-empty FIFO convention:** after each reset boundary at which the compared execution begins or restarts, every buffered message-passing channel with B(c)>0 has logical FIFO occupancy 0 at the chosen Σ boundary. Equivalently, the reset state contains no preloaded in-flight messages in the abstract channel realization used by Sys_ref / RTL_B. If a design requires preloaded channel contents, those contents must be modeled explicitly as post-reset producer actions or by a separate initialized-state extension outside the present empty-FIFO formal model. **Modeling note (multi-lane / burst transfers):** If an implementation can transfer k>1 messages per cycle on a logical resource, model it as k parallel point-to-point channels (or as a single widened message transferred by one Push_c/Pop_c per cycle) so that the Σ-level event model continues to satisfy the scalar constraint above.
occ_c(t)	Abstract FIFO occupancy at the chosen explicit abstract Sys_ref channel boundary: number of items committed by Push_c but not yet committed by Pop_c at that boundary, measured from the most recent reset boundary under the reset-empty FIFO convention. In the ordinary non-elaborated case this is the chosen Sys_B channel boundary; in elaborated cases Sys_ref = Sys_B^elab, this is the occupancy of the relevant explicit abstract staged/bundle/join channel boundary. Equivalently (E4), occ_c(t)=push(π_t)−pop(π_t), where π_t is the Σ-visible boundary-commit prefix for that explicit abstract channel strictly before cycle t (i.e., excluding any Push_c/Pop_c commits that occur in cycle t), so occ_c(t) is the abstract occupancy at the beginning of cycle t. For pure-FIFO channels with

Symbol / Rule	Definition — concise but complete
	B(c)>0, the FIFO availability predicates (not-full/not-empty) are evaluated against this begin-of-cycle occupancy; hence a Pop_c cannot commit in a cycle that begins empty solely because a Push_c also commits in that same cycle (no fall-through), and likewise a Push_c cannot commit in a cycle that begins full solely because a Pop_c also commits in that same cycle. Clarification (no "second Push/Pop"): movements of an item within the RTL channel realization (e.g., FIFO→staging/skid buffering, staging/skid buffering→pipeline-stage registers, pipeline-stage→pipeline-stage handoff) are internal ε-steps; they do not constitute additional Σ-visible committed Push_c/Pop_c events beyond the boundary events counted by push(π_t)/pop(π_t). Derived cutpoint shorthand (used in Appendices K and R): if σ is an ε-quiescent cutpoint occurring in cycle t_σ, write occ_c(σ) ≜ occ_c(t_σ). Likewise, when channel enablement is evaluated at an ε-quiescent cutpoint, write ChannelEnabled(q,σ) ≜ ChannelEnabled(q,t_σ) and ChannelDisabled(q,σ) ≜ ChannelDisabled(q,t_σ). This is only notational shorthand for cutpoint reasoning; the primary semantic definition of occupancy remains the cycle-boundary quantity occ_c(t). Note: occ_c(t) is an abstract analysis quantity derived from Σ-visible boundary commits; in both Sys_B and RTL_B, processes do not directly read occ_c(t) and instead proceed solely based on the per-channel ready/valid outcome of their own Push/Pop attempts.
P_push(c)	*Finite-progress invariant (producer):* this is a persistent-request premise, not a ChannelEnabled premise. In this paragraph, "the producer's request remains asserted continuously" means that the producer presents a persistent, non-withdrawable Push_c endpoint request from the starting cycle through the matching discharge event, with no deassert-before-commit and with stable payload while asserted. • Buffered case (B(c) > 0): If the producer's Push_c request remains asserted continuously from some cycle t_o onward while the channel is full (occ_c = B(c)) —with stable payload while asserted—and the complementary endpoint continuously requests the matching Pop_c (persistent request asserted; Pop_c boundary request remains true), then some cycle $t' \geq t_o$ commits a complementary Pop_c (i.e., the full condition is eventually discharged). • Rendezvous case (B(c) = 0): If the producer's Push_c request remains asserted continuously from some cycle t_o onward (with stable payload while asserted) and the complementary endpoint continuously requests the matching Pop_c (persistent request asserted; Pop_c boundary request remains true), then some cycle $t' \geq t_o$ commits the rendezvous transfer—i.e., a matching Push_c(v) and Pop_c(v) commit in the same cycle.
P_pop(c)	*Finite-progress invariant (consumer):* this is a persistent-request premise, not a ChannelEnabled premise. In this paragraph, "the consumer's request remains

Symbol / Rule	Definition — concise but complete
	asserted continuously" means that the consumer presents a persistent, non-withdrawable Pop_c endpoint request from the starting cycle through the matching discharge event, with no deassert-before-commit. • Buffered case (B(c) > 0): If the consumer's Pop_c request remains asserted continuously from some cycle t_o onward while the channel is empty (occ_c = 0)—and the complementary endpoint continuously requests the matching Push_c (persistent request asserted with stable payload; Push_c boundary request remains true)—then some cycle $t' \geq t_o$ commits a complementary Push_c (i.e., the empty condition is eventually discharged). • Rendezvous case (B(c) = 0): If the consumer's Pop_c request remains asserted continuously from some cycle t_o onward and the complementary endpoint continuously requests the matching Push_c (persistent request asserted with stable payload; Push_c boundary request remains true), then some cycle $t' \geq t_o$ commits the rendezvous transfer—i.e., a matching Push_c(v) and Pop_c(v) commit in the same cycle.
Trace-compatibility rules (E1–E5)	E1 Synchronization-order preservation: For each process P, the sequence of synchronization events executed by P (wait(), SyncChannel, START_P, FINISH_P) appears in the same source order in the pre-HLS and post-HLS traces. E2 Signal visibility: For each sequential process P, each signal Read in P occurs at the closest preceding synchronization event in P, and each signal Write in P occurs at the closest succeeding synchronization event in P, in both the pre-HLS and post-HLS traces (per R2/E2). For each combinational process or explicitly modeled combinational network component C_comb, observable combinational signal Read/Write labels appear only inside emitted R2C fixed-point signal-observation units for the selected R2C observation component, as defined by R2C. Each emitted R2C unit is evaluated at one ε-quiescent combinational fixed point, and all Read/Write labels inside that unit are atomic with respect to that fixed point. Emitted R2C units are matched by component identity and local emitted-occurrence ordinal, not by global observable-cycle bucket number. R2C stutter cycles whose selected observable slice is unchanged do not create separate Σ-visible units unless an explicitly modeled observer/sampler requires a fresh emitted unit. Interpretation (logical timestamping). For sequential processes, the equations in E2 define the logical timestamps of Read/Write Σ-events: the anchor points at which compatibility is checked. An implementation may physically sample a Read later than pred_S(r), or physically apply a Write earlier than succ_S(w), provided the value is stable across the anchor interval and the trace emitted for compatibility records the Σ-event at the anchor point. For combinational processes, the logical observation point is the emitted R2C fixed-point signal-observation unit, not a global observable-cycle bucket number; internal delta-cycle propagation before the fixed point and R2C stutter cycles with unchanged selected observable slices are ε/stutter and are not separately Σ-

Symbol / Rule	Definition — concise but complete
	visible. Direct-input pragmas (e.g., #pragma hls_direct_input and #pragma hls_direct_input_sync) do not change E2 and are not exceptions to E2. For sequential processes, they impose stability/timing contracts that allow later physical sampling, or controlled updates at a synchronization boundary, while the logical Read/Write Σ-events used for ≈ checking remain timestamped at the E2 synchronization anchor points and retain the anchored value labels. For combinational processes, ordinary direct-input pragmas do not introduce a separate anchoring rule; any observable combinational signal Read/Write label remains part of an emitted R2C fixed-point signal-observation unit under the R2C unit-generation and matching conventions. Thus the pragma affects only implementation sampling freedom under its stated contract, not the logical Σ observation used for compatibility. E3 (Safe message issue ordering). (Post-HLS preservation of R4). For any two executed message-call instances q1,q2 ∈ Q_exec,P of the same process P: (i) Same-interface order is always preserved: if q1 and q2 access the same message-passing interface/channel and q1 <_src q2, then issue_post(q1) ≤ issue_post(q2). (ii) Distinct-interface no-reverse: if q1 and q2 access distinct message-passing interfaces and appear in sequence in the source (q1 <_src q2), then issue_post(q1) ≤ issue_post(q2) (i.e., they shall not be issued in the reverse sequence). (iii) Prefix-closed issue obligation: at every cycle-aligned cutpoint σ of the post-HLS execution / witness, if q1 <_src q2 and q2 has issued by σ, then q1 has also issued by σ and Issue(q1) ≤ Issue(q2). This is an implementation-side obligation on issued sets, not merely an inequality over pairs whose timestamps are already defined. (Note: any pipelined-loop internal staging/dequeue discussed in “Pipelined Loops” is ε-microstate within the back-annotated channel realization and is not a Σ-visible boundary issue event associated with any q ∈ Q_exec,P nor a committed-transfer event in M; therefore it does not constitute a counterexample to (ii) or (iii).) E4 FIFO legality: for each channel c, the post-HLS committed (Push_c(v), Pop_c(v)) history is a legal bounded-capacity execution consistent with Sys_B for that channel (capacity B(c)), and reduces to rendezvous consistency when B(c)=0. In particular, on each channel, the committed Pop_c(v) events return exactly the FIFO-ordered sequence of values v previously committed by Push_c(v), with no drops, duplications, or reordering. E5 (Messages cannot cross their immediate synchronization boundary). For every synchronization call s (in the source order used by R3 / pref_S / suff_S): ∀ q ∈ pref_S(s) : issue_post(q) ≤ clk_post(s) ∀ q ∈ suff_S(s) : issue_post(q) > clk_post(s)

Frontier-item predicate convention. BoundaryReq, ChannelEnabled, and ChannelDisabled are authoritative in their frontier-item form. For a message-operation frontier item x ∈ Ω_msg,P, let q = MsgOf_P(x) and c = BoundaryOf_P(x). BoundaryReq(x,σ), ChannelEnabled(x,σ), and ChannelDisabled(x,σ) are evaluated at that specific explicit abstract boundary and are undefined for non-message frontier items unless the statement explicitly restricts x ∈ Ω_msg,P. In the ordinary unelaborated case, where q has a unique frontier item and a unique evaluated boundary, the q-forms BoundaryReq(q,σ), ChannelEnabled(q,σ), and ChannelDisabled(q,σ) may be used as shorthands for the corresponding x-forms. In Sys_B^elab, one source-level call q may own several frontier items at different explicit boundaries, so the x-form is authoritative.

No hidden cross-channel disablement. Any cross-channel dependency introduced by combinational couplers must be reflected only at explicit internal staged/bundle/join boundaries introduced by the elaboration, i.e., via BoundaryReq/ChannelEnabled/ChannelDisabled on message-operation frontier items x ∈ Ω_msg,P, with MsgOf_P(x)=q, at those explicit staged/bundle channels. At the chosen Σ-visible channel endpoints, BoundaryReq(x,σ) and ChannelEnabled(x,σ)/ChannelDisabled(x,σ) for a message-operation frontier item x ∈ Ω_msg,P on channel c may not depend on other channels' state beyond the per-channel E4 predicate for c, and no extra hidden predicate may block an otherwise E4-enabled boundary transfer.

K-observation architecture (KObs; normative Appendix J/K cutpoint interface)

The formal development has three explicit layers. First, the operational semantics define process evaluation, Issue/Commit, Policy L/S, channel commits, fairness, and the continuation-level Definition R.16/K.Drain discipline. Second, the safety/replay layer proves Theorem J.1 and Lemma J.HCanon over committed observable units and the fixed witness. Third, the K cutpoint layer combines the canonical replay history H with the current fully attributed residual-offer set O in KObs = ⟨E,w,H,O⟩. Obligation J.KObsConf and Corollary J.1 establish the common record for the selected RTL and reachable Sys_ref cutpoints; Lemmas K.0, K.2, and K.2a and Theorem K.1A consume it through the named reconstruction functions. Appendix K-P uses the same KObs/ResOffers interface only at terminal snapshots and retains operational Issue, fairness, nonwithdrawal, and DrainDiscipline for cross-execution and continuation reasoning. No alternate broad state relation or compatibility notation is normative.

(1) Operational Issue is retained. Issue transitions and the Issue/Commit linkage remain part of the operational semantics and continue to govern R4/E3/E5, Policy L, Policy S, Axiom R.1, Definition R.16/K.Drain, the applicable progress and fairness premises including K-O1, and Lemmas K.2b.R/K.2b.P/K.2b.Q. KObs removes historical issue timestamps only from the cross-model cutpoint-observation and certificate interface; it does not redefine Issue(q), identify Issue(q) with Commit(q), or permit an operational proof to ignore whether work was already issued before a block.

(2) KObs is an observation-layer quotient, not a normalized execution. The factoring theorems prove that K-relevant cutpoint predicates depend only on replay history plus the current attributed residual offers. They do not construct a replacement execution in which committed operations issue in their commit cycles, and no issue-erasure normal form is assumed.

(3) Residual offers are typed over frontier items and explicit boundaries. A residual offer identifies the owning process P, frontier item x ∈ Ω_msg,P, owning dynamic message-call instance q = MsgOf_P(x), evaluated explicit abstract boundary c = BoundaryOf_P(x), endpoint direction, and reset epoch. Its operation kind is the explicitly derived field kind(o) := kind(q), with well-typedness requiring producer-valid exactly for a Push instance and consumer-ready exactly for a Pop instance. A bare set of source calls q is insufficient in Sys_B^elab, where one source-level call may own several proof-internal frontier items at different explicit boundaries.

(4) The full elaboration package E is a parameter of KObs. E includes the selected Sys_ref form, explicit abstract channels, capacities B(c), Π_elab, ownership, boundary and projection maps, frontier-item universes and partial orders, and any bundle/join/epoch-gate structure. KObs is not parameterized by B alone.
(5) Cross-model residual-offer conformance is the bidirectional boundary fact J.OfferConf of Appendix J: every abstract residual offer has its attributed RTL boundary request asserted, and every relevant asserted RTL boundary request is attributed to exactly one stated residual offer. The record enforces at most one attributed outstanding source-level offer per endpoint direction at each explicit abstract boundary. This prevents a proof-internal realization transfer already counted in channel occupancy from also being represented as a second source-level offer.
(6) Residual-offer reachability remains separate from boundary conformance. J.OfferReach establishes that a proposed KObs record is in the image of an admissible reachable Sys_ref cutpoint with the same annotated replay history and witness w. J.OfferConf establishes agreement between the record's residual offers and the RTL requests at the selected cutpoint. J.KObsConf requires both; OfferConf alone cannot manufacture an unreachable source offer set, and replay matching alone cannot determine which uncommitted requests are presently asserted.
(7) Snapshot drain closure is distinct from the transition discipline. DrainClosedNow(D,KObs) means that, at the selected ε-quiescent cutpoint, no already-offered non-frontier blocking request owned by a process in D is channel-enabled. It is a function of the cutpoint observation. DrainDiscipline denotes the continuation-level no-new-later-work, non-withdrawal, finite non-replenishing drain, and sync-boundary conditions of Definition R.16/K.Drain; it remains an operational premise.
(8) The minimal K-specific residual-offer abstraction carries no pending Push payload. Payload stability while a blocking Push request is asserted remains required by Axiom R.1 and Appendix C, and the committed payload remains checked by the value-labeled replay/trace results. A proof or checker needing mid-flight data-wire correspondence may add that separate implementation obligation without enlarging the deadlock observation itself.
(9) Elaboration and issue policy remain separate. Pipeline storage, bundle/join structure, and internal stage movement are represented by E and the explicit Sys_B^elab channels. Policy L/S governs issue of the actual source-level operation instances and their elaboration-provided frontier items. KObs records current source-attributed offers but does not turn intra-realization movement into additional source calls or double-count capacity already represented by E4 occupancy.
(10) KObs-facing waiver classes are observationally closed. For any theorem, certificate, or checker that claims a waiver-parameterized predicate is determined by KObs, write KObsClosed_I,E,w(W) for the requirement that, for all reachable ε-quiescent cutpoints σ_1 and σ_2 of the same fixed instance, KObs_E,w(σ_1) = KObs_E,w(σ_2) implies ($\sigma_1 \in W$ iff $\sigma_2 \in W$). This observational closure is distinct from the Lemma K.2b.Q waiver-closure-under-extraction requirement. A waiver class that depends on historical Issue timing, implementation microstate, or any other fact not reconstructible from KObs cannot be used in KObs extensionality unless a separate proof establishes KObsClosed_I,E,w(W).
(11) Environment-terminal reconstruction and the selected serial route are explicit instance data. The ambient instance I records a route r ∈ {General, EnvOrdDone}: General selects Bad_S^W = Dead_KS^W; EnvOrdDone selects Bad_S^W = Dead_K^W and carries the required K.EnvOrd/K.Done evidence. For EnvStop reconstruction, I also records either the standard B1-avail, pull-based, consumption-ordered input model with default always-ready outputs, under which Definition K.StimX and Lemma K.EnvX-Auto reduce permanent input unavailability to fixed-stimulus exhaustion at the consumed ordinal, or an explicit EnvReplayAdequate certificate showing that the relevant environment state and permanent next-interaction availability are functions of H and w. Without one of these environment conditions, KObs still factors the ordinary internal Dead_K predicate but makes no extensionality claim for EnvStop-dependent Dead_KS or the General-route Bad_S predicate.

(12) The certificate boundary is layered and explicit. A proof-backed application records: (a) one common instance header identifying the selected theorem instance, elaboration, capacities, observation boundaries, fixed stimulus, environment/reset contracts, witness and WC provenance, waiver set, and route selection; (b) when Corollary J.1 or Theorem K.1A is used, a KObs cutpoint certificate carrying the canonical replay history H, the fully attributed residual-offer set O, and the evidence for J.HCanon, J.OfferReach, J.OfferConf, B-faithfulness/E4, reset alignment, and finite ε-normalization; and (c) an A5 discharge certificate identifying the Policy-S, direct Policy-L, structural, invariant, exploration, or tool-certificate route and its completeness/fairness evidence. Derived Front, WFG, DrainClosedNow, DeadKRec, DeadKSRec, and BadSRec values are checker-reconstructed from the certified KObs record and ambient instance data; they are not trusted certificate inputs. The cross-model KObs certificate need not contain a complete historical Issue timeline. Operational Issue, fairness, request persistence, and DrainDiscipline evidence remains required where consumed and may be carried in, or referenced from, the source witness and A5 route evidence.

Shared-memory lowering condition (J.Mem). The proof-internal observable alphabet Σ of Appendices I–K contains committed channel transfers, synchronization events, and signal Read/Write actions; it does not contain mapped memory read/write events, and the Theorem J.1 invariant carries per-process replay state and per-channel commit histories but no global shared-memory state. Accordingly, the formal results of Appendices I–K apply to cross-process shared-memory communication only under the following lowering condition, named J.Mem: every shared-memory dependency through which one process's memory write can affect another process's control flow, payload/value expressions, or the identity or order of its later operations shall be lowered by the array-access mapping layer onto operations already covered by the proof alphabet — explicit point-to-point message-passing channels governed by E4 (memory command/response channels), and/or Σ-visible anchored signal IO governed by E2 with the producing Writes observable — so that every memory-carried value is carried by a matched, value-labeled Σ event. Synchronization alone (E1) fixes access order but does not, within the present invariant, establish that the two models contain the same memory value; J.Mem closes that gap by making the value flow Σ-visible. Designs that instead rely on the non-lowered "declared memory semantics" branch of the shared-memory transformation entry in Appendix G are outside Theorem J.1, Corollary J.1, and Appendix K as proved here; for them, agreement under the declared memory semantics is a separate verification obligation (see the informative note below). J.Mem is a named entry in the side-condition discharge record. K.CV's "declared memory serialization" clause remains a liveness-side premise and does not substitute for J.Mem in the safety proof.

Informative note (direct memory-state extension, not developed here). The alternative repair — adding mapped memory Read(addr, val)/Write(addr, val) events to the proof-internal Σ, extending the required-order relation $\prec_J$ with the mapping layer's declared serialization constraints, and adding a global memory-state agreement clause (I4) to the Theorem J.1 invariant together with a memory-state component in the enriched replay/KObs reconstruction and the corresponding J.OfferReach/J.OfferConf interface — is compatible with the present proof architecture but is not developed in this document. A flow that requires direct coverage of non-lowered shared memory shall supply that extension and re-verify Lemmas I.DR/I.DR', J.HCanon, and R.21b–R.21f, together with Corollary J.1, with the added clause.

Reset-boundary observable unit (RESET) and reset well-formedness (RW1–RW5). When dynamic reset is used, each dynamic reset occurrence is represented in the proof-internal event model as an explicitly atomic observable unit RESET(S_p, S_c): a proof-internal boundary marker whose scope names the set S_p of processes and the set S_c of explicit abstract channels affected by that reset occurrence. RESET units may be projected away from the outer Σ_DUT alphabet, like other proof-internal units. The unit is ordered strictly after every observable unit of the ending epoch owned by the affected scope and strictly before every observable unit of the new epoch of that scope, and the first observable action of each $P \in S_p$ in the new epoch is its fresh dynamic START_P. The state-changing reset transition itself — which

occurs before the new epoch's START_P — is exactly what the RESET unit represents, so it receives its own correspondence obligation (Lemma J.RS below) rather than being folded into START_P matching. The following reset well-formedness conditions are implementation-side obligations for designs using dynamic reset:

RW1 (channel-consistent scope). For every explicit abstract channel c, either both endpoint processes of c are in S_p and c ∈ S_c — in which case c re-initializes to reset-empty occupancy (A10) at the boundary and any in-flight (accepted but not yet delivered) contents are discarded consistently on both sides in both models — or neither endpoint is in S_p and c ∉ S_c, in which case c's state and history are unaffected. A reset that would affect exactly one endpoint of a live channel is permitted only if the design models the cross-epoch disposition explicitly (for example, an explicit flush/abort protocol on that channel); that protocol's events are ordinary Σ events, and the channel is treated per this clause after the protocol completes.

RW2 (request disposition). Every outstanding endpoint-owned boundary request of a process in S_p is cleared at the RESET unit; its owning dynamic instance does not commit, is not re-attributed across the boundary, and any post-reset re-execution is a fresh dynamic instance in the new epoch's universe. The Appendix C non-withdrawal discipline is scoped per reset epoch.

RW3 (corresponding reset transitions). At the RESET unit, RTL_B's post-reset state at the chosen abstract boundary corresponds to Sys_ref's prescribed epoch-initial state: control at epoch-initial locations for every P ∈ S_p, source-visible values at their reset values, reset-empty occupancy on every c ∈ S_c, no stale synchronization state, no unaccounted pending endpoint request, no hidden accepted message absent from the source reference model, and direct-input signals obeying their per-epoch stability or sync-controlled contracts (main text, Direct Inputs).

RW4 (atomic boundary). The reset state transition is not interleaved with same-cycle observable actions of the affected scope. If the implementation asserts reset in a cycle in which affected-scope observable actions also commit, those actions belong to the ending epoch and precede the RESET unit in the required order; the two reset epochs of the affected scope share no observable cycle bucket ambiguity at the boundary.

RW5 (locality). The RESET unit changes no state of processes outside S_p and no history or occupancy of channels outside S_c. Cross-scope effects, if any, must be expressed by ordinary modeled events.

Lemma J.RS (MatchedResetEpochSplice). Assume RW1–RW5 and the fixed-stimulus interpretation. Let H be a matched Theorem J.1 prefix satisfying Inv(H) whose next unit in the fixed RTL-time enumeration is a RESET unit γ_R = RESET(S_p, S_c), matched between RTL_B and Sys_ref with the same scope. Then Sys_ref extends from σ_ref[H] by its corresponding reset transition, and Inv(H ∪ {γ_R}) holds: (I1) holds because γ_R is matched with the same scope and the only required orders at the boundary are the epoch-boundary orders above; (I2) holds because, for every P ∈ S_p, the P-local replay state after γ_R is the deterministic epoch-initial state prescribed by RW3 — identical in both models by the reset-correspondence obligation — while for P ∉ S_p the replay state is unchanged by RW5; RW2 guarantees that no request-attribution fact survives the boundary for S_p, so Lemma I.DR′ applies epoch-locally with γ_R delimiting the P-local projection; and (I3) holds because channels in S_c restart with reset-empty histories per RW1 and A10 — the occ_c(t) definition is already measured from the most recent reset boundary — while channels outside S_c retain their histories by RW5. ■ Subsequent induction steps within the new epoch proceed exactly as in the initial epoch; START_P for P ∈ S_p is matched as the new epoch's first ordinary unit. Open dispositions that are not covered by RW1–RW5 — for example, an un-modeled partial reset of one endpoint of a live channel, retention of in-flight messages across the boundary, or cross-epoch request re-attribution — place the design outside Theorem J.1, Corollary J.1, and Appendix K until modeled explicitly.

Additional Semantic Assumptions (B-rules)

The following B rules are process assumptions used within the formal proofs. These B rules are in addition to the R rules presented in Appendix G.

ID	Basic-process property (informal statement)	Why it is needed / how it is used
B1 Deterministic Trace Property	For any fixed test stimulus and initial state, the sequence of observable actions emitted by the post-HLS design is unique, including the channel identity and payload/value of each committed Push_c(v)/Pop_c(v) and the value of each Write(sig,val)/Read(sig,val). Internal ε-steps may differ between runs, but the externally visible Σ-trace (with labels) cannot. **Clarification:** B1 is assumed of **RTL_B** (post-HLS) only. The source-level models **Sys/Sys_B** may admit multiple ε-/latency-different executions with the same Σ-projection. When a single concrete witness execution of Sys_B is required for simulation/debug, Appendix L's snooping wrapper may be used to **select** one admissible witness whose latency-sensitive events align with the unique RTL Σ-trace. Derived per-process consequence. For every process P, the projection of the unique RTL_B Σ-trace onto the observable units owned by or attributed to P is also unique. This follows from the system-level B1 property; it does not assert that P is deterministic when considered in isolation under arbitrary peer, channel, arbitration, or environment behavior.	Ensures the per-process and system-level trace-compatibility proofs (App. I & J) can match one post-HLS trace to one pre-HLS trace without branching on scheduler nondeterminism.
B2 Weak Fairness of the Scheduler (WF)	If an action's enabling predicate remains continuously true from cycle t onward, the scheduler/arbiter must eventually select that action, and the selected action then commits in the selected cycle, within a finite but unspecified number of cycles. Applies to Push, Pop, and Sync operations. For message-passing operations, "enabled" means ChannelEnabled under E4 at the chosen abstract boundary. Enablement is	Required for all progress arguments, especially the liveness lemmas in Appendix K and the channel-progress corollaries.

ID	Basic-process property (informal statement)	Why it is needed / how it is used
	eligibility for selection; it is not itself a same-cycle commit. Thus B2 is not vacuous: an enabled Push/Pop may be delayed by fair arbitration, but it may not be delayed forever while it remains continuously enabled.	
B3 Channel Progress Invariants	B3 is exactly the following family of per-channel progress obligations. For every explicit abstract channel c with capacity B(c), B3 consists of both P_push(c) and P_pop(c), as defined below. Thus references such as "B3," "Appendix N's B3," and "the P_push/P_pop obligations" name the same channel-progress assumption; P_push(c) and P_pop(c) are not additional assumptions separate from B3. B3 is stated separately from B2 because the proofs often reason from the perspective of a currently stalled request. A blocked Push_c at occ_c = B(c), a blocked Pop_c at occ_c = 0, or a rendezvous endpoint waiting for its peer is not itself able to complete until the relevant complementary discharge occurs. B3 packages that discharge fact as a named per-channel progress obligation: under the persistent-request premises below, the full/empty/rendezvous-blocked condition is eventually discharged, after which ordinary E4 availability and B2 can be used for any still-pending enabled transfer. For a single explicit abstract boundary with ordinary E4 buffered or rendezvous semantics, the corresponding B3 discharge is implied by B2 plus E4 once the complementary endpoint request in the B3 premise remains continuously asserted. In the buffered full case, occ_c = B(c) > 0 makes the complementary Pop_c continuously ChannelEnabled; B2 then forces eventual selection and commit of that	Explicitly assumed in Appendix J to rule out permanent back-pressure loops in which both endpoints continuously request a transfer until the matching discharge event but that transfer never completes, which are logically independent of the cycle-by-cycle R-rules.

ID	Basic-process property (informal statement)	Why it is needed / how it is used
	Pop_c, freeing space. In the buffered empty case, occ_c = 0 < B(c) makes the complementary Push_c continuously ChannelEnabled; B2 then forces eventual selection and commit of that Push_c, providing data. In the rendezvous case, the two persistent endpoint requests make the rendezvous pair continuously ChannelEnabled, so B2 similarly supplies eventual rendezvous selection and commit. B3 is nevertheless retained as an explicit named obligation because Lemma I.2, Appendix K, and the channel-validation methodology consume that modular form directly. It is also the form checked per explicit abstract channel in elaborated Sys_B^elab boundary chains. Thus B3 should be read as the modular channel-progress assumption used by the proof and checked against channel realizations, not as a claim that the ordinary single-boundary buffered clauses are logically independent of B2+E4. In the definitions below, “persistent,” “continuously requests,” and “remains true” are interval-scoped. The relevant endpoint request must remain asserted from the starting cycle t_o through the first matching discharge event, if such a discharge occurs. If no matching discharge occurs, the request must remain asserted for all later cycles. A Push request also keeps its payload stable throughout the outstanding interval. B3 does not require an endpoint request or Push payload to remain asserted forever after the matching transfer has already committed. • P_push(c): If a process P is stalled at a blocking $Push_{(C)}$ with its boundary request true and a persistent request asserted with stable payload while the channel is full ($occ_{(C)}$ = B(c)), and the complementary endpoint process continuously requests the matching	

ID	Basic-process property (informal statement)	Why it is needed / how it is used
	$Pop_{(c)}$ until the matching discharge event ($Pop_{(c)}$ endpoint request asserted; $Pop_{(c)}$ boundary request remains true), then the transfer on c must eventually complete. In particular, for B(c)>0, some $Pop_{(c)}$ eventually commits, freeing space; for B(c)=0, the rendezvous completion eventually occurs. • P_pop(c): If a process P is stalled at a blocking $Pop_{(c)}$ with its boundary request true and a persistent request asserted while the channel is empty ($occ_{(c)} = 0$), and the complementary endpoint process continuously requests the matching $Push_{(c)}$ until the matching discharge event ($Push_{(c)}$ endpoint request asserted with stable payload; $Push_{(c)}$ boundary request remains true), then the transfer on c must eventually complete. In particular, for B(c)>0, some $Push_{(c)}$ eventually commits, providing data; for B(c)=0, the rendezvous completion eventually occurs.	
B4 Finite Stutter Bound	Every process P has a finite constant depth D_P such that whenever P is not externally stalled (i.e., P is advancing internal micro-state toward its next observable Σ-action, or its next Σ-action is locally enabled), P can execute at most D_P consecutive ε-steps before either (i) executing a Σ-action, or (ii) reaching a stable waiting state in which P has no further ε-step available until some external/peer/environment condition changes the enabling predicates.	This guarantees the ε-stutter closure needed to construct witness mappings in Appendix I and to bound internal micro-latency (pipeline/FSM bookkeeping) by ≤ D_P. Unbounded waiting due to back-pressure or missing peer stimulus is governed by WF/B2 and the progress obligations B3, not by B4.
B5 System Quiescence Closure	If, at some cycle t_o, no observable actions are enabled in any process, and no external or environmental change from t_o through the closure interval changes the relevant enabling predicates, then within at most **max_P D_P** further cycles the system reaches an ε-fixed point. Thereafter, while no external or environmental change re-enables observable or internal work, the system executes no additional ε-steps.	Needed to finish the starvation-vs-deadlock analysis in Appendix K: ensures an all-disabled state cannot hide behind an infinite tail of unobservable activity.
B6 Time-Divergence /	After the fixed point of B5 is reached — that is, after the system is ε-quiescent, no Σ-	For prefix/trace matching in E1–E5 and for the Exec_post / Exec_ref

ID	Basic-process property (informal statement)	Why it is needed / how it is used
Idle-Stutter Convention	visible action is enabled, and no further internal ε micro-step is enabled under the same stimulus — the global clock continues to advance. We model each subsequent idle clock tick as a stutter step that produces no Σ-event, performs no ε-prefix or ε-suffix work, and leaves all non-clock state unchanged.	conventions in Appendix J, these idle ticks are treated as ε-stutter. Thus B6 gives a time-divergent infinite extension for any reachable finite execution prefix/state that is already at a B5/global fixed point. B6 is not, by itself, an arbitrary deadlock-cutpoint extension principle. In particular, a partial internal message-passing deadlock cutpoint in Appendix K need not be a global fixed point, because outside-D activity, environment activity, or already-issued non-frontier drain work may still be possible. Such cutpoints are handled by the separate Appendix K fair drain-normalization argument: enabled drain-only transfers may commit as finite drain-only progress under the B2/B3 progress assumptions, while pure idle-stutter is used only once the relevant disabled frontier blockers and drain-closed closed component are preserved in the required execution-scoped continuation.

Side-condition discharge record

Before applying Theorem J.1, Corollary J.1 cutpoint transfer, or the Appendix K liveness-preservation theorem to a concrete design, the verification flow shall maintain an explicit side-condition discharge record. The record need not use one physical file, but every referenced artifact shall be versioned and bound to one selected theorem instance through the common certificate header of Definition K.CertHdr. For each theorem premise identified below, the record shall state whether the premise is inapplicable to the selected theorem scope, satisfied by a simple syntactic/design restriction, discharged by tool/RTL evidence, or discharged by a source/RTL witness construction. A condition required by the selected theorem scope may not be marked inapplicable. The named audit entries are K.CertPkg certificate-package integrity; WC; J.GWR global witness-realizability; Sys_B^elab; B-faithfulness; J.OfferReach residual-offer realizability; J.OfferConf residual-offer boundary conformance; the composite J.KObsConf KObs cutpoint-conformance wrapper; J.Mem shared-memory lowering; J.FE fair extension / environment receptiveness; RW reset well-formedness (RW1-RW5, when dynamic reset is used); K.Drain; Kε; A5-S selected-instance serial liveness; A5-L scheduler-robust liberal liveness; K.CV value determinism; NB-Align witness-selected non-blocking alignment; and L.Dir one-directional alignment / RTL-side non-interference (required whenever Appendix L snooping / co-simulation is used), as applicable to the selected theorem scope.

K.CertPkg — certificate identity, schema, and trust boundary.
Trigger: Required whenever a generated or manually assembled certificate package is used to discharge Corollary J.1, Theorem K.1A/K.1, A5-S/A5-L, or CF-1 through CF-5.
Discharge: Provide the common K.CertHdr fields: certificate-schema and document/theorem version; a digest or unambiguous identifier for the selected Sys_ref/Sys_K and RTL artifact; E, B_E, explicit boundaries, projections, ownership, and K.Low lowering where applicable; fixed stimulus, environment, reset policy, witness w, WC provenance, waiver set W and closure evidence, and selected Bad_S route; the theorem scope and side-condition evidence references; and the generator/checker versions and declared trusted base. Every subordinate KObs or A5 certificate shall cite the same header identity. Derived K-facing predicates shall be recomputed by the checker rather than accepted as authoritative fields.
Evidence examples: A signed or hashed manifest binding source, RTL, elaboration, witness, and evidence files; a tool-generated certificate bundle with an independently validated schema; or an equivalent reviewable instance record.

- WC — witness-compatible ε choices / non-blocking outcomes.
  Trigger: Required when failed non-blocking polls or other ε-modeled choices can affect later Σ-visible control flow, frontier membership, request attribution, guard values, payload values, or other later Σ-visible choices.
  Discharge: Provide evidence that proves the actual WC premise used by Appendices I–J. The audit entry shall identify the provenance route by which WC was established:
  – NB-CF route: show that failed PushNB/PopNB outcomes do not affect later Σ-visible control flow, frontier membership, request attribution, guard values, payload values, or other later Σ-visible choices. Under this route, the failed-poll witness may be the identity or arbitrary selector, subject to the applicable latency-sensitive local-protocol conditions.
  – Appendix L snooping route: provide a witness selector W satisfying Lemma L.2. Show that the snooped outcomes cover every ε-modeled source choice that can affect later Σ-visible behavior, that the wrapper changes only ε-choice selection, and that it does not add, remove, reorder, or relabel Σ-visible events.
  – Direct witness-construction route: otherwise construct and prove the μ_Q / WC witness used by Appendix I/J.
  – Mixed per-site route: partition the relevant non-blocking or ε-choice sites among the applicable NB-CF, snooping, or direct-construction routes, and show that the partition covers every relevant site.
  NB-CF and Appendix L snooping are therefore provenance routes for proving WC, not separate theorem premises that replace WC. When useful for review or automation, the WC audit entry may include optional per-site metadata identifying each relevant call site or ε-choice site, its selected provenance route, and the corresponding source restriction, tool/RTL evidence, wrapper evidence, or witness construction.
  Evidence examples: Source inspection or a static control/dataflow check establishing NB-CF; a generated WC witness or certificate; a wrapper proof or monitor specification satisfying Lemma L.2; a co-simulation/snooping harness, subject to the L.Dir entry below; or a complete per-site coverage report for a mixed discharge.

  - Sys_B^elab — elaborated source reference model.
    Trigger: Required when ordinary Sys_B capacities at the outer abstract boundaries are not sufficient to
    represent the RTL realization as ordinary E4 bounded-FIFO/rendezvous behavior; examples include

multi-input/join couplers, staged/bundle/join channels, and sync-boundary / epoch-gate capacity
withdrawal.
Discharge: Provide a well-formed elaboration package E defining the explicit abstract channels, capacities, projection Π_elab, and E4 ChannelEnabled/ChannelDisabled behavior used by Sys_B^elab(E).
Evidence examples: Tool-generated elaboration, structural back-annotation report, Appendix Q-style
construction, or manual elaboration justified against the RTL channel structure.

- B-faithfulness — faithfulness of each back-annotated capacity B(c) and chosen explicit abstract channel boundary.
  Trigger: Required for every back-annotated explicit abstract channel c whose B(c), occupancy occ_c(t), E4 ChannelEnabled/ChannelDisabled behavior, BoundaryReq facts, or Appendix K blocking/WFG behavior is used by the selected theorem instance. For such a channel, B-faithfulness is a required per-channel obligation and shall not be marked inapplicable.
  Discharge: Show all of the following for the chosen abstract boundary of c:
  Boundary completeness — every Σ-visible transfer of c in RTL_B crosses the chosen abstract boundary as the corresponding committed Push_c or Pop_c, with no hidden side entrance or exit.
  Exact boundary behavior — for every relevant execution state and boundary action, the concrete realization permits the action exactly when the E4 abstract channel permits it. In particular, realized unpopped occupancy is bounded by B(c), and the realization contains no hidden storage or credit beyond the capacity represented by B(c).
  No hidden enablement or disablement — no unmodeled grant, arbitration, throttling, or backpressure condition changes whether a boundary action is permitted. Any such condition that affects boundary behavior shall instead be represented by explicit Sys_B^elab boundary structure.
  ε-opacity of internal movement — movement of data within the realized channel between chosen boundary commits is internal ε-behavior and creates no additional Σ-visible Push/Pop event or Appendix K WFG blocking cause.
  Evidence examples: A trusted back-annotation algorithm; a per-channel capacity and boundary certificate; a structural RTL check; an RTL/interface monitor; a formal refinement check comparing the realized boundary behavior with E4; or equivalent tool/RTL-flow evidence establishing the complete B-faithfulness obligation.
- J.OfferReach / J.OfferConf / J.KObsConf — KObs cutpoint correspondence.
  Trigger: Required whenever Corollary J.1 cutpoint correspondence is used, and therefore required for Appendix K / Appendix R uses that transfer Pending_P, BoundaryReq, rendezvous endpoint-request, ChannelEnabled/ChannelDisabled, Front_P, WFG, or derived deadlock facts between RTL_B and Sys_ref. The evidence shall be packaged as, or mapped unambiguously onto, the CF-0 KObs cutpoint certificate of K.4e.
  J.OfferReach discharge: For the same selected Theorem J.1 source witness, witness w, and ε-normalization used by the application, exhibit H and O and show that the selected reachable ε-quiescent Sys_ref cutpoint σ_ref satisfies H = H_E,w(σ_ref) and O = ResOffers_E,w(σ_ref), with all operational Issue/Commit, E3/E5, reset-epoch, elaboration, and source-transition rules respected. This is an image/realizability obligation; well-typedness of ⟨E,w,H,O⟩ alone is insufficient. The complete Issue history may remain in the referenced source witness rather than being duplicated in the CF-0 record.
  J.HCanon discharge: Establish the hypotheses of Lemma J.HCanon for the selected source/RTL

annotated prefixes and fixed J.GWR package; record that both prefixes produce the same canonical enriched replay-history quotient H.
J.OfferConf discharge: At the selected ε-quiescent RTL_B cutpoint ρ_post, enumerate every KObs-relevant asserted endpoint-owned request at the explicit boundaries of E and prove bidirectional, unique attribution between that physical request set and O using J.GWR(ii), Appendix C, and I.A0. Record the physical RelevantReqSet_E(ρ_post), ReqProjection_E(O), their equality, request-to-offer completeness, no-hidden-request, boundary completeness, issued-summary prefix closure, reset-epoch typing, and endpoint cardinality. A monitor or certificate that reports only requests it can already attribute is insufficient unless it separately proves that no other KObs-relevant asserted request exists.
J.KObsConf discharge: For that same selected tuple (ρ_post, τ_ref, σ_ref, E, w, H, O), additionally establish B-faithful/E4 interpretation at every explicit boundary, reset alignment, and ε-opacity of normalization. J.KObsConf is the composite application wrapper; it no longer requires independent proof of a broad source-value/frontier/channel/enablement slice, because those facts are reconstructed from equal KObs by Lemmas R.21b-R.21e.
Audit priority: J.OfferConf is the authoritative RTL request-abstraction boundary. Its completeness and no-hidden-request directions shall receive the same explicit review and certificate coverage previously assigned to the request/pending clauses of the broad J.KObsConf relation.
Evidence examples: A CF-0 tool-generated paired KObs/cutpoint certificate; a source execution log proving OfferReach; an RTL/interface request-attribution monitor plus boundary-completeness proof proving OfferConf; structural back-annotation and B-faithfulness evidence; a co-simulation witness log; a bounded proof over generated RTL; or an equivalent combined certificate.

- K.Drain — drain-only blocked-frontier discipline. Trigger: Required for Appendix K whenever the selected liberal Sys_ref or RTL_B execution may have source-later same-interval blocking work already issued while an earlier minimal blocking frontier remains uncommitted. This includes pipelined-loop overlap, other explicitly modeled eager/overlapped issue under Policy L, and ordinary non-overlapped control when Lemma K.2b or drain normalization relies on the absence of newly launched work past the blocked point. Discharge: Show the full Definition R.16 discipline: blocked-frontier identification, no new later work, no request withdrawal, drain-only downstream progress, finite explicit boundary storage/credit, and any required sync-boundary capacity withdrawal in Sys_B^elab. Ordinary source operations on distinct interfaces remain source-ordered; any multi-frontier behavior used by Appendix K must be represented by explicit source/elaboration structure rather than by implicit incomparability.
- Kε — WFG-inert ε / no hidden micro-timing control decisions.
  Trigger: Required for Appendix K liveness/deadlock preservation when ε-steps occur between Σ-visible cutpoints.
  Discharge: Show that ε-only steps do not create, remove, or redirect the relevant Appendix K WFG blocking causes except through the explicit Σ-visible or E4-modeled channel state already represented in Sys_ref.
  Evidence examples: RTL structural argument, scheduler proof, bounded pipeline/drain argument, formal check over the WFG abstraction, or tool certificate.
- A5 — primary selected-instance Policy-S liveness (A5-S). Trigger: Required whenever Theorem K.1, the user-facing Policy-S preservation theorem, is invoked; the core Theorem K.1A instead consumes A5-L. A5-S is a property of the selected normalized instance Sys_K, including its capacities B(c), explicit abstract boundaries, Sys_B^elab structure, fixed stimulus, fixed witness choices, environment/reset contracts, waiver set, route-selected Bad_S predicate, and

Definition K.Low lowering. Discharge: Supply the CF-S Policy-S execution certificate of K.4e. It shall identify one designated complete maximal fair Policy-S execution and establish that none of its reachable ε-quiescent, drain-closed cutpoints satisfies Bad_S^W. The certificate records the transition-system identity, initial state, witness and schedule provenance, the route-selected reconstructed predicate, and sound evidence of maximality, fairness, completeness, and predicate avoidance. When the optional static K.EnvOrd and K.Done conditions are discharged, Corollary K.EnvOrd-1 permits the simpler check against Dead_K^W. A finite regression prefix of a nonterminating execution is supporting evidence but is not a complete discharge unless accompanied by an invariant, sound model-checking/recurrence argument, or other completeness evidence. The checker reconstructs each Bad_S value from the cutpoint KObs and ambient instance data; it does not trust a logged Boolean result.

- A5-L — scheduler-robust liberal source liveness. Statement: No admissible Policy-L execution of the selected Sys_ref instance, under the fixed stimulus and witness choices, reaches an ε-quiescent cutpoint containing a non-waived internal message-passing deadlock in the K.1 sense. A5-L is the premise consumed by Theorem K.1A. Trigger: Required whenever the Appendix K preservation conclusion is applied. Discharge: Primarily by a CF-S certificate and Corollary K.2c from one complete maximal fair Policy-S run satisfying A5-S; alternatively by Theorem K.Str-1, K.Str-2, or K.Str-3, direct Policy-L verification, an inductive invariant, exploration, or a validated CF-5 tool certificate. The A5 discharge package shall cite one K.CertHdr and record the route tag, selected normalized instance, capacities, elaboration, lowering, witness, stimulus, environment/reset model, waivers and closure evidence, target predicate, and route-specific completeness/fairness or inductiveness evidence. A direct Policy-L package targets Dead_K^W; a Policy-S reduction package targets the recorded Bad_S^W predicate and additionally cites the Lemma K.2b/Corollary K.2c applicability evidence.
- K.CV — design-level value determinism.
  Trigger: Required for Lemma K.2b, Corollary K.2c, and Theorem K.1 whenever a process can branch on, compute payloads from, or select later operations based on cross-process observations beyond blocking Pop values — anchored signal reads, direct-input(-sync) reads, shared/external-memory reads through the array-access mapping layer, emitted R2C fixed-point observation units, or synchronization outcomes.
  Discharge: Show that every such dependency is carried by an explicitly modeled causal mechanism and that the observed value is a deterministic function of the causal history feeding that mechanism, not of incidental schedule timing — i.e., the design satisfies the causal-dependency well-formedness rule with RULE 1 / RULE 2 anchored signal IO, proper handshakes or direct-input contracts on cross-process signals, mapping-layer discipline on shared memories, and R2C well-formedness; latency-sensitive sites are isolated or witness-aligned per Appendices D and L.
  Evidence examples: Source inspection or lint establishing RULE 1 / RULE 2 and handshake conformance; a coverage-justified Appendix L latency-robustness stress-testing result, a refinement argument; an Appendix D isolation argument; or a causal-dependency review showing no reliance on the ordering of causally independent events. Randomized stress testing by itself is supporting evidence, not a complete K.CV discharge, unless accompanied by a coverage/refinement argument showing that the tested executions cover the relevant value/control-dependence cases. This K.CV evidence is necessary but not sufficient for the separate NB-Align condition (next entry), which is discharged independently.
- NB-Align — witness-selected non-blocking alignment.Statement: NB-Align is the scope restriction stated normatively in the Lemma K.2b Statement (Appendix K), which is its authoritative wording: every witness-selected non-NB-CF non-blocking site that can participate

in a potential wait cycle must satisfy the Appendix L alignment conditions that convert its selected outcome dependency into an explicitly modeled message/synchronization obstruction. NB-Align is consumed as the applicability condition of the Definition K.Low witness-indexed blocking lowering: an aligned site is lowered to a blocking guard on an explicit point-to-point supplier channel, and a witness schedule containing a non-aligned such site admits no lowering and is outside the Lemma K.2b / Corollary K.2c / Theorem K.1 scope. Theorem K.1A does not consume NB-Align.Trigger: Required whenever Lemma K.2b or Corollary K.2c is invoked (including through Theorem K.1) and the design contains witness-selected non-blocking outcomes that are not NB-CF.Discharge: Show that each witness-selected non-blocking site is NB-CF, is Appendix L-aligned (so that Definition K.Low applies to it), or is structurally excluded from every potential wait cycle of the selected instance.Evidence examples: An NB-CF classification per Appendix L; a per-site Appendix L alignment argument identifying the modeled supplier used by the Definition K.Low guard channel; or a structural argument that no witness-selected site can participate in a wait cycle of the selected instance. NB-Align is distinct from, and not implied by, K.CV.

- K.Low — witness-indexed blocking lowering and cutpoint correspondence.Trigger: Required whenever the serial-reduction route (Lemma K.2b / Corollary K.2c / Theorem K.1) is applied to a design containing explicit synchronization waits or NB-Aligned witness-selected non-blocking sites in potential wait cycles.Discharge: Exhibit the lowered instance Λ_W(Sys_ref) of Definition K.Low — epoch-gate representation of synchronization per Definition R.16 clause (7), and a blocking guard channel per aligned site with its modeled supplier as unique producer — and discharge the Lemma K.Low.C correspondence obligations (i)–(iii) for the lowering rules used. The A5 / A5-L premises and the Policy-S check are then evaluated on the lowered instance.Evidence examples: The elaboration record for Λ_W(Sys_ref); per-site guard-channel construction tied to the Appendix L alignment; a mechanized or reviewed K.Low.C case proof per lowering rule. Guard channels carry the corrected default B(g_s) = 1 skewed timing, or a declared atomic rendezvous variant, under Definition K.Low.
- K.EnvOrd — optional static environment no-bypass condition (may-follow form), with completion-sufficiency companion K.Done. Trigger: Optional; when both are discharged, the general Dead_KS Policy-S check reduces to the ordinary internal K.1 predicate (Corollary K.EnvOrd-1). Static discharge of K.EnvOrd: Build the finite starvation graph over operation sites or process-boundary schemas. Seed it with environment-facing blocking sites and, unless K.Done is discharged, with possibly-under-supplied internal sites. Close it transitively through an internal blocking site b whenever the unique complementary supplying site for b may-follows a graph site of the counterpart process — may-follow meaning textually source-later or sharing a control-flow cycle, so loop-carried instances are covered. For every site q that may-follows a graph site b of the same process and can feed an internal wait component, prove that q is control- or data-dependent on the commit or selected outcome of the dynamically preceding pending instance of b; a pipelined loop containing a graph site shall gate iteration advance on that site's commit for independent feeding operations. K.CV/WC then imply the dynamic no-bypass property. Discharge of K.Done: prove that every possibly-terminating endpoint's total production covers its counterpart's demand, or declare every process perpetual. Caution (normative): the one-hop form (Example K.CE-2) and the site-level source-later form without may-follow (Example K.CE-3) do not suffice; without K.Done, completion-rooted starvation defeats the collapse (Example K.CE-4). Neither condition is required for the general Dead_KS route.
- W.Env — environment-rescue waiver. Trigger: Optional; used only for a specifically declared cutpoint class whose environment-facing co-frontier is intended and proved to rescue the

internal wait. Discharge: Record the component class, rescuing item, environment contract, and justification. Waived classes are removed explicitly from Dead_K^W and Dead_KS^W and propagate to Theorems K.1A/K.1. A broad end-of-test waiver is not sufficient to hide the Example K.CE pattern; a terminal class containing internal waiters must also carry a recorded transitive no-bypass argument (Condition K.EnvOrd cone form) or an equivalent safety argument; a one-hop no-bypass record does not suffice (Example K.CE-2), nor does a site-level record without may-follow (Example K.CE-3); a terminal class ending at a completed producer requires a Condition K.Done record or equivalent. Each entry shall also satisfy the waiver-closure requirement of Definition K.DeadW.

- L.Dir — one-directional alignment / RTL-side non-interference.
  Statement: L.Dir is the scope restriction stated normatively in Condition L.Dir (Appendix L), which is its authoritative wording: the source-side snoop sequence selected by W is exactly an ordinal-preserving consumption of the free-running RTL_B snoop sequence at the chosen boundary — each pre-HLS-side snoop commit consumes the next recorded RTL_B event of the same snoop point with matching kind and payload, no pre-HLS-side snoop commit occurs without such a next recorded RTL_B event, every recorded RTL_B snoop event within the compared run is eventually consumed or reported outstanding per the Requirement W4 bound, and no RTL_B-side signal or event is gated, delayed, or modified. Benign cycle skew is permitted; only order, kind, or payload divergence — non-existence of the WC witness at the chosen boundary — violates the condition. A run violating L.Dir is outside the Appendix I–K theorem scope for that comparison and carries no theorem-backed guarantee.
  Trigger: Required whenever WC is discharged through an Appendix L snooping / witness-selector co-simulation harness (Lemma L.2 / Lemma L.5).
  Discharge: Provide a per-run monitor per Lemma L.5(iv) — realized by a wrapper conforming to the Appendix L Wrapper Realization Requirements W1–W5 — that aborts loudly on violation, together with the run's clean-monitor result and a statement of whether the Requirement W4 watchdog bound is proved sufficient for the selected instance (bound-exceeded aborts under an unproved bound are inconclusive witness-realization failures, not proven violations); or a static/structural argument — for example, latency back-annotation or delay padding of the pre-HLS model, or latency-interval analysis at each snoop point in the spirit of the simplified snooping approaches — showing that the post-HLS side cannot run ahead in event order; or a model-checking proof that order divergence is unreachable for the selected instance.
  Evidence examples: A harness monitor log showing no L.Dir abort, with cycle-skew statistics reported separately as diagnostics; a static latency analysis at the simplified-snooping end of the Appendix L spectrum; or a wrapper certification against Lemma L.2 together with a watchdog configuration that converts witness non-existence into a reported failure rather than a hang.
  L.Dir violations are triaged per the Snooping Concerns section of Appendix L: the pre-HLS model is the default suspect in a certified-harness, trusted-tool flow, but harness incompleteness and tool non-compliance must not be excluded a priori.
- J.GWR — global witness-realizability.
  Trigger: Required whenever Theorem J.1's Post → Ref clause, Corollary J.1, Theorem R.21, or any result consuming their witnesses (including Appendix K) is applied.
  Discharge: Supply, for each compared RTL_B prefix, a witness/annotation package satisfying Definition J.GWR — in particular the realizable RTL-time enumeration and finite selected-unit extension properties, rendezvous / paired-synchronization endpoint co-presence, coherent μ_Q / operational Issue / request-attribution / E4-history annotations needed by the execution-level witness construction, and, when cutpoint transfer is requested, the enriched replay-history record H used by J.OfferReach/J.KObsConf. Historical Issue timestamps remain permitted inside

the operational witness package where Theorem J.1 needs them, but they are not fields of KObs and are not independently compared across cutpoints.
Evidence examples: tool-generated schedule certificate; an Appendix L snooping / witness-selection harness that records the package during co-simulation; a flow-specific global witness-realizability theorem proved once per tool version.

- J.Mem — shared-memory lowering.
  Trigger: Required whenever the compared design contains a cross-process shared-memory dependency that can affect control flow, payloads, or operation identity/order.
  Discharge: Show every such dependency is lowered by the array-access mapping layer onto covered IO operations (explicit point-to-point E4 channels and/or Σ-visible anchored signal IO), or supply the memory-state invariant extension described in the informative note in Common Formal Definitions.
  Evidence examples: use of pre-verified shared-memory library classes presenting channel/signal boundaries; mapping-layer configuration review; structural check that no un-lowered cross-process array access exists.
- J.FE — fair extension / environment receptiveness.
  Trigger: Required whenever an infinite fair execution is asserted to contain a given finite reachable prefix (Appendix I.5; the embedding step of Theorem K.1).
  Discharge: Show the fixed stimulus/environment is receptive — defined on every reachable Σ-causal history — so that Lemma J.FE applies; or discharge the required embedding directly for the flow.
  Evidence examples: testbench review showing input-enabled/receptive behavior; environment model with a total transition relation; direct construction of the continuation.
- RW — reset well-formedness (RW1–RW5).
  Trigger: Required whenever dynamic reset can occur in the compared executions.
  Discharge: Show every dynamic reset occurrence satisfies RW1–RW5 so that Lemma J.RS (MatchedResetEpochSplice) applies at each RESET boundary unit.
  Evidence examples: reset-tree/scope review; RTL check that both endpoints of each live channel reset together or that an explicit modeled flush/abort protocol is used; confirmation that outstanding requests clear and affected channels re-initialize reset-empty.

This record is not an additional semantic assumption beyond the named premises already used in Appendices I–K. It makes the required discharge of WC, J.GWR, elaboration, B-faithfulness, J.OfferReach, J.OfferConf, J.KObsConf, J.Mem, J.FE, RW reset well-formedness, K.Drain, Kε, A5-L scheduler-robust liveness, the selected A5-L discharge route—including A5-S and its route-selected Dead_KS/Dead_K check when the Policy-S route is used—and K.CV value determinism explicit for the selected theorem scope. Its purpose is to make theorem application auditable: realistic complications are allowed, but each complication must be mapped to a named condition and discharged rather than left implicit.

Appendix I – Per Process Trace Compatibility Proof

I.1 Overview

This appendix establishes only the per-process witness property in the direction needed by Appendix J's execution-level correspondence: under a fixed initial state and fixed external stimulus/environment

behavior, every reachable finite observable prefix of the post-HLS model RTL_B has a matching finite observable prefix in the prescribed source reference model Sys_ref (Appendix J, Remark 2), satisfying E1–E5 (Post → Ref).

Downstream citation discipline. Later appendices may cite Appendix I for this Post → Ref witness construction over Sys_ref, and for the local per-process ordering / anchoring facts proved along the way, including Implementation assumption I.A0 and Lemmas I.1–I.3. Appendix I does not, by itself, justify an unrestricted statement that every Sys_ref trace/prefix has a matching RTL_B trace/prefix.

Importantly, the converse direction ("every Sys_ref trace has a matching RTL_B trace") is NOT claimed in general when latency-sensitive / non-blocking effects are made Σ-visible (e.g., by observing non-blocking poll outcomes or other timing-sensitive events via an expanded observation boundary). In such cases, Sys_ref may admit additional Σ-traces (for example, extra unsuccessful poll observations, or in elaborated cases additional source-side micro-realizations with the same outer intent) that do not match the RTL_B Σ-trace under the same stimulus. When a reverse correspondence is required, Appendix L's snooping / witness-selection technique is used to restrict attention to annotated RTL-consistent Sys_ref executions in the sense of RTLConsistentRefPrefix (see Theorem J.1). The rendezvous specialization is recovered when Sys_ref = Sys_B and B(c)=0.

I.2 Formal Preconditions

- **Observable alphabet** Σ is as defined in *Common Formal Definitions for Appendices I–K*: it consists of the event schemas {Push_c, Pop_c, Sync, wait, START_P, FINISH_P, Write, Read} instantiated only for channels/signals designated observable at the chosen proof boundary. For Appendices I–K, Σ is the proof-internal observable alphabet of the prescribed Sys_ref / RTL_B comparison. Thus all message-passing channels that can participate in Appendix K deadlock reasoning are designated observable at that proof boundary and hence included in Σ. In elaborated cases Sys_ref = Sys_B^elab, this proof-internal Σ may include introduced staged/bundle/join channels even when the projection Π_elab hides those internal events from the outer DUT/testbench-visible alphabet Σ_DUT. Such events are therefore Σ-visible for the proof-internal comparison, but may be projected away from the outer observation boundary.
- **Non-blocking message-passing interpretation.** In both RTL_B and Sys_ref, a non-blocking call (PushNB / PopNB) is treated as a poll. A successful poll contributes the same observable event as the corresponding committed Push_c / Pop_c on that channel. An unsuccessful non-blocking call (returns "false" / "not completed") contributes no Σ-event and is modeled as an ε-step. A non-blocking poll may still assert a Σ-visible endpoint request in the cycle of that call. However, each executed non-blocking call is a distinct dynamic source-level instance $q \in Q_{exec,P}$ for its owning process P. Accordingly, continuous assertion of an endpoint request with no intervening commit does not by itself collapse later executed non-blocking call instances into an earlier q. Only a poll that succeeds contributes the corresponding committed transfer $m(q) \in M$ for that call instance. If a request assertion persists without a new executed source-level call instance being entered, its attribution remains with that same currently outstanding q; but repeated non-blocking polls in later loop iterations or later source-level call instances create additional $q \in Q_{exec,P}$ even if req does not deassert.
- **Silent step** Any internal RTL microstate transition is written **ε**. In particular, unsuccessful non-blocking polls, arbitration bookkeeping, and pipeline advances are ε-steps.
- **Finite-pipeline-depth premise (FPD)** There exists a finite constant $pipe\_depth(P) = D_P < \infty$ such that once P begins internal micro-progress toward its next observable Σ-action (or that next Σ-action is locally enabled), P executes at most D_P cycles of ε-steps before either committing a

Σ-action or reaching a stable waiting state with no further ε-step available until external/peer conditions change. In particular, a process cannot perform an unbounded number of ε-only failed non-blocking polls while making no progress toward any Σ-action; designs that can spin indefinitely without reaching either a Σ-action or a stable wait state violate FPD/B4 and are outside the model.

- **Weak-fairness premise (WF)** If a micro-operation remains continuously enabled, the scheduler eventually selects it for progress. For an already-issued blocking message request whose BoundaryReq is asserted and whose ChannelEnabled predicate remains true, this means the enabled transfer is eventually allowed to commit; it does not mean that the same source-level operation is "issued" again. This is an assumption on the execution/scheduling environment (see Appendix N), not an automatic guarantee of "clock-synchronous RTL." In practice it requires that any arbiters/back-pressure logic that can affect whether an enabled operation is selected or allowed to commit must be fair in the sense of B2/B3.
- **Finite-progress invariants (B3)** For every channel c, assume producer and consumer obligations P_push(c) and P_pop(c) hold, guaranteeing that data (or space) eventually becomes available. This is an assumption on the execution/scheduling environment (e.g., fairness of arbiters/back-pressure; see Appendix N), not a consequence of the cycle-by-cycle R-rules.

- **Channel enablement semantics.** Let $q \in Q\_exec,P$ be a dynamic source-level message-operation instance of its owning process P on message-passing channel c, and let $kind(q) \in \{Push\_c, Pop\_c\}$ denote its endpoint direction. If q has issued a persistent blocking request to transfer on c, then BoundaryReq(q,σ) holds at the ε-quiescent cutpoint σ: the endpoint-owned boundary request remains asserted, and the payload is stable if q is a Push. The channel-side commit-enabling predicate for q is determined as follows.

  Rendezvous case, B(c)=0. A Push_c operation instance q is channel-enabled exactly when BoundaryReq(q,σ) holds and the matching Pop_c endpoint boundary request is asserted in the same ε-quiescent cycle. A Pop_c operation instance q is channel-enabled exactly when BoundaryReq(q,σ) holds and the matching Push_c endpoint boundary request is asserted in the same ε-quiescent cycle. Thus, when both endpoint requests are asserted in that cycle, the matched Push_c(v) and Pop_c(v) operation instances are eligible for selection as a rendezvous pair. If the scheduler selects that enabled rendezvous pair in that cycle, the matched Push_c(v) and Pop_c(v) commit as a same-cycle rendezvous pair and contribute the corresponding Σ-events. If the pair remains continuously enabled, B2 supplies eventual selection.

  Buffered case, B(c)>0. The channel-side eligibility predicate for q is exactly the FIFO availability predicate conjoined with BoundaryReq(q,σ):
  for $kind(q)=Push\_c$: $ChannelEnabled(q,\sigma) \Leftrightarrow BoundaryReq(q,\sigma) \wedge occ\_c(\sigma) < B(c)$ ("not full"), and
  for $kind(q)=Pop\_c$: $ChannelEnabled(q,\sigma) \Leftrightarrow BoundaryReq(q,\sigma) \wedge occ\_c(\sigma) > 0$ ("not empty").
  Here ChannelEnabled means that the abstract channel boundary permits the transfer to be selected; it does not by itself assert that the transfer has already committed in the current cycle.

  Modeling note (fall-through / simultaneous empty/full cases). The buffered abstraction for B(c)>0 is cycle-boundary, non-fall-through. A Push and a Pop may both be selected and commit in the same cycle when the FIFO begins the cycle neither empty nor full ($0 < occ\_c(t) < B(c)$); in that case both commits are legal and the occupancy is unchanged. The abstraction does not permit a Pop to be selected in a cycle that begins empty solely because a Push is also selected in

that same cycle, nor does it permit a Push to be selected in a cycle that begins full solely because a Pop is also selected in that same cycle. If an implementation uses a fall-through FIFO (allowing same-cycle pass-through when empty), model that behavior explicitly (e.g., as an explicit bypass/rendezvous path composed with the bounded FIFO), so that the Σ-level committed-transfer trace matches the intended cycle semantics.

Operational interpretation (availability, selection, and ready/valid): occ_c(t) is an abstract quantity derived from Σ-visible boundary commits for B(c)>0 buffered channels; processes do not read occ_c(t) directly. For B(c)=0 rendezvous channels, the corresponding channel-side availability fact is the matching endpoint request predicate in the same ε-quiescent cycle. In both Sys_ref and RTL_B,
ChannelEnabled/ChannelDisabled records the settled abstract boundary availability/eligibility predicate. A commit occurs only when an enabled operation, or enabled rendezvous pair, is selected by the scheduler/arbiter for that cycle.

Here Sys_ref denotes the prescribed source reference model at the chosen abstract boundary: Sys_B in the ordinary case, or Sys_B^elab in the elaborated back-annotated cases. A concrete realization is correct only if its committed ready/valid transfers are legal with respect to the corresponding rendezvous or buffered availability predicate at that chosen boundary, and if every transfer that remains continuously eligible is eventually selected as required by B2. Thus E4 is a safety/legality condition on which transfers may commit, while B2 is the progress/fairness condition that prevents continuously eligible transfers from starving.

No additional hidden availability predicate is part of the channel semantics. Any cross-channel coordination, arbitration, grant, or back-pressure policy in the realization must be represented either
(1) as scheduler/arbiter selection among already ChannelEnabled actions, governed by B2, or
(2) as explicit abstract boundary structure in Sys_B^elab with its own B(·), occ_·(t), BoundaryReq, ChannelEnabled/ChannelDisabled, and B3 obligations. Such coordination must not be encoded as an unmentioned extra conjunct that silently changes the E4 availability predicate at an otherwise ordinary channel boundary.

Therefore, when the boundary request is true and the relevant rendezvous or FIFO availability predicate holds continuously, the transfer is continuously enabled; by weak fairness (B2), the corresponding commit occurs after a finite, though not necessarily bounded, delay.

- **No ill-formed signal IO.** Every observable signal Read/Write has a unique real bounding synchronization operation on each dynamic path on which it occurs; otherwise, the design is ill-formed. START_P may be used as an initial anchor only when it is included as a real synchronization event in S, and FINISH_P may be used as a terminal anchor only when the process actually terminates. The formal model does not use a virtual synchronization event at ∞. (From RULE 1 and RULE 2.)
- **Fixed-stimulus interpretation (reactive environments).** If the “same stimulus” includes a reactive testbench and/or peer processes, it is interpreted as the same global Σ-causal behavior (same externally supplied inputs, and the same peer/environment responses to the same prior Σ-visible history), not as a function of P-local history alone. In Appendix I this interpretation is used only as a witness-replay condition: when a next RTL_B event e is already fixed in the chosen post trace prefix, any peer/environment enabling needed for e is part of that same fixed global Σ-causal behavior and is therefore replayed consistently for the matching Sys_ref

construction. This is not a standalone assumption that arbitrary Sys_ref continuations are globally live.

- **WC (Witness-compatible non-blocking outcomes).** Because failed polls are modeled as ε (unobservable), all correspondence results in Appendices I–K are stated for witness-compatible comparisons: any ε-modeled non-blocking poll outcome (and any other ε-modeled internal choice that can affect the next Σ-visible control flow / next Σ-visible blocking frontier) is fixed/selected to be consistent with the compared RTL_B execution prefix τ_post (e.g., via the Appendix L witness-selection/snooping wrapper). WC also supplies the μ_Q matching for ε-modeled failed non-blocking call instances when E3/E5 or the proof construction refers to $q \in Q_{exec,P}$; those failed q's are not inferred from Σ alone.
- **Remark (NB-CF as a sufficient condition).** If a design satisfies the following non-blocking control-flow property—failed PopNB/PushNB polls do not influence the next Σ-visible control flow/frontier—then the witness is trivial and WC holds automatically. Equivalently: between two consecutive Σ-events of a process, inserting/removing any finite sequence of failed PopNB/PushNB polls may change only internal ε-behavior/timing, not which Σ-visible action (Push/Pop/synchronization anchor) is next in program order.

I.3 Auxiliary Lemmas

Implementation assumption I.A0 (RTL message-issue discipline).
The post-HLS RTL_B execution satisfies the full E3 obligation as an implementation-side scheduling obligation: (i) per-interface issue order is preserved for all message operations; (ii) for distinct interfaces, the no-reverse rule holds; and (iii) issue is prefix-closed under source order, so at every compared cutpoint a source-later issued message operation implies that every required source-earlier message operation has also issued, with Issue timestamps ordered accordingly. This condition is not derived inside Appendix I; it is the RTL/tool-compliance obligation corresponding to the scheduling rules stated in the main text and Appendix G. For hand-written RTL or other non-HLS implementations, I.A0 is the corresponding proof/qualification obligation on that implementation.

Lemma I.1 (Bounded ε-chain to Σ-action or stable wait).
Assume the finite-pipeline-depth / finite-internal-stutter premise FPD/B4. From any reachable well-formed cycle-boundary state of P in the Appendix I construction, if P is not already stably waiting on an external/peer enabling condition, then within at most D_P clock cycles of P-local ε-progress, P either:
(i) commits its next observable Σ-action, or
(ii) reaches a stable waiting state in which no further P-local ε-step is available until some external, peer, channel, or stimulus condition changes the relevant enabling predicates.
Equivalently, P cannot produce an infinite P-local ε-only execution segment from a reachable well-formed state while remaining internally enabled. Any unbounded delay that remains after the stable-wait point is therefore attributable to external/channel/stimulus enabling and is governed by the WF/B2 and B3 progress premises, not by local ε-stutter.
Proof. Direct from FPD/B4, applied only to reachable well-formed states of P in the Appendix I construction. ■

Lemma I.2 (Eventual space / data). Assume P is blocked on either:
• Push_c with occ_c = B(c), P's Push_c request remains asserted with stable payload for the outstanding transfer, and the complementary endpoint continuously requests the matching Pop_c until the matching discharge event (Pop_c endpoint request asserted; Pop_c boundary request remains true); or
• Pop_c with occ_c = 0, P's Pop_c request remains asserted for the outstanding transfer, and the complementary endpoint continuously requests the matching Push_c until the matching discharge

event (Push_c endpoint request asserted with stable payload; Push_c boundary request remains true). Then a complementary Pop_c (respectively Push_c) commits within finite time.

Proof. (By B3 / Appendix N.) In either case, P is stalled at a blocking channel call while its persistent request for that outstanding transfer remains asserted until commit, with stable payload if it is a Push. By the additional premise, the complementary endpoint also continuously requests the matching operation until the corresponding discharge event, with its boundary request remaining true. Therefore the interval-scoped premises of P_push/P_pop (Appendix N) hold, and the corresponding transfer must eventually complete: for Push_c at full, some complementary Pop_c eventually commits, freeing space; for Pop_c at empty, some complementary Push_c eventually commits, providing data. The premise does not require either endpoint request or Push payload to remain asserted after that matching transfer has committed.
■

Definition I.Obs (Σ-relevant source-state slice).
Fix a process P, a fixed external stimulus, and a fixed witness w satisfying WC for the chosen observable boundary. Define Obs_{Σ,P,w}(σ) to be the tuple of P-local source-level facts evaluated at the ε-quiescent source state σ that are relevant to future Σ-visible behavior or to the Appendix I–K proof predicates owned by P. This tuple includes: the source control position; source variables and latched values used to evaluate future control predicates Pred_x, payload/value expressions, anchored Write values, and anchored Read labels under the fixed-stimulus interpretation; witness-selected ε outcome information that can affect later Σ-visible control flow, frontier membership, or request attribution; Q_exec,P / Q_Ω,P attribution facts needed for E3/E5 or WFG reasoning; Pending_P, NextFront_P, and Front_P membership; and BoundaryReq values for message-operation instances owned by P. Define σ $\sim_{\{\Sigma,P,w\}}$ σ′ *iff Obs*{Σ,P,w}(σ) = Obs_{Σ,P,w}(σ′). The Σ-relevant source-state slice of σ is its equivalence class [σ]_{Σ,P,w}.
This quotient intentionally ignores purely internal diagnostic state and ε-only micro-timing state that cannot affect any future Σ-visible label, frontier predicate, pending/request-attribution fact, payload/value expression, BoundaryReq predicate, or Appendix K blocking/WFG-facing predicate. ChannelEnabled / ChannelDisabled facts are not independently added to this P-local quotient; they are obtained by combining the P-local slice, including BoundaryReq and pending/frontier facts, with the corresponding channel-state facts in the chosen Sys_ref boundary model, such as B(c), occ_c(σ), rendezvous complementary endpoint requests, and any explicit Sys_B^elab staged/bundle/join boundary state.
For cadence-sensitive non-blocking polling sites, such as retry/timeout counters whose externally visible behavior depends on the number or cadence of failed polls, this definition is applied only after the site has been handled as an isolated or snoop-aligned latency-sensitive local protocol block under Appendix D / Appendix L. The ordinary Appendix I source-core proof does not derive retry/timeout behavior from an unconstrained hidden failed-poll cadence.

Lemma I.Obs-Congruence (one-step closure of the Σ-relevant slice).
Fix P, the external stimulus, the chosen observable boundary, and a witness w satisfying WC. Let σ_1 and σ_2 be ε-quiescent source states of P such that Obs_{Σ,P,w}(σ_1) = Obs_{Σ,P,w}(σ_2). Suppose that, from σ_1 and σ_2, the same next P-local Σ label ℓ is produced under the same witness w and the same fixed-stimulus interpretation, where ℓ includes any committed Push/Pop value, successful non-blocking transfer value, synchronization/wait label, anchored Read value, anchored Write value,

START_P/FINISH_P event, or other Σ-visible boundary event. Let σ_1' and σ_2' be the resulting ε-quiescent cutpoint states after that same-label step and its following finite ε-closure. Then:
$Obs_{\Sigma,P,w}(\sigma_1') = Obs_{\Sigma,P,w}(\sigma_2')$.
Proof. By case analysis on the next source construct and on the kind of label ℓ.
For deterministic local computation, equality of the control position and of all variables/latched values feeding future predicates, payloads, anchored writes/reads, guards, and proof predicates implies equality of the updated members of Obs. Updates to variables or diagnostic state not included in Obs cannot affect any later Σ-visible label or Appendix I–K proof predicate by the definition of Obs and the source well-formedness obligations.
For anchored signal reads and writes, the relevant read/write value is either fixed by the common stimulus and anchor or is the common label ℓ; therefore all future-relevant latched values and anchored-label facts agree after the step.
For blocking Push/Pop and synchronization operations, the common label ℓ, together with equality of the current control position, relevant payload variables, Pending_P, NextFront_P, Front_P, Q_exec,P / Q_Ω,P attribution facts, and BoundaryReq facts, determines the same update to all P-owned frontier, pending, request-attribution, and BoundaryReq components of Obs. ChannelEnabled / ChannelDisabled is not an independent P-local component; any needed enablement fact is obtained from the common P-local request/frontier facts together with the chosen Sys_ref boundary state and E4.
For successful non-blocking PushNB/PopNB, the successful transfer is included in ℓ as the corresponding committed Push_c(v) / Pop_c(v) event. The common label and equal current slice determine the same control, data, attribution, frontier, pending, and BoundaryReq updates.
For failed non-blocking PushNB/PopNB, the failed poll is an ε-step. If NB-CF holds, failed polls cannot affect the next Σ-visible control flow, frontier membership, request attribution, guard values, or payload values, so any difference is outside Obs. If NB-CF does not hold but WC is supplied by Appendix L, the failed-poll outcome is fixed by the witness as part of selecting an admissible source execution at the chosen boundary. If the process computes externally visible behavior from the number or cadence of failed polls, the site must already have been isolated or snoop-aligned as a latency-sensitive local protocol block under Appendix D / Appendix L; otherwise the ordinary Appendix I source-core theorem is not applicable to that process at that boundary.
For any other ε-only implementation choice, WC requires that every outcome capable of affecting later Σ-visible behavior or Appendix I–K proof predicates is selected by w and is causally consistent with the matched prefix. Thus equal current Obs and the same witness-selected outcome produce equal successor Obs. ε-only state not included in Obs is, by the preceding cases and by the construction of Obs, inert with respect to all future labels and proof predicates owned by P.
Therefore equality of Obs is preserved by one same-label cutpoint-to-cutpoint step. ■

Lemma I.DR (Deterministic replay / Σ-relevant source-state determinacy under fixed stimulus + witness). Fix a process P, a fixed external stimulus, and a fixed witness w satisfying WC for the chosen observable boundary. Consider two executions of the same P source code that start from the same initial source-level state and are run with that same witness. If their chosen P-local observable-unit replay projections are the same, then for every k, the Σ-relevant source-state slice of P at the ε-quiescent cutpoint after the first k P-local observable units is identical in the two executions. Here “P-local observable-unit replay projection” means the sequence obtained from the document’s event-matching / Appendix J observable-unit convention: explicitly atomic same-cycle transactions are grouped as single units; independent same-cycle bucket members are not given an intrinsic order by the bucket itself; and any ordering used for deterministic replay is the selected witness/replay order, together with w and μ_Q for ε-modeled failed non-blocking $q \in Q_{exec,P}$ when those instances are needed by E3/E5. Thus Lemma I.DR is not a determinacy theorem from a bare unordered Σ-bucket projection alone.

Proof. At k = 0 the executions start from the same initial source-level state, so their Obs_{Σ,P,w} slices agree. For the induction step, assume the slices agree after k P-local Σ-events. The two executions have the same (k+1)-st P-local Σ label and are run under the same fixed stimulus and witness. By Lemma I.Obs-Congruence, the ε-quiescent cutpoint states after that label have equal Obs_{Σ,P,w} slices. Therefore, by induction on k, the Σ-relevant source-state slice agrees at every ε-quiescent cutpoint after the first k P-local Σ-events. ■

Lemma I.DR′ (Immediate post-unit deterministic replay under fixed stimulus + witness). Fix a process P, a fixed external stimulus, and a fixed witness w satisfying WC for the chosen observable boundary. Consider two executions of the same P source code that start from the same initial source-level state and are run with that same witness. Suppose that their chosen P-local observable-unit replay projections through the first k units are the same. Stop each execution at the canonical immediate post-unit semantic state: after the transition, or explicitly atomic transition group, that realizes the kth P-local observable unit, and before any optional following maximal ε-normalization. Then the two executions agree on the P-owned source-level replay state at that point, including:

• the source control position;
• all source variables and latched values that can influence future Pred_x predicates, payload/value expressions, or anchored Read/Write values;
• the dynamic-operation and request-attribution facts determined by the replay prefix; and
• the Pending_P, NextFront_P, Front_P, and BoundaryReq facts at that exact semantic point.

This lemma does not assert that the immediate post-unit state is ε-quiescent or drain-closed, and it does not identify that state with a later ε-normalized cutpoint. Lemma I.DR remains the corresponding theorem for explicitly ε-normalized cutpoints.

Proof. By induction on k. The base case follows from equality of the initial source-level state. For the induction step, equality of the prior immediate replay state, together with the same fixed stimulus, witness-selected ε outcomes, dynamic-instance attribution, and next observable unit, determines the same finite source transition segment through the transition or explicitly atomic transition group that realizes the next unit. Therefore the two canonical immediate post-unit states agree on every listed P-owned replay fact. No maximal ε-closure is invoked. ■

I.4 Inductive Construction for Finite Traces (Post → Ref)

Let

τ_post = τ_post[0..n–1] ∘ e (where |τ_post| = n + 1)

be the next post-HLS prefix of RTL_B for process P (under the fixed initial state and fixed stimulus). Assume by induction that we already have a matching Sys_ref prefix τ_ref[0..n–1] satisfying E1–E5 with τ_post[0..n–1]. We extend τ_ref by a finite ε-chain (written ε*) followed by an observable action e′ so that

τ_ref ∘ ε* ∘ e′ matches τ_post.

Here Sys_ref is the prescribed source reference model for this process/boundary, as fixed in Appendix J, Remark 2.

Non-blocking note. For a non-blocking PopNB/PushNB call, unsuccessful polls are treated as ε-steps. This is sound for Post→B construction under the witness-compatible (WC) premise (I.2): ε-modeled poll outcomes are either (a) irrelevant to the next Σ-visible control flow/frontier (NB-CF holds), or (b) fixed/selected consistently with τ_post (e.g., via Appendix L), so they do not introduce an unmatched Σ-visible control-flow/frontier divergence. Each executed PopNB/PushNB call still constitutes its own dynamic source-level message-call instance q ∈ Q_exec,P for the owning process P; WC/NB-CF fixes the ε-modeled outcomes of those calls, but it does not identify multiple executed non-blocking call instances as one q merely because the endpoint request stays asserted across cycles. The matching of

failed non-blocking q instances, when needed by E3/E5 or by the witness construction, is part of the chosen μ_Q witness and is not reconstructed from Σ. A successful non-blocking call may also be represented in Q_Ω,P when its committed transfer contributes a frontier/observable item; a failed non-blocking poll remains in Q_exec,P \ Ω_P.

Lemma I.3 (Finite ε*-extension step for I.4).

Let τ_post = τ_post[0..n–1] ∘ e (where |τ_post| = n + 1) be the next post-HLS prefix of RTL_B for process P (under the fixed initial state and fixed stimulus). Assume by induction that we already have a matching Sys_ref prefix τ_ref[0..n–1] satisfying E1–E5 with τ_post[0..n–1]. Also assume the global witness/channel-history invariant used by the Appendix I/J construction: the matched prefix fixes the same Σ-causal history, the same selected ε-only witness choices covered by WC, the same relevant channel histories/occupancies at the chosen Sys_ref abstract boundaries, and, for any message transfer whose matching RTL_B event is e, the necessary complementary endpoint request or concrete finite complementary-action sequence is supplied by the same matched global execution/witness. Also assume the finite selected-unit realization condition for e: after the finite local replay recorded by W, either the complete matching unit e' is enabled and may be chosen by the Sys_ref scheduler/arbiter in the next legal step, or W supplies a concrete finite legal sequence of matched complementary actions after which the complete unit is enabled; atomic counterpart availability is included in this condition. In the execution-level Theorem J.1 use, this is the per-step finite-extension clause of J.GWR. Then there exists a Sys_ref extension by a finite ε-chain ε* followed by an observable action e' such that:
(i) τ_ref ∘ ε* ∘ e' matches τ_post under the document's Σ-event matching convention; and
(ii) the extended prefix continues to satisfy E1–E5 (in particular, E3's message-operation issue-order clause holds by Implementation assumption I.A0).

Proof. By cases on the kind of Σ-event e. The finite selected-unit realization premise supplies a concrete finite Sys_ref transition segment for the selected complete unit. For Sync/wait and anchored signal I/O events, replay-state agreement and the fixed stimulus justify the recorded peer/environment enabling and anchored values. For message-passing events, the matched channel history, endpoint requests, and any recorded finite complementary-action sequence justify E4 legality at the chosen Sys_ref abstract boundary.

Choose the finite legal complementary-action sequence recorded by W, if any. Once the complete matching unit is enabled, the existential Sys_ref transition relation permits the scheduler/arbiter choice that selects and commits it; no fairness premise is needed to prove existence of this finite path. The selected witness supplies all required endpoint co-presence, E4 channel legality, finite internal replay, and atomic grouping. Hence ε* is finite under the lemma's witness and operational-semantic premises. ■

Case note (what differs across event kinds). For Sync/wait and anchored signal I/O events, the finite ε* step is a witness-replay argument, not an independent global-liveness claim. The lemma conditions on the already chosen next RTL_B event e and, via the fixed-stimulus interpretation above, replays the same peer/environment enabling from the same global Σ-causal history in the Sys_ref witness construction. Thus Lemma I.3 does not assume that peer/environment progress follows from P-local history alone, and it does not preempt the system-level liveness/deadlock arguments of Appendices J and K. For message-passing events, the corresponding channel availability is read from the matched global channel history and the finite selected-unit realization component of J.GWR, including any concrete finite complementary-action sequence and persistent counterpart request needed for a rendezvous or bounded-capacity transfer. B2/B3 are not used by this finite-prefix lemma. Thus Lemma I.3 is a conditional finite-extension step inside the chosen Appendix I/J witness construction, not a

standalone proof that P-local progress alone forces the next global message transfer. In the execution-level use of this lemma by Theorem J.1, the "matched global witness/channel-history invariant" and finite selected-unit realization premises are supplied by GlobalWitnessRealizable(τ_post, W) (Definition J.GWR); they are explicit implementation-side premises, not facts derived from E1–E5.

I.5 Extension to Infinite Traces (Post → Ref)

Fix a process P and a fixed external stimulus/initial state. (For reactive environments, interpret "fixed stimulus" per Appendix I.2, "Fixed-stimulus interpretation (reactive environments).")
Let ρ_post be any RTL_B execution under this stimulus that is within the intended Appendix J / Theorem J.1 scope (i.e., an execution whose finite prefixes lie in Exec_post; equivalently: an execution that is a fair/progress-satisfying infinite run, using the idle-stutter/time-divergence convention B6 when no further Σ-progress is enabled). Let τ_post,P be the Σ-observable trace of P induced by ρ_post.
Finite-P-local-Σ-event case. If τ_post,P contains only finitely many Σ-events $e_1 \dots e_n$ (that is, after e_n no further Σ-event of P occurs), apply the finite-prefix construction in I.4 to obtain a Sys_ref prefix τ_ref,P[0..n) whose Σ-projection matches $e_1 \dots e_n$ and whose correspondence satisfies E1–E5. The absence of further P-local events does not by itself justify an application of B6: other processes, the environment, already-issued work, or other observable actions may continue even though P emits no further Σ-event. There are therefore two cases. If the matched global Sys_ref execution reaches a B5/global fixed point after this prefix, B6 supplies a time-divergent idle suffix, and projection onto P yields only empty P-local buckets after e_n. Otherwise, the infinite P-local trace is obtained by projecting an admissible infinite global Sys_ref continuation under the same fixed stimulus; global cycles in which P emits no Σ-event again contribute only empty P-local buckets/stutter to P's projection. The existence and composition of that global continuation are supplied by Appendix J's Exec_ref witness construction, not by an arbitrary idle extension of the P-local prefix. In either case, the resulting infinite P-local projection has exactly the matched finite Σ-prefix $e_1 \dots e_n$ and no later P-local Σ-events.
Countably-infinite local-Σ-event case. Otherwise, τ_post,P has countably many P-local Σ-events. We construct an infinite Sys_ref trace τ_ref,P by an explicit inductive limit of the finite-prefix construction in I.4.
This paragraph is a per-process construction only. Its event index n ranges over the P-local Σ-events of a single process trace, after applying the document's per-process Σ-event matching convention. It is not the system-level prefix index used by Theorem J.1, where prefixes advance by required-prefix observable units under ≺_J rather than by requiring preservation of the RTL_B cycle-bucket decomposition.
Let τ_ref,P[0..0) be the empty local prefix. For each n ≥ 0, assume we have constructed a finite Sys_ref local prefix τ_ref,P[0..n) such that its P-local Σ-projection matches the first n observable P-local Σ-events of τ_post,P and the correspondence satisfies the applicable per-process parts of E1–E5. Let e_n be the (n+1)-st P-local Σ-event of τ_post,P. Apply the I.4 construction step to extend τ_ref,P[0..n) by a finite ε* segment followed by a matching observable event e'_n, obtaining τ_ref,P[0..n+1) while preserving the per-process E1–E5 obligations.
Because each extension step adds a finite segment, the increasing chain of local prefixes defines a countably infinite Sys_ref process trace τ_ref,P whose every finite local event prefix matches the corresponding local event prefix of τ_post,P. Therefore, τ_ref,P ≈ τ_post,P at the per-process trace level. When these per-process witnesses are composed in Appendix J, the system-level proof schedules the matched local events into some legal Sys_ref execution satisfying E1–E5, the chosen channel/synchronization semantics, and the explicitly atomic same-cycle obligations. The resulting Sys_ref execution need not use the same observable cycle-bucket decomposition as the RTL_B execution: independent events may be placed in different cycles or grouped differently whenever no E1–E5 rule, channel rule, signal-anchor rule, or explicit synchronization transaction fixes their relative

timing.
Scope clarification: this inductive-limit step establishes the per-process Σ-trace witness only (local prefix/trace matching under E1–E5); it does not by itself discharge global fairness/progress obligations (B2/B3) or prove that an arbitrary global Sys_ref continuation is fair/progress-satisfying. Those execution-level existence/scope obligations are handled in Appendix J / Theorem J.1 via the Exec_post / Exec_ref definitions and qualifiers. ■

Appendix J – System-Level Trace Compatibility Proof

Synchronization-pair well-formedness for ≈.
For SyncChannel/ac_sync operations with paired endpoints, the compared Sys_ref and RTL_B executions use the same dynamic synchronization-transaction pairing. That is, the two endpoint events of a matched sync_out/sync_in transaction are matched by dynamic sync-instance identity, commit as one two-party barrier transaction, and are treated as simultaneous events in the same observable cycle bucket. This synchronization transaction induces the same cross-process before/after ordering as described in the causal-dependency definition.
Equivalently, E1 preserves the per-process source order of synchronization endpoint events, but E1 does not weaken or replace the paired-barrier semantics of SyncChannel/ac_sync. The system-level trace-compatibility predicate $\tau_{ref,system} \approx \tau_{post,system}$ is defined only over synchronization-well-formed traces in this sense: if one endpoint of a dynamic SyncChannel/ac_sync transaction appears in the trace, then the matching endpoint appears in the same observable synchronization transaction, with the same dynamic pairing, in both Sys_ref and RTL_B.

Theorem J.1 (Execution-Level / Trace-Set Compositional Compatibility)

Fix an initial state and a fixed external stimulus/environment behavior (e.g., the same testbench-driven inputs) for both the prescribed source reference model Sys_ref and RTL_B, with Sys_ref as defined in Remark 2. The raw rendezvous model Sys is the special case obtained when Sys_ref = Sys_B and all effective capacities are zero.

Clarification (reactive environments). Interpret “fixed stimulus” per Appendix I.2, “Fixed-stimulus interpretation (reactive environments).”

Let Exec_post denote the set of finite execution prefixes of RTL_B that are reachable under this stimulus and that are prefixes of at least one (infinite) execution under the same stimulus that satisfies the Appendix J fairness/progress premises; and let Exec_ref denote the corresponding set of finite execution prefixes of Sys_ref (defined analogously).
Definition J.Reach (finite reachability sets Reach_post / Reach_ref). Let Reach_post denote the set of all finite execution prefixes of RTL_B reachable under the fixed stimulus, with no fair-extension qualifier; define Reach_ref analogously for Sys_ref. By construction Exec_post ⊆ Reach_post and Exec_ref ⊆ Reach_ref. The finite-prefix safety results of this appendix — the Post → Ref witness construction of Theorem J.1 and the cutpoint correspondence of Corollary J.1 — are stated over Reach_post / Reach_ref: their finite-ideal inductions never use the existence of a fair infinite extension, so a genuinely bad reachable prefix (for example, one ending at a deadlock cutpoint) is in scope for the state-transfer results regardless of whether a fair infinite extension exists. Exec_post / Exec_ref are used only where

an infinite fair execution is itself asserted (Appendix I.5, the B6 convention, and the embedding step of Theorem K.1); Lemma J.FE below relates the two sets.
Lemma J.FE (fair extension / machine closure). Premise (environment receptiveness): the fixed stimulus, interpreted per Appendix I.2 for reactive environments, is defined on every reachable Σ-causal history — the environment/testbench never becomes undefined or refuses to continue after a reachable finite prefix. Claim: under B2–B6 and this premise, every τ ∈ Reach_post (respectively Reach_ref) is a prefix of at least one infinite execution under the same stimulus satisfying the B2/B3 fairness/progress premises; hence Reach_post = Exec_post and Reach_ref = Exec_ref. Proof sketch: construct the continuation greedily. At each reached state, if any Σ-action or ε-step is enabled, select among the enabled actions by a fair discipline (for example, oldest-continuously-enabled first); B4 bounds consecutive ε-only work, so selection returns to Σ-eligible actions after finitely many steps, and no action remains continuously enabled without eventually being selected, so B2 holds. B3's interval-scoped channel-progress obligations hold on this schedule because their antecedents keep the relevant transfer continuously ChannelEnabled, and the fair discipline eventually selects it. If no action of any kind is enabled, the state is by definition a B5 global fixed point, and B6 supplies the time-divergent idle-stutter suffix, on which B2/B3 hold vacuously. The construction never blocks: at every reachable state either some action is enabled or B6 applies. ■ Status: J.FE is a named lemma with an explicit environment-receptiveness premise; that premise is an entry in the side-condition discharge record. Flows whose testbench can refuse further input after some reachable history must either restrict Reach_post/Reach_ref accordingly or discharge the required embedding directly.

Cycle-bucket and required-prefix convention for this theorem. A system execution may be represented as a sequence of observable cycle buckets $\beta_1\ \beta_2\ \ldots$, where β_i is the finite set of Σ-events committed in one observable clock cycle of that particular execution. Same-cycle events inside a bucket are unordered except for the explicit constraints imposed by E1–E5, the channel semantics, signal-anchor semantics, and explicit synchronization constructs.

The bucket decomposition records the clock timing of one execution; it is not, by itself, part of the cross-model compatibility obligation. The relation τ_ref,system ≈ τ_post,system does not require Sys_ref and RTL_B to use the same observable cycle-bucket decomposition. Events that are not ordered by the explicit required constraints may be placed in different cycles or grouped into different buckets in the two executions, provided the matched executions preserve all ordering, timestamp, FIFO-legality, and atomicity constraints required by E1–E5 and by the explicitly modeled channel, signal, and synchronization semantics.

The prefix order used by Theorem J.1 is required-prefix order over matched observable units, not RTL_B bucket-prefix order and not the full Appendix G causal-dependency graph →. An observable unit is either:
• a singleton Σ-event; or
• an explicitly atomic same-cycle transaction that the semantics requires not to be split, including a rendezvous Push_c(v)/Pop_c(v) transfer, the two endpoints of a paired SyncChannel/ac_sync transaction, an R2C combinational fixed-point signal-observation unit, a proof-internal RESET boundary unit when dynamic reset is used (Common Formal Definitions, reset well-formedness RW1–RW5), and any other same-cycle transaction explicitly identified by the signal/channel semantics.
E2 signal-anchor grouping convention: a synchronization commit together with the sequential-process signal Read/Write events that E2/R2 anchor to it (and therefore require to share its commit cycle) may — and, for mechanization, should — be grouped as one explicitly atomic same-cycle unit. This grouping replaces the same-cycle equality constraint between separately enumerated units by unit-internal

atomicity, and is consistent with Corollary J.1(2a), which advances a synchronization-call item together with its anchored signal action items. When anchored signal events are instead enumerated as singleton units, their E2 timestamp equality with the anchor is enforced by the Sys_ref realization semantics (R2) and the within-cycle listing convention, not by the strict order $\prec_J$, which never orders same-cycle-aligned events.
Let $\prec_J$ be the strict required-order relation over observable units generated only by the explicit constraints that must be preserved by $\tau_{ref,system} \approx \tau_{post,system}$:
• E1 synchronization source-order constraints within each process;
• E2 signal-anchor timestamp constraints, interpreted as required alignment with the relevant synchronization or R2C fixed-point unit rather than as raw ordering among all source actions;
• E4 channel/FIFO legality for explicit abstract channels, including same-channel FIFO order for B(c)>0 and atomic rendezvous grouping for B(c)=0;
• E5 immediate synchronization-boundary constraints;
• paired SyncChannel/ac_sync barrier semantics; and
• any other explicitly stated signal/channel/synchronization atomicity constraint.

Per-process source order contributes to $\prec_J$ only where it is reflected in one of these explicit required constraints, such as synchronization order, same-endpoint/same-channel order, signal anchoring, or immediate synchronization-boundary legality. E3 contributes issue-order and request-attribution obligations, but it does not by itself create a strict predecessor edge between distinct-interface committed observable units merely because their source calls are lexically ordered. The Appendix G causal-dependency relation $\rightarrow$ is not a generator of $\prec_J$.

A theorem prefix is a finite ideal of observable units under $\prec_J$, i.e. a finite down-closed set H such that $\gamma \in H$ and $\gamma' \prec_J \gamma$ imply $\gamma' \in H$. The proof below proceeds by induction along a fixed RTL-time chain of such finite ideals, chosen from the RTL_B witness prefix as described below. Acyclicity of $\prec_J$ on the fixed execution prefix. For the chosen legal RTL_B execution prefix, assign each observable unit $\gamma \in U(\tau_{post})$ its RTL_B commit cycle $clk(\gamma)$. If γ is an explicitly atomic same-cycle unit, such as a rendezvous Push_c(v)/Pop_c(v) transfer or a paired SyncChannel/ac_sync transaction, $clk(\gamma)$ is the common commit cycle of the endpoints in that atomic unit.

Every strict generator edge $\gamma_1 \prec_J \gamma_2$ implies a strict cycle inequality $clk(\gamma_1) < clk(\gamma_2)$ in that same legal execution prefix:
• E1 synchronization source-order constraints impose strict ordering between the relevant synchronization commits.
• E4 buffered-channel FIFO/order/capacity constraints impose strict commit-cycle ordering through reset-empty FIFO order, bounded occupancy, and non-fall-through cycle-boundary availability. In particular, for B(c)>0, a Pop_c can consume only a value already present at the relevant cycle boundary, and a Push_c that depends on space becoming available after a full condition is enabled only in a later cycle after the required Pop_c effect has occurred.
• E5 immediate synchronization-boundary constraints impose strict separation between a synchronization commit and later suff_S work whose issue/commit is after that synchronization boundary.
• Paired synchronization/barrier before/after constraints impose the corresponding strict before/after cycle relation between distinct observable units when such a relation is explicitly part of the synchronization semantics.
• E2 signal-anchor constraints contribute timestamp equality/alignment to the relevant synchronization or R2C fixed-point unit, or same-cycle atomicity when explicitly specified; they do not by themselves

introduce an independent strict edge between distinct observable units.
• Rendezvous Push/Pop endpoints, paired same-cycle synchronization endpoints, and other explicitly atomic same-cycle transactions are represented as single observable units, not as strict internal edges.
• Non-strict issue simultaneity permitted by E3 does not create a strict $\prec_J$ edge.

Therefore any cycle $\gamma_0 \prec_J \gamma_1 \prec_J \dots \prec_J \gamma_0$ over $U(\tau\_post)$ would imply $clk(\gamma_0) < clk(\gamma_1) < \dots < clk(\gamma_0)$, an impossibility. Hence $\prec_J$ is acyclic on $U(\tau\_post)$, which is the only acyclicity needed for the finite-ideal induction. For the proof of Theorem J.1, we use a particular topological enumeration that refines the chosen RTL_B witness's commit-cycle / issue-attribution order. Its prefixes are ideals under $\prec_J$, but the enumeration itself has no semantic significance and must not be read as adding an observable order among independent events. There is no semantic cutpoint after only one endpoint of an explicitly atomic multi-endpoint unit has been taken; such units are split neither in Sys_ref nor in RTL_B.

Scope note (intentional). The finite-prefix Post → Ref clause of Theorem J.1 and the cutpoint correspondence of Corollary J.1 are stated over Reach_post / Reach_ref and do not require an infinite fair extension. Their finite-ideal construction uses the per-step finite selected-unit realization supplied by J.GWR and the Sys_ref operational transition semantics. Weak fairness B2, channel progress B3, and the time-divergence / idle-stutter convention B6 are used only when an infinite fair execution is itself asserted, such as Appendix I.5, Lemma J.FE, or a later embedding into Exec_post / Exec_ref. B4/B5 remain separate normalization premises when ε-quiescent or fixed-point cutpoints are selected.

Assume the Appendix I / Appendix L witness machinery is available as follows—namely: the per-process Post → Ref witness construction from Appendix I, including the witness-compatibility premise WC for ε-modeled non-blocking polls (and any other ε-only internal choices that can affect the next Σ-visible control flow/frontier) as stated in Appendix I.2 and enforceable via Appendix L's snooping / witness-selection technique; with NB-CF as a sufficient condition that makes the witness trivial—together with the capacities/occupancies/enablement semantics of Sys_ref, finite capacities on every explicit abstract channel of Sys_ref, the per-step finite selected-unit realization clause of J.GWR, and the standing RTL_B determinism assumption B1 under the fixed stimulus. No B2/B3 premise is used by the finite-prefix clause. Use B4/B5 additionally when ε-normalization or uniqueness up to finite ε-stutter is invoked. When an infinite fair execution is asserted, separately require B2/B3 and the J.FE environment-receptiveness premise. Additionally assume the shared-memory lowering condition J.Mem for every cross-process shared-memory dependency in the compared design and, when dynamic reset is used, the reset well-formedness conditions RW1–RW5 (both in Common Formal Definitions). Then:

Compatibility-relation typing convention. The symbol ≈ is an ordered cross-model compatibility predicate, not a mathematical equivalence relation. It is intentionally directed from a Sys_ref execution/projection to an RTL_B execution/projection, and no symmetry or transitivity property is implied. Its carrier is not a bare Σ-label trace alone.
For each process P, the per-process carrier is an annotated observable projection record. Such a record contains the P-local bucketed Σ projection, the relevant observable-unit projection, the dynamic source/witness universe Ω_P and Q_exec,P used for that comparison, the Issue/Commit annotation for dynamic message-call instances, the selected witness w, and the μ_Q matching supplied by WC / Appendix L when ε-modeled failed non-blocking calls or other witness-selected choices are relevant. Forgetting these annotations yields the ordinary P-local observable Σ projection, but E3, E5, BoundaryReq attribution, and the witness construction may depend on the annotations and are not reconstructed from the Σ projection alone.
Accordingly, write

≈_P ⊆ AnnTraces_{Σ,P,w}(Sys_ref) × AnnTraces_{Σ,P,w}(RTL_B)
for the per-process annotated observable-compatibility predicate induced by the P-local R/E obligations: synchronization ordering, signal anchoring or R2C fixed-point agreement, message issue-order constraints, synchronization-boundary issue constraints, and the P-local witness/request-attribution facts needed for those obligations. Here AnnTraces_{Σ,P,w}(·) denotes the annotated projection records just described, not merely Traces_{Σ,P}(·).

Similarly, write

≈_sys ⊆ AnnTraces_{Σ,w}(Sys_ref) × AnnTraces_{Σ,w}(RTL_B)
for system-level annotated observable compatibility. A system-level annotated record includes the per-process annotated projections plus the explicit abstract-channel histories and boundary-state facts needed to check E4 at the chosen Sys_ref / RTL_B boundaries.

For system executions τ_ref,system and τ_post,system, the assertion τ_ref,system ≈_sys τ_post,system is therefore an abbreviation: there exist compatible annotation/witness records for the two executions such that every process projection satisfies the corresponding τ_ref,P ≈_P τ_post,P obligation and every explicit abstract channel in the chosen Sys_ref / RTL_B comparison satisfies the applicable E4 channel semantics at the agreed abstract boundary. When the annotations are clear from context, the same τ_ref ≈ τ_post notation may be used for readability; it should not be read as typing ≈ over bare Σ-traces alone.

Definition J.RTLConsistentRefPrefix (annotated RTL-consistent source prefix). A source-side prefix τ_ref ∈ Exec_ref is RTL-consistent for the Ref → Post clause only when it is equipped with a selected RTL_B prefix τ_post^* ∈ Exec_post and a witness/annotation package W such that: (1) τ_post^* is admitted by RTL_B under the same fixed stimulus; (2) the observable-unit projection of τ_ref matches a required-prefix projection of τ_post^*; (3) W provides compatible annotated projection records for τ_ref and τ_post^*, including the dynamic operation universes Ω_P and Q_exec,P, μ_Q matching, the operational Issue/Commit and E3/E5 annotations needed by the execution-level construction, request-attribution facts (including failed non-blocking poll witnesses where relevant), and the applicable per-channel E4 histories and abstract boundary facts; and (4) whenever Corollary J.1, Appendix K, or any cutpoint transfer uses the selected pair, the same selected witness and normalized source prefix satisfy J.OfferReach, the selected RTL cutpoint satisfies J.OfferConf, and the composite J.KObsConf relation holds for the same H and O. Label-projection matching alone is therefore not sufficient to establish Ref → Post; the compatible annotations and, when needed, the KObs cutpoint package are part of the RTLConsistentRefPrefix witness.

Then:

(Post → Ref existence) For every τ_post ∈ Reach_post together with a selected RTL_B witness/annotation package W satisfying GlobalWitnessRealizable(τ_post, W) (Definition J.GWR), there exists τ_ref ∈ Reach_ref and a matching Sys_ref witness/annotation package such that the annotated system projections satisfy τ_ref,system ≈_sys τ_post,system under E1–E5. Membership of τ_post in Exec_post (the fair-extendable subset of Reach_post) is not required for this finite-prefix clause; it is needed only where an infinite fair execution is asserted (Definition J.Reach, Lemma J.FE).

(Ref → Post existence, annotated RTL-consistent prefixes only) For every τ_ref ∈ Exec_ref for which there exist τ_post^* ∈ Exec_post and a witness/annotation package W satisfying RTLConsistentRefPrefix(τ_ref, τ_post^*, W) under the same fixed stimulus, there exists τ_post ∈ Exec_post—namely τ_post = τ_post^*—with the compatible witness/annotation package W such that the annotated system projections satisfy τ_ref,system ≈_sys τ_post,system. In Appendix L

snooping/witness-selection applications, the selected τ_post^* and W are the RTL_B execution prefix and witness package selected by the snooping wrapper/witness.

Moreover, because B1 is a standing assumption in this theorem instance (deterministic Σ-projected RTL_B trace under fixed stimulus), the matching observable Σ projection of a selected RTL_B prefix is unique (up to insertion/removal of finite ε-steps permitted by B4/B5). This uniqueness is only a uniqueness statement about the observable projection; it does not by itself determine μ_Q, failed-poll instances in Q_exec, issue annotations, request-attribution facts, the selected package W, or the full stateful Sys_ref witness. The corresponding execution prefix witness τ_ref need not be unique as a stateful prefix, since Sys_ref may admit multiple ε-different realizations with the same Σ projection. Sys_ref may also admit executions whose Σ projection does not match RTL_B's Σ trace when latency-sensitive / non-blocking effects are Σ-visible; such executions are outside the scope of (Ref → Post) unless constrained by RTLConsistentRefPrefix / the snooping/witness-selection technique (Appendix L).
Proof strategy. Theorem J.1 is not obtained from pairwise composition alone. Proposition J.0 below is only the conditional lifting step for a chosen compatible pair of system traces. The actual execution-level theorem is obtained by combining that conditional lifting step with the Appendix I / Appendix L witness machinery and the Exec_post / Exec_ref scope conditions, as proved later in this appendix under "Proof of Theorem J.1 (expanded execution-level witness construction)."

Proposition J.0 (Pairwise Composition Lemma)

Fix any test stimulus (environment inputs) and initial state. Let τ_ref,system ∈ Traces_Σ(Sys_ref) be an observable system trace of the prescribed source reference model Sys_ref under that stimulus, and let τ_post,system ∈ Traces_Σ(RTL_B) be an observable system trace of RTL_B under the same stimulus.
Definition J.GWR (GlobalWitnessRealizable(τ_post, W)). A selected RTL_B witness/annotation package W for a finite RTL_B prefix τ_post is globally witness-realizable, written GlobalWitnessRealizable(τ_post, W), when W supplies one coherent global package consisting of: (i) the WC-selected ε outcomes and the μ_Q matching for every process; (ii) operational Issue/Commit and request-attribution annotations for every dynamic message-call instance, consistent with Appendix C and Implementation assumption I.A0; (iii) the per-channel E4 histories and abstract boundary facts for every explicit abstract channel of the chosen Sys_ref; (iv) the explicitly atomic same-cycle groupings — rendezvous Push_c(v)/Pop_c(v) transfers, paired SyncChannel/ac_sync transactions with matched dynamic synchronization-transaction identities, and R2C fixed-point observation units — with matched dynamic-instance identities; and (v) the realizable RTL-time enumeration property: there exists an enumeration $\gamma_1 \dots \gamma_m$ of the observable units of τ_post refining RTL commit-cycle order such that, for every k and every process P participating in γ_{k+1}, the deterministic source-level replay state determined by the P-local projection of the first k units under W has the corresponding source operation either reached as a next-frontier operation, or recorded as already issued, still pending, and attributed to the same dynamic source-level operation instance; for rendezvous and paired-synchronization units this property holds simultaneously for both endpoint processes of the unit. In addition, for every k the reconstructed global Sys_ref operational state admits a finite legal extension to the complete matching unit γ_{k+1}: after a finite local ε replay recorded by W, W records either immediate enabled selection or a concrete finite sequence of matched complementary actions that makes the unit enabled, together with all scheduler/arbiter choices, channel-legality facts, and atomic counterpart availability required for its commit. When the prefix is later used for cutpoint transfer, W also supplies the enriched annotated replay-history component H and the dynamic identities needed to state J.OfferReach and J.OfferConf. This last cutpoint record does not put historical Issue timestamps into KObs; the operational annotations remain internal evidence for the execution-level witness construction.

Status of J.GWR (implementation-side premise, not a consequence of E1–E5). The existence of a globally witness-realizable package for every reachable RTL_B prefix is a scheduling/tool-compliance obligation of the implementation flow, of the same kind as I.A0 and B-faithfulness: it expresses the fact that the generated RTL commits observable units only in orders that the scheduled source program, replayed on the same observable history under the selected witness, could reach — including simultaneous endpoint availability for rendezvous and paired-synchronization units and a finite operational realization of every selected next unit. J.GWR is not derived in this document from the local E-rules, per-process replay, or channel-history matching, and Theorem J.1's Post → Ref clause is stated conditionally on it. A separate global witness-realizability theorem, deriving J.GWR for a particular tool flow from that flow's scheduling invariants, is identified as future work; until such a theorem is supplied for a given flow, J.GWR shall be discharged as tool/RTL compliance evidence and recorded in the side-condition discharge record. Lemma I.3's "matched global witness/channel-history invariant" and finite selected-unit realization premises are exactly the per-step content of J.GWR and are discharged by it in the execution-level construction below.

For each process P, let A_ref,P and A_post,P be the annotated P-local projection records induced by τ_ref,system and τ_post,system under the chosen witness/annotation package. Their forgotten observable components are the ordinary projections τ_ref,P and τ_post,P, but the records also carry the issue annotations, dynamic message-call universe, μ_Q matching, and request-attribution facts needed by E3/E5 and WC. Assume that for every process P, these annotated projections satisfy the per-process compatibility predicate A_ref,P ≈_P A_post,P. In readable prose below, this may be abbreviated as τ_ref,P ≈_P τ_post,P when the annotation package is fixed by context.

Thus Proposition J.0 is only a conditional composition result for a chosen compatible trace pair. By itself it is not yet an execution-set theorem and does not provide unrestricted source ↔ RTL correspondence. Theorem J.1 is the later execution-level result obtained by instantiating Proposition J.0 with the witness construction and scope restrictions stated in Appendix I / Appendix L and in the Exec_post / Exec_ref definitions.

Assume every explicit abstract channel c in the chosen Sys_ref / RTL_B comparison has finite capacity B(c) ≥ 0. Also assume, as a per-channel Sys_ref / RTL_B well-formedness condition, that both chosen traces satisfy the applicable E4 channel semantics at the chosen abstract boundaries. Thus, for B(c)=0 channels, committed Push_c / Pop_c events obey the same-cycle rendezvous semantics; for B(c)>0 channels, the committed Push_c / Pop_c history obeys the reset-empty, cycle-boundary, non-fall-through bounded-FIFO semantics with capacity B(c), including FIFO order, no underflow, no overflow, no duplication, and no loss. Proposition J.0 does not derive these per-channel E4 facts; it assumes them as part of the definition of the compared well-formed Sys_ref / RTL_B traces.

No weak-fairness or channel-progress premise is needed for this conditional pairwise lifting lemma: Proposition J.0 does not construct a matching trace, but only lifts already-compatible per-process trace projections and already well-formed per-channel communication histories to the aggregate system trace. The finite selected-unit witness-construction premises are used later in Theorem J.1 when proving the finite execution-prefix existence result; weak fairness and channel progress are used only when a corresponding infinite fair execution is asserted. Then the aggregate traces are compatible: τ_ref,system ≈_sys τ_post,system under rules E1–E5 (given the R rules and B rules).

Notation: The order relation <_src used below is the dynamic source-induced order from Appendix G / R1–R4: it ranges over dynamic operation instances in the compared execution / witness, not over all syntactic call sites in the source text. Equivalently, when the process must be explicit, write <_P. Untaken branches and unexecuted loop iterations are not members of the witness-specific dynamic universe.

Proof

We prove each compatibility property. The per-channel E4 facts are not derived inside this proposition; they are part of the per-channel Sys_ref / RTL_B well-formedness hypothesis for the chosen traces.

E1 (Synchronization-order preservation):

Fix any process P. By the proposition hypothesis, $\tau\_ref,P \approx\_P \tau\_post,P$, so the synchronization events of P occur in the same source order in both traces. Since this holds for every P, E1 holds for the system traces $\tau\_ref,system$ and $\tau\_post,system$.

E2 (Signal-visibility preservation):

Fix any process P. If P is a sequential process, then by the proposition hypothesis (using the E2 anchoring relation embodied in $\tau\_ref,P \approx\_P \tau\_post,P$), every signal Read in P is anchored to P's closest preceding synchronization event, and every signal Write in P is anchored to P's closest succeeding synchronization event, in both τ_ref,P and τ_post,P.

If P is a combinational process, then R2C rather than pred_S/succ_S anchoring applies: P's observable signal Read/Write events are represented by the R2C fixed-point signal-observation unit for the relevant combinational process/network component, matched by that component's local R2C occurrence ordinal as specified in the R2C unit matching/index convention. By the combinational well-formedness condition and the compatibility hypothesis for the fixed stimulus / selected witness, the pre-HLS and post-HLS combinational networks reach corresponding unique ε-fixed-point valuations for the matched R2C occurrence unit, so the matched Read/Write labels agree within that fixed-point unit. This local R2C-unit agreement is an explicit atomicity constraint; it does not require Sys_ref and RTL_B to use the same global observable cycle-bucket decomposition for unrelated independent events.

Since the appropriate sequential or combinational signal-visibility condition holds for every process P, E2 holds for $\tau\_ref,system$ and $\tau\_post,system$.

E3 (Safe message issue ordering):

Fix any process P. E3 is the post-side issue-order conjunct of the per-process compatibility relation $\approx\_P$, not an additional ordering property derived from the Sys_ref trace. Thus what must be checked here is that P's post-HLS execution satisfies E3 as stated in Appendix G: for any $q1,q2 \in Q\_exec,P$ with $q1 <\_src q2$, including ordinary distinct-interface message calls that appear in dynamic source sequence, $issue\_post(q1) \leq issue\_post(q2)$, and the issued set is prefix-closed under that source order. Same-cycle issue is permitted where allowed, and a later operation may be issued before an earlier operation commits under Policy L, but a source-later operation may not be issued before the source-earlier operation has issued. This post-side property is part of the local per-process hypothesis $\tau\_ref,P \approx\_P \tau\_post,P$; in the execution-level use of Proposition J.0 / Theorem J.1, the needed post-side witness/order fact is supplied by Appendix I / Implementation assumption I.A0. Since the E3 post-side conjunct holds for every P, the aggregate system pair satisfies E3.

E4 (Per-channel channel semantics):

E4 holds by the per-channel Sys_ref / RTL_B well-formedness hypothesis of Proposition J.0. That hypothesis states that, for every explicit abstract channel c in the chosen comparison, the Sys_ref and RTL_B traces already satisfy the applicable E4 channel semantics at the chosen abstract boundary: same-cycle rendezvous semantics when B(c)=0, and reset-empty, cycle-boundary, non-fall-through bounded-FIFO semantics when B(c)>0. Thus Proposition J.0 uses E4 as an assumed channel-realization condition for the chosen traces; it does not attempt to derive E4 from the per-process compatibility relation. Accordingly, Proposition J.0 should be read as a composition theorem under already-established process and channel conformance hypotheses. It does not prove that the implementation realizes the chosen abstract channel boundaries, nor that the back-annotated capacities B(c) satisfy B-faithfulness as defined in the Common Formal Definitions. Those facts are separate RTL/tool-flow compliance obligations required before the theorem is applied to a concrete generated design.

E5 (Messages cannot cross syncs):

For every synchronization call s (in the source order used by R3 / pref_S / suff_S over Q_exec,P):
∀ q ∈ pref_S(s) : issue_post(q) ≤ clk_post(s)
∀ q ∈ suff_S(s) : issue_post(q) > clk_post(s)
(Note: For blocking message transfers, the "no-withdraw"/persistence semantics imply that if a blocking operation is issued in the pre-side interval before s, then it must also commit no later than s (and similarly on the post side), because the process cannot complete s until all earlier-issued blocking requests in that interval have committed. Failed non-blocking polls in Q_exec,P are constrained at the issue level by this rule but contribute no committed Σ-event when they fail.)
This property is guaranteed per-process by the proposition hypothesis on the chosen compatible pair (with Appendix I supplying the witnesses in the execution-level Post → Ref use case) and requires no inter-process reasoning, so it lifts directly to the system level.
Conclusion:
All five compatibility properties hold at the system level for this chosen compatible trace pair. The composition is valid because:
· Intra-process properties are preserved by the per-process compatibility hypothesis.
· Inter-process communication respects the applicable E4 channel semantics by the per-channel Sys_ref / RTL_B well-formedness hypothesis for the chosen traces.
· No additional progress or fairness argument is needed inside Proposition J.0, because this lemma does not construct or extend executions; it only composes already-given compatible projections and already well-formed per-channel communication histories. Finite witness existence is supplied later by J.GWR in Theorem J.1; progress and fairness are needed only for separate infinite-execution claims.
Therefore, τ_ref,system ≈ τ_post,system. □

Remark 1. When Sys_ref = Sys_B and all B(c)=0, Sys_ref coincides with the rendezvous interpretation of Sys, so Proposition J.0 and Theorem J.1 both specialize to the rendezvous case. In the elaborated cases, the source-side proof target is Sys_ref = Sys_B^elab.

Remark 2 (Reference-model ladder: raw Sys, abstract Sys_B, and elaborated Sys_B^elab).
The Matchlib library provides a capacity back-annotation feature that extracts finite capacities B(c) from the RTL realization of each message-passing channel and makes them available to the source-level model. To avoid ambiguity, this document distinguishes three source-side models:

• Sys: the raw source-level rendezvous interpretation of the user-written processes, i.e., the model obtained when every effective channel capacity is zero and no additional abstract realization structure is introduced.

• Sys_B: the abstract bounded-FIFO source-level interpretation of the same user-written processes under E4, with finite capacities B(c) at the chosen abstract channel boundaries. When all B(c)=0 and no additional abstract realization structure is needed, Sys_B coincides with Sys.

• Sys_B^elab: the elaborated back-annotated source-level model used in the more complex cases discussed in Appendix B / Appendix Q, in which additional explicit abstract staged/bundle/join channels are introduced so that conjunctive/join dependencies, synchronization-boundary ramp-down / epoch-gating, occupancies, and enablement are represented at explicit abstract boundaries rather than as hidden predicates at the outer Σ-visible endpoints.

Formal proof target. Unless a statement explicitly says otherwise, the formal results of Appendices J–K target the prescribed source reference model Sys_ref, defined as follows:

(i) Sys_ref = Sys_B in the ordinary case where finite capacities B(c) at the chosen abstract boundaries suffice to capture the RTL realization; and
(ii) Sys_ref = Sys_B^elab in the elaborated cases where additional explicit abstract staged/bundle/join channels or explicit sync-boundary / epoch-gate channels are required.

In particular, if an RTL pipeline with an explicit synchronization boundary ramps down by draining already accepted work and withholding producer-facing availability for capacity usable only after the synchronization call commits, then the prescribed source reference model is Sys_B^elab. That withholding shall be represented as ordinary ChannelDisabled behavior at an explicit abstract boundary of Sys_B^elab. It shall not be modeled as an extra hidden disablement predicate on an outer bounded channel whose E4 occupancy test would otherwise make the transfer enabled.

For a simple pipelined loop with a single message-passing input and output, Sys_ref is ordinarily just Sys_B: the RTL pipeline's internal staging capacity is accounted for entirely by the back-annotated effective capacity B(c) at the chosen Sys_B channel boundary. Concretely, Sys_B still has exactly one Σ-visible Push_c(v) at the producer endpoint and one Σ-visible Pop_c(v) at the consumer endpoint. With overlapped iterations, the post-HLS RTL may accept/read-ahead up to B(c) values into internal pipeline staging before older iterations produce their corresponding outputs; these accept/read-ahead transfers are modeled as internal microstate (ε) within the concrete realization contributing to B(c). The RTL may realize this staging on either side of a particular module-local "Pop interface" handshake point; that handshake point need not coincide with the chosen Σ-visible Pop endpoint. By construction, the chosen Sys_B boundary encloses all staging counted in B(c) between the Σ-visible endpoints, so movements within that enclosed realization (including stage-to-stage handoff) do not create additional Σ-visible committed Push/Pop events beyond the single boundary Push/Pop events.

In more complex cases such as HW pipelines with multiple message-passing inputs, the back-annotation mechanism may elaborate the source-side simulation into Sys_B^elab by introducing additional explicit abstract realization structure. In those cases the formal target is Sys_ref = Sys_B^elab, and B(·), occ_·(t), E4, BoundaryReq, ChannelEnabled/ChannelDisabled, and the Appendix K blocking/WFG machinery are interpreted over the explicit abstract channels of that elaborated model. See Appendix B and Appendix Q.
Operational note. In all three cases, Sys_ref is a transition system with explicit process states, explicit abstract channel states, operation-attributive pending requests, and ε-microstate as defined in Appendix G's operational interpretation of Sys_ref. Sys_B and Sys_B^elab differ in the channel/elaboration component of that transition system, not in the ordinary dynamic source order of the user-written process code. Source-level message-call instances that appear in sequence in a process remain ordered by <_src; elaboration may introduce additional explicit proof-internal frontier items and abstract channels, but it does not make ordinary source-sequenced calls incomparable.

Note that in all cases the elaborated structure (staged/bundle/join channels, combinational couplers, and the projection map Π_elab) introduced for a particular pipelined realization is local to that process's side of the abstract channels at its boundaries. The complementary endpoint processes see only the abstract Σ-visible channels at the boundary and are unaffected by the elaboration choices.

For Lean-style formalization, the phrase "chosen Sys_ref" should be read as an explicit theorem parameter: either Sys_B itself, or Sys_B^elab(E) for a well-formed elaboration package E as defined in Appendix Q. Likewise, references to the Appendix I/L witness machinery assume that the actual WC premise has been discharged. The corresponding audit entry records whether WC was proved through

NB-CF, an Appendix L snooping / witness-selector construction satisfying Lemma L.2, another direct witness construction, or complete mixed per-site coverage. NB-CF and Appendix L snooping are provenance routes for proving WC, not independent theorem premises alongside WC.

Side-condition audit before theorem application. Applying Theorem J.1, Corollary J.1 cutpoint transfer, or the Appendix K liveness-preservation theorem to a concrete design requires instantiating the side-condition discharge record from the Common Formal Definitions for the selected theorem scope. In particular, the proof/application must identify whether Sys_ref is ordinary Sys_B or elaborated Sys_B^elab(E); provide a discharge of the actual WC premise, recording whether that proof was obtained through NB-CF, an Appendix L snooping / witness-selector construction, another direct witness construction, or complete mixed per-site coverage; and show that every back-annotated explicit abstract channel used by the theorem instance satisfies the required per-channel B-faithfulness obligation. When Corollary J.1 cutpoint transfer or Appendix K is used, the application must additionally discharge J.OfferReach, J.OfferConf, and J.KObsConf for the same selected witness, post prefix, reference prefix, canonical H, residual offers O, and cutpoints. When liveness/deadlock preservation is used, the application must also discharge K.Drain, Kε, K.CV, A5-L and the selected A5-L discharge route—including A5-S when the Policy-S route is used—and the applicable Appendix K/N progress assumptions. Thus these results should be read as conditional theorems over explicit assumption packages, not as claims that WC, J.GWR, Sys_B^elab, B-faithfulness, J.OfferReach, J.OfferConf, J.KObsConf, J.Mem, J.FE, RW1–RW5, K.Drain, Kε, K.CV, A5-L, the selected A5-L discharge route (including A5-S/Dead_KS where applicable), NB-Align, L.Dir, or their supporting evidence are automatically supplied by ordinary HLS synthesis. The record should also state, when K.2b/K.2c is invoked, how the NB-Align condition is discharged for witness-selected non-NB-CF non-blocking sites that could participate in frozen wait cycles. When WC is discharged through an Appendix L snooping / witness-selector co-simulation harness, the record shall additionally state how Condition L.Dir (Appendix L) is discharged for the compared runs, per Lemma L.5.

Remark 3 (Practical rendezvous liveness screening under determinism).
When B1 holds and the design's control flow does not branch on empty/full observations of non-blocking interfaces (i.e., it depends only on the ordered history of observable Push/Pop/synchronization actions, not on "probe" outcomes (NB-CF, Appendix I.2)), the rendezvous specialization B(c)=0 is often an effective practical screen for design-level deadlocks: it exercises the most constrained communication discipline and can expose true cyclic data-dependency deadlocks early. Because rendezvous is strictly more constrained than buffered/elaborated operation, it can also yield false positives: a deadlock observed under B(c)=0 may disappear once finite buffering or explicit elaborated staged/bundle/join structure is present. However, bounded buffering can still introduce capacity-induced deadlocks (FIFO-depth artifacts) when FIFO depths are underestimated; these deadlocks are real for that modeled capacity configuration but may disappear once capacities are increased to the intended design point. Increasing buffer capacity (or applying dynamic buffer-growth strategies) is a standard way to distinguish depth-limited deadlocks from true cyclic dependency deadlocks. Accordingly, the formal results of Appendices J–K deliberately target the prescribed source reference model Sys_ref of Remark 2, with the actual back-annotated capacities and explicit abstract boundaries required by the chosen realization, rather than relying on rendezvous alone.

Remark 4 (When raw Sys may be used instead of the prescribed source reference model for safety-only checks).
If verification is concerned only with safety properties over the chosen DUT-boundary projection Σ (denote it Σ_DUT) (and not with deadlock/liveness), the prescribed source reference model Sys_ref of

Remark 2 remains the default reference model. This is because Σ_DUT includes committed channel-transfer events, and bounded buffering / explicit staged-bundle-join structure generally changes the set/timing of those committed Push_c/Pop_c events relative to the raw rendezvous case.
Practical note (early bring-up): Before accurate capacities B(c) are available (or before back-annotation/elaboration can be used), running the rendezvous specialization Sys against the intended testbench is still useful to validate functional intent and catch gross protocol/ordering bugs early; however, it remains a screening check, not a proof about the buffered/elaborated RTL-correspondent model.
Important (soundness): When any channel or explicit abstract staged/bundle/join boundary that can influence Σ_DUT-observable behavior has positive effective capacity or nontrivial enablement structure, the raw rendezvous model Sys is often a restriction of Sys_ref / RTL_B at Σ_DUT (it can admit fewer Σ_DUT behaviors). Therefore, using Sys in place of Sys_ref is generally suitable only as a debug/screening check: a failure can be useful diagnostically, but a pass does not by itself prove Σ_DUT-safety of the buffered/elaborated implementation.
Sys may be used as a proof-equivalent substitute for Sys_ref only when Sys and Sys_ref are Σ_DUT-equivalent for the chosen Σ_DUT (i.e., their Σ_DUT-projected trace sets coincide under the intended stimulus). A simple sufficient condition is that every Σ_DUT-visible explicit abstract channel/boundary in Sys_ref has effective capacity zero and that buffering/elaboration elsewhere is Σ_DUT-opaque (it cannot enable/disable/reorder Σ_DUT-events or change when Σ_DUT-events occur). In all other cases—i.e., if any Σ_DUT-visible explicit abstract boundary has positive effective capacity, or if buffered/elaborated internal channels can affect when Σ_DUT-events occur—use the prescribed Sys_ref of Remark 2.
Formal soundness condition. Let Traces_Σ_DUT(M) denote the set of Σ_DUT-projected traces of model M under the intended fixed stimulus. Sys may replace Sys_ref for proving a universal Σ_DUT-safety property ϕ only if Traces_Σ_DUT(Sys) = Traces_Σ_DUT(Sys_ref); under that condition, Sys ⊨ ϕ iff Sys_ref ⊨ ϕ (and otherwise Sys is only a diagnostic/screening check as stated above).

Proof of Theorem J.1 (expanded execution-level witness construction)

We now prove the execution-level clauses of Theorem J.1 by explicit witness construction. We first prove (Post → Ref). The (Ref → Post) clause is not a symmetric arbitrary-source witness construction. It is a selected-prefix / RTL-consistency clause: it applies only when the source prefix has already been restricted by Theorem J.1 (or by Appendix L's snooping / witness-selection mechanism) to match a selected RTL_B execution prefix under the same fixed stimulus. The proof for that clause is given after the Post → Ref construction by choosing that selected RTL_B prefix and applying the compatibility and finite-ideal obligations relative to the chosen RTL_B witness, rather than constructing an RTL execution from an arbitrary Sys_ref run.

Fix τ_post ∈ Reach_post, together with its selected package W satisfying GlobalWitnessRealizable(τ_post, W) (Definition J.GWR), and consider a finite observable prefix of τ_post. No ε-normalization is required for Theorem J.1 itself. When corresponding ε-quiescent cutpoints are needed, Corollary J.1 separately applies B4/B5 normalization to the selected RTL_B and Sys_ref prefixes. The RTL_B execution may be displayed as observable cycle buckets $\beta_1\ \beta_2\ \ldots$, but those buckets are used here only to identify the timing of that RTL_B execution and to identify explicitly atomic same-cycle units. They are not the prefix index for the cross-model compatibility proof.

Let U(τ_post) be the finite set of observable units in the chosen RTL_B prefix, where a unit is either a singleton Σ-event or an explicitly atomic same-cycle transaction as defined in the cycle-bucket and required-prefix convention above. Let ≺_J be the strict required-order relation from Theorem J.1: it is

generated only by the explicit E1–E5 ordering/timestamp/issue-side obligations that impose required predecessors on observable units, by E4 channel/FIFO legality, by signal-anchor semantics, by paired synchronization/barrier semantics, and by explicitly stated signal/channel/synchronization atomicity constraints. It is not generated by raw per-process source order on distinct-interface message commits, and it is not generated by the full Appendix G causal-dependency graph →.

Let Ideals(U(τ_post),≺_J) be the finite ideals H ⊆ U(τ_post), i.e. the finite down-closed subsets of observable units under ≺_J. The empty set ∅ is the initial ideal, and the complete chosen RTL_B prefix U(τ_post) is itself the terminal ideal. Since ≺_J is acyclic on U(τ_post), topological enumerations exist.

The witness construction below does not require extension from every proper ideal, and it does not allow an arbitrary ≺_J-minimal unit to be selected out of RTL_B witness order. Instead, fix once and for all a realizable RTL-time enumeration
$\gamma_1, \gamma_2, \ldots, \gamma\_m$
of the observable units in U(τ_post), with the following properties:

- If $clk(\gamma_i) < clk(\gamma_j)$ in the chosen RTL_B execution prefix, then $i < j$.
  • Explicitly atomic same-cycle transactions, such as rendezvous Push_c(v)/Pop_c(v) transfers and paired SyncChannel/ac_sync transactions, appear as single units.
  • Within a single RTL_B commit cycle, units not ordered by ≺_J may be listed in any order compatible with the selected RTL_B witness/replay attribution: for each participating process P, the unit listed next is one whose source operation is either reached as a next-frontier operation in the P-local replay state determined by the already-listed P-local units, or is already issued and pending under the chosen WC / issue-attribution witness.
  • The enumeration is obtained from the chosen RTL_B witness prefix itself after grouping explicitly atomic same-cycle transactions; it is a proof device and does not add an observable order among units that are unordered by ≺_J.

Provenance of the enumeration (J.GWR). The existence of an enumeration with the properties above — in particular the third property, that each next-listed unit's source operation is frontier-reached or issued-and-pending in the corresponding replay state, simultaneously at both endpoints for rendezvous and paired-synchronization units — is precisely clause (v) of GlobalWitnessRealizable(τ_post, W) (Definition J.GWR) and is taken from that premise. It is not derived here from E1–E5, from per-process replay, or from channel-history matching. All uses of "witness-realizability" below refer to this premise.
For $0 \le k \le m$, define
$H\_k = \{ \gamma_i \mid 1 \le i \le k \}$.

By the acyclicity argument above, every strict ≺_J edge increases RTL_B commit cycle. Therefore the prefix consisting of all units before any given RTL_B commit cycle is down-closed under ≺_J. Within a single commit cycle, explicitly atomic same-cycle transactions are represented as single units, and the fixed enumeration lists any same-cycle units in an order compatible with the selected witness/replay attribution. This same-cycle listing is a witness-construction order only; it does not introduce additional strict ≺_J edges. Hence each H_k is an ideal of U(τ_post) under ≺_J.

We construct, by induction on k, a Sys_ref prefix τ_ref[H_k] ending in a terminal semantic state σ_ref[H_k], such that τ_ref[H_m] is the desired witness τ_ref. The construction may take the finite ε-steps needed to reach and commit the next matched observable unit, but it does not append a maximal ε-normalization suffix and does not require σ_ref[H_k] to be ε-quiescent. B4/B5 normalization belongs to the later cutpoint-correspondence construction of Corollary J.1.

Invariant Inv(H_k). The invariant below is stated for a prefix H_k of the fixed RTL-time ideal chain. When the notation Inv(H) is used locally below, H denotes one of these chain prefixes, not an arbitrary ideal of U(τ_post).

(I1) Required-prefix matching:
τ_ref[H],system has produced exactly the matched observable units in H, with the same Σ-labels/values and with all ≺_J required-order constraints among those units preserved. The Sys_ref bucket decomposition used to realize H may differ from the RTL_B bucket decomposition, except that explicitly atomic same-cycle units must remain atomic.

(I2) Per-process replay-state agreement:
For every process P, the source-level state of P at σ_ref[H] agrees with the deterministic source-level replay state determined by the P-local projection of H under the fixed-stimulus and WC witness assumptions. In particular, this agreement covers every source-level variable, control predicate, payload/value expression, and request-attribution fact that can influence future Σ-visible behavior of P or any Appendix K pending-request / drain-only fact of P.

(I3) Channel-history agreement:
For every explicit abstract channel c of Sys_ref, the committed Push_c/Pop_c history induced by H is the same on the Sys_ref witness side and on the RTL_B witness side under the event matching convention; when dynamic reset is used, this history is interpreted per reset epoch, restarting reset-empty at each matched RESET boundary unit in H (cf. the occ_c(t) definition, which is measured from the most recent reset boundary). Hence, for B(c)>0 channels, the FIFO occupancy/head facts computed from that history agree; for B(c)=0 channels, each committed transfer is represented only by an explicitly atomic rendezvous Push_c(v)/Pop_c(v) unit.

In particular, this agreement covers:
• any source-level variable read by BoundaryReq(·,·);
• any source-level control predicate Pred_x(·) that decides reachability/selection of the next observable action(s);
• any payload/value expression of such next action(s); and
• for blocking Push/Pop operations, the source-level Issue/Commit attribution facts needed to determine whether a dynamic operation instance q is already issued, not yet committed, and still the operation to which the asserted endpoint-owned request is attributed under Appendix C's Issue/Commit linkage.

Channel-side enablement facts are then determined at the chosen abstract boundary from this request-attribution state together with the channel-history facts preserved by Inv(H).(I3): for B(c)>0 channels, the logical occupancy/head state induced by H; for B(c)=0 rendezvous channels, the endpoint-request predicates determined by the replay states of the two endpoint processes.

For each process P, the RTL_B-side comparison state used in Inv(H).(I2) is the deterministic source-level replay state determined by the P-local projection of H under the fixed-stimulus and WC witness assumptions. This immediate post-unit replay object is governed by Lemma I.DR'. Lemma I.DR remains the corresponding theorem for explicitly ε-normalized cutpoint reasoning, as used later in Corollary J.1 and Appendix K. The corresponding Sys_ref state σ_ref[H] is the terminal semantic state after realizing

the matched required prefix H; no ε-quiescence or maximal normalization is asserted at this stage.

Base case (H = ∅).
Let τ_ref[∅] be the empty Sys_ref prefix under the fixed stimulus and σ_ref[∅] its initial state. The empty ideal ∅ contains no observable units. By the fixed-stimulus comparison setup, the initial source-level valuations, request-attribution facts, and reset-empty channel histories agree. Hence (I1), (I2), and (I3) hold. When dynamic reset occurs later in the compared executions, each subsequent reset epoch re-establishes this correspondence through the matched RESET boundary unit and Lemma J.RS (MatchedResetEpochSplice); the induction therefore has one base case per reset epoch rather than only an initial one.

Inductive step along the fixed RTL-time ideal chain (H_k → H_{k+1}).
Assume Inv(H_k), where k < m. Let γ = γ_{k+1} be the next observable unit in the fixed RTL-time enumeration, and let H^+ = H_{k+1} = H_k ∪ {γ}. By construction of the enumeration, H_k and H^+ are ideals under ≺_J, and every required ≺_J predecessor of γ has already been matched in H_k.
The choice of γ is not arbitrary. It is the next unit in the selected RTL_B witness order after grouping explicitly atomic same-cycle transactions. Consequently, for each process P participating in γ, the P-local replay state determined by the P-local projection of H_k has the corresponding source operation either:
• reached as a next-frontier operation; or
• recorded, under the WC / issue-attribution witness, as already issued, still pending, and still attributed to the same dynamic source-level operation instance.
This witness-realizability condition is what makes the case analysis below applicable. If multiple unordered same-cycle units are available, the proof may choose any one satisfying this condition; such a choice is only a construction order for the Sys_ref witness and does not add an observable order among units unordered by ≺_J. Sys_ref is still not required to reproduce RTL_B's cycle-bucket decomposition, except that explicitly atomic same-cycle units remain atomic.

Existence of a matching Sys_ref extension.
We show there exists a finite Sys_ref extension from σ_ref[H_k] that commits a matching unit γ′ for γ = γ_{k+1}, producing the terminal state σ_ref[H_{k+1}] of the extended matched prefix:

σ_ref[H_k] —ε*→ σ̃_ref —γ′→ σ_ref[H_{k+1}].

Here γ′ has the same Σ-label(s), value label(s), dynamic message-operation matching, and synchronization-transaction identity as γ. If γ is a singleton event, γ′ is a singleton event. If γ is an explicitly atomic unit, γ′ is the corresponding explicitly atomic Sys_ref unit.
The following cases use the witness-realizability property of the fixed RTL-time enumeration. Thus, when a case refers to the next-frontier information or to an already-issued pending request, that fact is supplied by Inv(H_k).(I2) together with the selected RTL_B witness / WC request-attribution state, not merely by γ being ≺_J-minimal.

If γ is a singleton process-local non-message Σ-event, such as a sequential-process anchored Read/Write event as in E2 (when not grouped with its anchor synchronization unit under the E2 signal-anchor grouping convention) or a one-sided synchronization endpoint that is not part of a paired atomic transaction, then the corresponding source-level operation of the relevant process P is selected from the next-frontier information determined by the P-local replay state in Inv(H_k).(I2). Since σ_ref[H_k]

agrees with that replay state on the control predicates Pred_x used for next-action selection and on the value expressions, the same local predicate/value facts hold in Sys_ref. For sequential Write events, the value label is determined by the same source-level expression evaluation; for sequential anchored Reads, the value label is determined by the fixed stimulus plus E2 anchoring. Thus the matching Sys_ref event has the same Σ-label.

If γ is an explicitly atomic R2C combinational fixed-point signal-observation unit, then γ is not selected by a process-owned synchronization anchor and has no pred_S/succ_S requirement. Instead, its formal occurrence is the cycle's ε-quiescent combinational fixed-point observation. Because H is down-closed under $\prec_J$, all causally prior sequential clock-boundary updates, stimulus values, and already-matched signal-driving facts needed to determine that fixed point have already been matched in H. By the R2C well-formedness premise, the relevant combinational network is deterministic and reaches a unique ε-fixed point; by Inv(H).(I2) and the fixed-stimulus hypothesis, the corresponding Sys_ref and RTL_B fixed-point valuations agree on the observed signal boundary. Intermediate delta-cycle propagation and redundant re-evaluation are ε-steps and are not separately Σ-visible. Therefore Sys_ref can extend by ε* to the corresponding fixed point and emit the matching atomic R2C unit containing the corresponding fixed-point Read/Write Σ-labels with the same value labels.

For a message-transfer event in γ, the matched dynamic source-level message-operation instance q is handled by a two-case classification.

Case 1: q is a not-yet-issued next-frontier message operation. Then q is selected from the message-operation slice of NextFront_P determined by the P-local replay state in Inv(H).(I2). Since σ_ref[H] agrees with that replay state on the source-level variables read by BoundaryReq(q,·), on the control predicates Pred_q used for next-action selection, and on the payload/value expressions, Sys_ref can issue the same boundary request for q with the same payload/value facts, subject to the same side-of-synchronization and same-interface ordering constraints.

Case 2: q is an already-issued pending message operation. Then q need not be a member of NextFront_P(σ_ref[H]); in particular, Appendix B's drain-only discipline allows an already-asserted non-frontier request to persist as protocol state while an earlier pending blocker remains frontier-minimal. In this case the relevant fact is not NextFront membership, but the request-attribution component of Inv(H).(I2): q has already issued, has not yet committed, remains the operation to which the asserted endpoint-owned boundary request is attributed under Appendix C's Issue/Commit linkage, and has the same stable payload facts when q is a Push. Therefore Sys_ref contains the same pending request instance q at the compared cutpoint.

For buffered channels with B(c)>0, the channel-side legality of the matching Push_c/Pop_c commit is checked against the FIFO history induced by H. By Inv(H).(I3), the corresponding FIFO occupancy/head facts agree in Sys_ref and in the RTL_B witness history for the same required prefix H. Therefore the matching buffered Push_c/Pop_c event has the same channel-side legality in Sys_ref as in RTL_B, subject to the same reset-empty, FIFO-order, no-underflow/no-overflow, and non-fall-through E4 rules. If a Push_c and a Pop_c occurred in the same RTL_B clock bucket but are not an explicitly atomic rendezvous unit, they may be realized in different Sys_ref cycles or in a different Sys_ref bucket grouping, provided the E4 history and all $\prec_J$ constraints are preserved.

For a rendezvous channel with B(c)=0, a committed transfer is an explicitly atomic observable unit containing the two endpoint events Push_c(v) and Pop_c(v). It is not matched by independently

scheduling the two endpoints. If γ is such a rendezvous transfer, then the endpoint processes P and Q have the corresponding source-level boundary requests in the replay states determined by H. Each endpoint request may be present either because its operation is the not-yet-issued next-frontier operation just selected, or because it is an already-issued pending request preserved by the request-attribution state described above. By Inv(H).(I2), the same endpoint request predicates and payload/value facts hold in Sys_ref. Therefore the matching Push_c(v)/Pop_c(v) endpoint pair is jointly enabled in Sys_ref and commits as one atomic rendezvous unit. The Appendix G causal-dependency edge Push_c(v) → Pop_c(v) is not used here as a strict edge internal to the unit; the unit is added to the ideal all at once.

For a paired SyncChannel/ac_sync transaction, γ is likewise an explicitly atomic unit containing the two matched endpoint events with the same dynamic synchronization-transaction identity. By the synchronization-pair well-formedness condition for ≈ and by Inv(H).(I2), both endpoint processes reach the corresponding synchronization frontier with the same before/after source-state facts. Therefore Sys_ref commits the matching paired synchronization transaction as one atomic synchronization unit, preserving the induced cross-process ordering.

By Lemma I.3's finite selected-unit realization construction, instantiated with the per-step finite-extension clause of J.GWR and the Sys_ref operational transition semantics, choose a finite ε* extension in which γ' commits. This is an existential finite-prefix construction: it selects the enabled action, or follows the concrete finite complementary-action sequence recorded by W, and does not invoke B2/B3 or a fair infinite extension. Let τ_ref[H_{k+1}] be τ_ref[H_k] extended by this finite ε* segment and the matching unit γ', with σ_ref[H_{k+1}] its resulting terminal semantic state. No maximal ε-normalization suffix is appended in Theorem J.1. Then (I1) holds for H_{k+1} by construction; (I2) follows from deterministic replay on each process's P-local projection of H_{k+1}; and (I3) follows from applying the same E4 channel-history update for the matched channel event(s). Thus Inv(H_{k+1}) is established.
If γ is a proof-internal RESET boundary unit RESET(S_p, S_c), the extension and the re-establishment of Inv(H^+) are given by Lemma J.RS (MatchedResetEpochSplice) under the reset well-formedness conditions RW1–RW5 (Common Formal Definitions): the base-case correspondence argument is re-applied per reset epoch at the matched boundary, which discharges the reset-splice obligation of Appendix T.2.1. No ε-normalization is asserted at the boundary beyond the finite reset transition itself, and processes/channels outside the reset scope are unaffected by RW5.

Under the fixed stimulus and the WC premise, apply Lemma I.DR' (Immediate Post-Unit Deterministic Replay) to: (a) P's execution within the chosen Sys_ref witness prefix τ_ref[H_{k+1}] reaching the terminal matched-prefix state σ_ref[H_{k+1}], and (b) the source-level replay determined by the P-local projection of H_{k+1} in the RTL_B witness. These executions start from the same initial source-level state and have the same P-local observable-unit replay projection under the document's event-matching convention, including the fixed witness w and μ_Q data where ε-modeled failed non-blocking q ∈ Q_exec,P are relevant. Therefore Lemma I.DR' implies agreement of the canonical immediate post-unit replay states for every source-level variable, control predicate, payload/value expression, completion fact, and replay-derived NextFront fact included in Inv(H_{k+1}).(I2). Operational facts saying that an already-issued uncommitted request exists and is attributed to a particular q are supplied by the J.GWR witness component used by the extension case, not inferred from the bare P-local observable sequence. No ε-quiescence or maximal ε-normalization is invoked in this step. Any later K-facing use of current uncommitted offers first establishes the applicable ε-normalized cutpoint and invokes J.OfferReach/J.OfferConf. This use of a sequence is a witness-selected observable-unit replay sequence, not an intrinsic linearization of an unordered same-cycle bucket.

This does not claim that raw RTL_B request wires or the identity of current uncommitted offers are determined from the bare P-local Σ-label sequence. Corollary J.1 obtains those facts only through the residual-offer component O: J.OfferReach identifies O with the current attributed offers of the selected reachable Sys_ref cutpoint, and J.OfferConf identifies that same O bidirectionally with the relevant RTL boundary requests. Process values, frontier candidates, abstract channel state, enablement, and WFG facts are then reconstructed as functions of the common KObs rather than copied as independent clauses of a broad state relation.
Because P was arbitrary, the per-process replay-state agreement holds for all processes; and because all channel-history updates are matched under E4, Inv(H_{k+1}).(I3) holds as well.

Conclusion.
By induction along the fixed RTL-time ideal chain $H_0 \subseteq H_1 \subseteq \dots \subseteq H\_m = U(\tau\_post)$, Inv(H_m) holds for the complete finite ideal U(τ_post) of observable units in the chosen RTL_B prefix. Taking τ_ref = τ_ref[H_m] yields the required Sys_ref witness prefix for (Post → Ref). The constructed τ_ref,system is compatible with τ_post,system under E1–E5 and the explicit channel, signal-anchor, and synchronization semantics, but it need not have the same observable cycle-bucket decomposition as τ_post,system. The construction uses RTL-time order only as a witness-construction schedule; it does not add an observable order among units unordered by ≺_J, and it does not require the broader Appendix G causal-dependency graph → to be acyclic. Bucket equality is required only inside explicitly atomic units, such as rendezvous Push/Pop transfers and paired SyncChannel/ac_sync transactions.

No ε-quiescence is asserted by this execution-level construction. When Corollary J.1 requires corresponding cutpoints, it separately applies B4/B5 normalization to the selected prefixes and then discharges J.KObsConf for those normalized cutpoints.

At the terminal matched prefix, the P-local projection of U(τ_post) is exactly the P-local observable-unit replay projection of the chosen RTL_B prefix, under the selected witness/annotation package. Therefore the replay-state agreement in Inv(U(τ_post)).(I2) gives
the deterministic-replay part of the K-observation bridge used by Corollary J.1 when it relates the selected
RTL_B cutpoint ρ_post to the selected Sys_ref cutpoint σ_ref via the J.KObsConf equal-KObs relation. The replay/committed-history portions of the record are supplied by the matched prefix and witness; the current residual-offer portion O is supplied by J.OfferReach and J.OfferConf. All frontier, guard, value, abstract-channel, endpoint-request, pending, enablement, and WFG facts used later are derived from that common record.

Ref → Post clause. The reverse clause is discharged asymmetrically. Let τ_ref ∈ Exec_ref satisfy the annotated RTL-consistency premise of Theorem J.1; that is, choose τ_post^* ∈ Exec_post and W with RTLConsistentRefPrefix(τ_ref, τ_post^*, W). By the definition of RTLConsistentRefPrefix, τ_post^* is admitted by RTL_B under the same fixed stimulus and τ_ref's observable-unit projection matches the required-prefix projection of τ_post^*. Because B1 is a standing assumption in this theorem instance, this selected observable projection is the corresponding prefix of the unique RTL_B Σ-trace, but B1 supplies only that projection uniqueness and not the annotation package. The package W supplies the compatible per-process annotated projection records, including dynamic operation universes, μ_Q matching, Issue/Commit annotations, E3/E5 issue-side facts, witness/request-attribution facts, and the per-channel E4 histories and abstract boundary facts needed to instantiate Proposition J.0 for the chosen pair. When cutpoint transfer is invoked, the same selected W supplies H and the operational identities needed for J.OfferReach; the cutpoint certificate additionally supplies O, J.OfferConf, B-

faithfulness/E4, reset alignment, and the composite J.KObsConf obligation for the selected (ρ_post, σ_ref, w). Applying Proposition J.0 yields τ_ref,system ≈_sys τ_post^*,system, so τ_post = τ_post^* is the required post-side witness. This argument does not construct an RTL execution from an arbitrary Sys_ref run, does not infer annotations from bare Σ-label projection matching, and does not rely on symmetry of ≈. □

Lemma J.HCanon (Canonical replay-history quotient agreement)

Fix the same theorem instance I, elaboration E, witness w, reset interpretation, dynamic operation identities, and J.GWR annotation package. Let τ_post be a finite RTL_B prefix and τ_ref the Theorem J.1 source witness prefix selected for τ_post. If τ_ref and τ_post satisfy the annotated observable-unit compatibility relation of Theorem J.1 under that fixed package, then their canonical enriched replay-history quotients are equal:
CanonHist_E,w(τ_ref) = CanonHist_E,w(τ_post).
Proof. The compatibility relation matches the same atomic observable units with the same labels, values, dynamic identities, and reset scopes. The fixed witness w and μ_Q/WC package identify the same witness-selected failed non-blocking, arbitration, signal, and ε-choice outcomes used by replay. Theorem J.1 preserves each process-local required order, each per-channel committed-transfer ordinal/FIFO order, every explicit atomic rendezvous or paired-synchronization grouping, every RESET marker, and every predecessor required by ≺_J. CanonHist retains exactly those fields. It discards only incidental source-versus-RTL clock-bucket placement and ordering among units that are independent under the retained constraints. Therefore both annotated prefixes map to the same typed quotient H. This lemma asserts equality of the canonical replay quotient, not equality of concrete cycle buckets or historical Issue timestamps. ■

Definition J.OfferReach (reachable source realization of a KObs record)

Fix the theorem instance I, elaboration package E, witness w, enriched replay history H, attributed residual-offer set O, and the particular Sys_ref witness prefix selected for the current Theorem J.1 / Corollary J.1 application. Write OfferReach_I,E,w(H,O; τ_ref,σ_ref) when all of the following hold: τ_ref is a finite reachable Sys_ref prefix under the same fixed stimulus; after the selected finite B4/B5 ε-normalization it ends in the ε-quiescent cutpoint σ_ref; H = H_E,w(σ_ref); O = ResOffers_E,w(σ_ref); and the prefix and cutpoint satisfy the operational Issue/Commit, E3/E5, Axiom R.1, reset-epoch, elaboration, and explicit-boundary rules of the selected Sys_ref instance. For an Appendix K application, the selected prefix/cutpoint package shall additionally be recorded as admissible under Policy L; this is the source-policy reachability fact consumed by Theorem K.1A.
OfferReach is an image/realizability predicate. It is not implied by type-correctness, source-prefix closure, or endpoint cardinality of an arbitrary tuple ⟨E,w,H,O⟩. In Corollary J.1 it is discharged for the same normalized source witness selected from Theorem J.1, not for an unrelated existential source state. A source execution log, mechanized reachability proof, or tool/witness certificate may discharge it.
Historical Issue cycles may be used internally to prove that τ_ref is admissible and that O is its current residual-offer set, but they are not fields of KObs and are not compared with RTL issue cycles. OfferReach records only the surviving attributed offers at the selected cutpoint.

Definition J.RelevantReq (KObs-relevant RTL endpoint request)

At an ε-quiescent RTL_B cutpoint ρ_post, an asserted request level r is KObs-relevant for elaboration E when: (i) r is endpoint-owned and lies at an explicit abstract boundary selected by E; and (ii) the

presence or absence of r can affect the reconstruction of BoundaryReq, a rendezvous endpoint-request/co-presence fact, ChannelEnabled/ChannelDisabled, Front, WFG, WFG+, or a derived Dead_K/Dead_KS/Bad_S predicate. Relevance is determined from the selected physical/abstract boundary and the reconstructed predicate interface, not from whether the flow happened to supply an attribution. Thus an asserted boundary request with missing attribution remains relevant and causes J.OfferConf to fail; it cannot be dropped from the abstraction by calling it unrecognized.

The dynamic operation/frontier/epoch attribution used below is supplied by the operational request-attribution annotations of J.GWR(ii), constrained by Appendix C's Issue/Commit linkage and Implementation Assumption I.A0. It is not inferred from the ready/valid wire level alone. A post-cycle request level proved by those annotations to belong only to an operation that committed in the selected cycle, with no next dynamic operation owning the continuing level, is not a residual offer. For a back-to-back continuing assertion, the annotation shall identify the next owning dynamic operation.

Obligation J.OfferConf (bidirectional RTL residual-offer conformance)

For an ε-quiescent RTL_B cutpoint ρ_post, fixed E and w, and candidate residual-offer set O, OfferConf_I,E,w(ρ_post,O) holds only if the following boundary conditions are satisfied at every explicit abstract boundary used by the theorem instance. All references to attribution use the J.GWR(ii)/Appendix C/I.A0 machinery specified by Definition J.RelevantReq; no attribution is inferred solely from an asserted wire level.
(1) Offer-to-request completeness. For every o = (P,x,q,c,dir,e) ∈ O, the corresponding endpoint-owned RTL request at c and direction dir is asserted at ρ_post, is KObs-relevant, and is attributed by the selected J.GWR(ii) package to that same frontier item x and source operation q in reset epoch e, with the operation kind derived from q.
(2) Request-to-offer completeness and uniqueness. Every KObs-relevant endpoint-owned RTL request asserted at the selected post-cycle ε-quiescent cutpoint is attributed by the selected J.GWR(ii)/Appendix C/I.A0 package to exactly one member of O, unless that package proves that the level belongs only to an operation that committed in the selected cycle and no next operation owns the continuing assertion. If the level remains high for a back-to-back operation, the attribution shall identify that next dynamic operation and the corresponding member of O. No relevant asserted request may be hidden, left unattributed, or omitted from O. At most one attributed outstanding source-level offer exists per explicit boundary and endpoint direction.
(3) Snapshot issue-order consistency. The issued-summary represented by the committed/replay history H, the WC witness information, and the current residual offers O is prefix-closed under the applicable dynamic source order and is consistent with E3, E5, reset epochs, and the selected elaboration. This is a cutpoint set invariant; OfferConf does not compare historical first-issue timestamps across models.
(4) Boundary and reset typing. Every x, q, c, direction, kind, owner, and epoch in O is supplied by E and agrees with the live RTL boundary attribution. Reset clears ending-epoch offers as required by RW2. Internal movement already represented as E4 occupancy is not also exposed as a second source-level offer.
(5) Payload scope. O need not contain pending Push payload values. Stable request payload is still required by Appendix C/Axiom R.1, and committed payload equality is checked by the value-labeled replay history. A flow that needs mid-flight data-wire correspondence may add it as a separate implementation obligation.
Audit-critical status of J.OfferConf(2). J.OfferConf is the authoritative RTL request-abstraction boundary for the KObs cutpoint result. In particular, request-to-offer completeness, no-hidden-request, explicit-boundary completeness, and attribution uniqueness are load-bearing obligations: omitting any asserted request that is KObs-relevant can remove a reconstructed wait-for edge or rendezvous-enablement fact

and invalidates J.OfferConf, J.KObsConf, and the equal-KObs conclusion. B-faithfulness/E4 and canonical-history agreement remain separate independent obligations; satisfying them does not compensate for an incomplete request abstraction.

Obligation J.KObsConf (composite KObs cutpoint conformance)

For every reachable ε-quiescent RTL_B cutpoint used by Corollary J.1 or Appendix K, the flow shall select one exact package (τ_ref, σ_ref, ρ_post, I, E, w, H, O) such that: (a) the selected source and RTL prefixes use the same fixed instance, elaboration, witness, stimulus, reset-epoch interpretation, and observation boundaries; (b) Lemma J.HCanon yields H as their common canonical replay-history quotient, without requiring equal incidental cycle bucketings; (c) J.OfferReach_I,E,w(H,O;τ_ref,σ_ref) holds and therefore realizes O at the reachable source cutpoint; (d) J.OfferConf_I,E,w(ρ_post,O) holds and therefore identifies O bidirectionally and uniquely with the complete KObs-relevant RTL request inventory; and (e) the selected RTL abstract boundaries satisfy E4, B-faithfulness, reset correspondence, and ε-opacity of the finite normalization. J.KObsConf is tied to the particular source witness chosen by Theorem J.1; an unrelated existential source cutpoint or an unrealizable residual-offer record does not discharge it. Define the J-facing RTL observation certified by that package as KObsPost_J(ρ_post;E,w,H,O) := ⟨E,w,H,O⟩. J.OfferReach and Definition R.18c give KObs_E,w(σ_ref) = ⟨E,w,H,O⟩. Hence J.KObsConf entails KObsPost_J(ρ_post;E,w,H,O) = KObs_E,w(σ_ref). This typed equality is the sole cutpoint-level correspondence used by Appendix K. It is not equality of arbitrary RTL and source microstates, and it creates no independent clause-based state relation.

Corollary J.1 (K-observation Correspondence at the End of a Matched Observable Prefix)

Purpose.
This corollary is the bridge between execution-level trace compatibility and the K-facing cutpoint observation used directly by Appendix K. Its primary and authoritative cutpoint conclusion is equality of the compact KObs record, not equality of arbitrary source and RTL state. K-relevant frontier, request, channel, enablement, WFG, drain, and deadlock facts are derived from that equality by the named reconstruction functions; no clause expansion is required by the Appendix K transfer layer.
Statement. Assume the hypotheses of Theorem J.1 for Sys_ref versus RTL_B under the fixed stimulus, including B1 for the RTL_B Σ projection, and use B4/B5 to choose finite ε-normalized cutpoints. Let τ_post ∈ Reach_post carry its selected J.GWR package. Choose an ε-quiescent RTL_B representative ρ_post by a maximal finite ε-extension of τ_post. Select the matching Theorem J.1 source witness prefix and the finite source ε-normalization recorded by J.OfferReach, ending in the reachable ε-quiescent cutpoint σ_ref. Fix the same I, E, w, H, and O throughout, and assume J.KObsConf for this exact package, namely: canonical-history agreement H from Lemma J.HCanon, J.OfferReach_I,E,w(H,O;τ_ref,σ_ref), J.OfferConf_I,E,w(ρ_post,O), B-faithful/E4 interpretation at E's explicit boundaries, reset correspondence, and ε-opacity. Then:
(1) τ_ref,system ≈ τ_post,system under E1–E5; and
(2) KObsPost_J(ρ_post;E,w,H,O) = KObs_E,w(σ_ref).
Derived K-facing consequence. Let K denote the common record in conclusion (2). Every typed reconstruction function in Definitions R.18d–R.18e has the same value on the selected post and source views. In particular, ProcRec, ChanRec, NextFrontRec, PendingRec, BoundaryReqRec, FrontRec,

ChannelEnabledRec/ChannelDisabledRec, WFGRec, and DrainClosedNowRec agree. Under the explicit premises stated in Definition R.18c-1 and Lemma R.21f, the corresponding waiver- and environment-dependent Dead_K^W, Dead_KS^W, and Bad_S^W reconstructions also agree. These are functional consequences of conclusion (2), not additional implementation-side state-equality assumptions.

Proof of Corollary J.1

Step 0 (Choose the normalized cutpoints). Extend the selected RTL prefix by one maximal finite ε suffix to the chosen ε-quiescent ρ_post; B4 makes the suffix finite and B5 supplies the quiescent/fixed-point interpretation. Use the source normalization recorded by J.OfferReach to obtain the ε-quiescent σ_ref. Neither side requires uniqueness of maximal ε extensions; all conclusions are relative to the selected package.

Step 1 (Obtain the matched replay history). Apply Theorem J.1 to the selected RTL prefix and J.GWR package to obtain the matching Sys_ref witness prefix under the same stimulus. Lemma J.HCanon then gives

CanonHist_E,w(τ_ref) = CanonHist_E,w(τ_post) = H.

Thus conclusion (1) holds, and the replay-history field of the two KObs records is equal even when their concrete cycle-bucket decompositions differ. The retained fields and the erased incidental ordering are exactly those stated in Lemma J.HCanon.

Step 2 (Establish the common residual offers). J.OfferReach identifies O exactly with ResOffers_E,w(σ_ref) at the selected reachable source cutpoint. J.OfferConf identifies that same O bidirectionally and uniquely with the relevant asserted endpoint-owned RTL requests at ρ_post. The reset-epoch, source-prefix-closure, boundary-cardinality, and no-hidden-request conditions ensure that O is a well-typed residual-offer component for both sides.

Step 3 (Conclude equal KObs). Both records have the same elaboration E, witness w, replay history H, and residual-offer set O. Therefore KObsPost_J(ρ_post;E,w,H,O) = ⟨E,w,H,O⟩ = KObs_E,w(σ_ref), proving conclusion (2). B-faithfulness and E4 ensure that the post-side interpretation of the committed history at each explicit boundary is the same abstract channel interpretation used by ChanRec; finite normalization is ε-opaque by J.KObsConf.

Step 4 (Derive the K-facing facts). Lemmas R.21b–R.21e reconstruct the process slice, explicit abstract channel state, NextFront, residual endpoint requests, Pending/BoundaryReq attribution, enablement, Front, WFG, and DrainClosedNow from the common record K. Lemma R.21f supplies the ordinary Dead_K factoring and, under its stated waiver-closure and environment-terminal premises, the corresponding Dead_K^W, Dead_KS^W, and Bad_S^W factoring. These conclusions follow by functional extensionality of the named reconstruction functions; no broad state correspondence is assumed. □

J.3 Causal Dependency and What System-Level Compatibility Guarantees (Informative)

The system-level result of Appendix J establishes compatibility of the aggregate observable behavior, i.e. $\tau_{ref,system} \approx \tau_{post,system}$ over the alphabet Σ of channel operations, synchronization events, and signal Read/Write actions.
This compatibility is intentionally *observational*: it constrains what an external environment can distinguish at Σ, not internal RTL micro-timing.

As a consequence, Appendix J should be read as preserving explicitly required observable ordering, atomicity, and FIFO/synchronization semantics, not incidental latency. The causal-dependency relation ($\rightarrow$) defined in the "Causal Dependency Between Processes" section remains useful for explaining information flow and for documenting which interactions a designer intends to be functionally significant. However, $\rightarrow$ is not itself the strict induction order for Theorem J.1, and the theorem does not require the unquotiented dependency graph $\rightarrow$ to be acyclic. The proof order is the required-order relation $\prec_J$ over observable units.

Accordingly, the system-level theorem implies the following required-order preservation principle:
• If two observable units γ_1 and γ_2 are ordered by the required-order relation $\gamma_1 \prec_J \gamma_2$, then the matched Sys_ref and RTL_B executions preserve that required order. This follows because each elementary $\prec_J$ generator is preserved by the corresponding rule: synchronization source order by E1; signal-anchor timestamping by E2; issue-side and request-attribution obligations by E3; channel FIFO legality and rendezvous atomicity by E4; immediate synchronization-boundary constraints by E5; and paired-barrier semantics by the explicit synchronization well-formedness conditions. In particular, for each explicit abstract channel c, the matched executions preserve the committed Push_c/Pop_c history: for B(c)=0, a committed transfer is one explicitly atomic rendezvous Push_c(v)/Pop_c(v) unit; for B(c)>0, the k-th committed Pop_c observes the k-th earlier unmatched Push_c according to reset-empty, FIFO-order, no-underflow/no-overflow, and non-fall-through E4 semantics.
• If two observable units are not ordered by $\prec_J$ and are not members of the same explicitly atomic unit, then their relative cycle placement or interleaving is not constrained by the methodology, and differences are treated as incidental latency variation. A designer who needs a particular cross-process ordering to be preserved must express it through an explicit Σ-visible mechanism such as message-passing FIFO order, a rendezvous/handshake atomic unit, explicit synchronization, or signal-anchor semantics.

This perspective is what makes the "single testbench / no surprises" implication precise: for the fixed-stimulus execution sets covered by Theorem J.1, any testbench that observes only Σ and encodes its expectations through causal dependencies (channels, barriers, or explicit handshakes) cannot distinguish the matched Sys_ref / RTL_B execution pairs provided by the theorem.
Equivalently, RTL_B introduces no new Σ-observable behavior relative to Sys_ref under that fixed stimulus, and source-to-RTL transfer is asserted only for the annotated RTL-consistent source prefixes covered by Theorem J.1. (Here Sys_ref is as fixed in Remark 2. The rendezvous specialization Sys with B(c)=0 is covered only under the explicit conditions of Remark 4.)

Appendix K – Preservation of Internal Wait-Cycle Freedom (Reachable-Cutpoint Deadlock-Freedom) for the Prescribed Source Reference Model

K.1 Scope, Notation, and Definition of Internal Message-Passing Deadlock

Terminology alignment

Throughout Appendix K, when evaluating predicates at an ε-quiescent cutpoint σ, let t_σ denote the cycle of that cutpoint and use the cutpoint shorthand $occ_c(\sigma) \triangleq occ_c(t_\sigma)$. For any frontier item x on an explicit abstract channel c = BoundaryOf_P(x) of Sys_ref, with owning dynamic source-level message instance q = MsgOf_P(x), use BoundaryReq(x,σ), ChannelEnabled(x,σ), and ChannelDisabled(x,σ) for the endpoint-owned request, enablement, and disabledness facts at that specific evaluated boundary. In the ordinary unelaborated case, where q has a unique evaluated boundary and a unique frontier item, BoundaryReq(q,σ), ChannelEnabled(q,σ), and ChannelDisabled(q,σ) may be used as shorthands. In Sys_B^elab, however, the frontier-item form is authoritative because one source-level operation may own multiple proof-internal frontier items at different explicit boundaries. Thus "boundary request is asserted" means BoundaryReq(x,σ), "channel-enabled" means ChannelEnabled(x,σ), and "channel-disabled" means ChannelDisabled(x,σ), all evaluated at the ε-quiescent ready/valid fixed point of the cutpoint cycle and read through q = MsgOf_P(x) and BoundaryOf_P(x). These predicates are not applied to already-committed Σ-event labels except through the operation/frontier item that contributes m(q) when it commits.

We work with a fixed design:

System Sys_ref: The collection of pre-HLS processes obeying the R-rules and B-rules, interpreted under the prescribed source reference model of Appendix J, Remark 2. In this appendix, capacities/occupancies/enablement predicates are interpreted at the chosen abstract boundaries of Sys_ref; in elaborated cases, this means the explicit abstract staged/bundle/join channels of that model. Each explicit abstract channel c of Sys_ref has finite capacity $B(c) \geq 0$, chosen to match the corresponding RTL realization at that abstract boundary.

Post-HLS implementation RTL_B: The hardware implementation of the same processes in which each chosen abstract channel c is realized with finite capacity $B(c) \geq 0$. In this appendix, we compare Sys_ref (the prescribed source-side proof target) to RTL_B (the micro-architectural realization). The proof does not require mapping buffered RTL states to a distinct raw rendezvous (B(c)=0) state.

Witness-selection and post-view convention. Corollary J.1 supplies a selected reachable Sys_ref cutpoint σ_ref and a common record K with KObs_E,w(σ_ref) = KObsPost_J(ρ_post;E,w,H,O) for the selected ε-normalized RTL cutpoint ρ_post. Appendix K does not compare arbitrary RTL microstate with source state. Define the K-facing post abstract view by applying the reconstruction functions of Definitions R.18d–R.18e to KObsPost_J. Unless a raw RTL signal is expressly named as part of J.OfferConf, every post-side occurrence of NextFront, Pending, BoundaryReq, channel state, ChannelEnabled/ChannelDisabled, Front, WFG, DrainClosedNow, or Dead_K in Appendix K denotes that reconstructed view. If a reproducible source witness is desired for debugging, the Appendix L snooping

wrapper may select one aligned to the RTL latency-sensitive events; uniqueness is not required by the theorem.

A global state σ of either system consists of:

1. For each process P: a control location (program point) and local state.
2. For each explicit abstract channel c of Sys_ref: its abstract FIFO state at the chosen Sys_ref boundary—its current occupancy occ_c(t) and (for buffered channels) its ordered contents. Here occ_c(t) counts all items that have been committed by Push_c but have not yet been committed by Pop_c at that chosen abstract boundary. In RTL_B, this count includes items residing anywhere in the concrete sub-realization that contributes to that particular abstract channel's back-annotated capacity B(c). Thus internal movement within the concrete realization of one abstract channel c is ε and does not create additional committed Push_c/Pop_c Σ-events beyond the boundary events counted by occ_c; however, transfers between distinct explicit abstract channels introduced in Sys_B^elab are represented in Sys_ref by those channels themselves and are not collapsed back into one outer Sys_B boundary.
3. Any additional architectural state not representing buffered messages on any channel (e.g., computation pipeline registers, arbitration state, bookkeeping, etc.).

We adopt the usual Wait-For Graph (WFG) abstraction, restricted to internal message-passing channels. Internal means: the producer and consumer of channel c are both processes of Sys_ref / RTL_B (i.e., both endpoints are in the modeled system). Channels whose complementary endpoint is the external environment/testbench (DUT boundary) are excluded from the WFG and from the deadlock definition in Appendix K. We write WFG_ref for the wait-for graph of Sys_ref and WFG_post for that of RTL_B.

- **Vertices:** Processes.
- Edges: There is a directed edge P→Q via channel c in state σ if there exists some frontier item x ∈ Front_P(σ) on channel c, with owning message instance q = MsgOf_P(x), such that x is channel-disabled (i.e., ChannelDisabled(x,σ) holds), and Q is the unique process that is the complementary endpoint for q (producer/consumer partner for c). This edge definition applies uniformly for both buffered channels (B(c)>0, enable is space/data availability via occ_c) and rendezvous channels (B(c)=0, enable requires the complementary endpoint's boundary request to be true in the same cycle), hence WFGs may contain a mix of edges via both kinds of channels. Here Front_P(σ) is the set of ≤_P-minimal pending blocking message-operation frontier items x ∈ Ω_msg,P whose owning message instances q = MsgOf_P(x) have kind(q) ∈ {Push_c, Pop_c}, have issued, and have not yet committed. Under ordinary source-sequenced message order, Front_P(σ) has at most one ordinary source-level message item for a process. Multiple message-operation frontier items may appear only when Sys_B^elab explicitly introduces proof-internal staged/bundle/join/epoch-gate frontier items and declares them incomparable in the elaboration package. P is "blocked on message passing" in σ as defined in Definition (Blocked on message passing) (K.1). Note: WFG edges may exist even when P is not blocked (e.g., one frontier item is channel-disabled while another explicitly incomparable frontier item is channel-enabled); however, internal message-passing deadlock (K.1) considers only sets D in which every P∈D is blocked, so all WFG edges relevant to deadlock reasoning originate from blocked processes with no channel-enabled frontier item. When Front_P(σ) has multiple members, P may have multiple outgoing WFG edges (one per blocked frontier item). **(No deassert-before-commit assumption.)** Blocking Push_c/Pop_c requests are non-withdrawable: once the request corresponding to a frontier item x ∈ Ω_msg,P is asserted through its owning message instance q = MsgOf_P(x), it is not deasserted before q commits (and Push_c payload remains stable while asserted). This rules out "try-and-withdraw" behavior for blocking transfers within the WFG abstraction.

All WFG reasoning is over ε-quiescent (ε-normalized) global states: internal pipeline/arbitration micro-steps are treated as ε and are not represented as separate WFG blocking points. Here, "ε-quiescent" includes the fixed point of purely combinational ready/valid propagation, so K.1 enablement is evaluated only on those settled ready/valid values.
(WFG-inert ε premise.) Appendix K uses ε-steps only after the WFG-relevant boundary-request / attribution cutpoint has been fixed. More precisely, consider an ε-path $\sigma \xrightarrow{\varepsilon^*} \sigma'$ that contains no Σ-visible committed Push_c/Pop_c event, no synchronization event, no signal Read/Write event, and no source-level issue/attribution transition that creates, removes, or reassigns a pending blocking boundary request. Such ε-steps are observationally silent with respect to the WFG abstraction: they do not change (i) the potentially-blocking frontier Front_P(σ) for any process P (i.e., which guarded blocking Push/Pop frontier items are minimal and pending), nor (ii) the truth or attribution of any boundary requests relevant to those frontier items, nor (iii) the channel-side enabling status (as defined in K.1) of any $x \in$ Front_P(σ), read through q = MsgOf_P(x). Equivalently, these WFG-inert ε-steps do not change the channel state relevant to K.1 enabling (e.g., occ_c(t) for B(c)>0 buffered channels), and they do not create/remove rendezvous enablement for a frontier transfer except via Σ-visible committed Push_c/Pop_c events.
This premise does not say that every non-Σ microstep is WFG-inert. In particular, an ε-normalization segment may include source-level issue/attribution steps that establish a persistent blocking request, such as moving from a cutpoint before a Pop has issued to a cutpoint where the Pop request is pending and BoundaryReq is true. Those steps may change Front_P, BoundaryReq, or request attribution, and are therefore not the ε-steps quantified over by the WFG-inert premise. Appendix K's WFG construction is applied only at the resulting ε-quiescent cutpoints, after such request/attribution state has been fixed.
In particular (pipelined loops / overlapped iterations): when HLS introduces overlap that may initiate later-iteration message reads "early," we assume the design obeys the Automatic flush (drain-only blocking discipline) defined in Appendix B. Under that discipline, once the process has reached a stable blocked frontier, overlapped execution does not introduce any later-iteration blocking Push/Pop frontier item into Front_P(σ), so transient internal pipeline activity that merely moves already-accepted work inside the concrete realization remains WFG-inert ε and does not contribute wait-for edges. Overlapped execution in pipelined loops is modeled only as a potential retiming of ordinary Σ-visible boundary commits (Push_c/Pop_c) relative to the unpipelined source, and is accounted for by the back-annotated semantics at the chosen abstract boundary of Sys_ref (E4). In particular, the channel facts relevant to K.1 enabling/WFG edges—e.g., occ_c(t) for B(c)>0, and rendezvous enablement for B(c)=0—change only on Σ-visible committed Push_c/Pop_c events at that boundary, or on explicit source-level issue/attribution steps before the WFG cutpoint is formed; internal movement within the concrete realization of a single abstract channel after that cutpoint is WFG-inert ε and does not affect K.1 enablement or introduce/remove WFG edges.
Precondition (Kε: WFG-inert ε / no hidden micro-timing control decisions). All results in Appendix K are stated under the precondition Kε that the narrowed WFG-inert ε premise above holds for the design at the ε-quiescent cutpoints used for WFG reasoning. If an implementation intentionally uses internal micro-timing after such a cutpoint to change Front_P membership, boundary-request attribution, or ChannelEnabled/ChannelDisabled facts without a corresponding Σ-visible event or explicit source-level issue/attribution transition, then Kε is violated unless that behavior is made Σ-observable (e.g., via explicit modeling/snooping) or is witness-selected under the Appendix I.2 framework.

A common instance is branching on the success/failure of a non-blocking poll in a way that changes the next Σ-visible candidate frontier **NextFront_P(σ)** (Definition R.6; see Lemma R.21b and Corollary J.1); in that case, either NB-CF must hold (Appendix I.2), or the poll outcomes must be witness-selected as stated there.
Non-blocking polling attempts and other purely internal actions do not contribute edges to the WFG; they are modeled as internal ε-steps. When a non-blocking call succeeds, that success is already represented at the Σ level as the corresponding committed Push_c/Pop_c event, but it does not itself add a wait-for edge because it is not a blocking operation.

SyncChannels and barrier well-formedness.
SyncChannel (barrier) operations are treated as pure synchronization events and are omitted from the Wait-For Graph, which tracks only blocking Push/Pop dependencies. We therefore interpret "deadlock" in this appendix as **internal message-passing deadlock** (i.e., a deadlocked set in which all processes are blocked on some internal Push/Pop dependency captured by the WFG). Any global stall at a barrier caused by mismatched or conditional participation (e.g., a process can permanently bypass the barrier or reach it a different number of times) is a specification-level error already present in the pre-HLS model; such stalls are not created by the HLS transformation and are excluded by an explicit barrier well-formedness assumption (A8 below), not by the internal message-passing deadlock premise. Under this assumption, omitting SyncChannels from the WFG is sound: a post-HLS deadlock implies the existence of a Push/Pop wait-cycle as characterized below.

Definition (Blocked on message passing). Using Front_P(σ) as defined above, we say that P is blocked on message passing in state σ iff Front_P(σ) is non-empty and, for every frontier item x ∈ Front_P(σ), BoundaryReq(x, σ) holds and ChannelDisabled(x, σ) holds (equivalently, BoundaryReq(x, σ) ∧ ¬ChannelEnabled(x, σ)), read operation-attributively through the owning message instance q = MsgOf_P(x). Note (ALL disabled ⇒ stall). If any frontier item x ∈ Front_P(σ) is channel-enabled at the chosen Σ-visible boundary, then P is not "blocked on message passing" for WFG purposes. Consequently, an implementation may not be physically stalled at the chosen Σ boundary by a cross-channel "join" predicate while some frontier item x ∈ Front_P(σ) at that boundary remains channel-enabled; any such conjunctive/join dependency must be represented as an explicit staged/bundle/join boundary channel so that, when the process is physically stalled, the relevant frontier items on that explicit boundary are all channel-disabled.

Definition (Drain-closed cutpoint for deadlock extraction). Let D be a non-empty set of processes. An ε-quiescent state σ is drain-closed for D iff no already-issued non-frontier blocking Push/Pop request of any process P ∈ D is channel-enabled at σ. Equivalently, after the frontier members Front_P(σ) are all disabled, there is no remaining already-issued downstream transfer of any process in D that can still commit as drain-only progress.
Here "already-issued downstream transfer" means an already-issued non-frontier blocking message-operation frontier item x ∈ Pending_P(σ) \ Front_P(σ), with owning message instance q = MsgOf_P(x), interpreted at the chosen abstract Sys_ref / RTL_B boundary and within the current source interval, i.e., the interval between the adjacent synchronization calls that bracket σ in P's current execution; hidden internal pipeline staging or later-iteration prefetch absorbed into B(c) is not a separate source-level operation for this definition unless it has been made an explicit abstract channel in Sys_B^elab.
If such a non-frontier drain transfer is enabled, σ is not yet a stable deadlock cutpoint for D; either that drain transfer must be allowed to commit finitely often until a drain-closed cutpoint is reached, or the resulting commit changes the WFG-relevant channel facts so that some frontier operation becomes enabled and the candidate deadlock disappears.

Finite drain note. At a blocked frontier satisfying the applicable drain-only blocked-frontier discipline, no new later blocking message-operation frontier item may be issued. Therefore the only downstream work that can still drain is work already accepted before the blocked condition. Because every explicit abstract channel has finite capacity B(c), every process has finite ε-depth D_P when not externally stalled, and no new work may cross the blocked frontier, the number of D-owned downstream drain commits before reaching a drain-closed ε-quiescent cutpoint is finite. A coarse bound may be taken as a function of the sum of the relevant in-flight occupancies on explicit abstract channels plus the finite per-process ε-depths D_P; the exact numeric bound is not used by Theorem K.1, only finiteness.

An internal message-passing deadlock is a reachable ε-quiescent state σ in which there exists a non-empty set of processes D such that σ is drain-closed for D and:

- Every process P in D is blocked on message passing, and in particular there exists at least one frontier item x ∈ Front_P(σ) on an internal message-passing channel c (i.e., a channel included in the WFG), with owning message instance q = MsgOf_P(x), such that BoundaryReq(x, σ) holds and ChannelDisabled(x, σ) holds.
- The vertices in D, together with the internal channels they access, form a strongly connected component in the WFG that has no outgoing edges (a closed cycle): processes in D can only wait on each other through internal WFG channels. (Members of D may additionally hold environment-facing frontier items, which are excluded from the WFG; see the Terminology note below.)

Intuitively, D is a set of processes that are waiting only on each other (via internal message-passing channels tracked by the WFG), with no remaining enabled drain-only transfer of any process in D that could change the WFG-relevant channel facts. Thus D cannot be unblocked by internal modeled message-passing progress represented in the closed WFG component, nor by already-issued downstream drain from within D. This is a reachable-state predicate, not a permanence claim: a member of D may also hold environment-facing frontier items (excluded from the WFG), and later environment/testbench activity under the fixed stimulus may dissolve such a state; the Appendix K results and the A5 premise quantify over reachability of these states, not over their permanence. A process that is blocked solely on channels whose complementary endpoint is the external environment/testbench (excluded from the WFG) does not witness an internal message-passing deadlock in this appendix.

Terminated-process convention. For purposes of Appendix K, termination is interpreted per reset epoch. After a process P emits FINISH_P for a given reset epoch, the infinite execution is padded only by process-local idle stutter for P for the remainder of that epoch. This padding creates no new P-owned Σ-events, no new BoundaryReq, no new Front_P or NextFront_P item, and no new outgoing WFG edge from P. This epoch-local padding does not preclude a later reset boundary affecting P. If such a reset occurs and P later exits reset, the subsequent START_P is a new dynamic occurrence that begins a fresh reset epoch, and the earlier FINISH_P has no ordering, anchoring, or WFG force across that reset boundary. Any later termination of P is represented by a distinct dynamic FINISH_P occurrence. Thus a process blocked waiting for a complementary Push/Pop from a peer that has terminated in the current reset epoch is not, by itself, an internal message-passing deadlock in the K.1 sense. It is instead outside the K.1 deadlock notion unless the design includes an explicit modeled termination protocol that makes such waiting part of the internal channel protocol being analyzed.

Terminology (adopted reading: reachable closed internal wait-cycle cutpoint). The predicate just defined is, precisely, a reachable, drain-closed, closed internal wait-cycle cutpoint. It is deliberately weaker than a conventional (permanent) deadlock: a member of D may hold environment-facing

frontier items excluded from the WFG, and — in a declared timed or reactive environment outside the B1-avail convention — a fair continuation in which the environment later enables such an item can leave the state. (Under the default B1-avail pull-based stimulus convention, an environment-facing input item that is disabled at a cutpoint is stimulus-exhausted and remains disabled forever, per Lemma K.EnvX-Auto in K.4b; the later-enablement possibility, and with it the input-side W.Env rescue class, arises only for declared timed/reactive environments.) Three consequences are adopted normatively throughout this appendix. (1) The property preserved by Theorem K.1 — absence of reachable states satisfying this predicate — is an invariance (safety-style) reachability property over admissible executions, not a temporal liveness property in the eventually/always sense; the historical word "liveness" is retained in titles for continuity but shall be read with this meaning. (2) The phrase "processes in D can only wait on each other" means, wherever it appears, "only wait on each other through internal WFG channels"; it does not exclude coexisting environment-facing waits. (3) The A5 premise is correspondingly conservative: it can be violated by a transient internal wait-cycle cutpoint even in a design whose fixed environment is guaranteed to release an environment-facing frontier item so that the system always progresses; such a report is a true positive for the defined predicate and a deliberate conservatism with respect to permanent deadlock. Designs for which this conservatism is unacceptable may instead be assessed against the stronger persistent-deadlock predicate — every blocking frontier item of every member of D internal, with its complementary owner in D, and the state a trap under all admissible continuations — but the theorems of this appendix neither require nor establish results for that stronger predicate; adopting it would require re-proving Lemmas K.1a–K.2b and Theorem K.1 with an added permanence argument.
Scope warning. This exclusion is only a scope convention for Theorem K.1. It does not imply that waiting forever for a terminated peer is benign or acceptable design behavior. If termination, EOF, done, cancellation, or end-of-stream behavior is part of the intended protocol, that behavior must be modeled explicitly in Sys_ref and RTL_B, or verified separately outside the Appendix K internal message-passing-deadlock theorem.

Our notion of the preserved property (historically labeled "liveness") in this appendix is: no reachable internal message-passing-deadlock state (reachable closed internal wait-cycle cutpoint) as defined above, under the fixed stimulus and fair scheduling. Formally this is an invariance (safety-style) reachability property over admissible executions — see the Terminology note above — and it does not assert that a reached deadlock state is permanent when environment-facing frontier items are present. (Starvation of a single process in an otherwise live system is ruled out separately by weak fairness.)
Implication note (relation to persistent deadlock). The reachable-cutpoint predicate implies absence of persistent internal communication deadlock: a K.1 component is internally closed, so no internal continuation can dissolve it, and escape requires environment action on a co-frontier item excluded from the WFG. The preserved property is therefore at least as strong as the operational guarantee that the design never hangs permanently on internal communication. The stronger reachability form is retained as the normative predicate deliberately: it is safety-shaped — an invariance over reachable states — which is what makes it checkable by inductive invariants (K.4e) and preservable through finite-prefix correspondence (Definition J.Reach, Corollary J.1). A persistence-only predicate would be liveness-shaped: its preservation would require continuation and fairness correspondence between RTL_B and Sys_ref, and its per-design certification would require co-reachability rather than reachability arguments. Weakening the predicate would make both the theorem and its discharge harder, not easier, while yielding a weaker guarantee that the present predicate already implies. Designs with intentional environment-rescued wait cycles are handled by the Waiver W.Env mechanism (K.2), not by weakening the predicate.

K.2 Assumptions and Imported Results

K.2.1 Standing Assumptions We assume throughout Appendix K:

A1. R-rules and B-rules. The prescribed source reference model Sys_ref and the post-HLS implementation RTL_B satisfy the process- and channel-level semantic rules defined earlier (R-rules and B-rules).

A2. Finite Pipeline Depth (FPD) / B4 finite ε-stutter. For each process P there is a finite bound D_P such that, along any consecutive ε-only evolution in which P is internally advancing toward its next observable Σ-action or its next Σ-action is locally enabled, and P is not externally stalled by a peer/environment/back-pressure condition, P executes at most D_P consecutive ε-steps before either (i) executing a Σ-action, or (ii) reaching a stable waiting state in which no further ε-step is available until some external/peer/environment condition changes the relevant enabling predicates. This is not a bound on time spent waiting for external stimulus, peer action, or back-pressure relief; such waiting is governed by WF/B2 and the channel-progress obligations B3. The system-level all-disabled quiescence bound used later in Appendix K is supplied by B5, with bound max_P D_P.

A3. Weak Fairness (WF) (B2). If an action remains continuously enabled (its guard remains true), the scheduler eventually selects it.

A4. Channel Progress (B3 (Additional Semantic Assumptions (B-rules)); Appendix N). For each channel c with capacity B(c), assume the progress invariants of Appendix N hold. This is an obligation on the scheduler/environment (e.g., fairness of arbiters/back-pressure) and is not implied by the local R-rules alone.

A5. Primary selected-instance Policy-S liveness premise (A5-S). For the selected Sys_K instance, fixed capacities B(c), explicit abstract boundaries and elaboration, fixed stimulus, fixed witness choices, and any Definition K.Low lowering, designate one complete maximal fair execution under Policy S. No reachable ε-quiescent, drain-closed cutpoint of that execution may satisfy the applicable serial bad-state predicate Bad_S^W. The general route uses Bad_S^W = Dead_KS^W. If K.EnvOrd and K.Done are discharged, the standard route uses Bad_S^W = Dead_K^W.

For A5, Policy S is the conservative serial-blocking interpretation of Appendix G. Within each source interval, a process does not issue a source-later blocking operation until every source-earlier blocking operation required by the serial discipline has committed in a strictly earlier cycle. The condition is process-facing: proof-internal staged/bundle/join/epoch-gate transfers do not satisfy it unless the elaboration identifies them as the commit that releases the source-level call. Channel-side E4 movement, FIFO drainage, independent-process execution, and explicit elaborated transfers remain concurrent.

Completeness and fairness of the selected run are part of A5. A finite simulation discharges A5 only when it reaches a declared maximal terminal state with no enabled non-stutter continuation; a finite prefix of a still-running system is insufficient. For a nonterminating design, the selected run may be covered by an inductive invariant, sound model checking, or another complete argument showing that Bad_S^W is never reached.

Instance dependence. A5 is not a property of the raw user-written SystemC model and is not discharged by an arbitrary ordinary simulation. It is re-established, or carried by an explicit parametric proof, whenever B(c), a chosen boundary, elaboration, lowering, fixed stimulus, witness assignment, or selected fair Policy-S scheduler changes.

Practical discharge route (Corollary K.2c). Lemma K.2b is universal over maximal fair Policy-S executions: if any admissible Policy-L execution reaches a non-waived internal K.1 deadlock, every complete maximal fair Policy-S execution reaches Bad_S^W. Its contrapositive makes one passing complete fair Policy-S run sufficient. No exploration over alternate issue policies or Policy-S scheduler interleavings, and no Policy-S confluence theorem, is required.

A source region whose liveness depends on favorable same-cycle or overlapped issue of multiple blocking operations does not pass the Policy-S route merely because a particular Policy-L or RTL schedule is live. It must pass the applicable Policy-S bad-state check, be rewritten or buffered, be placed under explicit protocol structure, or use a direct A5-L certificate outside the serial reduction.
A5 and Appendix K do not assert freedom from specification-level barrier errors, reset stalls, or external-environment stalls that do not contain the internal wait structure represented by the operative predicate.
A6. Point-to-point channels (single-producer/single-consumer). For each message-passing channel c (including buffered channels with B(c)>0 and rendezvous channels with B(c)=0), exactly one process performs all Push_c operations on c (the producer) and exactly one process performs all Pop_c operations on c (the consumer). Hence the complementary endpoint for any blocked Push_c/Pop_c is unique, and WFG edges are well-defined even for mixed WFGs that include both buffered and rendezvous channels.
If a design has a shared resource that would violate this uniqueness, it must be modeled via an explicit arbiter process plus point-to-point channels (as noted in “Common Formal Definitions” B(c)).
A7 (Determinism / witness selection / equal-KObs cutpoint observation and certificate correspondence). Assume B1 holds for RTL_B (post-HLS), so the Σ-labeled execution under the given stimulus is unique up to finite ε-stutter. Do not require B1 for Sys_ref: there may be multiple Sys_ref executions whose selected cutpoints have the same KObs as the RTL cutpoint. Appendix K uses the existence of one selected Sys_ref witness supplied by Corollary J.1 and consumes the typed equality KObsPost_J(ρ_post;E,w,H,O) = KObs_E,w(σ_ref) directly. J.OfferReach, J.OfferConf, and J.KObsConf must refer to that same selected RTL_B cutpoint, Sys_ref cutpoint, canonical history H, residual-offer set O, elaboration E, and witness w The selected source witness used by Appendix K shall also be admissible under Policy L.
When a deterministic, reproducible Sys_ref witness is desired for simulation/debug, apply the Appendix L snooping wrapper to select a particular witness aligned with the latency-sensitive ε-choices. B4 and B5 justify the selected ε-normalized endpoints required by Corollary J.1. Every Appendix K transfer of a reconstructed frontier, request, channel, enablement, WFG, drain, or deadlock predicate uses the same equal-KObs package. A discharge for an unrelated source cutpoint, different witness, different O, or noncanonical unmatched history is insufficient.
A8. Barrier well-formedness (SyncChannels). For each SyncChannel instance, the set of designated participant processes reaches the barrier the same number of times under the fixed stimulus/initial state; equivalently, no participant can permanently bypass a barrier call while another participant is waiting at that barrier. (Barrier stalls are therefore outside the behaviors considered in Appendix K’s internal message-passing deadlock analysis.)
A9. Single-transfer-per-cycle endpoint constraint (scalar channel interface). For every message-passing channel c, in any clock cycle t, at most one Push_c commit and at most one Pop_c commit may occur on c. If a design uses multi-lane/burst transfers on a logical resource, it must be modeled explicitly (e.g., k parallel point-to-point channels or a widened message) so that the Σ-level per-channel event model satisfies this constraint.
A10. Reset-empty FIFO assumption. After each reset boundary at which the compared execution begins or restarts, every buffered message-passing channel with B(c)>0 has logical occupancy 0 at the chosen Σ boundary. The Appendix J/K occupancy equations and the per-channel Push/Pop precedence lemma are interpreted relative to that reset-empty initial channel state.
A11. WFG-inert ε (Kε). The design satisfies Precondition Kε (WFG-inert ε / no hidden micro-timing control decisions) as stated in K.1.

No further standing assumption is introduced here beyond A1–A11. The conservative serial-blocking

source-liveness requirement stated in A5 is part of the selected-Sys_ref liveness obligation, not a separate theorem conclusion derived from the local R-rules or from the no-reverse distinct-interface scheduling rule.
Drain-only blocked-frontier discharge for Appendix K (K.Drain). The K.2b dominance/extraction bridge and K.1 drain-normalization arguments require the complete seven-clause discipline of Definition R.16 for every interval in which Policy L can leave source-later same-interval work outstanding behind a disabled earlier frontier. Pipelined or otherwise overlapped control discharges this through Appendix B automatic flush. Ordinary non-overlapped control may discharge it through a structural proof of the same discipline. In every case, the discharge must establish the blocked point, no new source-later dynamic-operation launch, request non-withdrawal, exclusion of non-frontier requests as WFG sources, drain-only downstream progress, finite non-replenishing B-faithful boundary storage or credit, and explicit sync-boundary / epoch-gate treatment for any withheld producer-facing capacity. This is the K.Drain application-specific discharge of the existing scheduling and channel assumptions, not an additional numbered A-assumption.

Premise A5-L (scheduler-robust liberal source liveness). No admissible Policy-L execution of the selected Sys_ref instance, under the fixed stimulus, capacities B(c), elaboration, and fixed witness-selected outcomes, reaches an ε-quiescent cutpoint satisfying Dead_K^W. Theorem K.1A consumes A5-L directly. Where synchronization waits or witness-selected non-NB-CF sites can participate in a potential wait cycle, the serial route uses the normalized blocking instance Sys_K defined in K.4b.
Policy-S-primary architecture. Lemma K.2b quantifies over every maximal fair Policy-S execution. Its contrapositive therefore makes one complete passing fair Policy-S execution sufficient; no Policy-S confluence theorem or universal externally supplied Policy-S premise is required. Example K.CE explains why the general Policy-S bad-state predicate includes internally chained stalls ending at starvation terminals. The optional K.EnvOrd/K.Done fragment, in its transitive may-follow starvation-cone form, reduces that check to the ordinary K.1 predicate; the one-hop form is insufficient (Example K.CE-2).
Normative organization of Appendix K. The spine is: the K.1 cutpoint/WFG predicate; Lemma K.0 as the source-local frontier reconstruction theorem; Lemma K.2 as WFG extensionality under equal KObs; Lemma K.2a as DrainClosedNow extensionality under equal KObs; Theorem K.1A from A5-L; the normalized instance Sys_K; the universal serial-dominance Lemma K.2b; Corollary K.2c; the primary A5-S Policy-S check; and Theorem K.1. Appendix K-P contains the operational companion proof, now expressed at terminal cutpoints through the same KObs/ResOffers interface. Its issue-fairness, reachability, no-new-launch, and owner-walk arguments remain operational rather than being reconstructed from KObs. Structural and direct certificates remain alternate A5-L routes rather than prerequisites for the primary route.
Waiver W.Env (declared environment-rescued cutpoints). The side-condition record may exclude a precisely identified cutpoint class when a declared environment-facing co-frontier is intended and proved to rescue it. The record shall identify the component class, rescuing item, environment assumption, and design justification. A waiver does not alter the default predicates and does not excuse an Example K.CE-style bypass without an accompanying no-bypass or equivalent argument.
Definition K.DeadW (waiver-parameterized predicates). Let W be the union of declared waived cutpoint classes. Dead_K^W(σ) means Dead_K(σ) and σ ∉ W. Dead_KS^W is defined analogously from Dead_KS. Bad_S^W denotes Dead_K^W in the K.EnvOrd standard route and Dead_KS^W in the general route. W is an explicit parameter of the mechanized theorem statements. Waiver closure under extraction (normative): because the serial obstruction extracted by Lemma K.2b.Q may differ from the liberal component, each W.Env entry shall either cover both the waived liberal class and every serial obstruction class derivable from it under the owner-walk extraction, or carry a recorded argument that extraction cannot produce a cutpoint of the waived class. KObs-facing waiver closure (normative):

because Theorem K.1A now transfers the waiver-parameterized predicate through equal KObs, W shall satisfy KObsClosed_I,E,w(W) whenever W is non-empty. This observational closure is separate from extraction closure. A waiver depending on historical Issue timing or implementation microstate not reconstructed by KObs is outside this route unless a separate closure proof is supplied.
Scope of the preservation theorem. Theorem K.1A transfers a reachable non-waived RTL_B internal deadlock to a reachable Policy-L Sys_ref deadlock through the common KObs record and contradicts A5-L. It consumes neither K.CV, NB-Align, K.Low, nor serial dominance. Theorem K.1 adds the serial-route premises and obtains A5-L from one complete fair Policy-S execution via Corollary K.2c. All RTL/tool-flow obligations, including J.OfferReach/J.OfferConf/J.KObsConf, K.Drain, Kε, B-faithfulness, scheduler/environment progress, and KObs waiver closure where applicable, remain explicit side conditions.
The no-reverse rule is not by itself a liveness theorem. Its practical force is captured by the Policy-S reduction: Policy S withholds source-later requests and is used as the conservative deadlock-stress execution. The serial theorem does not claim that Policy S reproduces the Policy-L trace or the same deadlock component; it claims that any Policy-L internal deadlock forces every complete fair Policy-S execution to reach some prohibited serial obstruction.
Example: blocking rendezvous operations whose liveness depends on same-cycle/eager issue. Consider two rendezvous channels a and b with B(a)=B(b)=0 and two processes:

```
P1: b.Pop(); a.Push(v)
P2: a.Pop(); b.Push(w)
```

Under Policy S's conservative serial-blocking issue discipline, each process issues only its first source-order blocking operation: P1 waits on Pop_b and P2 waits on Pop_a; neither process reaches the complementary Push, so the system has a closed internal rendezvous WFG deadlock. If, instead, each process uses an admissible eager Policy-L issue schedule that has both distinct-interface blocking requests asserted by the same cycle while preserving source order, then both rendezvous transfers can commit in that cycle. The Policy-S/A5-S discharge route therefore cannot establish the Appendix K guarantee for this example unless it is rewritten, buffered, explicitly synchronized, or otherwise made to pass the route-selected Dead_KS/Dead_K check. A direct Policy-L proof, structural proof, or another recorded alternate A5-L discharge could still be used. The example also illustrates the role of K.2b: the Sys_ref cutpoint obtained from the RTL_B witness may contain overlapped pending requests, whereas A5-S is checked on a complete maximal fair Policy-S execution.

The Front/NextFront bridge used later in Appendix K is the source-local reconstruction result of Lemma K.0. Cross-model frontier, raw rendezvous-request, enablement, and WFG agreement are obtained by applying the same reconstruction functions to the equal KObs record of Corollary J.1. No asserted rendezvous request is identified with frontier membership merely because it is present in O.

K.2.2 Lemma K.0 — Frontier Reconstruction from KObs

Purpose. Appendix K forms WFG edges from the minimal pending blocking frontier. The KObs layer already separates the candidate source frontier reconstructed from H from the currently asserted attributed offers reconstructed from O. This lemma identifies the operational source frontier with the corresponding pure reconstruction and states the cross-model consequence by equal-KObs extensionality.
Statement. Fix I, E, w and a reachable ε-quiescent Sys_ref cutpoint σ, and let K = KObs_E,w(σ). For process P define

BlockingMsgNextFrontRec_P(K) := { x ∈ NextFrontRec_P(K) | x ∈ Ω_msg,P and MsgOf_P(x) is a blocking Push or Pop instance }.
Then
Front_P(σ) = FrontRec_P(K) = { x ∈ BlockingMsgNextFrontRec_P(K) | BoundaryReqRec(x,K) }.
Here NextFrontRec is witness-relative over the fixed Ω_P and ≤_P supplied by E; it is a candidate frontier, while BoundaryReqRec selects the currently outstanding blocking offers.
Cross-model consequence. Let K_post = KObsPost_J(ρ_post;E,w,H,O) and K_ref = KObs_E,w(σ_ref) be the exact records selected by Corollary J.1. If K_post = K_ref, then for every process P, FrontRec_P(K_post) = FrontRec_P(K_ref), and BoundaryReqRec(x,K_post) = BoundaryReqRec(x,K_ref) for every typed blocking message-operation frontier item x. Thus the reconstructed post frontier is exactly the operational source frontier at σ_ref. This consequence uses only the common record; it does not compare historical Issue times or raw RTL control state.
Mixed blocking/non-blocking clarification. Successful non-blocking transfers may contribute committed Push/Pop events to H, and failed polls may contribute WC-selected ε outcomes, but neither creates a pending blocking frontier member. A blocking endpoint may nevertheless wait on a complementary process whose source endpoint uses a non-blocking call; the non-blocking side can change occupancy or complete the complementary transfer, but does not itself become a WFG source unless it also owns a separate pending blocking frontier.
Proof. For a reachable source cutpoint, Lemma R.21d identifies Pending and BoundaryReq with PendingRec and BoundaryReqRec and therefore identifies Front_P with FrontRec_P. The displayed BlockingMsgNextFrontRec characterization is the Definitions R.14–R.15 classification of the ≤_P-minimal pending blocking items; source-prefix issue order, synchronization anchoring, and the exclusion of failed polls ensure that no earlier unrealized nonblocking, signal, or synchronization item invalidates the candidate-frontier characterization. The cross-model conclusion follows by substituting K_post = K_ref into the pure reconstruction functions. □
Corollary K.0a (Rendezvous endpoint-request extensionality). For a point-to-point rendezvous channel c with B(c)=0, define ReqProdRec_c(K) and ReqConsRec_c(K) as the producer-direction and consumer-direction projections of O. If K_post = K_ref as above, then ReqProdRec_c(K_post) = ReqProdRec_c(K_ref) and ReqConsRec_c(K_post) = ReqConsRec_c(K_ref). This raw endpoint-request equality is independent of frontier membership: an already-asserted non-frontier blocking offer remains represented in O and can participate in rendezvous enablement.

K.2.3 Example K.0b — Front_P vs. BlockingMsgNextFront_P Example

The following example illustrates the distinction between Front_P and BlockingMsgNextFront_P. In this example all message-operation items under discussion are blocking Push/Pop operations, so BlockingMsgNextFront_P is the relevant WFG-facing slice of the source-level NextFront_P frontier over Ω_P. The broader notation MsgNextFront_P, when used elsewhere, denotes the general message-operation-item slice and may also include successful non-blocking transfer items; it is not the object used by the Front/NextFront bridge for WFG construction.

```
void P() {
  while (1) {
    wait();          // sync point
    int x = in.Pop();   // blocking Pop
    out.Push(x + 1);    // blocking Push
  }
}
```

Consider two ε-quiescent cutpoints for this same process:

Cutpoint σ0: just after wait() has committed, before in.Pop() has issued
· BlockingMsgNextFront_P(σ0) = { Pop_in }
· BoundaryReq(Pop_in, σ0) = false
· Front_P(σ0) = ∅
Cutpoint σ1: in.Pop() has issued, rdy is asserted, but no data has arrived yet
· BlockingMsgNextFront_P(σ1) = { Pop_in }
· BoundaryReq(Pop_in, σ1) = true
· Front_P(σ1) = { Pop_in }
So the difference is:
· BlockingMsgNextFront_P says which blocking message-operation items are next candidates in the source-level frontier over Ω_P for WFG/frontier classification.
· Front_P says which of those next blocking message-operation items are actually outstanding pending blockers right now.

K.3 Lemma K.1a — Characterization of Internal Message-Passing Deadlock (Buffered and Rendezvous Cases)

Elaboration prerequisite. The channel boundary c used in this lemma is the chosen abstract boundary of the prescribed Sys_ref / RTL_B comparison after applying any required elaboration from Appendix J, Remark 2. In particular, synchronization-boundary ramp-down / epoch-gating is not treated as a hidden exception to E4 on an otherwise ordinary buffered channel. If such gating can withhold producer-facing availability while outer projected capacity would otherwise appear available, the relevant blocking point is represented by an explicit abstract channel of Sys_B^elab.

Statement: Let σ_post be a reachable ε-quiescent global state of the post-HLS system RTL_B (equivalently, the ε-normalization of some reachable state, which exists as a finite extension by B4/B5). Suppose there is a non-empty set of processes D that is internally message-passing deadlocked (as defined in K.1). Then**:**
Every process P in D is blocked on a blocking channel operation (Push or Pop), not on an internal step.
Buffered-channel case (B(c) > 0):
• If P is blocked on a frontier item x ∈ Front_P(σ_post), with owning message instance q = MsgOf_P(x), kind(q)=Push_c, and B(c)>0, then occ_c(σ_post) = B(c) (channel is full at that cutpoint).
• If P is blocked on a frontier item x ∈ Front_P(σ_post), with owning message instance q = MsgOf_P(x), kind(q)=Pop_c, and B(c)>0, then occ_c(σ_post) = 0 (channel is empty at that cutpoint).
Rendezvous case (B(c) = 0):
• If P is blocked on a frontier item x ∈ Front_P(σ_post), with owning message instance q = MsgOf_P(x), kind(q)=Push_c, and B(c)=0, then the complementary endpoint is not simultaneously requesting the matching Pop_c in σ_post (i.e., the complementary Pop_c endpoint request is not true in that cycle, so rendezvous enabling for x is false).
• If P is blocked on a frontier item x ∈ Front_P(σ_post), with owning message instance q = MsgOf_P(x), kind(q)=Pop_c, and B(c)=0, then the complementary endpoint is not simultaneously requesting the matching Push_c in σ_post.
The processes in D form a closed strongly connected component in the WFG.

Proof:
Internal Steps: Because σ_post is ε-quiescent (i.e., an ε-normalized endpoint that exists as a finite extension by B4/B5), no process has any enabled internal ε-step in σ_post. In particular, if some process had an internal step whose guard is true in σ_post, then σ_post would not be ε-quiescent: the ε-

normalization could be extended by at least one additional ε-step. Therefore, no process in D is blocked on an internal step with a true guard; since each P ∈ D is blocked on message passing (K.1), the blocking operation must be a blocking channel operation (Push or Pop).
Blocked Push: By the definition of "blocked on message passing" in K.1, if P is blocked due to a frontier item x ∈ Front_P(σ_post), with owning message instance q = MsgOf_P(x) and kind(q)=Push_c, then BoundaryReq(x,σ_post) is true and ChannelEnabled(x,σ_post) is false.
• If B(c)>0, the channel-side enabling condition for x is space availability, i.e., occ_c(σ_post) < B(c). Hence, if P is blocked on such a Push_c frontier item x in σ_post, necessarily occ_c(σ_post)=B(c) (channel is full).
• If B(c)=0 (rendezvous), the channel-side enabling condition for x is that the complementary Pop_c endpoint request is true in the same cycle. Since P is blocked, this rendezvous enabling condition is false; equivalently, the complementary endpoint is not simultaneously requesting Pop_c in σ_post.
Blocked Pop: Symmetric. By the definition of "blocked on message passing," if P is blocked due to a frontier item x ∈ Front_P(σ_post), with owning message instance q = MsgOf_P(x) and kind(q)=Pop_c, then BoundaryReq(x,σ_post) is true and ChannelEnabled(x,σ_post) is false.
• If B(c)>0, the channel-side enabling condition for x is data availability, i.e., occ_c(σ_post) > 0. Hence occ_c(σ_post)=0 (channel is empty).
• If B(c)=0 (rendezvous), the channel-side enabling condition for x is that the complementary Push_c endpoint request is true in the same cycle. Since P is blocked, this rendezvous enabling condition is false; equivalently, the complementary endpoint is not simultaneously requesting Push_c in σ_post.
Closed WFG cycle: By the definition of internal message-passing deadlock, every WFG edge from a process in D points to a process in D. Hence no process in D waits, through the literal WFG edge relation, on a complementary endpoint owner outside D.

K.4 Lemma K.2 — WFG Extensionality under Equal KObs

Statement. Let ρ_post and σ_ref be the selected ε-quiescent RTL_B and Sys_ref cutpoints of one Corollary J.1 application, and write
K_post := KObsPost_J(ρ_post;E,w,H,O),
K_ref := KObs_E,w(σ_ref).
If K_post = K_ref, then
WFGRec(K_post) = WFGRec(K_ref).
Equivalently, the K-facing reconstructed post wait-for graph and the operational source wait-for graph have exactly the same vertices and directed edges. The statement applies uniformly to buffered and rendezvous boundaries and to Sys_B^elab explicit staged, bundle, join, and epoch-gate boundaries supplied by E.
Proof. WFGRec is a typed pure function of K. FrontRec and BoundaryReqRec are reconstructed from O by Lemma K.0/R.21d; buffered channel state is reconstructed from H and E by Lemma R.21c; rendezvous complementary requests are direction projections of O by Corollary K.0a; ChannelEnabledRec/ChannelDisabledRec and the edge relation are then reconstructed by Lemma R.21e. Substitution of K_post = K_ref therefore gives graph equality by functional congruence. No separate case split over full, empty, or rendezvous channels and no clause-by-clause state correspondence is required. Lemma K.1a remains the local semantic characterization of what a disabled frontier means at a concrete boundary, but is not needed to transfer the graph. □

K.4a Lemma K.2a — DrainClosedNow Extensionality under Equal KObs

Statement. Let K_post and K_ref be the equal records of Lemma K.2, and let D be a non-empty process set. Then

DrainClosedNowRec(D,K_post) = DrainClosedNowRec(D,K_ref).
Consequently, if the reconstructed post cutpoint is drain-closed for D in the K.1 snapshot sense, the selected operational source cutpoint σ_ref is drain-closed for D; the converse snapshot implication also holds.
Proof. DrainClosedNowRec(D,K) is the pure predicate that no residual offer owned by D and lying outside FrontRec is ChannelEnabledRec. All of its arguments are fields or reconstruction functions of K, so equality follows by functional congruence from K_post = K_ref and Lemmas R.21d–R.21f. This lemma transfers only the cutpoint predicate. It does not transfer, erase, or reconstruct the continuation-level Definition R.16/K.Drain discipline, which remains an operational premise for the executions used by Appendix K and Appendix K-P. □

K.4b Universal Serial-Source Dominance and the Primary Policy-S Route

Organization. The serial route first normalizes the selected instance to explicit point-to-point blocking structure, then proves a universal dominance statement: a Policy-L internal deadlock implies that every maximal fair Policy-S execution reaches the general serial obstruction Dead_KS. This universal conclusion, rather than Policy-S confluence, is what permits one complete passing Policy-S run to discharge the user-facing premise. The optional static conditions K.EnvOrd and K.Done are only simplifiers that let the implementation check the ordinary K.1 predicate instead of Dead_KS.
Definition K.BF (blocking-only fragment). Every wait relevant to the WFG is a persistent blocking Push/Pop at an explicit point-to-point boundary with finite B-faithful capacity; control, payloads, and later operation identities are deterministic functions of committed causal history, fixed stimulus, and fixed witness choices under K.CV; shared arbitration is remodeled as explicit point-to-point subchannels through an arbiter whose choices are deterministic or witness-fixed. Bounded local progress: computation between message operations terminates, so at every point each process either normally completes, reaches its next message operation, or is blocked at one; a process that diverges in pure computation is outside the fragment. Lemma K.2b.O consumes this assumption, and a mechanization shall state it as a fragment axiom.
Definition K.Low and normalized instance Sys_K. Let Λ_W(Sys_ref) be the witness-indexed lowering that represents synchronization by explicit epoch-gate channels and every aligned non-NB-CF non-blocking dependency by a fresh blocking guard channel. The default guard has B(g)=1 with ordinary non-fall-through E4 timing; the finite skew after supplier commit is WFG-inert and cannot remain at a drain-closed cutpoint. Define Sys_K ≜ Λ_W(Sys_ref) where lowering is needed, and Sys_K ≜ Sys_ref otherwise. All serial-route predicates below are evaluated on Sys_K.
Lemma K.Low.C (normalization correspondence) — statement. Projection erasing guard/epoch transitions maps executions of Sys_K to executions of the original selected instance with the same external Σ history; witness-consistent executions lift to Sys_K; and internal wait cutpoints correspond to literal blocking wait cutpoints of Sys_K. The local proof obligations are per lowering rule and are collected in Appendix K-P.
Example K.CE (why the general serial predicate is stronger than K.1). Let P execute Pop_e; Push_c(v0), where e is an exhausted environment input and Push_c is independent of Pop_e. Let downstream processes use c and then form an internal wait cycle. Policy L may issue Push_c while Pop_e remains pending and reach the downstream internal cycle. Policy S withholds Push_c and instead reaches a chain of internal waiters ending at exhausted e. Thus absence of a Policy-S internal cycle alone does not imply A5-L.
Definition K.StimX. Under convention B1-avail, the next fixed input item is available as soon as earlier items on that environment channel have been consumed. An environment-facing Pop is stimulus-exhausted exactly when no item remains; an environment-facing Push is exhausted only under a declared permanent-stop contract, and never under the default always-ready output environment.

Definition K.WFG+ and K.DeadKS (general serial obstruction). Extend the ordinary wait-for graph with starvation-terminal nodes of two kinds. Environment terminals: one node EnvStop_c for each environment-facing boundary c whose next required interaction is permanently unavailable under Definition K.StimX; if blocked process P has an environment-facing disabled frontier on c and that frontier is stimulus-exhausted, add P → EnvStop_c. Completion terminals: one node DoneStop_c for each internal point-to-point channel c whose unique complement endpoint process has normally completed; if blocked process P has a disabled internal frontier on c and the actual unique complement owner of that frontier has normally completed, add P → DoneStop_c — the awaited operation lies beyond the completed owner's dynamic operation sequence and, by the completion-permanence property of Lemma K.2b.O, commits in no admissible execution. Ordinary internal dependencies between blocked processes remain process-to-process edges P → Q. At a reachable ε-quiescent, drain-closed Policy-S cutpoint σ, Dead_KS(σ) holds when K.WFG+(σ) contains a nonempty closed blocked component D+ that contains at least one process-to-process edge. The ordinary predicate Dead_K is the special case in which D+ contains no EnvStop and no DoneStop node. Requiring at least one process-to-process edge excludes a process that is merely idle at a starvation terminal with no internal waiter. Without the DoneStop extension the predicate is unsound as a serial check: Example K.CE-4 below exhibits a Policy-L-reachable internal cycle for which every Policy-S execution stalls in a chain ending at a normally completed producer, a stall that no environment terminal and no closed process component captures. Benign end-of-test states in which a terminated producer leaves a looping sink waiting are DoneStop chains of the same kind as EnvStop chains and are dispositioned identically: testbench restructuring so that sinks terminate, review, or a recorded Waiver W.Env-style entry satisfying the Definition K.DeadW closure requirement.

Definition K.Cone, Condition K.EnvOrd (optional static collapse condition — may-follow form), and Condition K.Done. K.Cone is a finite graph over blocking operation sites or process-boundary schemas, not an enumeration of unbounded dynamic loop instances. Define the may-follow relation over the sites of one process: site q may-follows site b when q is textually source-later than b, or q and b lie on a common control-flow cycle — in a shared loop, a later iteration's instance of any body site is dynamically source-later than a pending instance of any other body site, so both textual orders must be covered. Seed Cone with every environment-facing blocking site and — unless Condition K.Done below is discharged instead — with every possibly-under-supplied internal blocking site, one whose demand is not statically proved to be covered by the counterpart endpoint's total production. Close Cone transitively: an internal blocking site b enters Cone when the unique complementary supplying site for b may-follows a Cone site of the counterpart process. Condition K.EnvOrd holds when every internal operation site q of a process that may-follows a Cone site b of the same process and can feed an internal wait component is control- or data-dependent on the commit or witness-selected outcome of the dynamically preceding pending instance of b; operationally, a pipelined loop containing a Cone site shall gate iteration advance on that site's commit for every independent feeding operation. By K.CV and WC, this static dependence implies the dynamic no-bypass property consumed by Corollary K.EnvOrd-1: no dynamic instance of q can issue while a dynamically source-earlier instance of b remains pending. Caution (normative): the site-level source-later form without may-follow is insufficient — Example K.CE-3 defeats it through a loop-carried crossing — as is the earlier one-hop form (Example K.CE-2). Condition K.Done (completion sufficiency): every process that can normally complete is completion-audited — for every internal channel with a possibly-terminating endpoint, the terminating endpoint's total dynamic operation count is statically proved to cover every demand of the counterpart endpoint — or every process is perpetual. K.Done makes completion terminals (DoneStop) unreachable; without it the Corollary K.EnvOrd-1 collapse fails at completion-rooted starvation (Example K.CE-4), which can arise with no environment operation anywhere in the starving chain. These conditions are optional because the general Policy-S theorem checks Dead_KS directly.

Example K.CE-2 (transitive starvation — insufficiency of one-hop no-bypass). Let e be an environment input channel with empty stimulus, and let c_E, c_V, a, b be internal point-to-point channels. E: Pop_e; Push_cE. W: Pop_cE; Push_cV(v0), with v0 independent of the Pop_cE payload. V: Pop_cV; Push_a; Pop_b. P2: Push_b; Pop_a. Only E has an environment-facing operation, so the one-hop rule binds only E: E never issues Push_cE while Pop_e pends, and c_E is therefore supplied in no execution. W's pending earlier blocker Pop_cE is internal, so the one-hop rule is vacuous for W: an admissible Policy-L schedule lets W issue and commit the payload-independent Push_cV while Pop_cE remains pending; V receives it, V issues Push_a, P2 issues Push_b, neither complementary Pop is launched before the frontiers form, and Definition R.16 then forbids launching them — so {V, P2} is a reachable, non-waived, single-frontier internal K.1 cycle and A5-L fails. Under Policy S with the same stimulus, Push_cV is withheld behind Pop_cE, which never fills, so every maximal fair Policy-S execution stalls in the chain P2 → V → W → E → exhausted environment, with internal wait edges but no internal cycle: Dead_K is never reached, and the one-hop collapse to Dead_K would certify a genuinely deadlocking design. The general predicate Dead_KS does flag this stall. Under the transitive Condition K.EnvOrd, Pop_cE lies in the environment-starvation cone — its supply Push_cE is source-later than the cone operation Pop_e — so W's crossing is forbidden in every admissible Policy-L execution, the liberal cycle is unreachable, and the check and the premise agree.

Example K.CE-3 (loop-carried bypass — insufficiency of site-level source order). Let e and c_E be as in Example K.CE-2, with E: Pop_e; Push_cE(f(e)) fully dependent and therefore compliant, and let W now be the pipelined loop W: loop { Push_cV(v0); Pop_cE }, with v0 independent of the Pop_cE payload. The site Pop_cE correctly enters the cone — its supplying site Push_cE is source-later than the seed Pop_e — but the site Push_cV is textually earlier than Pop_cE, so a site-level source-later rule imposes no dependence on it, and the site-level condition passes. Dynamically, ordinary loop overlap issues iteration i+1's Push_cV while iteration i's Pop_cE remains pending — a dynamically source-later crossing, admissible under Policy L when issued before the frontier settles — feeding V and P2 exactly as in Example K.CE-2: the internal cycle {V, P2} is Policy-L-reachable and A5-L fails, while every Policy-S execution withholds iteration i+1's Push_cV behind the serial frontier Pop_cE of iteration i and stalls in the chain to EnvStop_e, never reaching Dead_K. The may-follow rule of Definition K.Cone covers the pair: Push_cV may-follows Pop_cE through the shared loop, the required dependence fails for the constant payload, and the repaired Condition K.EnvOrd correctly rejects the instance.

Example K.CE-4 (completed-producer starvation — necessity of DoneStop terminals). Let Z: Push_c(w); end produce exactly one item on internal channel c and normally complete in every execution, and let Y: Pop_c; Pop_c; Push_d(v0) demand two, with v0 independent; V: Pop_d; Push_a; Pop_b and P2: Push_b; Pop_a as before. By completion permanence (Lemma K.2b.O), Y's second Pop_c awaits an operation beyond Z's dynamic sequence and commits in no admissible execution. Under an admissible Policy-L schedule, Y issues the independent Push_d before its frontier settles and commits it; V and P2 then form a reachable, non-waived, single-frontier internal K.1 cycle, so A5-L fails. Under Policy S, Push_d is withheld behind the second Pop_c, and every execution stalls in the chain P2 → V → Y → DoneStop_c. Without the DoneStop extension no closed component exists at that stall — the owner of Y's internal dependency is completed, not blocked — so the previous Dead_KS was unsound as a serial check; with DoneStop the component {P2, V, Y} with its terminal edge is closed, contains process-to-process edges, and Dead_KS fires. The example also defeats the previous Lemma K.2b.O claim that normal completion before the required complement is excluded, and it shows the Corollary K.EnvOrd-1 collapse needs Condition K.Done: with no environment operation anywhere in the starving chain, the cone is unseeded while the completed producer starves it.

Lemma K.EnvX-Auto (environment co-frontiers). Under B1-avail and the default always-ready output environment, every environment-facing frontier item held by a blocked member of a K.1 component is stimulus-exhausted. Proof. Blockedness requires every frontier item to be channel-disabled. A remaining

input item or ready output environment would enable the environment-facing item. Hence no separate K.EnvX premise is needed. Nonstandard timed/reactive environments require a recorded substitute argument or direct A5-L verification. Scope emphasis (normative): B1-avail is a genuine environment-model restriction, not a bookkeeping convention — it asserts that stimulus availability is consumption-ordered and untimed. A testbench that delivers stimulus at declared clock times (for example, a configuration token released at a fixed cycle) is outside B1-avail: under timed delivery an environment-facing item can be disabled now and enabled later, this lemma does not apply, and the instance shall either declare the timed channels and supply a substitute exhaustion argument or discharge A5-L by a non-serial route.

Lemma K.2b (universal serial-source dominance). Fix Sys_K, the same fixed stimulus, capacities, elaboration, environment model, and witness choices. Assume A1-A4 with issue fairness, A6-A11, K.CV, point-to-point ownership, nonwithdrawable blocking requests, Kε, and the applicable Definition R.16/K.Drain discipline. If some admissible Policy-L execution reaches an ε-quiescent, drain-closed cutpoint σ_L, write K_L := KObs_E,w(σ_L). If K_L satisfies DeadKRec^W(K_L,I,W) — equivalently, under the recorded R.21f waiver-closure premises, σ_L is a non-waived internal K.1 deadlock cutpoint — then every maximal fair Policy-S execution of the same selected instance has a finite prefix reaching an ε-quiescent, drain-closed cutpoint σ_S. Writing K_S := KObs_E,w(σ_S), the general route satisfies DeadKSRec^W(K_S,I,W), equivalently Dead_KS^W(σ_S) under the recorded environment-terminal and waiver premises. The optional static Conditions K.EnvOrd (may-follow form) and K.Done imply, through Corollary K.EnvOrd-1, that K_S instead satisfies the ordinary DeadKRec^W predicate. This is only a terminal-interface restatement: the dominance proof still reasons operationally about Issue, Commit, fairness, and continuation-level DrainDiscipline.

Proof status. Appendix K-P gives the identity-aware progress-dominance and stuck-state extraction proof, with terminal liberal and serial facts exposed through K_L and K_S. The remaining mechanization obligations are the local K.Low.C rules, K.2b.P1 determinacy, the explicit serial-prefix request-reachability lemma used uniformly for buffered and rendezvous transfers, the missing-supply-owner lemma used by the owner walk (with its completion-permanence clause), the issue-fairness reading K-O1, the separate multi-frontier composition lemma K-O2, the may-follow cone construction and dynamic no-bypass derivation K-O6, the completion-sufficiency audit K-O7, and the KObs/ResOffers drain-bag interface and descent theorem K-O8. The former K.EnvC refinement, K-O3, K.Conf, scalar-progress arguments, and Petri-net equivalence claims are not part of the primary theorem stack.

K.4c Corollary K.2c — One Complete Policy-S Run Suffices

Definition K.BadS. The primary general Policy-S route uses Bad_S^W ≜ Dead_KS^W. If the optional static Conditions K.EnvOrd and K.Done are discharged, Corollary K.EnvOrd-1 permits the equivalent simplified check Bad_S^W ≜ Dead_K^W. The selected predicate and the K.EnvOrd / K.Done evidence, if used, are recorded in the side-condition discharge record.

Premise A5-S (primary Policy-S check). There exists one designated complete maximal fair Policy-S execution of Sys_K under the fixed stimulus and witness choices such that no reachable ε-quiescent, drain-closed cutpoint on that execution satisfies Bad_S^W. A concrete discharge is recorded in the CF-S payload of the A5 certificate package defined in K.4e.

Corollary K.2c (Policy S is a sufficient worst-case check). Under Lemma K.2b's premises, A5-S implies A5-L. Proof. If A5-L were false, Lemma K.2b would force every maximal fair Policy-S execution, including the designated A5-S execution, to reach Bad_S^W, a contradiction. Therefore no Policy-L internal deadlock is reachable. □

Definition K.CompleteS and practical completeness rule. A complete Policy-S execution is a maximal fair execution of the selected transition system: a terminating execution must reach a declared maximal terminal state, while an indefinitely running execution requires invariant, model-checking, recurrence,

lasso, or other sound completeness evidence. The CF-S certificate shall make this evidence machine-checkable or cite a separately checkable proof artifact. A finite regression prefix of a nonterminating system is supporting evidence but is not a proof. Different capacities, elaborations, witnesses, stimuli, environment contracts, reset policies, waiver sets, or transition semantics define different selected instances and require separate certificate headers. Merely choosing a different fair scheduler or interleaving within the same Policy-S transition system does not define a new instance and does not require another run, because Lemma K.2b is universal over all maximal fair Policy-S executions.
No confluence requirement. Corollary K.2c does not require Policy-S executions to have identical traces, limits, or terminal WFG facts. The reason one run suffices is stronger and simpler: if a Policy-L deadlock existed, every maximal fair Policy-S execution would encounter a prohibited obstruction.

K.4d Structural Sufficient Conditions for A5-L

The theorems of this subsection are alternate direct discharges of A5-L. They do not use Policy S, Lemma K.2b, or Corollary K.2c. They remain useful for nonterminating systems, designs outside the serial fragment, or flows that prefer a compact structural certificate.
Definition K.Dep (full static dependency graph). G_dep is the directed graph over the processes of the selected instance with an edge P → Q for every way P can, at some reachable cutpoint, hold a disabled blocking frontier item whose actual unique complement owner is Q: a data edge from the consumer to the producer of every internal channel; a capacity (back-pressure) edge from the producer to the consumer of every finite-capacity internal channel; both edges for every rendezvous channel; both directions between synchronization partners through their epoch-gate channels; and, for every arbiter process, edges in both directions between the arbiter and each client along the remodeled point-to-point sub-channels. Caution (normative): G_dep shall be computed on this full relation. The data-flow topology alone is insufficient: a feed-forward data graph with finite capacities, synchronization, or arbitration can still contain dependency cycles through capacity, epoch-gate, or arbiter edges.
Theorem K.Str-1 (acyclic dependency). If G_dep is acyclic, then A5-L holds for the selected instance.
Proof. At every reachable ε-quiescent cutpoint under every admissible issue policy, every literal K.1 WFG edge is an instance of a G_dep edge, by the E4 evaluation of ChannelDisabled at point-to-point boundaries and the K.1 edge definition; a WFG cycle would therefore yield a G_dep cycle. ■
Theorem K.Str-2 (ranked dependency). If there is a function r from processes (or from process–boundary pairs) to a well-ordered set such that every G_dep edge realizable as a WFG edge at a reachable cutpoint strictly decreases r, then A5-L holds, since a WFG cycle would yield an infinite descent in r. Edges provably unrealizable at reachable cutpoints may be excluded from the obligation, with each exclusion proved as an invariant of the selected instance and recorded in the side-condition discharge record. ■
Theorem K.Str-3 (credit / initial-token invariant). If every cycle of G_dep contains at least one edge that is unrealizable as a WFG edge at any reachable cutpoint — for instance, an edge of a channel with a proved occupancy invariant $0 < occ_c < B(c)$ whenever both endpoints are pending, as arranged by an initial token or a credit protocol — then A5-L holds. ■ Each such invariant is itself a proof obligation over the selected instance, including its capacities and elaboration, and shall be recorded in the side-condition discharge record.

K.4e Certificate and Discharge Formats (Normative)

A concrete theorem application uses one versioned certificate package. The package has one common instance header, may contain one or more CF-0 KObs cutpoint certificates for the Corollary J.1 / Theorem K.1A boundary, and contains one A5-L discharge payload: the primary CF-S Policy-S route, a direct Policy-L proof reference, or one of CF-1 through CF-5 below. A certificate evaluated directly over

Policy L targets Dead_K^W. A certificate evaluated over the Policy-S transition relation uses Corollary K.2c and targets the recorded Bad_S^W. No canonical/confluence reduction is assumed.
Definition K.CertHdr (common certificate header). The header binds every subordinate artifact to one theorem instance. It records: (1) certificate-schema, document, theorem, and semantic-model versions; (2) stable identities or digests for the source, selected Sys_ref/Sys_K, generated RTL when relevant, and any wrapper/snooping model; (3) E, Chan_elab(E), B_E, Π_elab/π_Σ,E, explicit observation and proof boundaries, point-to-point ownership, and Λ_W/K.Low lowering where applicable; (4) fixed stimulus, environment contract, reset policy and epochs, witness w and WC provenance, waiver set W with KObs-closure/extraction-closure evidence as applicable, and route r in {General, EnvOrdDone}; (5) the selected theorem scope and references to every required side-condition discharge; and (6) generator/checker versions, validation status, and the declared trusted computing base. Two artifacts with different header identities are evidence for different selected instances and shall not be combined without an explicit equivalence proof.
CF-0 — KObs cutpoint correspondence certificate. This certificate supports Corollary J.1 and Theorem K.1A; by itself it does not discharge A5-L. It records or reproducibly commits to: (1) the K.CertHdr identity; (2) the selected source and RTL prefixes/cutpoints and the finite ε-normalization evidence; (3) the canonical history H, including committed dynamic identities, values, per-channel ordinals, reset markers, atomic observable units, required-order information, and WC-selected outcomes retained by CanonHist; (4) the fully typed residual-offer set O with owner, frontier item, operation identity/kind, explicit boundary, direction, and reset epoch; (5) J.HCanon and J.OfferReach evidence; (6) the complete physical RelevantReqSet_E(ρ_post), ReqProjection_E(O), their equality, unique attribution, endpoint-cardinality, prefix-closure, and no-hidden-request evidence required by J.OfferConf; and (7) B-faithfulness/E4, reset-alignment, and ε-opacity evidence required by J.KObsConf. H or large logs may be stored by digest only when the checker can retrieve and replay the committed artifact. Derived process slices, Front, BoundaryReq, enablement, WFG, WFG+, DrainClosedNow, DeadKRec, DeadKSRec, and BadSRec are recomputed by the checker. A complete historical Issue timeline is not a required CF-0 field; the operational source witness referenced by J.OfferReach must nevertheless contain enough Issue/Commit evidence to validate reachability, E3/E5, persistence, and any triggered K.Drain premise.
CF-S — Primary Policy-S execution certificate. The payload records: (1) the K.CertHdr identity and route tag PolicyS; (2) the selected Policy-S transition relation, initial state, fixed witness/stimulus/environment, capacities, elaboration, lowering, waivers, and Bad_S route; (3) one designated execution artifact or a symbolic representation of it; (4) evidence that the execution is maximal and fair and satisfies the applicable B2/B3/B5, K.Drain, Kε, and nonwithdrawal requirements; (5) sound completeness evidence under Definition K.CompleteS; and (6) a check that every reachable ε-quiescent drain-closed cutpoint on that execution avoids the route-selected BadSRec predicate, reconstructed from its KObs rather than accepted as a logged Boolean. The payload also cites K.CV, NB-Align, K.Low, K.Low.C, point-to-point, K.PSF/multi-frontier, K-O1/K-O2/K-O6/K-O7/K-O8, and other Lemma K.2b applicability evidence as triggered. A valid CF-S payload discharges A5-S and, through Corollary K.2c, A5-L.
Definition K.A5Cert (A5-L discharge package). The package consists of K.CertHdr plus exactly one declared route payload: CF-S; a proof reference for K.Str-1; CF-1; CF-2; CF-3; CF-4; CF-5; or a direct Policy-L invariant/exploration/proof artifact. The package records the target predicate and route, and shall expose enough data for an independent checker to validate the route without trusting a tool-emitted conclusion string. Policy-S route packages target Bad_S^W and include the dominance applicability evidence; direct Policy-L packages target Dead_K^W. When environment terminals or waivers are used, the package also carries EnvTerminalAdequate and KObsClosed/extraction-closure evidence as applicable.

CF-1 — Rank certificate. A function r from processes (or process–boundary pairs) of the selected instance to a declared well-ordered set, together with the Definition K.Dep edge enumeration, such that every realizable edge strictly decreases r (Theorem K.Str-2). Checker obligations: the edge enumeration covers data, capacity back-pressure, rendezvous, epoch-gate, and arbiter edges of the selected instance including its elaboration and lowering; every non-excluded edge strictly decreases r; every excluded edge carries a recorded invariant proof of unrealizability.

CF-2 — Credit / cycle-breaking certificate. For each cycle of the full dependency graph G_dep, provide an invariant—such as token presence, occupancy bounds, or a circulating-credit invariant—establishing that at least one edge of the cycle is unrealizable as a WFG edge at any reachable cutpoint (Theorem K.Str-3). Checker obligations: the dependency-cycle enumeration is complete; each invariant is inductive over the selected instance; and the proved edge exclusion applies at the explicit capacities and elaborated boundaries. Petri-net or siphon analyses may be used by a tool as one way to generate this certificate, but no Petri-net equivalence theorem is part of normative Appendix K.

CF-3 — Inductive-invariant certificate. Supply an abstraction map α from the selected operational transition system into a finite abstraction A. The observation component of A shall be KObs, or an explicitly proved quotient sufficient to reconstruct the same Bad predicate. A may additionally carry only the auxiliary operational state needed to soundly define Post — for example issue-eligibility phase, fairness/ranking state, or drain-discipline control that KObs deliberately omits. Supply an invariant Inv such that α(Init) is contained in Inv, Post_A(Inv) is contained in Inv, and Inv has empty intersection with BadRec. Here BadRec = DeadKRec^W for Policy L and BadRec = BadSRec for Policy S. Checker obligations: alpha and Post_A soundly overapproximate the selected operational relation; every Bad fact is reconstructed from the observation component and ambient instance data; the abstraction includes the reset/environment/waiver distinctions needed by the selected predicate; witness-selected non-blocking outcomes are source-describable through NB-CF or K.Low; and all invariant inclusions are mechanically validated. Equality of KObs records alone shall not be treated as transition equivalence or as proof of future Issue legality, fairness, or DrainDiscipline.

CF-4 — Exploration certificate. Record an exhaustive or soundly bounded model-checking result over the selected Policy-L relation establishing unreachability of DeadKRec^W, or over the selected Policy-S relation establishing unreachability of BadSRec, with K.CertHdr, the explored transition-system identity, abstraction map, cutpoint normalization rule, tool/version, bound or completeness justification, fairness interpretation, and coverage argument. The checker shall recompute the bad predicate from KObs at explored canonical cutpoints. A bound without a proof that it is complete for the claimed result is supporting evidence only.

CF-5 — Tool-emitted certificate. Any CF-S or CF-1 through CF-4 emitted by the HLS tool or a companion tool over the selected instance may exploit the tool's knowledge of the scheduled control graph, stage allocation, actual FIFO and skid-buffer capacities, request ownership, join and arbitration structure, automatic-flush implementation, and hidden storage or credits. It shall use K.CertHdr and the applicable CF-0/A5 payload schema and be validated by an independent checker whose trusted base is recorded. The checker shall validate structure, evidence references, and reconstructed K-facing predicates, not merely the certificate generator's final claim. Labeling rule (normative): a certificate stated over the generated RTL itself, rather than over the selected Sys_ref instance, is admissible evidence of deadlock-freedom for that design but bypasses the Appendix K preservation architecture — it is output verification, not a discharge of A5-L — and shall be labeled as such in the record.

Soundness restriction (normative). Admissible certificates are invariant-shaped. Fairness- or eventuality-based cycle-breaking arguments — for example, “the arbiter eventually grants a member of the cycle” or “the epoch gate eventually opens an escape transition” — do not discharge A5-L: A5-L prohibits reaching the closed wait-cycle cutpoint, not merely remaining in it forever, and an eventual rescue does not prevent the cutpoint from being reached. Such arguments bear only on the weaker

persistent-deadlock property, which the normative predicate already implies (see the K.1 implication note); where a design's justification is genuinely of this form, the Waiver W.Env mechanism, not a certificate, is the recorded instrument.

K.4f The Policy-S Standard Fragment (K.PSF)

Definition K.PSF (Policy-S standard applicability fragment). A selected instance is in K.PSF when: (1) it is in normalized blocking form Sys_K; (2) evaluated channels are point-to-point with explicit finite B-faithful capacities; (3) K.CV holds and arbiters are deterministic or witness-fixed; (4) potential deadlock members are single-frontier, or a separately certified multi-frontier extension is supplied; (5) blocking requests are nonwithdrawable; and (6) Kε and the applicable Definition R.16/K.Drain discipline hold. K.PSF contains model and semantic applicability conditions only; completeness and fairness evidence for the designated Policy-S execution belongs to A5-S, not to the fragment definition.
Theorem K.PSF-1. For a K.PSF instance, one designated complete maximal fair Policy-S execution that reaches no non-waived Dead_KS cutpoint establishes A5-L and permits Theorem K.1. If the optional static Conditions K.EnvOrd and K.Done are also discharged, Corollary K.EnvOrd-1 permits the execution to be checked only for ordinary non-waived internal K.1 cutpoints. Proof. Apply Lemma K.2b and Corollary K.2c; use K.EnvOrd-1 only for the optional predicate collapse. □ This theorem inherits the proposed-reduction status of Lemma K.2b pending mechanization.
Intent. K.PSF is the recommended standardized applicability target for new latency-insensitive designs. The general user-facing evidence is the conservative Policy-S execution checked against Dead_KS. K.EnvOrd is an optional static simplifier for common data-dependent streaming designs; structural certificates remain alternate evidence, especially for indefinitely running systems or instances outside the single-frontier fragment.

K.5 Theorems K.1A and K.1 — Reachable-Cutpoint Deadlock-Freedom Preservation ("Liveness Preservation")

Theorem K.1A (core preservation from the scheduler-robust premise). Statement:
Assume A1–A4 and A6–A11, the Appendix I/J correspondence prerequisites including WC, the exact J.OfferReach/J.OfferConf/J.KObsConf package yielding equal KObs for the compared cutpoints, the K.Drain discharge of Definition R.16 where it triggers, point-to-point evaluated boundaries, and Kε. If W is non-empty, also assume KObsClosed_I,E,w(W). If the scheduler-robust premise A5-L holds for the selected Sys_ref, then RTL_B cannot reach an internal message-passing-deadlock cutpoint classified as non-waived by the KObs-facing waiver predicate of the selected theorem instance. With W = ∅, RTL_B is internally message-passing-deadlock-free. The theorem consumes neither K.CV, NB-Align, nor Lemma K.2b; those attach only to the serial-reduction route of Theorem K.1.
Proof (by contradiction).

1. Assume RTL_B reaches a finite ε-quiescent cutpoint ρ_post whose K-facing reconstructed observation K_post satisfies DeadKRec^W(K_post,I,W). By Definition K.Dead and the reconstruction interface, this includes a non-empty blocked closed WFG component and DrainClosedNow for that component. The asserted RTL requests and channel facts used in this observation are covered by J.OfferConf and B-faithfulness/E4; no unrecorded raw RTL request may be omitted.
2. Apply Corollary J.1 to this same finite prefix and certificate package. It supplies a reachable ε-quiescent Sys_ref cutpoint σ_ref and K_ref = KObs_E,w(σ_ref) such that K_post = K_ref. J.OfferReach ensures that σ_ref is the endpoint of the selected admissible source witness rather than an arbitrary or unrealizable record.

3. By Lemma K.0, Lemma K.2, and Lemma K.2a — equivalently, by functional extensionality of the reconstruction functions — DeadKRec^W(K_ref,I,W) has the same truth value as DeadKRec^W(K_post,I,W). When W is non-empty, KObsClosed_I,E,w(W) makes the waiver predicate well-defined on the common observation, and Lemma R.21f identifies the reconstructed source predicate with Dead_K^W(σ_ref). Hence Dead_K^W(σ_ref) holds.
4. The source witness selected by Corollary J.1/J.OfferReach is admissible under Policy L for the fixed stimulus, elaboration, and witness choices: Policy L permits the source-order-respecting outstanding-offer pattern represented by O, while the operational Issue, request-persistence, and K.Drain rules remain in force. Thus σ_ref is a reachable Policy-L cutpoint prohibited by A5-L.
5. This contradicts A5-L. Therefore RTL_B cannot reach a non-waived internal K.1 deadlock cutpoint. □

Mechanization note. The proof core is now the finite-prefix Corollary J.1 construction, equality of two typed KObs records, pure reconstruction of Dead_K^W, and the Policy-L reachability supplied by J.OfferReach. No separate graph-edge case split, broad state relation, or clause-by-clause transfer of frontier, request, occupancy, and drain facts remains in Theorem K.1A. The continuation-level K.Drain premise is retained and is not derived from KObs.

Theorem K.1 (user-facing Policy-S preservation). Assume the premises of Theorem K.1A and the serial-route conditions of Lemma K.2b: K.CV; NB-Align and K.Low where applicable; point-to-point evaluated boundaries; nonwithdrawable requests; Kε; and the applicable Definition R.16/K.Drain discipline. If A5-S holds for the selected Sys_K—one complete maximal fair Policy-S execution avoids Dead_KS^W—then RTL_B cannot reach a non-waived internal K.1 deadlock. When K.EnvOrd and K.Done are discharged, the A5-S run may instead be checked for Dead_K^W by Corollary K.EnvOrd-1. Proof. Corollary K.2c gives A5-L; apply Theorem K.1A. □ Until the Appendix K-P companion obligations are mechanized, this theorem retains proposed-reduction status (Appendix U.13).

Discharge routes for A5-L (normative summary). Primary: a CF-S certificate and Corollary K.2c from one complete maximal fair Policy-S execution checked against Dead_KS^W. Optional simplification: Corollary K.EnvOrd-1, under the static Conditions K.EnvOrd and K.Done, reduces that check to Dead_K^W. Alternatives: K.Str-1; CF-1 through CF-4 over Policy L or Policy S with the appropriate reconstructed bad predicate; a CF-5 tool package; or a direct Policy-L proof artifact. Every route is wrapped by K.A5Cert and cites one K.CertHdr. The record shall identify the selected normalized instance, elaboration/lowering, witness, stimulus, environment/reset model, predicate, completeness/fairness or inductiveness evidence, waivers, and checker/trust boundary.

Appendix K-P – Proofs Companion for the Serial-Reduction Route

Status. This companion collects the serial-reduction proof at document rigor: Lemma K.Low.C; Definition K.Id; Lemma K.2b.P1; the serial-prefix request-reachability lemma; Lemma K.2b.P; the missing-supply-owner lemma; Lemma K.2b.Q Steps 1-4; the optional K.EnvOrd collapse corollary; the proof of Lemma K.2b; and optional Lemma K.Ser. The terminal-cutpoint notation and proof interface use KObs: σ_L and σ_S are represented by K_L and K_S, current issued-but-uncommitted requests by the attributed residual-offer component O, and finite drain work by the DrainBag projection defined below. The operational proof rules and theorem strength are unchanged. K.Conf is not part of the theorem stack: one complete Policy-S run suffices by the universal quantification of Lemma K.2b, not by confluence. Proof status: Lemmas K.2b.P1, K.2b.R, K.2b.P, K.2b.O, K.Ser, and K.Low.C are believed complete at document rigor and are the first mechanization targets; Lemma K.2b.Q carries obligations K-O1 (issue fairness), K-O2 (multi-frontier composition), and K-O8 (KObs/ResOffers drain-bag interface and descent), and Corollary K.EnvOrd-1 additionally requires K-O6 (may-follow cone and no-bypass derivation) and K-O7 (completion-sufficiency audit). Until these are discharged, Lemma K.2b, Corollary K.2c, Theorem K.1, and Theorem K.PSF-1 retain proposed-reduction status; the structural and direct A5-L routes are independent of all of them.

K-P KObs interface convention. For every reachable ε-quiescent source cutpoint σ used below, write K_σ := KObs_E,w(σ) = ⟨E,w,H_σ,O_σ⟩, where O_σ = ResOffers_E,w(σ). For K = ⟨E,w,H,O⟩, use owner(o), item(o), op(o), boundary(o), dir(o), and epoch(o) for the projections of the attributed residual-offer tuple o = (P,x,q,c,dir,e) of Definition R.18b. Write Proc(I) for the finite process set of the selected instance. For a process set D define Offers_D(K) := {o ∈ O | owner(o) ∈ D}; FrontOffers_D(K) := {o ∈ Offers_D(K) | item(o) ∈ FrontRec_owner(o)(K)}; and DrainBag_D(K) as the finite multiset of offers o ∈ Offers_D(K) whose item lies outside FrontRec_owner(o)(K). DrainCandidates_D(K) is the submultiset of DrainBag_D(K) whose item is ChannelEnabledRec at K. CmtRec_P(K), PushOrdRec_c(K), and PopOrdRec_c(K) denote the committed-identity and per-endpoint ordinal projections of H. These names are pure snapshot selectors; at an operational cutpoint K_σ they agree with the corresponding source facts by Lemmas R.21b-R.21f.

Operational and elaboration boundary. The graph and residual-offer arguments in this appendix range over the full E-typed frontier items and tuples, including explicit Sys_B^elab staged, bundle, join, guard, and epoch-gate boundaries. When the proof uses an operation symbol f_P, g_P, or q, it abbreviates MsgOf_P(x) only after the corresponding typed item x and boundary have been fixed; the item identity, boundary, direction, epoch, and owner remain part of the formal object. KObs deliberately omits historical Issue times and does not determine future issue legality. Definition R.16/K.Drain remains the continuation-level DrainDiscipline used to forbid replenishment past a blocked frontier. DrainClosedNowRec and DrainCandidates are only cutpoint predicates. Channel-side E4 movement inside the selected elaboration is handled by B4/B5 ε-normalization and is not duplicated as a source residual offer.

Lemma K.Low.C (normalization correspondence) — restated with proof. For the same fixed stimulus and witness choices: (i) every admissible execution of Λ_W(Sys_ref) projects, by erasing guard-channel and epoch-gate transitions, to an admissible execution of Sys_ref with the same issue-policy class (S or L), the same committed message/synchronization/observation history, and the same Σ projection; (ii) every admissible execution of Sys_ref in which each witness-selected site takes its W-selected outcome lifts to an admissible execution of Λ_W(Sys_ref) in which each guard Pop commits within a finite WFG-inert ε-delay of the commit of its modeled obstruction — in the default B(g_s) = 1 form, in the cycle after that commit once the guard request is asserted; exactly at that commit only in a declared atomic rendezvous variant; (iii) a reachable ε-quiescent cutpoint of Λ_W(Sys_ref) contains an internal K.1 deadlock if and only if its projection is a reachable ε-quiescent cutpoint of Sys_ref containing a closed

internal wait cycle in the extended sense — a non-empty process set closed under waiting through internal message frontiers, infeasible NB-aligned selected outcomes, and pending explicit synchronizations. Proof. Parts (i) and (ii) are per-rule stuttering simulations: the added transitions touch only the fresh channels; by NB-Align and K.CV, W's selected outcome at each lowered site is a deterministic function of the commit of its modeled obstruction, so the guarded process's post-guard behavior is identical in both models; clause (7) already requires synchronization refusal to be represented as ChannelDisabled at an explicit boundary, so the synchronization/epoch-gate lowering rule is representational rather than semantic; point-to-point ownership and E3 prefix closure identify the unique g_s producer; Kε keeps the added microsteps WFG-inert. The guard skew of the default B(g_s) = 1 form is absorbed by the same stuttering simulations: during the skew the guard Pop is channel-enabled with its unique producer already committed, so it is WFG-inert and is a drain candidate, and therefore no ε-quiescent drain-closed cutpoint analyzed by part (iii) contains a half-completed guard; an exactly-at reading would be inconsistent with the non-fall-through E4 semantics for positive-capacity channels. For (iii), left to right: a member blocked at a guard or epoch-gate frontier projects to a process whose W-selected outcome or synchronization is infeasible, and the unique complement owner of that frontier projects to the modeled supplier, so a lowered literal WFG cycle projects to a closed wait cycle in the extended sense; right to left: each frozen or waiting node of an extended cycle is, in the lowered instance, message-blocked at its guard or epoch-gate frontier with its complement owner the image of its modeled supplier, so the extended cycle is a literal all-message-blocked K.1 cycle in Λ_W(Sys_ref). ■ Mechanization note. Lemma K.Low.C is the single successor to the former obstruction-pointer conversion obligation. It shall be discharged per lowering rule — one case lemma for the epoch-gate rule and one for the guard rule — as a local correspondence at each lowered site, not as a global argument over terminal cutpoints. Part (iii) is the load-bearing claim; a development that mechanizes only the blocking-only Lemma K.2b without K.Low.C(iii) covers only designs natively in the blocking-only fragment.

Definition K.Id (committed-identity state and serial frontier). For an execution prefix ρ and process P, Cmt_P(ρ) is the set of dynamic source-operation identities of P that have process-facing committed in ρ, and Push_c(ρ), Pop_c(ρ) are the per-endpoint commit ordinals at channel c. At a terminal cutpoint σ with K_σ = ⟨E,w,H_σ,O_σ⟩, write CmtRec_P(K_σ), PushOrdRec_c(K_σ), and PopOrdRec_c(K_σ) for the same facts reconstructed from H_σ. Because operations on a single message-passing interface issue and commit in source order (R4/E3 with FIFO transfer order and point-to-point ownership), the per-channel per-endpoint commit ordinal of any fixed dynamic operation is the same in every admissible execution in which it commits; per-channel counts therefore identify committed transfers by identity through Lemma K.2b.P1. Across distinct interfaces, however, Policy L may commit a source-later operation while a source-earlier operation remains uncommitted, so Cmt_P(ρ_L) need not be a source-order prefix, and a scalar commit count identifies neither the committed set nor the frontier under Policy L. Under Policy S, Cmt_P(ρ_S) is always a source-order prefix of P's dynamic operation sequence; in the blocking-only fragment every commit advances the frontier, and SerFront_P(ρ_S) — P's serial frontier — is its source-minimal uncommitted blocking operation, with every source-earlier operation committed. All cross-policy progress comparisons remain statements over operational prefixes, committed-identity sets, and per-channel ordinals; the reconstructed KObs projections are used only when a terminal cutpoint is named. Scalar counts are consumed only as inequalities (Lemma K.2b.P). The earlier identification of a process's liberal frontier as operation prog_P + 1 was unsound and remains retired.

Lemma K.2b.P1 (dataflow determinacy) — restated with proof. In the blocking-only fragment, for the fixed instance, stimulus, and witness choices: for every channel c and ordinal n, the payload of the n-th committed transfer at c, and for every process P and index k, the identity of P's k-th dynamic source operation, are the same in every admissible execution — of either policy — in which they occur. Proof. By strong induction on the causal order of commits. By K.CV, P's k-th operation identity and its payload

expressions are deterministic functions of the values P observed at its committed operations of index below k, the fixed stimulus, and the fixed witness choices. The value observed at P's j-th pop of c is the payload of the transfer of a fixed ordinal at c, by FIFO order and point-to-point ownership; that payload is, by the induction hypothesis, the same in every execution in which it occurs. Environment payloads are fixed by B1, and arbiter selections are deterministic or witness-fixed by Definition K.BF. ■ This is the Kahn-determinacy fact for the fragment, and it is the instance of K.CV actually consumed below. At any named terminal cutpoint, the determined committed identities, payload ordinals, and process-local replay facts are exactly the corresponding projections of H in its KObs record; this observation does not replace the operational induction.

Lemma K.2b.R (serial-prefix request reachability). Immediately before a first alleged Policy-S overtake of a liberal comparison execution, every source operation committed before the current Policy-S operation has also committed, by dynamic identity, in the liberal execution. K.CV therefore places the liberal owner at or beyond the same dynamic operation. Because Policy L permits every issue allowed by Policy S, issue fairness eventually asserts the corresponding endpoint request, and nonwithdrawal keeps it asserted until commit. This lemma supplies the owner-side request fact used uniformly in the buffered-Pop, buffered-Push, and rendezvous cases of Lemma K.2b.P.

Lemma K.2b.P (serial progress dominance) — restated with proof. Fix the selected instance Sys_K under the fixed stimulus and witness choices, a finite admissible Policy-L prefix ρ_L, and any maximal fair Policy-L extension ρ_L∞ of ρ_L within that same instance (no stimulus truncation is used). Then for every maximal fair Policy-S execution ρ_S of the same instance under the same fixed stimulus and witness choices, every channel c, and every process P, the limits satisfy push_c(ρ_S) ≤ push_c(ρ_L∞), pop_c(ρ_S) ≤ pop_c(ρ_L∞), and prog_P(ρ_S) ≤ prog_P(ρ_L∞). Proof. Lemma K.2b.R first establishes that the corresponding liberal owner has reached and persistently issued the same dynamic operation; the remaining case analysis establishes channel-side enablement. Suppose not, and consider the first commit step of ρ_S that takes some count strictly beyond its ρ_L∞ limit; say it commits the transfer of ordinal n at channel c (the per-process claim is the projection of the per-channel claims onto the owning endpoints). Immediately before that step, every count in ρ_S is within its ρ_L∞ limit, and by Lemma K.2b.P1 the operations and payloads committed so far in ρ_S agree, ordinal for ordinal, with their occurrences in ρ_L∞. The step is enabled in ρ_S, and each enabling condition transfers to ρ_L∞. For a buffered Pop of ordinal n: enabledness gives push_c ≥ n in ρ_S, hence push_c(ρ_L∞) ≥ n by the first-violation choice, so the ordinal-n pop is data-enabled in ρ_L∞ from some finite point on, and point-to-point ownership makes that enabledness persistent. For a buffered Push of ordinal n at capacity B(c): enabledness gives pop_c ≥ n − B(c) in ρ_S, hence in ρ_L∞, so space for the ordinal-n push is persistently available in ρ_L∞. For a rendezvous transfer of ordinal n: both complementary ordinal-n requests are asserted in ρ_S, each owner having committed its source prefix below the request; by Lemma K.2b.P1 and the first-violation choice the same per-process prefixes commit in ρ_L∞, whose owners therefore reach and — Policy L never being more restrictive than Policy S about issuing a reached blocking request — assert the same non-withdrawable ordinal-n requests. In every case the ordinal-n transfer at c is, from some finite point of ρ_L∞ on, persistently enabled with the required endpoint requests asserted; by fairness (A1–A4, B2/B3) and non-withdrawal, ρ_L∞ commits it, so the ordinal-n count at c lies within the ρ_L∞ limits — contradicting the choice of a first violating step. ■ Note. The lemma formalizes the monotone-enabler character of blocking requests in this fragment: asserting a request can only enable transfers, never disable them, so the conservative serial policy can never overtake the liberal comparison run. This is the semantic reason Policy S is the worst case; contrast resource-acquisition settings, where eager acquisition rather than conservative acquisition is the deadlock-prone policy. The per-process inequality prog_P(ρ_S) ≤ prog_P(ρ_L∞) is consumed below only as an inequality; it is never used to identify which operations committed or where a frontier lies (Definition K.Id). The proof

compares two executions of one shared instance and nowhere uses truncation of the stimulus; issue fairness (obligation K-O1) supplies the reached-implies-eventually-issued step of each case.
Lemma K.2b.O (missing-supply owner trichotomy). Let x be a typed blocking frontier item at an explicit boundary c and per-channel ordinal n in a maximal fair Policy-S execution, and let y be the actual unique complementary item at that same boundary and ordinal, owned by process Y; write q := MsgOf_Y(y). Suppose the ordinal-n transfer never commits. Under bounded local progress (Definition K.BF) exactly one of the following holds. (i) Stuck: Y has uncommitted operations. In the blocking-only fragment every commit of Y advances its serial frontier, and only finitely many operations of Y precede q's position, so Y's typed frontier is eventually constant at a source-minimal uncommitted blocking item z_Y, with g_Y := MsgOf_Y(z_Y); were z_Y ever persistently ChannelEnabledRec at a stabilized KObs cutpoint, fairness would commit it, so Y is permanently non-advancing and blocked there. Moreover z_Y is not y: a supplying Push toward the empty channel of a blocked Pop (occ = 0 < B(c)), a supplying Pop from the full channel of a blocked Push (occ = B(c) > 0), and a rendezvous with both complementary requests asserted are each channel-enabled and would commit; hence g_Y is strictly source-earlier than q. (ii) Completed: Y normally completes, with q beyond its dynamic operation sequence. Completion permanence: by K.CV and Lemma K.2b.P1, a process's dynamic operation sequence is a deterministic function of its consumed payloads, which are execution-independent ordinal for ordinal; a process that normally completes in one admissible execution commits, in every admissible execution, a prefix of that same sequence or all of it, and never more — so y does not arise within the completed sequence, q commits in no admissible execution, and the waiter's typed frontier is unfillable in every admissible execution. (iii) Compute-divergent: excluded by the bounded-local-progress clause of Definition K.BF. Case (i) supplies each item-labelled process-to-process owner-walk edge of Lemma K.2b.Q Step 3, and case (ii) supplies each DoneStop terminal.
Lemma K.2b.Q (universal stuck-state extraction) — Statement. Assume the premises of Lemma K.2b, and let the finite admissible Policy-L prefix ρ_L reach the ε-quiescent, drain-closed cutpoint σ_L. Write K_L := KObs_E,w(σ_L) = ⟨E,w,H_L,O_L⟩ and suppose DeadKRec^W(K_L,I,W) holds, witnessed by a non-empty process component D; equivalently, σ_L contains the corresponding non-waived internal K.1 deadlock under Lemma R.21f's recorded premises. Fix any maximal fair Policy-S execution ρ_S of the selected instance Sys_K under the same fixed stimulus and witness choices. Then ρ_S has a finite prefix ending at a reachable ε-quiescent, drain-closed cutpoint σ_S. Writing K_S := KObs_E,w(σ_S) = ⟨E,w,H_S,O_S⟩, the general route satisfies DeadKSRec^W(K_S,I,W), witnessed by some non-empty component D_S; if the static Conditions K.EnvOrd and K.Done hold, K_S satisfies DeadKRec^W(K_S,I,W) instead (Corollary K.EnvOrd-1). Because ρ_S is arbitrary, the conclusion is universal over maximal fair Policy-S executions, which is the form consumed by Corollary K.2c. Throughout, over-time comparisons are stated over operational committed-identity sets and per-channel per-endpoint ordinals (Definition K.Id); terminal frontier, request, channel, graph, and drain-closure facts are read from K_L or K_S. The fairness premises A1-A4 include issue fairness — a reached source operation whose issue preconditions are persistently satisfied is eventually issued; this reading is obligation K-O1 and shall be confirmed against the A1-A4 statements during mechanization.
Proof, Step 1 (joint frontier freeze and permanent serial blockage of D). Let K_L = ⟨E,w,H_L,O_L⟩. For P ∈ D in the single-frontier case, let x_P^L be the unique K.1-relevant typed item in FrontRec_P(K_L), and write f_P := MsgOf_P(x_P^L). Because WFGRec(K_L) carries an internal edge from P, x_P^L is internal. Source-prefix closure of the issued summary, together with the exact residual-offer set O_L, shows that every operation of P source-earlier than f_P committed by σ_L: otherwise its attributed request would also occur in O_L and would precede x_P^L in FrontRec, contradicting minimality. Let n_P be the per-channel per-endpoint ordinal of x_P^L at BoundaryOf_P(x_P^L), fixed across executions by same-interface source order and the H_L ordinal projections. Extend ρ_L to one maximal fair Policy-L execution ρ_L∞ of the selected untruncated instance in which no f_P, for any P ∈ D, ever commits — a

single joint extension, not one extension per item. Admissibility of the joint freeze: (a) each x_P^L stays disabled. Enabling it requires the typed complementary item y_Q at the same boundary and ordinal, owned by the actual unique complement process Q ∈ D. O_L contains no attributed offer for y_Q: if it did, that pair would be an enabled buffered drain candidate or an enabled rendezvous transfer at σ_L, contradicting DrainClosedNowRec(D,K_L) or the disabledness of x_P^L. The operation MsgOf_Q(y_Q) is source-later than Q's own frontier item x_Q^L, so the continuation-level Definition R.16 DrainDiscipline forbids newly asserting y_Q while f_Q remains pending; the frozen frontiers therefore preserve one another as an invariant of the extension. This use of R.16 is not an inference from DrainClosedNowRec. (b) Each environment-facing frontier item held by a member of D is stimulus-exhausted at σ_L by Lemma K.EnvX-Auto under B1-avail and the default output environment, hence permanently disabled in every admissible continuation; no stimulus truncation is used. (c) Each already-present non-front offer in DrainBag_D(K_L) may later become enabled through outside-D or environment activity and, under fairness, commit as finite drain-only progress. R.16 forbids inserting new source-later offers past the blocked frontier, nonwithdrawal preserves each current offer until commit, and the finite O_L therefore bounds each member's remaining drain work; by (a), no such drain commit supplies a frozen channel. The extension is fair because no frozen x_P^L is ever persistently enabled. Consequently the ρ_L∞ limit at x_P^L's channel on P's side is strictly below n_P. By Lemma K.2b.P applied to ρ_S and ρ_L∞ over the same instance, ρ_S never commits the ordinal-n_P transfer. Under Policy S every operation of P source-later than f_P is withheld behind it, so each P ∈ D becomes permanently non-advancing, blocked at a typed serial frontier x_P^S with g_P := MsgOf_P(x_P^S), g_P ≤_src f_P, and committed identities equal to the source prefix strictly below g_P. Multi-frontier elaborated members are handled itemwise over FrontRec_P(K_L) and then composed; this is obligation K-O2.

Proof, Step 2 (stabilization at a serial cutpoint). Let NA be the set of processes permanently non-advancing in ρ_S; D ⊆ NA by Step 1. Every channel with an endpoint in NA receives only finitely many further commits: if the consumer is in NA its pops are finite, so pushes are bounded by pops + B(c); if the producer is in NA, pops are bounded by its finite pushes. Hence the KObs-reconstructable boundary facts for NA are eventually confined to finitely many changes induced by the finite already-present drain offers: FrontRec and the blocked requests stabilize, while ChanRec occupancy, O, and the relevant H ordinals can change only when one of those offers commits. Choose a stabilization cutpoint σ_0 and write K_0 = KObs_E,w(σ_0). The measure is the finite multiset M_0 := DrainBag_NA(K_0), not a global remaining-work count. Along the subsequent drain-normalization segment, Definition R.16/K.Drain is the operational non-replenishment premise: no NA member asserts a new source-later offer past its stabilized frontier, so the corresponding DrainBag never increases; nonwithdrawal ensures an existing entry disappears only when its process-facing operation commits; and each commit of an enabled member of DrainCandidates_NA strictly removes one entry. B4/B5 provide finite ε-normalization between such commits, while channel-side E4 movement within the elaboration is handled as ε behavior and does not create a new source-attributed offer. Therefore the multiset descent terminates after finitely many drain commits. Let σ_S be a subsequent ε-quiescent cutpoint and K_S := KObs_E,w(σ_S) at which the stabilized NA facts hold and DrainCandidates_NA(K_S) is empty. Then DrainClosedNowRec(D',K_S) holds for every candidate component D' drawn from NA; processes outside NA may retain ongoing activity, because recurring ε-quiescence at a cycle boundary does not require global termination. This finite ResOffers-derived multiset is the termination measure and replaces the retired global remaining-work count, which was not provably finite because an autonomous internal loop can commit infinitely many transfers with no further stimulus. Obligation K-O8 is the mechanized proof that the operational R.16 discipline implies non-increase of DrainBag and that each selected drain commit strictly decreases it.

Proof, Step 3 (stuck walk with starvation terminals). Call a process stuck when it is permanently non-advancing in ρ_S and, at K_S, has a typed item x_X ∈ FrontRec_X(K_S) with

ChannelDisabledRec(x_X,K_S); under bounded local progress (Definition K.BF) there is no other permanent stall, and a normally completed process is not a walk node. Each member of D is stuck at its stabilized serial item x_P^S by Step 1. For a stuck process X: (internal case) x_X is internal. Let y be the actual unique complementary typed item at the same explicit boundary and ordinal, let Y be its owner, and let q := MsgOf_Y(y). That ordinal never commits in ρ_S, so Lemma K.2b.O applies. In case (i), Y is itself stuck at a typed serial frontier z_Y whose operation g_Y is strictly source-earlier than q; add the item-labelled owner-walk edge X → Y, which is exactly the corresponding process edge of WFGPlusRec(K_S,I). In case (ii), Y has normally completed, y lies beyond its dynamic sequence, and WFGPlusRec adds X → DoneStop_c. (environment case) x_X is environment-facing. Every operation of X source-earlier than MsgOf_X(x_X) committed in ρ_S, so its stimulus ordinal is reconstructed from H_S; if the fixed stimulus still contained that item, the asserted offer in O_S would be persistently enabled under B1-avail, point-to-point ownership, and issue fairness, and fairness would commit it. Hence EnvStopRec_c(K_S,I) holds and WFGPlusRec adds X → EnvStop_c. Single-frontier members of D never carry terminal edges at their own stabilized frontier: an environment-facing g_P <_src f_P would have committed at σ_L by Step 1's all-below-f_P fact and would subsequently commit under fairness; a DoneStop edge at an internal g_P <_src f_P is impossible because that demand was supplied in the admissible prefix ρ_L, so completion permanence places the complementary item within its owner's sequence and Lemma K.2b.O case (i) applies; and g_P = f_P is internal with its complement owner in D by the original WFGRec(K_L) closure, stuck by Step 1. Hence every single-frontier member of D contributes a process-to-process edge. Since the set of stuck processes is finite, every maximal item-labelled walk either repeats, yielding an internal wait cycle among stuck processes, or terminates at an EnvStop or DoneStop node. In the multi-frontier extension, the construction is applied to every item in FrontRec_X(K_S), subject to obligation K-O2.

Proof, Step 4 (closure in the extended wait graph). Let D+ be the finite closure of D in WFGPlusRec(K_S,I) under the Step 3 item-labelled process-owner edges and EnvStop/DoneStop terminal edges, applied to every K.1-relevant item of every member. Every process node of D+ is stuck; every internal frontier dependency lands either on a stuck owner inside D+ or on its DoneStop terminal; every environment-facing frontier lands on its EnvStop terminal. For members of D, exhaustion also follows from Lemma K.EnvX-Auto at K_L and monotonicity of fixed-stimulus exhaustion. D+ contains at least one process-to-process edge because each single-frontier member of D contributes one by Step 3. By Step 2, K_S is ε-quiescent and DrainClosedNowRec holds for the process nodes of D+; equivalently no offer in their DrainBag is a current drain candidate. Therefore DeadKSRec(K_S,I) holds, and with the recorded waiver-closure premise the same component establishes DeadKSRec^W(K_S,I,W). Because the chosen maximal fair Policy-S execution was arbitrary, every such execution reaches the general serial obstruction. Where an ordinary internal K.1 component is wanted, Corollary K.EnvOrd-1 proves that no starvation terminal is reachable and extracts a terminal strongly connected component of the process-to-process subgraph, yielding DeadKRec(K_S,I). ■ (general branch)

Corollary K.EnvOrd-1 (optional collapse to an ordinary internal cycle) — statement and proof. Assume additionally the static Conditions K.EnvOrd (may-follow form) and K.Done of K.4b, and retain K_S = KObs_E,w(σ_S) from Lemma K.2b.Q. Then no Step 3 walk from D reaches a starvation terminal; every maximal walk repeats internally, the Step 4 terminal strongly connected component is a literal internal K.1 component, and DeadKRec(K_S,I) holds — equivalently σ_S satisfies Dead_K, and with KObs waiver closure it satisfies Dead_K^W when the extracted component is non-waived. Proof. Condition K.Done makes DoneStop terminals unreachable outright: every possibly-terminating endpoint's production is audited to cover its counterpart's demand, or every process is perpetual, so Lemma K.2b.O case (ii) cannot arise at any walk node (obligation K-O7). For EnvStop terminals, suppose for contradiction that some walk P → Y_1 → … → Y_n → X from P ∈ D ends at an environment terminal X with exhausted serial frontier g_X. Each awaited missing operation along the walk is strictly source-later than its owner's

serial frontier, by Lemma K.2b.O case (i). Backward induction along the walk. (base) Y_n's awaited operation q_X of X is strictly source-later than g_X, so its site may-follows g_X's site, which is environment-facing and hence a cone seed; q_X can feed the blocked waiter Y_n, so the K.CV/WC-derived dynamic no-bypass property of Condition K.EnvOrd forbids every admissible Policy-L execution from issuing the corresponding dynamic q_X while g_X pends; g_X is stimulus-exhausted and commits in no execution, so q_X commits in no admissible execution, Y_n's frontier g_{Y_n} is unfillable in every execution, and g_{Y_n}'s site lies in the cone, its supplying site may-following the cone site of g_X. (step) If Y_i's frontier g_{Y_i} lies in the cone and is unfillable in every execution, then the operation q_{Y_i} of Y_i awaited by the previous walk node is strictly source-later than g_{Y_i} — its site may-follows g_{Y_i}'s site — and can feed that blocked waiter, so the dynamic no-bypass property forbids issuing it while g_{Y_i} pends; since g_{Y_i} never commits in any execution, q_{Y_i} commits in no admissible execution, and the previous node's frontier is unfillable in every execution with its site in the cone. (conclusion) The induction propagates unfillability-in-every-execution back to the walk's exit from D. Walk edges taken at frozen liberal frontiers f_P land inside D by K.1 closure, and single-frontier members of D never carry terminal edges at their own frontier (Step 3); so a walk that reaches a terminal must leave D at a member P* blocked strictly earlier than its liberal frontier, g_{P*} <_src f_{P*}. By Step 1's all-below-f fact, g_{P*} committed at σ_L in the admissible execution ρ_L — its supply arrived, so the awaited operation of the next walk node committed in ρ_L — while the induction shows that same operation commits in no admissible execution. Contradiction. Multi-frontier members are covered by obligation K-O2 as in Step 1. This corollary is optional; Lemma K.2b itself needs only Dead_KS. Mechanized validation is required: the may-follow cone fixed point and the K.CV/WC no-bypass derivation are obligation K-O6, and the K.Done audit is obligation K-O7. ■ (K.EnvOrd branch)

Proof of Lemma K.2b. If the design is not natively in the blocking-only fragment, lift ρ_L and σ_L into Sys_K = Λ_W(Sys_ref) by Lemma K.Low.C; the corresponding lowered cutpoint has the same projected history and a literal message-frontier obstruction, and its terminal data are represented by the lowered K_L. Fix any maximal fair Policy-S execution ρ_S of Sys_K under the same fixed stimulus and witness choices; no stimulus truncation is used. Lemma K.2b.Q Step 1 uses K_L to identify and jointly freeze the original typed frontier items, while Lemmas K.2b.R and K.2b.P remain operational cross-execution arguments showing that every member of D is permanently non-advancing in ρ_S. Step 2 forms the stabilized K_S and discharges finite drain normalization through the ResOffers-derived DrainBag descent; Steps 3-4 follow actual typed complementary owners in WFGPlusRec(K_S,I) and establish DeadKSRec^W(K_S,I,W). Corollary K.EnvOrd-1 strengthens the reconstructed conclusion to DeadKRec^W under Conditions K.EnvOrd and K.Done. Because ρ_S was arbitrary, every maximal fair Policy-S execution reaches the route-selected BadSRec predicate — the universal conclusion consumed by Corollary K.2c. The KObs records expose only the terminal snapshots; request reachability, Issue fairness, nonwithdrawal, and no-new-launch remain operational premises. σ_S lies on an admissible finite Policy-S prefix of the selected instance under the fixed stimulus, so it is a reachable Policy-S cutpoint; no surrounding infinite fair execution is required because the prohibited predicates are reachability properties over admissible executions. Where an embedding into an infinite fair execution is wanted it is supplied by Lemma J.FE under its environment-receptiveness premise, not by B6, which applies directly only at a B5/global fixed point. Where lowering was applied, A5-S is evaluated on Sys_K per Definition K.Low, so no back-projection is required; Lemma K.Low.C(iii) relates the lowered obstruction to the original extended wait structure where desired. No stimulus truncation, scalar frontier identification, global remaining-work measure, issue-at-commit normalization, or Policy-S confluence theorem is used. □ Proof status: Steps 1-2 and the Step 3 walk framework are proved above at document rigor; obligations K-O1, K-O2, and K-O8 remain, Corollary K.EnvOrd-1 additionally carries K-O6 and K-O7, and mechanization is required before this lemma is cited as certified.

Lemma K.Ser (conditional serial trace replay). Statement. Fix the selected instance Sys_K, the fixed stimulus, and the fixed witness choices, and let ρ_L be a finite admissible Policy-L prefix ending at an ε-quiescent cutpoint σ_L with K_L = ⟨E,w,H_L,O_L⟩. Suppose: (a) commit-prefix condition — for every process P, CmtRec_P(K_L) is a source-order prefix of P's dynamic operation sequence; (b) serial-precedence acyclicity — the union of the causal dependencies of ρ_L (per-channel transfer order and capacity dependencies, rendezvous atomic pairing, epoch-gate and guard dependencies, and process-local source order) with the Policy-S precedence constraints, requiring each source-earlier blocking operation of a process to commit strictly before a source-later one issues, is acyclic; and (c) terminal offer admissibility — every o ∈ O_L is attributed to its owner's source-minimal uncommitted blocking item, equivalently DrainBag_Proc(I)(K_L) is empty, so σ_L contains no source-later asserted request that Policy S would have withheld. Then there exists a finite admissible Policy-S execution of the same instance, stimulus, and witness with the same per-channel committed transfer sequences and payloads ordinal for ordinal, the same committed-identity sets and canonical process-local replay history H_L, and therefore the same ProcRec, ChanRec, NextFrontRec, and source-relevant terminal values as K_L. If a separate terminal-offer replay check also establishes the same residual-offer set O_L, the two terminal cutpoints have equal KObs; equal O is not inferred from the committed history alone. Proof sketch. Linearize the finite acyclic combined precedence relation and replay commits in that order, inserting idle cycles to satisfy the strictly-earlier-cycle serial condition. The induction invariant — equal per-channel counts and payloads, equal process-local committed histories, equal channel state — is preserved at each step: source-earlier blockers have committed by the added precedence constraints, so each issue is Policy-S-legal; channel enabling conditions transfer by the equal-channel-state invariant; Lemma K.2b.P1 supplies operation-identity and payload agreement; and I.DR supplies per-process state agreement from equal replay histories. The optional equal-O conclusion requires one additional finite check that the terminal serial requests are asserted at the same typed boundaries and epochs. Mechanization note: a finite structural induction with no fairness or limit reasoning; mechanization is required before the lemma is cited as certified.

Engineering note (serial replay of recorded traces; scope of Lemma K.Ser). Lemma K.Ser characterizes when a recorded liberal or RTL-matched execution can be replayed under the conservative serial scheduler with identical committed histories — and, with the additional terminal-offer replay check, identical KObs. KObs alone is not a sufficient replay certificate. The recorded artifact must additionally contain the historical dynamic operation identities, Issue events, persistent-request intervals and attribution, failed non-blocking outcomes as witness selections, elaborated-boundary transfers, and arbiter selections needed to justify the acyclic operational replay; a committed-event history determines neither those issuance obligations nor the intermediate enablement facts. O_L records only requests still outstanding at the terminal cutpoint. Conditions (a) and (c) fail exactly where the recorded run exploited eager issue; Examples K.CE and K.CE-2 are minimal failures, and their liberal histories are reproducible by no Policy-S execution. Lemma K.Ser is therefore deliberately not a substitute for Lemma K.2b: the non-serializable case is where the dominance content of the serial-reduction route resides. Mechanization obligations. K-O1: formalize issue fairness for a reached source operation whose issue preconditions remain satisfied. K-O2: prove the separate multi-frontier owner-walk and joint-freeze composition theorem for elaborated joins and bundles over the full attributed item type; the primary certified theorem may initially be K.2b-SF for single-frontier processes. K-O4: mechanize Lemma K.2b.R uniformly across buffered and rendezvous cases. K-O5: mechanize Lemma K.2b.O as the typed stuck / completed / compute-divergent trichotomy, including the completion-permanence clause and the bounded-local-progress exclusion; the previous exclusion of normal completion is withdrawn (Example K.CE-4). K-O6: mechanize the finite static K.Cone construction in its may-follow form, including loop-carried instances (Example K.CE-3), and the K.CV/WC derivation of the dynamic no-bypass property. K-O7: mechanize the Condition K.Done completion-sufficiency audit and its consequence that DoneStop

terminals are unreachable. K-O8: formalize the KObs interface selectors and prove the drain-bag descent: at a stabilized non-advancing set NA, ResOffers supplies a finite DrainBag_NA; Definition R.16/K.Drain and nonwithdrawal prevent insertion or withdrawal without commit; every selected enabled drain commit strictly decreases the multiset; and B4/B5 ε-normalization preserves the source-attributed measure while permitting internal E4 movement. K.Low.C must state preservation of projected history modulo the permitted stuttering relation introduced by positive-capacity guard channels; no claim of identical cycle timing is required. Each operational obligation shall be proved from Definition R.16, the Appendix C at-most-once Issue/Commit linkage, nonwithdrawal, Kε, K.CV, B1/B2/B3/B4/B5, issue fairness per K-O1, bounded local progress per Definition K.BF, and point-to-point E4 ownership — not from finiteness of an envelope or state space, since finite systems can oscillate. The KObs equalities and selector facts are supplied by Definitions R.18a-R.18e and Lemmas R.21b-R.21g; they do not replace the operational premises. Where earlier text cites K.2b.S, K.2b.T, or K.2b.N, they correspond to Step 2, Step 4, and Step 3 of Lemma K.2b.Q respectively.

Appendix L – Snooping Introduction, Implementation and Concerns

Snooping Introduction

If designs are completely latency insensitive, verification of the pre-HLS versus post-HLS models is straightforward. However, if the design contains some components such as arbiters which have latency-sensitive behavior, and if that latency-sensitive behavior is in some cases externally visible to the DUT, then verification becomes somewhat more complex. Techniques can be applied to simplify verification.

Consider a design which has an arbiter like Matchlib toolkit example 09*. The arbiter uses non-blocking PopNB() on its inputs, and blocking Push() on its output to emit the winner of the arbitration. Since the latency between the pre-HLS and post-HLS designs will differ, the order of transactions presented to the arbiter in the two scenarios will differ, and thus the order of the winners will differ. This may result in verification mismatches between the two models if the order of the winners is externally visible to the DUT.

To force the pre-HLS and post-HLS simulations to match, we can run the two designs side-by-side. We can snoop the inputs to the arbiter in the post-HLS model, and only allow the inputs to the pre-HLS arbiter to proceed when the corresponding inputs are seen in the post-HLS model. This will force the order of the inputs to be equivalent between the pre-HLS and post-HLS models, and thus both arbiters will pick the same winners. When this technique is used, the pre-HLS simulation will be throttled by the post-HLS simulation, but the overall verification will still work properly.

Another related example involves interrupt request signals feeding into an interrupt controller within a CPU model. Typically, each interrupt request signal is a single bit sc_signal<>, indicating that an interrupt request is pending. If the requests originate from accelerator blocks that are being synthesized through HLS, then because of the latency differences between the pre-HLS and post-HLS models, the order of interrupt requests arriving at the interrupt controller will differ between the two models, and this will likely result in verification mismatches if the differences are externally visible to the DUT. To force the order of the requests to match, we snoop the requests arriving at the controller in the post-HLS model, and only then allow the requests to be seen in the pre-HLS model.

Simplified Snooping Approaches

In the snooping example directly above in which the arbiter is snooped, the entire post-HLS RTL DUT is simulated alongside the pre-HLS model to force the arbiter requests to be aligned in time. This is the most general case and assumes the input delays to the arbiter are difficult to determine via analysis.

Sometimes there are simpler cases where the input delays to the arbiter in the RTL DUT are easier to determine. For example, each input to the arbiter might arrive a fixed number of clock cycles after one of the primary inputs to the RTL DUT is pushed by the testbench. In this case, a much simpler model can be used to align the pre-HLS model with the post-HLS model. We can simply monitor the primary inputs to the pre-HLS model and then apply the fixed cycle delays to the pre-HLS arbiter inputs.

The two cases above illustrate two ends of a spectrum of possible approaches for extracting the delays from the RTL DUT. Between these two points there exist other possible approaches.

Snooping Implementation

To enable perfect matching between the pre-HLS and post-HLS system simulations, latency-sensitive global signals and latency-sensitive non-blocking message-passing operations need to be synchronized between the two simulations if their latency differences are externally visible to the DUT.

As explained earlier in this appendix, in the general case the entire post-HLS DUT RTL must be run alongside the pre-HLS system to achieve alignment. Examples of signals that need to be synchronized include global interrupt request signals (as discussed earlier in this appendix) and rdy/vld signal pairs for non-blocking operations on message-passing channels. All such synchronization signals and their corresponding handshake signals must be level-stable:

- A latency-sensitive signal s is **level-stable** if, once s becomes 1 in cycle t, it must remain 1 until the corresponding handshake signal h is 1 in some cycle $t' \geq t$.

Once the set of signals that need to be aligned between the two simulations is identified, the general rule to implement snooping is simple:

- For each polarity-normalized Boolean commit-enabling snoop signal (for example, the vld/rdy handshake signals of snooped non-blocking operations, or level-stable request lines), make all readers in the pre-HLS simulation see the logical AND of the value driven in the pre-HLS simulation and the recorded post-HLS observation of the same dynamic instance ordinal — captured at the post-HLS-side event and retired at the pre-HLS-side handshake of that ordinal, per Wrapper Realization Requirements W1–W4 below. Payload/data values are never ANDed or substituted; they are checked for equality at the matched ordinal (Requirement W4). The post-HLS simulation runs free: its readers see only its own driven values, and the wrapper observes but never gates, delays, or modifies any post-HLS-side signal.

Often the post-HLS side will not run ahead of the pre-HLS side at the snoop points, because HLS typically only adds (rather than subtracts) latency within each process. This tendency is a heuristic, not an invariant: HLS can also subtract latency, for example by removing conservative waits present in the source, by improving loop initiation intervals, or by realizing channels with fall-through timing. The one-sided rule above is therefore normative, and a symmetric variant — driving the logical AND of the two simulations' nets into the readers on both sides, so that the pre-HLS side can gate the post-HLS side — must not be used, even as a fallback when the post-HLS side runs ahead. Under B1 the post-HLS Σ-trace under fixed stimulus is unique, so there is no post-HLS nondeterminism for a wrapper to select among: gating a post-HLS-side reader (in particular an internal net such as an arbiter input or an interrupt request line) does not select an admissible RTL_B behavior; it produces a different circuit RTL_B' with different internal latencies, and any result obtained about RTL_B' does not transfer to the RTL_B that ships. Gating the pre-HLS side, by contrast, is witness selection over an intentionally under-specified specification (see Snooping Concerns below) and is always permitted.

Two situations must be distinguished when the post-HLS side does run ahead at a snoop point. (a) Benign cycle skew: the RTL asserts a snooped level-stable signal some cycles before the pre-HLS side does, but the order of Σ-visible events at the snoop points is unchanged. Because the compatibility

results of Appendices G–K are stated over Σ-event order and values, with finite ε-stutter tolerated (B4/B5), a matching pre-HLS witness still exists. The wrapper handles this case by recording the RTL assertion (level-stability makes the capture clean; the recorded assertion is held until the pre-HLS-side handshake of the same ordinal, per Requirement W1 below) and letting the pre-HLS side catch up, arbitrarily behind in host time. Cycle skew is not an error, and a cycle-locked per-cycle "perfect match" comparison must not be used as a validity gate, since it would misclassify a mere HLS speedup as a failure. (b) Divergence: no admissible pre-HLS execution can consume or produce the RTL's event sequence — its ordinal order, kinds, and payloads — at the chosen boundary; the witness required by WC does not exist. This is the genuine failure mode; it violates Condition L.Dir below, must cause the wrapper to abort loudly, and is triaged as described under Snooping Concerns. Comparison of the pre-HLS-side AND against the value driven on the post-HLS side remains a mandatory monitor (Requirement W4 below), but its role is diagnostic: a per-cycle mismatch reports skew, while the L.Dir violation itself is defined at the event level — ordinal order, kind, and payload — and is detected by an order/kind/payload check together with a watchdog/timeout that converts witness non-existence, or an inconclusive failure to realize the witness within an unproved bound (Requirement W4), into a reported failure rather than a silent gate or an indefinite hang.

Wrapper Realization Requirements (Recorded-Observation AND)

The AND rule above is stated over a recorded observation, not over the live post-HLS net. A naive realization in which the pre-HLS reader sees the AND of the two currently driven values is a cycle-window mechanism: it delivers a snooped event to the pre-HLS side only if the post-HLS assertion is still asserted when the pre-HLS side is ready to commit. Because the post-HLS side runs free, its assertion window ends at its own handshake, which is unrelated to whether the pre-HLS side has caught up; level-stability guarantees that the signal remains asserted until the post-HLS-side handshake, not until the pre-HLS-side handshake. The live-AND realization is therefore faithful exactly when, for every dynamic instance ordinal k at every snoop point, the pre-HLS-side commit of instance k falls inside the post-HLS-side assertion window of instance k. This per-ordinal window-overlap condition is strictly stronger than Condition L.Dir, and it is an empirical per-run property: it shall be checked, not assumed.

Where the window-overlap condition fails — including on runs that are L.Dir-clean, i.e., runs exhibiting only benign cycle skew — the live AND fails in three ways. First, it drops events: the post-HLS assertion retires before the pre-HLS side arrives, the AND never rises for the pre-HLS side, and the event is silently lost (for example, a lost interrupt downstream, or a pre-HLS process waiting forever on a request that never becomes visible), converting an L.Dir-clean run into an apparent failure or value divergence — precisely the misclassification this appendix forbids. Second, it aliases ordinals: if the post-HLS side runs two or more transactions ahead on the same signal, the pre-HLS side's k-th commit can be enabled by the post-HLS side's (k+1)-th assertion, and pulse-type signals held across two post-HLS instances can merge into one pre-HLS observation, or drop. Dropped and aliased events violate the no-add/remove/reorder premise of Lemma L.2 and break the per-instance μ_Q matching supplied by WC: the harness is then sampling a wire, not replaying the witness. Third, it detects nothing: genuine order divergence (a real L.Dir violation) manifests as a hang indistinguishable from a dropped-event hang, so the triage of Snooping Concerns cannot begin; moreover, the harness-induced infinite wait appears as a wait-for dependency in any Appendix K analysis — a phantom deadlock manufactured by the harness rather than by the design, and outside the finite-delay tolerance that B2 fairness extends to the wrapper on L.Dir-clean runs.

W1 (Recorded per-ordinal observation). For each snoop point, the wrapper shall capture each post-HLS snooped event — the committed transfer for message-passing snoop points, the assertion edge for level-stable global signals such as interrupt request lines — indexed by its dynamic instance ordinal, and shall hold the captured observation for the pre-HLS side until the pre-HLS-side handshake of the same ordinal, at which point the observation is retired. For a single-outstanding interface this reduces to one sticky flop, set by the post-HLS event and cleared by the pre-HLS-side handshake; in the general case it is a small per-snoop-point event FIFO. The AND shall consume the captured observation, never the raw net; level-stability is what makes the capture clean, and pulse-like signals shall be queued as distinct edge events so that dynamic instances are neither merged nor dropped.

W2 (Gate only the commit-enabling mirror signal). Per operation kind, the wrapper shall gate only the mirror signal that enables the pre-HLS-side commit: the observed vld for a pre-HLS-side Pop, and the observed rdy for a pre-HLS-side Push. The endpoint-owned request driven by the pre-HLS side itself is never modified, consistent with the Appendix C non-withdrawal discipline.

W3 (Registered sampling point). In a shared-clock co-simulation, the post-HLS observation shall be registered by at least one cycle before it enters the AND, so that delta-cycle ordering races between the two simulations cannot affect the observed value. This is safe by construction: it adds only pre-HLS-side latency, which is witness selection.

W4 (Monitor and watchdog; Lemma L.5(iv)). At each pre-HLS-side commit at a snoop point, the wrapper shall compare the committed event against the recorded post-HLS event of the same ordinal, in order, kind, and payload. Payloads must match (K.CV); the wrapper shall never substitute a post-HLS payload into the pre-HLS side, since substitution relabels Σ-events and masks value divergence. A pre-HLS-side snoop commit with no recorded post-HLS counterpart, or an order/kind/payload mismatch at a matched ordinal, shall abort loudly as an L.Dir violation. A recorded event outstanding beyond a configured watchdog bound, or a pre-HLS-side request that remains unmatched beyond that bound, shall likewise abort loudly, but as an L.Dir/WC discharge failure: if the bound has been proved sufficient for the selected instance, the abort may be classified as an L.Dir violation; otherwise it shall be triaged as an inconclusive witness-realization failure — the compared run remains outside the discharged theorem scope, but the abort shall not be converted into a claimed design deadlock or a claimed proven violation. In all cases the abort converts witness non-existence, or failure to realize the witness within the bound, into a reported failure rather than a silent gate or an indefinite hang. Per-cycle skew shall be logged separately as diagnostics and shall not be treated as a failure.

W5 (Coverage remains a separate obligation). Requirements W1–W4 concern the fidelity of the replay mechanism at each snoop point; they do not discharge the first premise of Lemma L.2. If the snooped set misses an ε-modeled choice that can affect Σ-visible control flow, frontier membership, request attribution, guard values, or payload values, the replayed witness is underdetermined regardless of the recording discipline, and the resulting failure shall be triaged as harness incompleteness under Snooping Concerns.

Remark (relation to the naive live AND). The live-AND and recorded-observation realizations coincide on single-outstanding snoop points at which the pre-HLS consumer commits within every post-HLS assertion window — for example, a non-blocking polling consumer that polls every cycle while the pre-HLS side remains at-or-ahead in cycle time at that snoop point throughout the run. Because W4 requires the W1 recording in any case as its comparison reference, gating with the live net offers no economy;

the recorded observation is therefore the normative gating term, and the live net may be used only as an additional diagnostic tap.

Shared reactive testbenches. Once the pre-HLS side is permitted to lag by more than a cycle, a single shared reactive testbench instance is problematic: its reactions are driven by whichever model it is wired to, so the two models no longer receive the same stimulus, contrary to the fixed-stimulus setting of the formal results. The flow shall therefore either replicate the stimulus/testbench per model, or structure the shared testbench as a function of the DUT-visible trace so that the wrapper's recorder can replay its reactions to the lagging side under the same recorded-observation discipline of W1–W4.

Definition L.1 (Witness selector / Snooping).

Fix an initial state, fixed external stimulus, and an RTL_B execution prefix τ_post. A witness selector W for a source process P is a partial strategy that, at each ε-quiescent source cutpoint σ_ref and each source-level choice point whose outcome is ε-modeled but may affect later Σ-visible behavior, returns the outcome selected by the matched RTL_B execution.
Such choice points include failed/successful outcomes of non-blocking PushNB/PopNB calls when the outcome affects later control flow, frontier membership, request attribution, or payload/guard values; and any other ε-only implementation choice that can affect the next Σ-visible frontier.
Cadence-sensitive non-blocking polling. If a source process computes externally visible behavior from the number or cadence of failed PushNB/PopNB polls — for example, retry counters, timeout counters, or arbitration policies whose winner depends on how long an input remained empty/full — then that polling logic is latency-sensitive in the sense of Appendix D. Such a process is not covered by the ordinary Appendix I/J latency-insensitive source-core argument merely by treating the failed polls as hidden ε-steps. To bring such a design into scope, the cadence-sensitive polling logic shall be isolated as a latency-sensitive local protocol block, or shall be aligned by an Appendix L snooping/witness construction that makes the selected source execution an admissible execution of the same isolated protocol behavior observed in RTL_B. The observable interface to the rest of the system shall then be the chosen boundary of the isolated block, not the internal failed-poll cadence itself.
The selected outcome must be causally available from the matched RTL_B prefix and the fixed global Σ-causal history. W is not allowed to depend on future RTL_B events beyond the matched prefix needed to identify the current source choice. For cadence-sensitive polling logic, the witness must select an admissible execution of the isolated latency-sensitive block; it may not manufacture a retry/timeout outcome that is inconsistent with the RTL_B behavior being used as the witness, the fixed stimulus, or the chosen observable boundary of that block.

Lemma L.2 (Snooping establishes WC).
Let τ_post be an RTL_B execution under fixed stimulus, and let W be the witness selector induced by the Appendix L snooping wrapper for the corresponding source execution. Assume:

1. the snooped signals / outcomes include every ε-modeled source choice whose result can affect the next Σ-visible control flow, frontier, request-attribution, guard value, or payload value;
2. the snooping wrapper does not add, remove, reorder, or relabel any Σ-visible event except by selecting ε-only source outcomes at the chosen observable boundary;
3. each selected ε-only outcome is consistent with the matched RTL_B prefix and the fixed global Σ-causal history;
4. failed non-blocking polls remain ε-events and successful non-blocking transfers produce exactly the corresponding committed Push_c / Pop_c Σ-event; and

5. if the source contains cadence-sensitive non-blocking polling whose externally visible behavior depends on the number or cadence of failed polls, that polling logic has been isolated or snoop-aligned as a latency-sensitive local protocol block as described in Appendix D and Definition L.1. The WC witness is then applied to the chosen observable boundary of that block, not to an unconstrained hidden failed-poll cadence inside the ordinary latency-insensitive source core.

Then the source execution selected by W satisfies the witness-compatibility premise WC used in Appendix I.2 and Appendix J.

Corollary L.3 (NB-CF identity witness).
If NB-CF holds for process P, then failed non-blocking poll outcomes do not affect the next Σ-visible control flow, frontier membership, request attribution, guard values, or payload values. Therefore the witness selector W may be taken to be the identity / arbitrary selector on failed non-blocking outcomes, and WC holds without additional snooping for those polls.
Corollary L.4 (Cadence-sensitive polling is handled by isolation).
If NB-CF does not hold because the source observes the number or cadence of failed PushNB/PopNB polls — for example, retry counters, timeout counters, or arbitration policies whose later Σ-visible behavior depends on how long a channel remained empty/full — then the design is latency-sensitive at that polling site. Such a site must be isolated or snoop-aligned under Appendix D / Appendix L before applying the ordinary Appendix I/J source-core proof. If no such isolation, snooping, or equivalent explicit timing model is supplied, the process is outside the scope of the Appendix I/J trace-compatibility theorem for the chosen observable boundary.

Condition L.Dir (One-directional alignment / RTL-side non-interference).
Fix the free-running RTL_B execution τ_post under the fixed stimulus (B1) and the Appendix L wrapper for a chosen set of snoop points. L.Dir holds for a compared run iff the source-side snoop sequence selected by the witness selector W of Definition L.1 is exactly an ordinal-preserving consumption of the free-running RTL_B snoop sequence at the chosen boundary: (i) each pre-HLS-side snoop commit consumes the next recorded RTL_B event of the same snoop point, with matching kind and payload; (ii) no pre-HLS-side snoop commit occurs without such a next recorded RTL_B event; and (iii) every recorded RTL_B snoop event within the compared run is eventually either consumed by the source witness or reported outstanding according to the proved or declared comparison bound of Requirement W4 above. Throughout, the alignment never gates, delays, or modifies any RTL_B-side signal or event (RTL-side non-interference): the wrapper never drives a post-HLS-side reader with a value different from the value the free-running RTL_B drives. Cycle-level lag of the pre-HLS side (benign skew) is permitted under L.Dir provided the ordinal-preserving consumption above is maintained via recorded/buffered replay of the level-stable assertions; only order, kind, or payload divergence — non-existence of the WC witness at the chosen boundary — violates L.Dir. L.Dir is a named side condition with its own entry in the side-condition discharge record (Common Formal Definitions). A run that violates L.Dir provides no theorem-backed guarantee. The harness shall abort loudly on any detected divergence and on any exceeded Requirement W4 watchdog bound, with the classification of bound-exceeded aborts governed by Requirement W4, and every abort shall be triaged as described under Snooping Concerns below.

Lemma L.5 (Online realization of W / harness soundness).
The equivalence and liveness results of Appendices I–K are stated over the free-running RTL_B execution τ_post under fixed stimulus (B1), together with the source-side witness selector W of Definition L.1. This statement is one-directional by construction: no premise of those results requires RTL_B to be stallable by the source, and τ_post is a fixed object rather than a participant that can be gated. Separately, suppose the co-simulation wrapper (i) observes but never gates, delays, or modifies any post-HLS-side

signal or event; (ii) records snooped level-stable assertions and replays them to the pre-HLS side in RTL order, permitting arbitrary finite host-time skew (Wrapper Realization Requirements W1–W3); (iii) satisfies the premises of Lemma L.2 (see Requirement W5); and (iv) monitors Condition L.Dir and aborts loudly upon violation (Requirement W4). Then on every run on which the wrapper does not abort, the wrapper computes W online, the selected source execution satisfies the WC premise used by Appendices I–J (by Lemma L.2), and the results proved over free-running RTL_B + W apply to that run. Bidirectional stalling therefore never enters any theorem whose conclusion is transferred to the shipping RTL: the post-stalls-pre direction is witness selection over the intentionally under-specified source, and the pre-stalls-post direction is excluded by the monitored Condition L.Dir rather than assumed, endorsed, or repaired by gating the RTL.

Snooping Concerns

The snooping technique is used to align pre-HLS and post-HLS latency-sensitive global signals and latency-sensitive non-blocking message-passing operations in cases where their latency differences are externally visible at the DUT boundary. This includes cadence-sensitive polling blocks, such as retry/timeout or polling-arbiter logic, when their externally visible behavior depends on the number or cadence of failed non-blocking polls. When such a wrapper is applied, the pre-HLS model is throttled to follow the latency choices made by the post-HLS RTL, and the resulting traces at the observable interfaces are forced to agree. Snooping does not make failed polls into committed message-transfer events; failed polls remain ε-steps. Rather, snooping selects an admissible execution of the latency-sensitive local protocol block, while the rest of the system is compared at the block's chosen observable boundary.

Readers with a formal background may be concerned by this technique, because it appears to use the post-HLS RTL implementation to modify the behavior of the pre-HLS specification. From a formal point of view, the important observation is that the pre-HLS model is intentionally under-specified with respect to micro-timing/latency: subject to the R-rules, B-rules, weak fairness, and the explicit Σ-visible ordering, FIFO, signal-anchor, synchronization, and atomicity constraints used by ≈, it can admit multiple ε-/latency-different executions that respect the same required-order constraints over the chosen observable units. This is compatible with B1, which asserts that the post-HLS Σ-projected trace is unique under a fixed stimulus; the pre-HLS side may still have multiple admissible realizations that (when projected to Σ) match that unique RTL Σ-trace.

Snooping does not change what executions are allowed by the pre-HLS specification. Instead, it selects—purely for verification purposes—one admissible pre-HLS execution whose latency-sensitive events align with those observed in the RTL. In other words, the implementation is used to pick a witness execution of the specification, not to redefine the specification.

Triage of L.Dir violations. A violation of Condition L.Dir (order, kind, or payload divergence at a snoop point, including a pre-HLS-side snoop commit with no recorded RTL_B counterpart) is a relational failure: the compared run is outside the discharged theorem scope, and no theorem-backed claim attaches to it. It shall not be defined a priori as a bug in the pre-HLS model, because witness non-existence has three possible root causes, and defining the event as a model bug by fiat forecloses detection of the third: (1) an ill-formed pre-HLS model — for example, reliance on the ordering of causally independent events, cadence-sensitive polling that was not isolated per Corollary L.4, or a snooped signal that is not level-

stable — in which case the failure is of the same character as a standalone pre-HLS test failure and the model is outside the well-formedness envelope of the stress-testing section below; (2) an incomplete snooping harness — the snooped set misses an ε-modeled choice covered by the first premise of Lemma L.2, so the replayed witness is underdetermined; or (3) a non-compliant tool/RTL — the RTL exhibits behavior outside the admissible envelope (for example, an R/B-rule or B-faithfulness violation), in which case the Post → Ref result itself fails and the divergence is evidence of a synthesis defect. In a flow in which the harness has been certified against Lemma L.2 and the tool is trusted, the pre-HLS model is the default suspect and the failure should be filed against it first; the formal status of the run, however, is "L.Dir/WC undischarged — outside theorem scope," not "pre-HLS model incorrect," until triage assigns the root cause. A watchdog abort under a bound that has not been proved sufficient for the selected instance is triaged in the same way but is recorded as an inconclusive witness-realization failure per Requirement W4, not as a proven L.Dir violation. Benign cycle skew (unchanged Σ-event order with the RTL merely earlier in cycle time) is not an L.Dir violation and shall not be classified as a model defect. Experienced SoC architects tend to view this slightly differently, in terms of design tradeoffs and verification cost.

First, it is usually possible at the architectural level to avoid latency-sensitive behaviors entirely, for example by insisting that all communication be latency-insensitive and fully synchronized. However, the quality-of-results (QOR) costs of such designs may be unacceptable. Introducing latency-sensitive behavior—non-blocking arbiters, latency-sensitive global interrupt signals, and so on—is a deliberate choice made by the designer to meet performance, power, or area goals.

Second, in many practical systems, latency-sensitive behavior is not functionally observable at the DUT boundary under the trace-compatibility predicate ≈ with Σ instantiated to include only the DUT-boundary observables (Appendix G; Common Formal Definitions above). As an example, consider a DUT that contains a single-port RAM used to exchange data between two internal processes. An arbiter is required to control access to that RAM port, and the arbiter uses latency-sensitive non-blocking Pop operations on its request channels. During HLS, internal latencies will change, so the order in which the arbiter sees pending requests and chooses winners may differ between the pre-HLS and post-HLS models.

In this example:

- The individual RAM read/write operations and the internal arbitration decisions are not directly visible at the DUT interface.
- Only the higher-level IO of the two processes (for example, their message-passing interfaces to the rest of the system) is externally visible.

One might initially conclude that it is necessary to snoop the arbiter's inputs to make the pre-HLS and post-HLS traces match. However, the modeling rules impose two additional constraints:

1. **Weak fairness for the arbiter.** The arbiter must satisfy the weak fairness assumptions, so that any request that remains pending is eventually granted in both the pre-HLS and post-HLS models.
2. **Causal-dependency discipline on shared memory.** The system must make any functionally significant shared-memory dependence explicit, either through synchronization that orders the relevant accesses or through declared array-access mapping / memory-serialization semantics. This discipline ensures that sufficient synchronization or declared serialization exists between the processes so that there are no read/write races through the RAM in either model.

Under these conditions, the different arbitration orders merely change the relative timing of internal operations and of externally visible actions that are not ordered by the explicit required Σ-constraints. They do not change the FIFO, synchronization, signal-anchor, atomicity, or other required ordering constraints at the DUT boundary. In the terminology of Appendices G and J, the differences are incidental variations in latency rather than changes to the required-order relation $\prec_J$ over the chosen observable units, and therefore they are not considered observable differences in behavior at the DUT boundary**. In such cases, snooping is not required**. (See also Note 1 below).

(More precisely: the trace-compatibility predicate ≈ is parameterized by the chosen observable alphabet Σ. If internal functionality (such as the above RAM arbiter) uses channels/signals that are not designated observable in the chosen instantiation of Σ (see Common Formal Definitions), then mismatches on those internal actions are outside ≈ by construction: they are not in Σ. If those internal channels/signals are designated observable (e.g., by snooping for debug, or by expanding Σ for Appendix K deadlock analysis), then they are Σ-events and must match under ≈. In addition, even when B1 holds (unique post-HLS Σ-trace under fixed stimulus), the pre-HLS model may still admit multiple ε-/latency-different executions consistent with the same required Σ-ordering, FIFO, signal-anchor, synchronization, and atomicity constraints; Theorem J.1 captures the intended quantification ("existence of a matching witness execution prefix") rather than requiring a single pre-chosen trace.)

Experienced SoC architects therefore try to avoid designs in which latency-sensitive internal behaviors leak directly into the observable behavior of the system under test, since this both complicates verification and makes RTL behavior less predictable. When they do introduce such behaviors—examples include non-blocking arbiters whose winners affect externally visible ordering, global interrupt request signals at the DUT boundary—they typically know exactly where those latency-sensitive interfaces are and can isolate them.

A useful analogy is a subsystem whose clock frequency is adjusted dynamically based on on-die temperature. As temperature changes, the externally visible behavior of the SoC can change (for example, in terms of throughput or timing of events), and designers may choose such a temperature-dependent scheme to meet overall system requirements. To verify such a system, we may wish to compare a concrete RTL trace captured at a specific temperature profile with a higher-level reference model. To do so, the reference model must be driven with the **same** temperature evolution as the RTL saw. If that temperature profile is not otherwise under direct control, the simplest way to obtain it is to "snoop" the temperature as observed in the RTL trace and replay it into the reference model.
In this analogy, temperature is an external parameter of the environment rather than a fundamental part of the functional specification, but it still influences observable behavior. Snooping simply provides a mechanism to recover that parameter from the implementation when it cannot be easily prescribed a priori. Similarly, snooping of latency-sensitive IO provides a mechanism to supply implementation-specific latency choices — subject to the fairness assumptions and the explicit required Σ-ordering, FIFO, signal-anchor, synchronization, and atomicity constraints — back to the pre-HLS specification so that the two models can be compared under a common environment.

Note 1 (Unobservable non-blocking micro-timing).
Safety / functional equivalence. The trace-compatibility predicate ≈ is parameterized by the chosen observable alphabet Σ (Appendix G). If, for safety-only checking, we instantiate Σ to include only DUT-boundary observables (Σ_DUT) and we assume that any latency-sensitive internal behavior is not visible at that boundary under ≈—i.e., it may change only internal micro-timing, but it does not change the values or presence of Σ_DUT events, and does not change the ordering of causally *dependent* Σ_DUT

events—then snooping of those internal latency-sensitive interfaces is not required for proving Σ_DUT-safety. Any mismatches on actions outside Σ_DUT are outside ≈ by construction.
Liveness / deadlock analysis. For message-passing deadlock analysis (Appendix K WFG), any latency-sensitive interface whose ε-level choices can affect whether a blocking Push/Pop is enabled, or can create/remove a wait-for dependency without an intervening committed Σ-event, must be accounted for. Practically, the default is to snoop and replay every such latency-sensitive (typically non-blocking) arbitration/probe interface in the DUT. Snooping can be avoided only if the design is certified "WFG-inert" with respect to that interface: it cannot change WFG edges except via committed Push_c/Pop_c (or explicit synchronization) events.

Stress Testing Pre-HLS Models for Latency Robustness

The formal trace-compatibility results in Appendices G–K establish that, under the scheduling rules and fairness assumptions, the post-HLS RTL is trace-compatible with the pre-HLS/source-reference model at all observable interfaces.
These results implicitly assume that the pre-HLS specification is *well-formed*: it must not rely on incidental, implementation-specific timing alignments for functional correctness. Designs that do rely on such incidental alignments fall outside the formal guarantees.

Designs that use non-blocking message-passing operations (e.g., PushNB/PopNB, ac_channel nb_read/nb_write) or latency-sensitive global signals are particularly vulnerable to this kind of fragility. For such designs, it is strongly recommended to subject the pre-HLS model to aggressive *latency stress tests* in which message and handshake latencies are varied across executions. The intent is to validate that the design behaves correctly across the range of weakly fair schedules permitted by the modeling rules, not just under one convenient execution order.

More concretely, verification should check that:

- **Causal dependencies are explicit.** All functionally significant causal-dependency relationships are enforced via message-passing, SyncChannel/ac_sync operations, explicit signal handshakes/anchors, declared memory-serialization constraints, or Appendix L snooping/witness-selection boundaries, rather than by relying on the incidental execution order of concurrent processes. In the terminology of Appendix G, the design shall not depend on the ordering of causally independent events for correctness.
- **Weak fairness does not hide latent deadlocks or starvation.** The design continues to make progress under adversarial but *weakly fair* scheduling—i.e., enabled actions may be delayed arbitrarily long but not forever—consistent with the B2/B3 obligations on the scheduler and environment summarized in Appendix N.

If the pre-HLS model fails under such randomized-latency stress scenarios, then the specification itself is outside the formal model of this document: it violates the design rule that well-formed systems must not rely on the relative ordering of causally independent events. In that situation, the trace-compatibility theorems still apply to well-formed traces, but they no longer guarantee that the "golden" specification is robust under the full range of latency variations allowed by the environment.

The Matchlib library provides practical mechanisms to automate these stress tests in pre-HLS simulation, including:

- **Random stall injection on message-passing channels**, to simulate variable communication delays while preserving rendezvous semantics.
- **Latency and capacity back-annotation**, to model specific per-channel buffer depths and delays, including the capacities B(c) chosen during HLS.

Worked examples of this methodology are provided in Catapult Matchlib examples 60* and 72*, which illustrate how to configure randomized stalls and back-annotated latencies when validating that a design remains well-formed and latency-robust.

Appendix M – Document Abstract

This document defines a precise user-level scheduling model for high-level synthesis (HLS) that enables system-level verification and debug to be carried out primarily on the pre-HLS SystemC model, while still permitting aggressive RTL optimizations in the synthesized design. It introduces a small set of uniform rules governing three classes of I/O operations — message-passing channels, signal I/O, and explicit synchronization — and organizes them into a "basic conceptual model" that preserves source-order synchronization, anchors signal reads and writes to their nearest synchronization points, and constrains the reordering of message-passing operations so that HLS cannot introduce new deadlocks merely by reversing source-order message calls. Deadlock behavior can still depend on bounded-capacity effects, overlapped execution, and progress assumptions; the full liveness/deadlock-preservation result is therefore stated separately and remains conditional on the Appendix K and Appendix N assumptions.

These rules are extended to pipelined loops, shared and external memories through an explicit array-access mapping layer and conflict-free reordering discipline, and direct-input pragmas that allow stable or explicitly synchronized signals to avoid unnecessary internal storage while preserving the intended Σ-visible correspondence under the stated stability and synchronization contracts.

The methodology is latency-insensitive by construction but accommodates latency-sensitive islands such as cycle-accurate transactors, non-blocking arbiters, and one-way handshake protocols through encapsulation, explicit synchronization, and the Appendix L snooping / witness-selection mechanism when needed. The document also provides concrete modeling guidelines, including rules for placing signal reads and writes around wait statements, coding rolled loops with signal I/O, and using Matchlib Connections and SyncChannel, so that digital verification engineers can write a single verification intent in SystemVerilog UVM or SystemC/C++ and reuse it across pre-HLS and post-HLS models with "no surprises," subject to the stated well-formedness and side-condition obligations.

A major contribution is the formalization, in Appendix G and subsequent appendices, of an ordered observable trace-compatibility predicate between the prescribed source reference model Sys_ref and the post-HLS RTL model RTL_B, expressed as partial-order constraints on observable I/O actions and encapsulated in trace-compatibility rules E1–E5. The unrestricted safety direction is Post → Ref: every compliant RTL_B behavior has a matching Sys_ref behavior under the fixed stimulus. The converse use of a particular source-side execution is not claimed for all Sys_ref traces; when it is needed, it is restricted to annotated RTL-consistent source prefixes or to source executions selected by the Appendix L snooping / witness-selection mechanism.

For channels whose RTL implementations introduce finite buffering or staged realization structure, the correspondence is stated against the chosen source reference model Sys_ref: in ordinary bounded-FIFO cases, Sys_ref = Sys_B with capacities B(c) faithful to the selected abstract boundaries; in elaborated cases, Sys_ref = Sys_B^elab with explicit staged, bundled, join, or epoch-gate abstract channels. The raw rendezvous case is recovered as the special case B(c)=0 at the relevant ordinary boundaries.

The trace-compatibility proofs are compositional in a strong local sense: each process is verified against the obligations at its own observable boundaries, while each abstract channel boundary supplies the methodology-defined rendezvous or bounded-FIFO ChannelEnabled/ChannelDisabled behavior required by E4. A producer or consumer process need not know whether the connected ordinary channel is implemented as rendezvous or with positive capacity; it need only satisfy its local endpoint scheduling and handshake obligations at the chosen boundary.

The liveness/deadlock result is more conditional. Corollary J.1 and J.KObsConf establish a common KObs record for a selected reachable Sys_ref cutpoint and the corresponding certified RTL cutpoint. The KObs record combines the canonical committed/replay history with the complete attributed residual-offer set; pure reconstruction functions derive channel state, frontier requests, WFG/WFG+, snapshot drain closure, and the selected deadlock predicate. The core Theorem K.1A preserves absence of reachable internal wait-cycle cutpoints from the selected Sys_ref to RTL_B under A5-L and the stated correspondence, fairness, progress, drain, and abstraction premises. The primary user route discharges A5-L with Policy S: universal serial dominance says that any Policy-L internal deadlock would force every complete maximal fair Policy-S execution to reach the applicable serial bad-state predicate. Therefore one complete passing Policy-S execution suffices, without a confluence theorem. The optional K.EnvOrd/K.Done fragment checks the ordinary K.1 predicate; the general route checks Dead_KS over the extended wait graph, which also detects internal wait chains terminating at exhausted environment inputs or normally completed producers. Structural and direct certificates remain alternatives.

This trace-compositional methodology covers key HLS transformations including loop pipelining, added FSM states, memory-access reordering, direct-input late sampling under explicit contracts, and configurable channel buffering. It also explains why the methodology is not inherently limited to HLS-generated RTL: hand-written or otherwise generated RTL can use the same proof framework when its processes, channel realizations, selected abstract boundaries, and tool/RTL evidence satisfy the same implementation-side obligations. The proofs do not by themselves certify any particular HLS tool invocation or RTL instance; those obligations must be discharged by tool documentation, generated certificates, static checks, RTL/interface checks, or another validation method appropriate to the flow. Collectively, the scheduling rules, coding guidelines, and formal guarantees provide a tool-agnostic, Catapult-compatible foundation for industrial HLS use, a scalable verification methodology for RTL design more generally, and a concrete starting point for standardization efforts within bodies such as the Accellera Synthesis Working Group.

Keywords: High-level synthesis (HLS); SystemC; scheduling rules; latency-insensitive design; message-passing channels; ready/valid protocols; signal I/O; loop pipelining; direct input pragmas; shared memory; Sys_ref; RTL_B; trace compatibility; witness selection; formal verification; bounded FIFOs; B-faithfulness; liveness preservation; deadlock preservation; fairness; Catapult HLS; Matchlib; Accellera Synthesis Working Group.

Appendix N – Scheduler and Environment Fairness Assumptions

The formal results in Appendices I–K rely on several semantic assumptions about the post-HLS scheduler and its environment. The B-rule table in "Common Formal Definitions for Appendices I–K" gives the proof-level names B1–B6. This appendix restates the fairness/progress rules in operational ready/valid form. In particular, Appendix N's "B3 Channel Progress Invariants" are exactly the P_push(c) and P_pop(c) obligations for each explicit abstract channel c; they are not additional assumptions beyond B3.

- **B2 (Weak fairness of the scheduler).** If the enabling predicate for an action (Push, Pop, Sync) remains continuously true from some cycle onward, the scheduler must eventually select that action within a finite, but unspecified, number of cycles.
- **B3 Channel Progress Invariants (ready/valid form).**
  For every channel c with capacity B(c), interpret "continuously enabled" in terms of continuous endpoint request, not as an immediate commit.
  (**No deassert-before-commit assumption**.) For blocking Push_c/Pop_c transfers, these requests are non-withdrawable during the outstanding transfer attempt: once asserted for a transfer attempt, they are not deasserted before the corresponding commit, and Push_c payload remains stable while vld_c is asserted for that outstanding transfer. After the corresponding commit, the endpoint may deassert or may present a different request/payload for a later transfer.

  Let vld_c denote the producer's persistent request to transfer (e.g., valid=1 with a stable payload for a Push_c), and let rdy_c denote the consumer's persistent request to transfer (e.g., ready=1 for a Pop_c). In the definitions below, "remains asserted continuously" is scoped to the outstanding interval from t_o until the first matching discharge event. If the matching discharge never occurs, then the request must remain asserted for all later cycles.
  • **P_push(c):** If vld_c is asserted at cycle t_o while the channel is full (occ_c = B(c)), and both endpoint requests continue until the matching discharge event — i.e., vld_c remains asserted with stable payload and the complementary Pop_c endpoint request remains asserted with BoundaryReq(Pop_c,t) true until the first Pop_c commit on c — then some Pop_c commit must eventually occur, discharging the full condition. Equivalently: when both endpoints continuously request the matching transfer until discharge, the system cannot remain stuck forever in the full state without a Pop_c commit.
  • **P_pop(c):** If rdy_c is asserted at cycle t_o while the channel is empty (occ_c = 0), and both endpoint requests continue until the matching discharge event — i.e., rdy_c remains asserted and the complementary Push_c endpoint request remains asserted with stable payload and BoundaryReq(Push_c,t) true until the first Push_c commit on c — then some Push_c commit must eventually occur, discharging the empty condition. Equivalently: when both endpoints continuously request the matching transfer until discharge, the system cannot remain stuck forever in the empty state without a Push_c commit.
  For B(c)=0 rendezvous channels: if both endpoint requests are asserted at cycle t_o and both remain asserted until the matching rendezvous discharge event, then the rendezvous completion eventually occurs. If no rendezvous completion occurs, both requests must remain asserted for all later cycles.
  These obligations rule out "permanent back-pressure loops" that are logically independent of the local R-rules. They do not require an endpoint request or Push payload to remain asserted forever after the matching transfer has already committed.

- **B5 (System quiescence closure).** If at some cycle no observable actions are enabled in any process, and no external or environmental change during the closure interval changes the relevant enabling predicates, then within a bounded number of cycles the system reaches an ε-fixed point. Thereafter, while no external or environmental change re-enables observable or internal work, the system executes no additional ε-steps. This prevents a dead, all-disabled state from being hidden behind an infinite tail of internal activity, without requiring the system to remain quiescent after a later reset, input change, externally supplied message, or other environmental event re-enables work.

These B-rules are **assumptions on the execution environment**, not guarantees automatically provided by the HLS tool. In practice they place concrete requirements on arbiters, back-pressure, and external masters that interact with the post-HLS RTL.

Priority arbiter example: fair vs unfair priority

Consider a simple two-input priority arbiter inside the post-HLS RTL. Each input is driven by a message-passing channel; the arbiter issues Pop operations to select which request to serve in each cycle.

- Input 0: high-priority channel c_high
- Input 1: low-priority channel c_low

Both channels may have pending requests at the same time.

1. **Fair priority arbiter (satisfies B2 / B3).**

A fair implementation might still give c_high strict priority in *individual* decisions, but it ensures that a continuously pending c_low request cannot starve. In RTL terms, this can be realized by, for example:

- A **round-robin or rotating-priority** arbiter that periodically moves the "highest" priority to the next requester, or
- A **bounded-burst priority** scheme in which c_high may win at most N consecutive cycles while c_low is requesting; after that, the next grant is forced to c_low.

Under these implementations:

- If Pop_low's ChannelEnabled predicate remains true indefinitely (the low-priority channel has a pending request and the relevant abstract boundary availability predicate holds), the scheduler/arbiter must eventually select Pop_low. The selected Pop_low then commits in the selected cycle. This satisfies B2 for that action.
- If the low-priority FIFO is full and keeps requesting service, the system ensures that some complementary Pop_low eventually commits, discharging data and freeing space. This is consistent with P_push/P_pop in B3; it is a channel-progress obligation that may be needed before a blocked producer's Push becomes ChannelEnabled.

Intuitively, the arbiter is still "priority-based," but its micro-architecture ensures that every continuously enabled requester is eventually selected, and that full/empty channel states covered by B3 cannot persist forever under the stated persistent-request premises.

2. **Unfair priority arbiter (violates B2 / B3).**

A more naive design might implement:

```
if (req_high) grant_high();
else if (req_low) grant_low();
```

combined with an environment in which req_high can remain asserted indefinitely. In this case, even if req_low is also continuously asserted:

    - Pop_low's ChannelEnabled predicate may be true forever, but Pop_low is never selected by the scheduler/arbiter.
    - Low-priority requests can starve indefinitely, even though they are continuously eligible for selection.

This behavior violates B2, because B2 requires eventual selection of any continuously enabled action. If c_low's FIFO is full and the only way to make space is to service it, permanent starvation can also violate B3's progress expectations for that channel, because the full condition may never be discharged under the stated persistent-request premises.
In such a design, the safety properties E1–E5 may still hold (trace-compatibility on actions that do occur), but the liveness/results that depend on B2/B3—especially the starvation-vs-deadlock arguments in Appendix K—no longer apply.

Patterns that typically satisfy the fairness assumptions

The following design patterns normally satisfy B2 and are compatible with B3/B5, provided the rest of the system is well-behaved:

- **Round-robin or rotating-priority arbiters.** Any requester that remains enabled will eventually be at the front of the rotation and will be granted.
- **Age-based or credit-based arbiters.** Requesters accumulate age or credits while waiting; the arbiter prefers older or more-starved requests, guaranteeing that long-pending actions eventually win.
- **Bounded-burst fixed priority.** Fixed priority is combined with a counter that limits how long a higher-priority requester can dominate while lower-priority requesters are pending. After the burst limit is reached, lower-priority requests are forced to win until the system is "caught up."
- **Back-pressure with bounded stall.** When ready/valid or similar handshake signals are used, the design (or its environment) must enforce that ready cannot remain low forever while valid remains high, and vice versa. For example:
    - Downstream consumers are required (by specification) to service queues at least once every N cycles when data is present.
    - External bus masters are configured such that they cannot indefinitely defer reading from full DUT FIFOs that are logically part of the interface contract.
- **Clock-gating and power-management schemes that respect B5.** Once no observable actions are enabled, the design either:
    - Quietly stabilizes (no further internal toggling), or
    - Explicitly enters a low-power state from which it only wakes when some observable action again becomes enabled.

In each case, the intent is the same: continuous ChannelEnabled status implies eventual scheduler/arbiter selection and commit under B2; persistent full/empty/rendezvous-blocked request premises imply eventual discharge under B3; and once nothing is enabled, the system converges rather than oscillating internally.

Patterns that tend to violate the fairness assumptions

Conversely, the following patterns are likely to **break B2/B3/B5** and therefore fall outside the scope of the liveness arguments in this document:

- **Pure fixed-priority arbitration with unbounded high-priority traffic.** A static priority tree with no rotation or aging, combined with workloads in which a high-priority master can keep its request asserted indefinitely, can starve lower-priority channels forever.

- **“Best-effort” external masters with no progress guarantee.** For example:
    - A software driver that reads from a DUT output FIFO only when it happens to poll, and that may be pre-empted indefinitely by higher-priority threads.
    - An external bus or DMA engine that is architecturally allowed to ignore some requesters forever if higher-priority traffic remains heavy.
      Such environments can violate **P_push/P_pop** by allowing FIFOs to remain full or empty indefinitely while the DUT is continuously requesting progress.
- **Circular back-pressure loops with no escape.** Two or more processes form a cycle in which each is waiting for the other’s channel to change state, and no other process can break the cycle. Without an explicit design-level guarantee that some process in the loop will eventually act (for example, by dropping priority or issuing a compensating Pop/Push), **B3** is not satisfied.
- **Internal oscillation without quiescence.** A scheduler or control FSM that can toggle internal ε-level state forever, even after all observable actions are disabled, violates **B5**. While such designs are uncommon in practice, patterns involving mis-configured clock gating, asynchronous feedback, or “keep-alive” timers that never expire can have this effect.

In all these cases, the trace-compatibility theorems **still describe what happens when actions do occur**, but the liveness claims that depend on B2/B3/B5 (for example, “a continuously enabled channel will not starve”) are no longer guaranteed. Designers and verification engineers should therefore either:

- Architect their arbiters and environments to satisfy these B-rules, or
- Treat the liveness guarantees in Appendix K as *out of scope* for those particular interfaces, and rely only on the safety-oriented parts of the model.

Appendix O – Support for Multiple Clock Domains (Informative Sketch)

This appendix is an informative sketch of how the single-clock formalism of Appendices G–K could be organized in a multi-clock design. The formal results of Appendices G–K are stated for a single global clock unless an explicit multi-clock semantic layer is added.

Within a single clock domain d, one may treat that domain as a synchronous island with a local cycle counter clk_d and apply the existing R- and E-rules to processes wholly inside that domain, interpreting clk(·) as clk_d(·) for local synchronization, signal I/O, and message-passing events. Inter-domain communication should be represented by explicit clock-domain-crossing components, such as CDC FIFOs, whose source-side and destination-side commit semantics are specified at their respective clock-domain boundaries.

A full formal lift requires additional definitions not supplied by Appendices G–K themselves: a global event order or partial order relating local-domain cycles, precise CDC commit semantics, occupancy sampling rules across source and destination clocks, synchronizer fairness/metastability abstraction assumptions, and progress obligations for CDC components. Once those additional definitions are supplied, the intended proof strategy is to treat each synchronous island using the existing single-domain rules and to treat each CDC component as an explicit channel/refinement boundary with its own stated E4-style legality and progress obligations. Without those additional definitions, Appendix O should not be read as claiming that the single-clock proofs apply unchanged to arbitrary multi-clock systems.

Appendix P – AI Discussion of Alternative Formal Frameworks

The following links provide an AI discussion of alternative formal frameworks compared to the one presented in this document:

https://drive.google.com/file/d/1ccU-eRW7H2ggh-AZyKnCOzO-KLPejob_/view?usp=sharing

https://chatgpt.com/share/69457119-ec30-8006-8684-9a2a8b19f96f

https://drive.google.com/file/d/1R3-46DtZLsBn1hdTbLEXyC0FVB50Gymz/view?usp=sharing

Appendix Q – Multiple Input Pipelines Back-Annotation

Appendix J Remark 2 discusses back-annotation of pipelined loops with a single message passing input. In more complex cases such as HW pipelines with multiple message-passing inputs, the back-annotation mechanism may elaborate the Sys_B (source-level) simulation by introducing additional internal realization structure. In this document, we adopt the following simplified modeling discipline:

1. Terminology (elaboration and channel accounting). In an elaborated realization, the quantities B(·), occ_·(t), and the E4 availability predicate are interpreted over the explicit abstract channels present in the elaborated model (including any introduced staged/bundle channels). When we continue to write a source-level channel name c at the outer DUT/testbench observation boundary, we mean the chosen outer abstract boundary channel associated with c in that elaboration; downstream staging that is reachable only through a join (and therefore may be withheld when "join not met") is represented by distinct internal channels with their own B(·) and occ_·(t), rather than as independent "slack" for c in isolation.
2. All buffering / capacity effects are represented explicitly as finite-capacity message channels (annotated with bounded capacity) that may appear upstream and/or downstream of any internal glue logic; and
3. Any internal "coupler" used to enforce mutual-stall between multiple inputs is purely combinational (stateless) handshake logic.

Observation-boundary convention for Appendix Q. In elaborated cases, distinguish the outer projected observation boundary from the proof-internal abstract boundaries. The outer Σ_DUT / testbench projection records only the chosen outer Push_c / Pop_c history after applying Π_elab. By contrast, Appendix K WFG reasoning is carried out over the explicit abstract channels of Sys_ref = Sys_B^elab, including introduced staged/bundle/join channels. Thus an introduced internal channel may be hidden by Π_elab from the outer DUT-visible trace while still being an explicit proof-internal boundary with its own B(·), occ_·(t), BoundaryReq, ChannelEnabled/ChannelDisabled, Front_P, and WFG facts.

Concretely, a combinational coupler is a stateless component that (a) reads the upstream channels' valid and data signals and computes/drives the downstream channels' valid and data signals, and (b) reads downstream ready signals and upstream valid signals and then computes/drives upstream ready signals. The coupler contains no internal state and has no separate credit/capacity state; any shared pipeline

capacity, per-input staging, skid/elastic buffering, or credit-like effects are modeled only by the explicit finite-capacity channels inserted by the back-annotation realization.

Example (two inputs, staged Pops, II=1, latency=4):
Consider a 4-stage RTL pipeline that (i) performs Pop_c1 in stage 1, (ii) performs Pop_c2 in stage 2, and (iii) performs Push_cout in stage 4, with II=1. To model the pipeline's internal staging and the stage-2 input coupling in the source-level Sys_B^elab simulation while keeping the same outer projected Σ boundary, the back-annotation elaboration may introduce the following internal realization structure:

- An explicit bounded **channel F1 of capacity 1** on the c1 path to represent stage-1 in-flight storage of values that have been accepted from c1 but have not yet reached the stage-2 join point.
- An explicit internal **rendezvous channel F2** on the c2 path at the stage-2 join boundary, with B(F2)=0. F2 represents the c2-side handoff into the stage-2 join. It is not additional c2-side buffering or slack, and it does not permit a c2 value to be accepted independently before the join is ready. Its purpose is to make the c2-side join blocking point an explicit proof-internal abstract boundary of Sys_B^elab, so that any mutual-stall condition involving c1/F1 and c2/F2 is represented as ordinary BoundaryReq / ChannelEnabled / ChannelDisabled facts on explicit internal channels rather than as a hidden cross-channel disablement predicate on the outer projected c2 boundary.
- A **purely combinational coupler/join** at the stage-2 boundary that enforces mutual stall: it allows the joined transfer to proceed only when (a) a token is available from F1, (b) the c2-side rendezvous over F2 is requested/enabled, and (c) downstream staging has space; it then presents the paired value(s) to the downstream staging. The join's possible refusal to proceed is therefore visible to Appendix K through the explicit internal boundaries F1, F2, and F34, even though those internal events are projected away from the outer DUT/testbench-visible Σ_DUT trace by Π_elab.
- An explicit bounded **channel F34 of capacity 2** after the coupler to represent the in-flight capacity of stages 3–4 for the joined/paired work.

This example illustrates why multi-input pipelines cannot always be captured by a single per-channel capacity number in isolation: c1 and c2 do not have independent "slack" because acceptance across the stage-2 join boundary is coupled. In this particular example, the c1 side has one stored stage-1 token available through F1, the c2 side has no independent pre-join storage and is therefore modeled by the rendezvous channel F2 with B(F2)=0, and the joined/paired work has two downstream storage slots through F34.

All storage-bearing buffering/capacity resides in the explicit bounded channels F1 and F34, and in any additional positive-capacity stages the realization uses. The explicit rendezvous channel F2 has B(F2)=0 and contributes no storage; it is an explicit proof-internal join boundary used to type the mutual-stall condition. The coupler itself remains stateless combinational handshake logic.

This elaboration is an implementation/refinement of the abstract bounded-FIFO channels at the chosen outer DUT/testbench observation boundary; it does not change the outer interface Σ_DUT or E4's per-channel FIFO meaning at that outer boundary. It may, however, introduce additional proof-internal Σ boundaries in Sys_ref = Sys_B^elab for Appendix K blocking/WFG reasoning. Concretely, any back-annotation elaboration (including extra FIFO stages and/or combinational couplers) must satisfy:
• No new outer DUT-boundary observables: It introduces no new Σ_DUT-observable actions at the chosen DUT/testbench boundary, and it does not change the labeling of existing outer Σ_DUT events (Push_c(v), Pop_c(v), etc.).

• Boundary/E4 preservation at the outer projection: At the chosen outer DUT/testbench boundary, each outer Σ_DUT-visible committed Push_c/Pop_c event still denotes a committed transfer on the same abstract channel c, with per-channel FIFO order and the same E4 capacity/enablement interpretation under the annotated B(c) and occ_c(t). Any additional internal channels introduced by the elaboration (e.g., staged/bundle/join channels) are separate proof-internal abstract channels in Sys_B^elab with their own B(·), occ_·(t), and E4 availability predicates; if they participate in Appendix K, their committed transfers are proof-internal Σ-events even when Π_elab hides them from Σ_DUT.
• No double-counting of internal staging: internal transfers within the concrete realization of a single explicit abstract channel that contributes to that channel's effective capacity (extra FIFO stages, skid/elastic buffering, FIFO→pipeline-stage handoffs, and value propagation through purely combinational couplers that does not cross an explicit abstract channel boundary) are ε-steps and do not create additional committed Push_c/Pop_c events at that abstract boundary. By contrast, a transfer across an explicit staged/bundle/join channel introduced in Sys_B^elab is a committed Push/Pop event at that proof-internal abstract boundary. Such an event may be projected away from the outer DUT/testbench-visible Σ_DUT trace by Π_elab, but it is not ε for the Appendix K proof-internal WFG boundary.
• Conservation / projection of internal staging (normative): For each outer Σ_DUT-visible abstract channel c at the chosen DUT/testbench boundary, the elaborated realization shall admit a fixed projection/accounting map from the occupancies of the explicit internal channels introduced by the elaboration (including staged/bundle/join channels, as applicable) to the outer boundary occupancy occ_c(t). This projection/accounting map is part of the realization and witnesses that the proof-internal occupancy bookkeeping is only a refinement of the outer boundary model: (i) every ε-step internal transfer and every committed transfer across an introduced proof-internal abstract channel preserves the projected outer boundary occupancy for each outer Σ_DUT-visible channel, except when it corresponds under Π_elab to an outer boundary commit, and (ii) each outer Σ_DUT-visible committed Push_c/Pop_c updates the projected outer boundary occupancy exactly as required by E4 (equivalently, exactly as the outer boundary occ_c(t) defined from Σ_DUT-visible boundary commits). For bundled/joined work items, the projection may assign one proof-internal token to multiple outer Σ_DUT-visible channels as specified by the realization. Therefore, the internal capacities/occupancies B(·), occ_·(t) do not introduce an additional independent outer Σ_DUT-visible state; they refine the same bounded-FIFO accounting already represented at the chosen outer boundary, while still serving as proof-internal Σ state for Appendix K when those internal channels are explicit abstract boundaries.
• **Multi-input pipelines / combinational couplers (mutual stall + explicit capacity):**
To preserve per-channel E4 at the Σ boundary, any downstream staging that is reachable only through a join (and therefore may be withheld when "join not met") shall be modeled as distinct internal finite-capacity channels with their own B(·) and occ_·(t), rather than being treated as additional independent "slack" for an upstream Σ-visible channel in isolation; the back-annotation elaboration may use combinational couplers to enforce the intended mutual-stall behavior when some required inputs are unavailable. Any "shared capacity" or "distributed staging" effects needed for such a pipeline shall be represented only by explicit finite-capacity channels in the realization (e.g., bounded staging FIFOs, bounded slot/credit channels, bounded bundle channels), not by coupler state. The coupler itself remains stateless combinational handshake logic and does not redefine the per-channel E4 capacity/availability predicates at the chosen observable boundary.

Projection/accounting invariant for Sys_B^elab (normative). Any elaborated reference model Sys_B^elab used for multi-input / coupler cases shall come with a projection map Π_elab from the explicit staged/bundle/join channels of the elaboration back to the chosen outer Σ-visible channel boundaries. This projection map has these roles:

• Observable-boundary preservation. When the internal staged/bundle/join events introduced by elaboration are hidden or projected away, the remaining outer-boundary Push_c(v) / Pop_c(v) history is the same boundary history that the non-elaborated model would expose at the chosen Σ-visible endpoints. Elaboration may add explicit internal abstract channels for proof and WFG purposes, but it may not create extra outer-boundary Push_c / Pop_c events, drop outer-boundary events, duplicate them, or change their payload labels.

• Capacity/occupancy accounting. For each outer channel c represented by an elaborated subnetwork, the outer-boundary availability facts used by E4 are preserved by the aggregate accounting of the explicit internal channels in the preimage/fiber Π_elab^{-1}(c) = { d : Π_elab(d) = c }. Internal staged/bundle/join occupancies may refine where storage and blocking occur, but after projection they must implement the same externally visible capacity/acceptance behavior at the chosen boundary. Conversely, any cross-channel dependency or join condition that can block progress must appear as ordinary BoundaryReq / ChannelEnabled / ChannelDisabled facts on some explicit internal abstract channel of Sys_B^elab, not as a hidden extra predicate on an otherwise E4-enabled outer-boundary operation.

All examples and constraints in Appendix Q are to be read relative to this projection/accounting invariant.

Explicit join boundary requirement (normative): The mutual-stall point shall be an explicit staged/bundle/join boundary inside the elaborated realization (i.e., the endpoint of an explicit finite-capacity message channel). “Join not met” shall therefore be expressed as ordinary ChannelDisabled at that explicit internal boundary (because that boundary’s own E4 availability predicate fails in the ε-quiescent handshake fixed point). The realization must not force a Σ-visible operation on an abstract channel c at the chosen boundary to be ChannelDisabled solely due to the state of some other channel; any cross-channel dependency shall be expressed only via the explicit staged/bundle channels introduced by the elaboration. Operationally, the coupler is invisible at the outer DUT/testbench observation boundary: it only propagates ready/valid between the explicit staged/bundle channels inside the elaborated realization, and it does not change the projected outer occ_c(t), which is defined solely by projected outer-boundary commits under Π_elab. This invisibility is only with respect to the outer projection; it does not hide any explicit staged/bundle/join channel from Appendix K’s proof-internal WFG boundary.

- WFG compatibility (Appendix K): The realization must not introduce a new blocking point that is invisible to the proof-internal abstract boundary of Sys_B^elab. In particular, if the consuming process cannot advance the pending blocking frontier Front_P(σ) (Appendix K.1), then at least one blocking frontier operation (Push/Pop) at either the chosen outer boundary or an introduced explicit staged/bundle/join boundary is asserted with a true boundary request and is channel-disabled by the K.1 enablement test; the process is therefore “blocked on message passing” exactly as defined in K.1. Because couplers are purely combinational, they have no internal state transitions; any changes in readiness/validity induced by the coupler are purely a function of (i) downstream readiness and (ii) the same channel occupancies / Σ-visible committed transfers already accounted for by E4/K.1, with “downstream readiness” interpreted in the ε-quiescent (ε-normalized) state used for WFG evaluation.

Pure internal data movements that do not change any proof-internal BoundaryReq, ChannelEnabled/ChannelDisabled, Pending_P, or Front_P fact are ε-steps and are WFG-inert under the

same pending-blocking-frontier / Front_P(σ) discipline used elsewhere in Appendix K. By contrast, a staged/bundle/join channel boundary that can block progress is not hidden from Appendix K; it is part of the explicit Sys_B^elab proof boundary, even if it is projected away from the outer Σ_DUT trace by Π_elab.
• **Combinational well-formedness:** The network of combinational couplers and ready/valid connections shall be combinationally acyclic (no pure combinational ready/valid loops). Any required feedback that would otherwise create a combinational loop must pass through an explicit finite-capacity channel stage (and is therefore represented in the back-annotated realization and its effective capacities).

Accordingly, the formal development need not depend on the particular back-annotation construction, provided it yields an explicit source-side reference model Sys_ref = Sys_B^elab (Appendix J, Remark 2) equipped with a projection/accounting map Π_elab satisfying the invariant above. The elaborated model must respect E4, BoundaryReq, ChannelEnabled/ChannelDisabled, and the Appendix K blocking/WFG semantics at its explicit proof-internal abstract boundaries, while its projection back to the chosen outer Σ_DUT-visible boundaries must preserve the externally visible Push_c / Pop_c history and aggregate capacity/occupancy behavior. In these cases the proofs do not target the raw outer-boundary Sys or the non-elaborated outer-boundary Sys_B directly; they target the proof-internal explicit abstract channels of Sys_B^elab together with their Π_elab projection back to the outer DUT/testbench-visible boundary.

Lean-oriented clarification / proof-parameter convention

The formal theorems do not require a unique algorithm that constructs Sys_B^elab from RTL_B. Instead, the chosen comparison boundary Sys_ref is part of the theorem instance. For ordinary bounded-FIFO cases, Sys_ref may be Sys_B. For cases involving proof-relevant internal staging, multi-input joins, combinational couplers, sync-boundary epoch gates, or shared-capacity effects, Sys_ref shall be supplied as Sys_B^elab(E) for some well-formed elaboration package E.

Thus elaboration is not an extra semantic freedom inside the proof. It is an explicit input to the proof instance, and all Appendix I–K definitions are interpreted over that chosen Sys_ref.

Definition Q.1 (Elaboration package for Sys_B^elab).

For a chosen outer source reference boundary Sys_B and outer observable alphabet Σ_DUT, an elaboration package E consists of:

1. an elaborated source reference model Sys_B^elab(E);
2. a finite set of explicit abstract channels Chan_elab(E), each with finite capacity B_E(d) ≥ 0 and cycle-boundary occupancy occ_d(t);
3. a distinguished subset of outer-boundary channels Chan_outer and a projection/accounting map

$$\Pi_{elab} : Chan_{elab}(E) \to Chan_{outer} \cup \{\bot\};$$

4. a trace projection map π_Σ,E from proof-internal Σ_elab traces to outer Σ_DUT traces; and
5. a state-accounting map A_E that computes each outer occ_c(t) from the occupancies and token/accounting state of the elaborated channels in Π_elab^{-1}(c).

The package is well formed only if the following obligations hold:

(a) Outer-trace preservation. π_Σ,E neither creates, drops, duplicates, nor relabels any outer-boundary Push_c(v) / Pop_c(v) event.

(b) Occupancy preservation. For every outer channel c and every ε-quiescent cycle-boundary state σ at time t,

$$occ_c(\sigma) = A_E(c, \sigma),$$

and projected Push_c / Pop_c commits update A_E exactly as required by E4 for the outer channel c.

(c) Internal refinement. Any internal staged/bundle/join transfer either is projected away by π_Σ,E or projects to a unique corresponding outer-boundary commit. Projected-away internal transfers preserve all outer occupancies A_E(c, σ).

(d) No hidden cross-channel disablement. If progress can be blocked by a join, mutual-stall condition, sync-boundary gate, or shared-capacity condition, then that blocking condition appears as ordinary BoundaryReq / ChannelEnabled / ChannelDisabled status at some explicit abstract channel $d \in Chan_{elab}(E)$. It is not an additional hidden predicate on an otherwise E4-enabled outer-boundary operation.

(e) WFG visibility. Every proof-relevant blocking point introduced by the elaboration is represented at an explicit abstract boundary of Sys_B^elab(E), and therefore may be used by Appendix K's Front_P, ChannelDisabled, and WFG definitions, even if its events are projected away from the outer Σ_DUT trace.

(f) Combinational well-formedness. Any combinational coupler logic in the elaboration is stateless and acyclic after ε-settling; any feedback or storage needed for correctness is represented by an explicit finite-capacity abstract channel.

(g) Point-to-point elaborated channels. Every explicit abstract channel in Chan_elab(E) at which BoundaryReq, ChannelEnabled/ChannelDisabled, Front_P, or Appendix K WFG facts are evaluated has exactly one producer process and exactly one consumer process at that boundary of evaluation. A join, bundle, or shared-capacity structure that would multiplex several producers or several consumers at an evaluated boundary shall be remodeled with explicit per-endpoint point-to-point sub-channels (for example, via an explicit arbiter process), as required by the point-to-point premise of Lemma K.2b.

(h) Frontier-item declaration and boundary attribution. The elaboration package declares every proof-internal frontier item introduced by Sys_B^elab(E), including its owning process, its membership in Ω_P / Ω_msg,P when applicable, its owning source-level message instance MsgOf_P(x), its evaluated abstract boundary BoundaryOf_P(x) ∈ Chan_elab(E), where P is the owning process of x, and the partial order ≤_P among ordinary source items and introduced elaborated frontier items. For ordinary unelaborated message calls there is a unique frontier item and a unique evaluated boundary. For elaborated calls, one source-level message instance q may own multiple frontier items at different explicit abstract boundaries; in that case BoundaryReq, ChannelEnabled, ChannelDisabled, Pending_P, and Front_P are interpreted for the pair (q, BoundaryOf_P(x)) represented by the frontier item x, not for q alone. Thus BoundaryReq(q,σ) remains only a shorthand for the ordinary unique-boundary case, while BoundaryReq(x,σ) means the endpoint-owned request for MsgOf_P(x) at BoundaryOf_P(x). The package also states how these introduced items project through π_Σ,E / Π_elab back to outer Σ_DUT events and occupancy accounting.

When Sys_ref = Sys_B^elab(E), all references to B(·), occ_·, BoundaryReq, ChannelEnabled/ChannelDisabled, Front_P, and the Appendix K WFG are interpreted over the explicit channels of Chan_elab(E), with outer observations recovered only through π_Σ,E and Π_elab.

Locality of elaboration

The elaboration package E is constructed locally, per process and per channel boundary. Each elaboration pattern in this appendix (multi-input pipelined-loop staging, sync-boundary epoch gating, R2C combinational coupler, etc.) is determined by the local channel structure of the process at the boundary in question — its number of input channels, its sync-boundary discipline, its pipelining configuration — and does not depend on the structure or elaboration of any other process. Independently chosen per-process elaborations compose into a system-level Sys_B^elab(E) without further global modeling. Π_elab then projects the introduced internal channels back to the outer Σ_DUT alphabet on a per-channel basis. In this sense, the elaboration step preserves the per-process compositionality stated in the introduction: it refines each process's abstract boundary, it does not introduce cross-process modeling obligations.

Appendix R – Compact Formal Core for Appendices B–K

R.1 Purpose and scope

Numbering convention. Definitions in this appendix use the numbers R.1–R.20, with letter suffixes (for example, Definition R.1a) for attached definitions. Theorems, lemmas, and corollaries use the numbers R.21 onward, continuing the same sequence, so that every type-plus-number pair in this appendix is unique; a letter suffix such as Corollary R.21a denotes attachment to the same-numbered parent item (here, Theorem R.21). Axiom R.1 and the Remarks retain their per-type numbering.
This appendix packages the common formal core already used by Appendices B, C, J, and K into a compact definition/lemma/theorem form. Its purpose is only to factor out the shared semantic skeleton and finite-prefix cutpoint view of the formal story. All terminology, assumptions, and proof targets remain those of the main document. In particular, the prescribed source-side semantic target remains exactly the Appendix J, Remark 2 target: Sys_ref = Sys_B in the ordinary abstract back-annotated presentation using the case-split channel semantics of E4 (B(c)=0 as rendezvous, B(c)>0 as cycle-boundary non-fall-through bounded FIFO), and Sys_ref = Sys_B^elab in the elaborated/back-annotated / coupler cases. Thus the proofs here do not introduce any proof target separate from the Appendix J target family.

Definition R.1a (Elaboration projection/accounting map Π_elab).
In the elaborated case Sys_ref = Sys_B^elab, Π_elab denotes the fixed projection/accounting map associated with the chosen elaboration. It maps proof-internal executions, events, and channel-occupancy state of Sys_B^elab to the chosen outer DUT/testbench-visible boundary view. Π_elab has the following roles:
(1) Trace projection. Π_elab preserves the labels and order of the chosen outer Σ_DUT-visible boundary events, such as outer Push_c(v) and Pop_c(v), and hides or projects away introduced proof-internal staged/bundle/join channel events that are not part of the chosen outer observation boundary.
(2) Occupancy projection. For each chosen outer Σ_DUT-visible abstract channel c, Π_elab maps the occupancies of the explicit internal channels introduced by the elaboration to the outer occupancy occ_c(σ) used for the outer E4 interpretation.

(3) Conservation/accounting. Internal ε-steps and committed transfers across introduced proof-internal abstract channels preserve the projected outer boundary history and projected outer occupancy, except when the elaboration designates such a transfer as corresponding to an outer boundary commit under Π_elab. Each projected outer Push_c/Pop_c commit updates the projected outer occ_c exactly as required by E4. For bundled or joined work items, Π_elab may assign one proof-internal token to one or more outer channels as specified by the elaboration.
(4) WFG visibility. Projection by Π_elab does not make an introduced proof-internal blocking boundary invisible to Appendix K. Any staged/bundle/join channel used for proof-internal blocking, BoundaryReq, ChannelEnabled/ChannelDisabled, Pending_P, Front_P, or WFG reasoning remains an explicit abstract channel of Sys_B^elab, even if its committed events are projected away from the outer Σ_DUT trace.

Thus Π_elab witnesses that Sys_B^elab is a refinement of the chosen outer boundary model: it may expose additional proof-internal abstract channels for Appendix K, but it does not add independent outer DUT-visible behavior or alter the projected outer Push/Pop history and E4 occupancy accounting.

This appendix should therefore be read as a compact companion to the existing appendices, not as a replacement for the full modeling assumptions, witness constructions, fairness/progress premises, or elaboration conventions already stated there.
Appendix B continues to supply the general drain-only blocked-frontier discipline, with automatic flush as its pipelined or otherwise overlapped implementation-side instance and a structural no-new-dynamic-operation-launch argument as the ordinary non-overlapped discharge. Appendix C supplies the Issue/Commit linkage and non-withdrawal discipline, Appendix I the witness construction machinery, Appendix J the full execution-level / cutpoint-correspondence results, Appendix L the witness-selection / snooping wrapper where needed, Appendix N the cited progress premises, and Appendix K the wait-for-graph and deadlock-preservation layer.
Corollary J.1 is the authoritative equal-KObs cutpoint theorem used by Appendices J–K. Appendix R introduces no alternate clause relation or compatibility symbol: cross-model arguments cite the common KObs record and the named reconstruction functions directly. Whenever abbreviated wording in this appendix about Issue, BoundaryReq attribution, or channel state appears less precise than Appendices C, J, or K, the authoritative definition in those appendices governs.

R.2 Prescribed reference model

Definition R.1 (Prescribed source reference model Sys_ref).

For a fixed design and fixed external stimulus, let Sys_ref be exactly the prescribed source reference model of Appendix J, Remark 2:
• Sys_ref = Sys_B in the ordinary abstract back-annotated presentation using the case-split channel semantics of E4: B(c)=0 is rendezvous, and B(c)>0 is cycle-boundary non-fall-through bounded FIFO.
• Sys_ref = Sys_B^elab in the elaborated/back-annotated case.
All source-side semantic notions used below are interpreted with respect to this prescribed source reference model.

Remark R.1 (Interpretation at explicit abstract channels).

Whenever Sys_ref = Sys_B^elab, the quantities B(·), occ_·(t), E4, BoundaryReq, ChannelEnabled, ChannelDisabled, and the Appendix K blocking / wait-for-graph machinery are interpreted per explicit abstract channel, including any staged, bundle, or join channels introduced by elaboration. Any conjunctive/join dependency shall therefore be represented only via such explicit abstract channels, not as cross-channel disablement at the outer Σ-visible endpoints.

Remark R.1a (Operational Sys_ref transition system and issue policies).

Sys_ref is the selected source-reference transition system described in Appendix G. Its states contain process-local control/variable state, reached/issued/committed dynamic message-call instances, explicit abstract channel state for every Sys_B or Sys_B^elab boundary, operation-attributive pending endpoint requests, synchronization/signal-anchor state, and ε-microstate. Its steps are pure source-evaluation ε-steps, message issue steps, E4 channel-commit steps, synchronization/signal-anchor steps, and admissible ε-settling/drain steps. Policy L and Policy S are two issue policies over this same transition system. Policy L permits source-later blocking requests to be issued before source-earlier same-interval blockers process-facing commit, subject to source-order prefix closure and all R/E/E4/E5 obligations. Policy S adds the conservative serial-blocking restriction: a later source-order blocking Push/Pop may issue only after every earlier source-order blocking Push/Pop in that interval has process-facing committed in a strictly earlier cycle. In Sys_B^elab, process-facing commit is the commit that releases the owning source-level operation at its selected boundary, not an arbitrary internal staged/bundle/join transfer.

R.3 Observable alphabet and executions

Definition R.2 (Observable alphabet Σ).

Let Σ be the observable event alphabet consisting of:
(1) committed message-transfer events Push_c(v) and Pop_c(v),
(2) synchronization events selected into S, and
(3) signal events Read(sig, v) and Write(sig, v), interpreted as anchored signal events for sequential processes under R2 and as ε-fixed-point cycle-bucket events for combinational processes under R2C. Under the pipelined-loop body no-wait condition, an HLS-pipelined loop body contributes no additional in-body wait/synchronization events to S or Σ.

Definition R.3 (ε-steps and compact trace notation).

Let ε denote internal micro-steps that are not observable at the proof-internal alphabet Σ, including internal staging/hand-off within the concrete realization of a single explicit abstract channel, late direct-input sampling between anchors, and failed non-blocking polls. In elaborated cases Sys_ref = Sys_B^elab, transfers across introduced staged/bundle/join channels are not ε merely because Π_elab hides them from the outer DUT/testbench-visible alphabet Σ_DUT; if such a channel is an explicit abstract boundary used by Appendix K, its committed Push/Pop transfers are Σ-events for the proof-internal comparison.
A finite execution prefix τ is understood in the same sense as in Appendix J: it carries the inherited cycle partition / per-cycle observable buckets of the execution, together with any ε-steps between successive Σ-visible cutpoints. For compact notation one may write a linearization over (Σ ∪ {ε}), but only modulo permutation of Σ-events that occur in the same cycle and are not additionally ordered by the document's explicit Σ-visible constraints. Accordingly, τ|Σ denotes the observable projection together with that inherited cycle partition; same-cycle observable events are not assigned any extra order merely by this compact notation.

Definition R.4 (ε-quiescent cutpoint).

A state σ is ε-quiescent if no further ε-step is enabled from σ. For any finite prefix τ, let $\bar{\tau}$ denote a maximal finite ε-extension to an ε-quiescent endpoint, preserving the same observable cycle partition / bucket structure on τ|Σ. The Appendix J / K state-comparison reasoning is always performed at such ε-quiescent cutpoints.

R.4 Source order and source-level frontier items

Definition R.5 (Per-process execution-call universe, frontier-item universe, observable projection, and source order).

For each process P, let Q_exec,P denote its execution-level set of dynamic source-level message-call instances for the particular execution / witness under the fixed external stimulus being analyzed. Q_exec,P includes blocking Push/Pop calls, successful non-blocking PushNB/PopNB calls, and failed non-blocking PushNB/PopNB polls.

Let Ω_P denote P's local dynamic source-level frontier-item universe for that same execution / witness. Thus Ω_P is stimulus-dependent: it contains exactly the local synchronization-call items, signal-action items, and message-operation frontier items that belong to the realized execution being compared, and excludes items on untaken control-flow branches, unentered cases, and unexecuted loop iterations. For sequential processes, signal-action items are positioned by the R2 anchoring rule; for combinational processes, signal-action items are positioned in the observable cycle bucket determined by the R2C ε-fixed-point rule. Message-operation frontier items include reached blocking Push/Pop instances, whether pending or committed, and successful non-blocking PushNB/PopNB instances that contribute committed Push_c(v) / Pop_c(v) events. Failed non-blocking polls are ε-steps: they are members of Q_exec,P when executed, but they are not members of Ω_P merely because they executed.

Because Q_exec,P is a universe of dynamic message-call instances while Ω_P is a universe of source-level frontier items, their message-operation overlap is not a literal set intersection. Let MsgOf_P(x) be the partial projection from a frontier item x ∈ Ω_P to its owning dynamic message-call instance, defined exactly for message-operation frontier items and undefined for pure synchronization or signal-anchor items. Define the message-frontier slice

Ω_msg,P := { x ∈ Ω_P | ∃ q ∈ Q_exec,P : MsgOf_P(x) = q }.

For any message-operation frontier item x ∈ Ω_msg,P, let BoundaryOf_P(x) denote the explicit abstract channel boundary at which x is evaluated for BoundaryReq, ChannelEnabled, ChannelDisabled, Front_P, and Appendix K WFG reasoning. In the ordinary unelaborated case, each source-level message-call instance q has a unique corresponding frontier item x and a unique evaluated boundary. In Sys_B^elab, MsgOf_P need not be injective: one owning source-level message-call instance q may project to a chain or bundle of proof-internal frontier items x at several explicit abstract boundaries. The elaboration package must then provide BoundaryOf_P(x), MsgOf_P(x), and the partial order ≤_P for each introduced item.

Define the corresponding message-instance slice induced by Ω_P as

Q_Ω,P := { q ∈ Q_exec,P | ∃ x ∈ Ω_msg,P : MsgOf_P(x) = q }.

Let Σ_P denote the corresponding observable event/label projection: synchronization and signal items contribute their Σ labels when observed, and a message-operation frontier item x ∈ Ω_msg,P with MsgOf_P(x)=q contributes the committed Σ-event m(q) only if q commits. Pending blocking message-operation frontier items may therefore be in Ω_P and Ω_msg,P, and their owning message-call instances may be in Q_Ω,P, without yet contributing any committed Σ-event. Let ≤_P denote the source order on Ω_P, i.e. the order generated by the document's scheduling rules for synchronization calls, signal actions as positioned by R2 or R2C, message-operation frontier items, and any explicit partial-order structure

declared by the selected Sys_ref elaboration. The execution-level order constraints E3/E5 use the corresponding source-induced order on Q_exec,P.

Definition R.6 (Done_P, Remain_P, and NextFront_P relative to the fixed witness).
Fix the particular execution / witness under analysis, hence the associated Q_exec,P, Ω_P, Ω_msg,P, Q_Ω,P, Σ_P, and ≤_P from Definition R.5. For a state σ occurring as a cutpoint of that compared execution, let
Done_P(σ) ⊆ Ω_P
be the set of local frontier items already realized by σ. For synchronization and signal items, "realized" means that the corresponding Σ-event has occurred. For a message-operation frontier item x ∈ Ω_msg,P with MsgOf_P(x)=q, "realized" means that q has committed and its committed Σ-event m(q) has occurred. A pending issued-but-uncommitted blocking message-operation frontier item x is therefore not in Done_P(σ), even though BoundaryReq(x,σ), ChannelEnabled(x,σ), and ChannelDisabled(x,σ) may be defined at σ, read operation-attributively through q = MsgOf_P(x). A failed non-blocking poll q ∈ Q_exec,P that induces no frontier item in Ω_msg,P is not considered by Done_P, Remain_P, or NextFront_P, because it is an ε-step with no frontier item and no committed Σ-event.
Let
Remain_P(σ) := Ω_P \ Done_P(σ).
Then define the next source-level frontier-item set by
NextFront_P(σ) := min_≤P(Remain_P(σ)).
This is the Appendix J next-frontier notion used by Theorem J.1 and reconstructed from KObs by Lemma R.21b, interpreted relative to the fixed witness universe Ω_P. It may contain synchronization-call items, signal-anchored action items, and message-operation items. It is a frontier of source-level items, not a set of already-committed Σ-event labels.

Definition R.7 (Message-operation slice of the next frontier).
For the same fixed compared execution / witness, define
MsgNextFront_P(σ) := { x ∈ NextFront_P(σ) | x ∈ Ω_msg,P and, for q = MsgOf_P(x), q is defined and kind(q) ∈ {Push_c, Pop_c} for some channel c }.
This is only the message-operation frontier-item slice of the candidate next frontier; it is not yet the pending blocking frontier of Appendix B / K.
Clarification R.5a (ordinary source order versus elaborated partial order). For ordinary user-written sequential source code, dynamic message-call instances that appear in sequence in one process are ordered by dynamic program order, including distinct-interface calls, repeated loop iterations, and unrolled instances whose selected source expansion is ordered. Same-cycle issue, same-cycle commit, or issue before an earlier operation commits is a timing/issue-policy property, not incomparability. A message-operation multi-frontier exists only when an explicit Sys_B^elab elaboration package introduces incomparable proof-internal frontier items and supplies the corresponding projection, boundary, order, and attribution facts. Ordinary distinct-interface source calls are not made incomparable merely because they access different channels.

R.5 Commit, issue, and boundary requests

Definition R.8 (Commit).

For a message transfer at a chosen Σ-visible boundary endpoint, Commit(q) is the cycle in which an external clocked observer sees both valid and ready high simultaneously at that endpoint. The committed payload is the value seen in that same cycle.

Definition R.9 (Issue).

For a source-level message operation instance q at a chosen Σ-visible boundary endpoint, Issue(q) is the first cycle in which the calling process begins requesting the transfer corresponding to q at that endpoint. Issue(q) is an auxiliary scheduler/witness timestamp used for rules such as R4, E3, and E5.

Definition R.10 (Endpoint-owned boundary request).

Fix a Σ-visible endpoint direction at an explicit abstract boundary. For any message-operation frontier item x ∈ Ω_msg,P with owning source-level message instance q = MsgOf_P(x) and evaluated boundary c = BoundaryOf_P(x), let req_x(t) denote the endpoint-owned boundary request signal for that endpoint in cycle t, namely valid for a Push endpoint and ready for a Pop endpoint. The predicate BoundaryReq(x, σ) records that this endpoint-owned request is asserted for frontier item x at state σ. BoundaryReq(x,σ) is operation- and boundary-attributive: it is true only when the asserted endpoint-owned request at σ is attributed to the specific frontier item x, its owning source-level operation q, and its evaluated boundary BoundaryOf_P(x). In particular, if Issue(q) has not yet occurred at σ, then BoundaryReq(x,σ) is false. In the ordinary unelaborated case, where q has a unique frontier item and a unique evaluated boundary, BoundaryReq(q,σ) may be used as shorthand for BoundaryReq(x,σ). In Sys_B^elab, the x-form is authoritative.

Axiom R.1 (Issue/Commit linkage).

Fix a Σ-visible endpoint direction. For source-level message operation instances on that endpoint direction, the following all hold:
(1) Boundary-request soundness / non-withdrawal.
If q is blocking, then once issued its endpoint-owned request remains asserted until q commits, with stable payload while asserted for Push.
(2) Stable attribution while the same source-level request remains outstanding.
If the request remains asserted across a cycle in which no transfer commits on that endpoint direction, then the continued assertion is attributed to the same outstanding q only so long as the requesting source-level operation remains that same outstanding instance. For blocking Push/Pop, this is the mandatory non-withdrawable case. For non-blocking PushNB/PopNB, continuous assertion of the endpoint request signal does not collapse later executed non-blocking call instances into the same q; each later executed source-level poll is a distinct q′ ∈ Q_exec,P unless the witness explicitly identifies the same still-outstanding source-level request instance.
(3) Back-to-back issuance.
If q_o and q_1 are same-direction source-level message operation instances, q_o commits in cycle t, and q_1 is the next such operation issued back-to-back within the same source interval, then Issue(q_1) = t + 1, even if the request signal does not deassert between commits.
(4) Sync-boundary discipline.
If q_o and q_1 are same-direction source-level message operation instances and q_o ∈ pref_S(s) and q_1 ∈ suff_S(s) for a synchronization call s, then Issue(q_1) occurs strictly after the commit of s. Any request assertion that persists while the process is still pending at s is attributed to the outstanding operation(s) in pref_S(s), not to issuance of any operation in suff_S(s).

Remark R.2 (Non-blocking polls).

For non-blocking message passing, a failed poll produces no Σ event and is modeled as an ε-step, while a successful poll contributes the corresponding committed Push_c(v) or Pop_c(v) event. Each executed PushNB/PopNB call is nevertheless a distinct dynamic source-level instance q ∈ Q_exec,P for its owning process P; thus repeated failed polls in later loop iterations are distinct q's even if the endpoint request

remains continuously asserted. The ε-modeling concerns observability in Σ, not the dynamic-instance identity of the underlying source-level calls.

R.6 Channel enablement

Definition R.11 (ChannelEnabled / ChannelDisabled).

Let x be a message-operation frontier item on channel c = BoundaryOf_P(x), and write q = MsgOf_P(x) with kind(q) ∈ {Push_c, Pop_c} for its endpoint direction. At an ε-quiescent cutpoint σ, let t_σ denote the cycle of that cutpoint. Define:
ChannelEnabled(x,σ) to mean that BoundaryReq(x,σ) holds and that the channel-side condition required for x's owning operation q to commit at BoundaryOf_P(x) is satisfied after combinational settling at that cutpoint, and
ChannelDisabled(x,σ) to mean that BoundaryReq(x,σ) holds but ChannelEnabled(x,σ) does not.
In the ordinary unique-boundary case, ChannelEnabled(q,σ) and ChannelDisabled(q,σ) may be used as shorthand for the corresponding x-form.

Definition R.12 (Buffered channels).

If B(c) > 0 and x is a source-level or elaborated message-operation frontier item on channel c = BoundaryOf_P(x), with q = MsgOf_P(x), then the buffered-channel enabling condition is:
• ChannelEnabled(x,σ) ⇔ BoundaryReq(x,σ) ∧ occ_c(σ) < B(c), when kind(q) = Push_c
• ChannelEnabled(x,σ) ⇔ BoundaryReq(x,σ) ∧ occ_c(σ) > 0, when kind(q) = Pop_c
Thus, if x is a buffered Push_c frontier item that is channel-disabled, then BoundaryReq(x,σ) holds and occ_c(σ) = B(c); and if x is a buffered Pop_c frontier item that is channel-disabled, then BoundaryReq(x,σ) holds and occ_c(σ) = 0.

Definition R.13 (Rendezvous channels).

If B(c) = 0 and x is a source-level or elaborated message-operation frontier item on channel c = BoundaryOf_P(x), with q = MsgOf_P(x), then the rendezvous enabling condition is:
ChannelEnabled(x,σ) ⇔ BoundaryReq(x,σ) ∧ (the complementary endpoint request on c is asserted at σ).
Equivalently:
• if kind(q) = Push_c, then
ChannelEnabled(x,σ) ⇔ BoundaryReq(x,σ) ∧ (the complementary Pop_c endpoint request is asserted at σ)
• if kind(q) = Pop_c, then
ChannelEnabled(x,σ) ⇔ BoundaryReq(x,σ) ∧ (the complementary Push_c endpoint request is asserted at σ)
Hence, if a rendezvous frontier item x is channel-disabled, its own BoundaryReq holds but the complementary endpoint is not simultaneously requesting the matching partner action at that cutpoint.
In the ordinary unique-boundary case, the equivalent q-form may be used as shorthand.

R.7 Pending blocking frontier

Definition R.14 (Pending blocking operations).

Fix an ε-quiescent state σ of process P within the current interval I_P(σ) between the adjacent synchronization calls that bracket σ in P's current execution. Define

Pending_P(σ) := { x | x ∈ Ω_msg,P is a blocking source-level message-operation frontier item in that same interval I_P(σ), and for q = MsgOf_P(x), q is defined, Issue(q) has occurred, and Commit(q) has not yet occurred in σ }.
By Appendix C's Issue/Commit linkage together with the definition of BoundaryReq, each member x of Pending_P(σ) has its endpoint-owned request asserted at the cutpoint, read operation-attributively through its owning dynamic message-call instance q = MsgOf_P(x).

Definition R.15 (Pending blocking frontier Front_P(σ)).
Define
Front_P(σ) := min_≤P(Pending_P(σ))
If several ≤_P-minimal pending blocking message-operation frontier items are simultaneously outstanding, they form a frontier. This is the Appendix B / K notion of the pending blocking frontier (or minimal pending blocking frontier). It is distinct from NextFront_P(σ), which is the broader source-level frontier-item set over Ω_P.

R.8 Drain-only blocked-frontier discipline and automatic flush

Definition R.16 (Drain-only blocked-frontier discipline).

A source interval satisfies the drain-only blocked-frontier discipline if, whenever Front_P(σ) is non-empty and every frontier item x ∈ Front_P(σ) is channel-disabled after ε-settling, the following all hold:
(1) Block point.
P is blocked on message passing at Front_P(σ).

(2) No new later work.
While Front_P(σ) remains non-empty and fully disabled, no new later blocking message-operation frontier item y ∈ Ω_msg,P is issued, where "later" means that, for some x ∈ Front_P(σ), x ≤_P y and x ≠ y, with issue understood through the owning message instance q_y = MsgOf_P(y). More generally, no new source-later dynamic operation is launched past the blocked point.
(3) No withdrawal.
Already-asserted blocking requests are not withdrawn before the owning message instances commit.
(4) Frontier-minimality.
An already-asserted blocking request whose frontier item is not in Front_P(σ) is not treated as a wait-for source unless and until that frontier item later becomes frontier-minimal.
(5) Drain-only downstream progress.
Work already past the blocked frontier may drain and commit when channel-enabled, but no new work advances past the blocked frontier while that frontier remains fully disabled.
(6) Finite non-replenishing drain.
Every selected explicit abstract boundary that can hold or credit drainable work has finite B-faithful storage or credit, directly or through a finite Sys_B^elab boundary chain. Together with clause (2), this makes the drain set finite and non-replenishing.
(7) Sync-boundary capacity withdrawal — elaborated-boundary interpretation.
While a process is pending at an explicit synchronization call s, producer-facing availability for any back-annotated capacity belonging only to suff_S(s) is withheld until s commits; work already accepted before s may still drain. For the Sys_ref proof model, this withdrawal is not an additional hidden enablement predicate on an otherwise ordinary E4 channel. It is represented only by choosing Sys_ref = Sys_B^elab with explicit sync-boundary / epoch-gate abstract channels, or equivalent explicit staged realization structure, so that the refusal appears as ordinary ChannelDisabled behavior at an explicit abstract boundary.

For a pipelined or otherwise overlapped implementation, Appendix B's automatic-flush rules are the implementation-side construction that supplies Definition R.16. For ordinary non-overlapped control, the same definition may be discharged by a structural proof that no new source-later dynamic operation can be launched after the blocked point is reached. Thus the K.2b dominance/extraction premise ranges over every liberal interval that can contain eager issue, not only loop pipelines.

Remark R.3 (ALL disabled ⇒ stall).

The document's intended policy is exactly the "ALL disabled ⇒ stall" interpretation: the process stalls only when none of the frontier items in Front_P(σ) is channel-enabled. Downstream drain is compatible with that stall provided no new later work is issued past the blocked frontier.

R.9 Observable compatibility and cutpoint correspondence

Definition R.17 (System-level observable compatibility).

For a witness prefix τ_ref of Sys_ref and a post-HLS prefix τ_post of RTL_B, define
τ_ref ≈ τ_post :⇔ E1 ∧ E2 ∧ E3 ∧ E4 ∧ E5
This is an ordered compatibility predicate, not a mathematical equivalence relation. The left argument supplies the Sys_ref witness prefix, and the right argument supplies the RTL_B post-HLS prefix constrained by E1–E5. This appendix treats E1–E5 as primitive named conjuncts, exactly as used by Appendix J.
Definition R.18 (Cross-model KObs correspondence).
For the exact selected package of Corollary J.1, cross-model cutpoint correspondence means only the typed equality KObs_E,w(σ_ref) = KObsPost_J(ρ_post;E,w,H,O) established by J.KObsConf. This definition introduces no additional notation and no independent state clauses. All process, channel, frontier, request, enablement, WFG, drain, terminal, and deadlock facts are obtained by applying Definitions R.18d–R.18e to the common record.

R.9a K-observation quotient and residual-offer layer

Status of this subsection. Definitions R.18a–R.18e and Lemmas R.21b–R.21g provide the source-side KObs record and its reconstruction theory. Appendix J now constructs the corresponding J-facing RTL record and uses J.OfferReach/J.OfferConf/J.KObsConf to prove equal KObs at matched cutpoints. The subsection still does not characterize the image of arbitrary well-typed records beyond the explicit J.OfferReach predicate, and it does not alter the operational Issue, Policy-L/Policy-S, or DrainDiscipline semantics.

Definition R.18a (Enriched replay history H).

Fix a theorem instance I consisting of the design, fixed stimulus, declared environment model, reset policy, chosen observation boundaries, waiver set W, selected serial route r ∈ {General, EnvOrdDone}, and — for r = EnvOrdDone — the recorded K.EnvOrd/K.Done evidence. I also records an environment-reconstruction mode: either StandardEnv, meaning the normative B1-avail pull-based, consumption-ordered input model with default always-ready outputs, or EnvReplayAdequate, meaning a certificate that supplies a typed reconstructed environment state EnvStateRec_I(H,w) and proves that Definition K.StimX permanent next-interaction availability is determined by H, w, and that reconstructed state. Fix the full elaboration package E and witness w. For a reachable ε-quiescent cutpoint σ of Sys_ref, let H_E,w(σ) be the finite canonical annotated replay history through σ. It contains the observable units and annotations needed by Theorem J.1 and I.DR/I.DR': committed Push_c(v)/Pop_c(v) events with dynamic identities and per-channel ordinals; synchronization and wait units; anchored Read/Write units; START_P, FINISH_P, and RESET(S_p,S_c) units where applicable; and the WC/witness-selected non-

blocking, arbitration, signal, and ε-choice outcomes required to replay the selected execution. H retains explicitly atomic unit grouping, per-process replay order, per-channel FIFO order, reset-epoch structure, and the required-prefix constraints used by $\prec_J$. It quotients away incidental model-specific clock-bucket placement and ordering among independent units not required by those constraints. Raw source or RTL cycle buckets may be retained as certificate metadata, but they are not fields compared by KObs equality. $H_{E,w}(\sigma)$ contains no historical Issue(q) timestamps merely because requests were asserted before commit.

Definition R.18b (Attributed residual offer and residual-offer set).

At a reachable ε-quiescent Sys_ref cutpoint σ, an attributed residual offer is a tuple o = (P,x,q,c,dir,e) satisfying all of the following: P owns x; $x \in \Omega_{msg,P}$; q = MsgOf_P(x); c = BoundaryOf_P(x); q is a blocking Push or Pop instance that has issued but not committed in reset epoch e; dir identifies the endpoint-owned producer-valid or consumer-ready direction; and BoundaryReq(x,σ) is asserted and attributed to that same x, q, c, direction, and epoch. Define the derived operation-kind projection kind(o) := kind(q), with the typing invariant dir = producer-valid iff kind(q) = Push and dir = consumer-ready iff kind(q) = Pop. Let $ResOffers_{E,w}(\sigma)$ be exactly the set of such tuples.
$ResOffers_{E,w}(\sigma)$ is well typed and issue-prefix consistent under the existing E3/Axiom R.1 semantics. In particular: (a) its item and boundary fields are supplied by E; (b) if a source-later residual offer exists while an earlier blocking item in the applicable source prefix is also issued, uncommitted, and still requesting at an evaluated boundary, that earlier offer is also represented; (c) reset clears all offers in the ending epoch under RW2; and (d) for each explicit abstract boundary and endpoint direction there is at most one attributed outstanding source-level offer. A concrete token or transfer already accounted for as explicit channel occupancy is not additionally represented as an independent source-level residual offer unless E declares a distinct source-attributed frontier item at that boundary.

Definition R.18c (K-observation KObs).

For the fixed instance I, elaboration E, witness w, and reachable ε-quiescent Sys_ref cutpoint σ, define the K-observation record by
$KObs_{E,w}(\sigma) := \langle E, w, H_{E,w}(\sigma), ResOffers_{E,w}(\sigma) \rangle$.
The fixed stimulus, environment model and reconstruction mode, reset policy, observation-boundary selection, waiver set W, selected serial route r, and any K.EnvOrd/K.Done route evidence are ambient parameters of the theorem instance I rather than duplicated in every record. Equality of KObs records means typed equality of E and w and equality of the canonical replay-history quotient and residual-offer components. It does not require equality of the source and RTL clock-bucket decompositions permitted to differ by Theorem J.1. It is not equality of arbitrary process, scheduler, channel-realization, environment, or RTL microstate.

Definition R.18c-1 (Observational waiver closure and environment replay adequacy).

For fixed I,E,w, define KObsClosed_I,E,w(W) to mean: for all reachable ε-quiescent Sys_ref cutpoints σ_1 and σ_2 of the same fixed instance, $KObs_{E,w}(\sigma_1) = KObs_{E,w}(\sigma_2)$ implies ($\sigma_1 \in W$ iff $\sigma_2 \in W$). Define EnvTerminalAdequate(I) to hold when I records either StandardEnv or an EnvReplayAdequate certificate. Under StandardEnv, EnvStop reconstruction uses Definition K.StimX together with Lemma K.EnvX-Auto and the default output-readiness assumption; under EnvReplayAdequate, it uses EnvStateRec_I(H,w) and the certificate's permanent-availability theorem. These are extensionality premises, not claims that every well-typed KObs record is reachable.

Definition R.18d (Process and channel reconstruction from KObs).

For $K = \langle E,w,H,O \rangle$, define ProcRec_P(K) to be the source-relevant process slice reconstructed by deterministic replay of P's projection of H under w: the source control location and reset epoch, source-

visible local values needed by the next frontier, branch/guard outcomes, operation identities and per-channel ordinals, and the source-level completion status. Define ChanRec_c(K) for every explicit abstract channel c declared by E to be the E4 abstract state reconstructed from its reset/initial state and the committed Push/Pop history in H: for B(c)>0, occupancy, empty/full status, and logical ordered contents/head information; for B(c)=0, capacity-zero occupancy together with endpoint-request information supplied separately by O. For an outer channel represented by Sys_B^elab, projected outer occupancy is obtained through Π_elab, while every proof-internal boundary used by Appendix K retains its own explicit ChanRec state.

Definition R.18e (K-relevant derived functions).

For K = ⟨E,w,H,O⟩ define the following typed functions. DoneRec_P(K), RemainRec_P(K), and NextFrontRec_P(K) are computed from Ω_P/≤_P supplied by E and the realized-item history reconstructed from H. PendingRec_P(K) is the set of frontier items x appearing in O for P, and BoundaryReqRec(x,K) holds exactly when O contains the attributed offer for x. FrontRec_P(K) := min_≤P(PendingRec_P(K)). ChannelEnabledRec(x,K) and ChannelDisabledRec(x,K) are computed at BoundaryOf_P(x) from ChanRec and the complementary endpoint offers in O using Definitions R.11–R.13. WFGRec(K) is Definition R.20 applied to FrontRec and ChannelDisabledRec. DrainClosedNowRec(D,K) holds exactly when no already-offered non-frontier blocking item owned by a process P ∈ D is ChannelEnabledRec at the cutpoint.

Define DoneStopRec_c(K,I) from the replay-derived normal-completion status of the unique complementary endpoint owner and the demanded per-channel ordinal, exactly as in Definition K.WFG+. When EnvTerminalAdequate(I) holds, define EnvStopRec_c(K,I) as follows: under StandardEnv, use the fixed stimulus and consumed ordinals reconstructed from H, with Definition K.StimX and Lemma K.EnvX-Auto (and default output readiness); under EnvReplayAdequate, use EnvStateRec_I(H,w) and its certified permanent-availability predicate. Define WFGPlusRec(K,I) to have the process nodes and ordinary edges of WFGRec(K), together with the typed EnvStop_c and DoneStop_c terminal nodes and edges generated by EnvStopRec and DoneStopRec. WFGPlusRec does not depend on W.

Define DeadKRec(K) by applying Definition K.Dead to WFGRec(K), the reconstructed frontier facts, and DrainClosedNowRec. When EnvTerminalAdequate(I) holds, define DeadKSRec(K,I) by applying Definition K.DeadKS to WFGPlusRec(K,I). Under KObsClosed_I,E,w(W), define DeadKRec^W(K,I,W) and, when EnvTerminalAdequate(I) also holds, DeadKSRec^W(K,I,W) by excluding the corresponding KObs-closed waiver classes. Finally define BadSRec(K,I,W,r) by the selected route recorded in I: r = General selects DeadKSRec^W; r = EnvOrdDone selects DeadKRec^W and requires the recorded K.EnvOrd/K.Done evidence. At this stage all functions are interpreted on records extracted from reachable source cutpoints; no image characterization is asserted.

Historical issue-time quotient. The record deliberately retains whether an operation survives as a current attributed offer, but not the earlier cycle in which that request first became asserted. Therefore two reachable source cutpoints with the same E, w, H, and O have the same KObs even if their operational executions used different legal historical issue times. This is only an observation equivalence at the selected cutpoint; it does not assert that the two executions have the same admissible continuations or satisfy the same transition-level DrainDiscipline for reasons independent of the common cutpoint.

R.10 Prefix-matching theorem

Theorem R.21 (Finite-prefix witness consequence of Theorem J.1)

Fix a finite post-HLS execution prefix τ_post ∈ Reach_post of RTL_B under a fixed external stimulus, where Reach_post is the Appendix J finite-reachability scope (Definition J.Reach) under the same witness premises used there (in particular J.GWR); no fair-extension qualifier is required for this finite-prefix result. Then there exists a finite execution prefix τ_ref of Sys_ref under that same stimulus such that

τ_ref ≈ τ_post

This theorem is a compact finite-prefix consequence of Appendix J's full execution-level result, not a replacement for that full execution-level statement. In particular, the controlling hypotheses are exactly Appendix J's execution-set scope conditions together with the relevant Appendix I / Appendix L witness machinery and the cited fairness/progress premises. Appendix K uses only the existence of such a matching source-side witness at the compared cutpoint, not uniqueness of that witness.

Corollary R.21a (Equal-KObs cutpoint correspondence; compact finite-prefix form of Corollary J.1).

This corollary is the compact finite-prefix restatement of Corollary J.1. Under the same selected normalized cutpoints and the same J.OfferReach/J.OfferConf/J.KObsConf package, there exist H and O such that

KObsPost_J(ρ_post;E,w,H,O) = KObs_E,w(σ_ref).

No second correspondence relation or derived clause package is introduced.

Let τ_post ∈ Reach_post be as in Theorem R.21, let ρ_post be its selected ε-quiescent RTL cutpoint, and let σ_ref be the selected reachable ε-quiescent source cutpoint. If the same package satisfies J.OfferReach, J.OfferConf, and J.KObsConf, equal KObs follows. Every K-facing fact listed in Definition R.18e is then obtained by applying the same pure function to the common record.

In particular, the compact form yields equal reconstructed NextFront, process replay facts, explicit abstract channel state, residual endpoint requests, Pending/BoundaryReq attribution, enablement, Front, WFG, and DrainClosedNow. Under the premises of Lemma R.21f it also yields the corresponding Dead_K^W, Dead_KS^W, and Bad_S^W equality.

R.10a K-observation reconstruction lemmas

The lemmas in this subsection are source-side reconstruction and factoring results. Corollary J.1 now consumes them after J.OfferReach/J.OfferConf establish a common KObs record across the selected RTL_B and Sys_ref cutpoints. They remain functions over a source-realizable record and do not by themselves prove RTL boundary conformance.

Lemma R.21b (Process replay reconstruction).

Let σ be a reachable ε-quiescent Sys_ref cutpoint under I,E,w and let K = KObs_E,w(σ). For every process P, ProcRec_P(K) equals the source-relevant process slice of σ used by Lemmas I.DR/I.DR' and the Appendix J finite-ideal construction. Consequently DoneRec_P(K), RemainRec_P(K), NextFrontRec_P(K), all next-item selection predicates, all values needed to determine future committed Σ labels, and P's normal-completion status agree with their operational definitions at σ. Proof. Project H to P, include the WC/witness choices and RESET epoch markers, and apply deterministic replay I.DR/I.DR'; the replayed process state together with Definition R.6 determines the resulting value and frontier facts. □

Lemma R.21c (Abstract channel reconstruction).

Let σ and K be as in Lemma R.21b. For every explicit abstract channel c declared by E, ChanRec_c(K) equals the E4 abstract channel state of σ. For B(c)>0, occupancy and logical contents are uniquely obtained from reset/initial contents and the committed Push/Pop sequence and per-channel ordinals in H. For B(c)=0, occupancy is identically zero and current rendezvous enablement additionally uses the endpoint offers in O. In Sys_B^elab, the same statement holds at each explicit proof boundary, and Π_elab supplies the declared outer projection. Proof. Induction over the committed channel-history projection of H, using E4, B-faithfulness, RESET/RW1–RW3, and Π_elab conservation. □

Lemma R.21d (Residual-offer, BoundaryReq, and frontier reconstruction).

Let σ and K = ⟨E,w,H,O⟩ be as above. For every process P and blocking message-operation frontier item x, x ∈ Pending_P(σ) iff x ∈ PendingRec_P(K), and BoundaryReq(x,σ) iff BoundaryReqRec(x,K). Hence Front_P(σ) = FrontRec_P(K). For every point-to-point rendezvous channel, Req_prod and Req_cons at σ are exactly the existence of the corresponding producer-direction and consumer-direction offers in O. Proof. O was defined as the exact set of current operation- and boundary-attributed blocking endpoint requests. Apply Definitions R.14–R.15 and take the ≤_P-minimal subset. The one-offer-per-endpoint-direction invariant makes the rendezvous projection functional. □

Lemma R.21e (Enablement and wait-for-graph reconstruction).

For every residual offer x at σ, ChannelEnabledRec(x,K) = ChannelEnabled(x,σ), and likewise for ChannelDisabled. Consequently WFGRec(K) = WFG(σ). Proof. Use Lemma R.21c for buffered occupancy/content state, Lemma R.21d for rendezvous endpoint requests and attributed frontier membership, and Definitions R.11–R.13 and R.20 at the explicit boundary supplied by E. The no-hidden-cross-channel-disablement convention ensures that no additional enabling fact is required. □

Lemma R.21f (DrainClosedNow and K-predicate factoring).

For every non-empty process set D, DrainClosedNowRec(D,K) holds iff the operational cutpoint σ is drain-closed for D in the snapshot sense used by Appendix K. The unwaived ordinary predicate Dead_K(σ) is therefore determined by K. If KObsClosed_I,E,w(W) holds, Dead_K^W(σ) is determined by DeadKRec^W(K,I,W). If EnvTerminalAdequate(I) holds, WFG+(σ), EnvStop, DoneStop, and the unwaived Dead_KS(σ) are determined by WFGPlusRec(K,I) and DeadKSRec(K,I); if KObsClosed_I,E,w(W) also holds, Dead_KS^W(σ) is determined by DeadKSRec^W(K,I,W). Under the selected route r recorded in I and its required evidence, Bad_S^W(σ) is determined by BadSRec(K,I,W,r).

Proof. Lemmas R.21b–R.21e reconstruct process completion, consumed ordinals, frontier membership, endpoint requests, channel enablement, and ordinary WFG edges. DoneStopRec uses replay-derived FINISH/completion and the demanded ordinal. Under StandardEnv, Definition K.StimX and Lemma K.EnvX-Auto reduce an input-side EnvStop to permanent exhaustion of the fixed consumption-ordered stimulus at the reconstructed ordinal, while default output readiness excludes permanent output-side terminals; under EnvReplayAdequate the recorded environment-state reconstruction supplies the same terminal classification. KObsClosed_I,E,w(W) supplies σ ∈ W iff the reconstructed KObs belongs to the corresponding waiver class. The snapshot drain clause is exactly the absence of an enabled non-frontier residual offer owned by D. No continuation-level claim about Definition R.16/DrainDiscipline follows from this factoring result. □

Corollary R.21g (KObs extensionality).

Let σ_1 and σ_2 be reachable ε-quiescent Sys_ref cutpoints of the same fixed instance with KObs_E,w(σ_1) = KObs_E,w(σ_2). Then their reconstructed process slices, explicit abstract channel states, NextFront and

Front sets, operation-attributive BoundaryReq values, channel-enable predicates, ordinary WFGs, DrainClosedNow predicates, and unwaived Dead_K truth values are equal. If KObsClosed_I,E,w(W) holds, their Dead_K^W truth values are equal. If EnvTerminalAdequate(I) holds, their EnvStop/DoneStop classifications, WFGPlusRec graphs, and unwaived Dead_KS truth values are equal; if KObsClosed_I,E,w(W) also holds, their Dead_KS^W truth values are equal. Under the same premises and the same selected route r and route evidence recorded in I, their Bad_S^W truth values are equal. The conclusion is independent of any difference in legal historical Issue timestamps not retained in the common residual-offer set. Without waiver closure or environment-terminal adequacy, the extensionality conclusion is correspondingly limited to the unwaived, reconstructible predicates stated above. Proof. Functional extensionality of the named reconstruction definitions and Lemmas R.21b–R.21f. □

Cross-model-use note. Corollary R.21g is the source-side extensionality theorem for reachable records. Appendix J supplies J.OfferReach and J.OfferConf and proves through J.KObsConf that the selected post certificate and source cutpoint share one record. Lemmas R.22/R.23/R.24 and their Appendix K counterparts state their cross-model consequences directly from that equality.

R.11 Explicit frontier-classification bridge

Definition R.19 (Derived blocking-frontier slice at ε-quiescent cutpoints).

At an ε-quiescent cutpoint σ and for a process P, define the asserted blocking message-operation-item slice of the next source-level frontier-item set by

FrontFromNext_P(σ) := { x ∈ BlockingMsgNextFront_P(σ) | BoundaryReq(x, σ) },

where

BlockingMsgNextFront_P(σ) ≜ { x ∈ NextFront_P(σ) | x ∈ Ω_msg,P and, for q = MsgOf_P(x), q is defined and q is a blocking source-level message-operation instance with kind(q) ∈ {Push_c, Pop_c} for some channel c }.

Here BoundaryReq(x,σ) is read operation-attributively for the specific source-level blocking message-operation frontier item x, through its owning dynamic message-call instance q = MsgOf_P(x), exactly in the Appendix C sense: the asserted endpoint-owned request at the cutpoint is attributed to x only if q is the currently outstanding requesting source-level operation instance on that endpoint direction. For blocking Push/Pop, that attribution cannot change before commit. Non-blocking PushNB/PopNB call instances are excluded from BlockingMsgNextFront_P unless represented by a frontier item in Ω_msg,P for non-WFG committed-transfer reasoning; successful non-blocking calls may contribute Push_c/Pop_c Σ-labels, and failed polls are ε, but neither creates a pending blocking frontier member for WFG purposes. This definition is over source-level message-operation frontier items in Ω_msg,P ⊆ Ω_P, not over the message-instance slice Q_Ω,P itself and not over already-committed Σ-event labels.

Remark R.4 (derived bridge, not extra premise).
Corollary R.21a aligns NextFront_P and the relevant operation-attributive BoundaryReq values only on the common blocking message-operation-item slice BlockingMsgNextFront_P of that frontier. No BoundaryReq predicate is defined or required for non-message frontier items, and successful non-blocking frontier items do not create pending blocking frontier members. Appendix K.2.2 (Lemma K.0) then derives, from the Appendix B drain-only blocking discipline and the Appendix C Issue/Commit linkage, that this asserted blocking message-operation-item slice is exactly the pending blocking frontier.

R.12 Front / NextFront classification lemma

Lemma R.22 (Front reconstruction; compact restatement of Lemma K.0).

For every reachable ε-quiescent Sys_ref cutpoint σ and K = KObs_E,w(σ),
Front_P(σ) = FrontRec_P(K) = { x ∈ BlockingMsgNextFrontRec_P(K) | BoundaryReqRec(x,K) }.
If K_post = K_ref under Corollary J.1, then FrontRec_P(K_post) = FrontRec_P(K_ref), and the corresponding operation-attributive BoundaryReqRec values agree. This is a classification of current pending blocking offers only; younger already-asserted non-frontier offers remain in O but outside FrontRec while an earlier blocker is minimal.

Corollary R.22a (Rendezvous endpoint-request extensionality; compact restatement of Corollary K.0a).

For a point-to-point rendezvous channel c, ReqProdRec_c and ReqConsRec_c are the direction projections of O. Equal KObs implies equality of both projections. No equivalence is asserted between a raw endpoint request and frontier membership.

R.13 Wait-for graph refinement

Definition R.20 (Wait-for graph).

At an ε-quiescent cutpoint σ, define the wait-for graph WFG(σ) exactly as in Appendix K: its vertices are the modeled processes, and its directed edges are induced from the frontier members Front_P(σ) by the corresponding channel-disabled dependencies on internal message-passing channels only. Equivalently, a directed edge P → Q is present when some frontier item x ∈ Front_P(σ), with owning message instance q = MsgOf_P(x), is channel-disabled at the chosen boundary and the unique complementary endpoint needed to enable q belongs to process Q.
Here “internal” means that both endpoints of the channel are modeled processes of Sys_ref / RTL_B. Channels whose complementary endpoint is the external environment/testbench (the DUT boundary) are excluded from the WFG and from the internal message-passing deadlock notion used in Appendix K. Buffered and rendezvous channels are both included, with B(c), occ_c(σ), BoundaryReq, ChannelEnabled, and ChannelDisabled interpreted at the chosen ε-quiescent cutpoint on the chosen abstract channels of Sys_ref.

Lemma R.23 (WFG extensionality; compact restatement of Lemma K.2).

Let K_post and K_ref be the equal records selected by Corollary J.1. Then
WFGRec(K_post) = WFGRec(K_ref).
The result is immediate from the functional definition of WFGRec using FrontRec, ChannelDisabledRec, ChanRec, and the residual-offer projections. It applies uniformly to buffered, rendezvous, and explicit Sys_B^elab boundaries.

R.14 Deadlock preservation

Theorem R.24 (KObs-level deadlock-predicate preservation; compact restatement of Theorem K.1A’s transfer step).

Let K_post and K_ref be the equal records selected by Corollary J.1. Then the ordinary WFG, every closed-component classification, and DrainClosedNowRec(D,·) are equal for each non-empty process set D. Therefore the unwaived ordinary predicate DeadKRec has the same truth value on both records. If KObsClosed_I,E,w(W) holds, DeadKRec^W also has the same truth value. Under EnvTerminalAdequate(I), the same statement extends to WFGPlusRec and DeadKSRec, and with waiver

closure to DeadKSRec^W and the route-selected BadSRec. This is a cutpoint-observation result only and makes no claim that the two executions have the same continuation-level DrainDiscipline.

R.15 Summary of dependency structure

For convenient reference, the compact dependency chain is: (1) Appendices B and C define the operational pending/frontier, Issue/Commit, request-persistence, and drain-only semantics. (2) Appendices G-J construct matched prefixes and prove, through J.HCanon, J.OfferReach, J.OfferConf, and J.KObsConf, that the selected CF-0 RTL cutpoint certificate and reachable Sys_ref cutpoint share one KObs record. (3) Definitions R.18d-R.18e and Lemmas R.21b-R.21f reconstruct all K-facing cutpoint facts from that record. (4) Lemmas K.0/R.22 reconstruct the pending blocking frontier; Lemmas K.2/R.23 give WFG extensionality; Lemmas K.2a/R.24 give DrainClosedNow and deadlock-predicate extensionality. (5) Theorem K.1A maps a non-waived RTL deadlock observation to a reachable Policy-L Sys_ref deadlock and contradicts the A5-L route recorded in K.A5Cert. (6) Appendix K-P and Lemma K.2b retain the operational Policy-S reduction used to discharge A5-L from the CF-S/A5-S evidence, but expose the terminal Policy-L and Policy-S cutpoints through K_L/K_S, use the attributed ResOffers component for current requests, and use a finite DrainBag projection as the Step-2 termination measure.

Appendix S – Lean Proof Strategy Document

A separate Lean proof strategy document is under development. It should import, rather than duplicate, the normative interfaces fixed here: KObs and its reconstruction functions, J.HCanon/J.OfferReach/J.OfferConf/J.KObsConf, K.CertHdr/CF-0/K.A5Cert, the operational Policy L/S and DrainDiscipline semantics, and the scheduling-specific mechanization obligations summarized in Appendix T.

Appendix T – Scheduling-Specific Obligations Beyond Standard Simulation Theory

T.1 Purpose and Nature of the Formal Result

Standard formal-methods libraries provide reusable foundations for labeled transition systems, weak or ε-abstracted transitions, forward simulation, finite and infinite executions, trace inclusion, well-founded induction, fixed-point reasoning, and related proof techniques. Those foundations can substantially simplify a mechanization of the results in this document. They do not, however, establish the HLS scheduling contract or its safety and liveness guarantees by themselves.

In particular, the top-level safety result in this document is not ordinary trace inclusion or trace equivalence. It is the document's witness-relative ordered trace-compatibility relation, written ≈, over the selected proof-internal observable alphabet Σ. This relation is defined over complete observable units and bucketed executions, preserves the required ≺_J ordering constraints, and permits independent observable units to commute or to appear in different cycle groupings in Sys_ref and RTL_B. It also treats the Post → Ref and restricted Ref → Post uses differently. Consequently, a generic result of the form "forward simulation implies trace inclusion" does not directly yield the compatibility theorem proved here.

The principal proof obligations arise from features specific to HLS scheduling and hardware communication: source-order constraints, same-cycle and atomic observable behavior, witness-selected replay, explicit channel-boundary semantics, back-annotated capacity, process composition, KObs cutpoint observation and certificate correspondence, fair channel progress, pipeline drain behavior, and the connection between overlapped execution and conservative serial-source deadlock. This appendix

summarizes the obligations that must remain explicit in any faithful formalization, even when standard transition-system and simulation libraries are reused.

The obligations divide into two structurally different groups. Safety is largely compositional: process-local scheduling compatibility and explicit channel-boundary properties are combined into a system-level trace-compatibility result. Liveness is a conditional global preservation result: it ranges over complete admissible executions and global wait-for components and cannot in general be obtained by composing independent per-process liveness theorems.

The purpose of this appendix is not to prescribe a particular theorem prover, transition-system library, or Lean architecture. Rather, it identifies the scheduling-specific semantic and proof obligations that a mechanization must discharge in addition to any generic infrastructure supplied by standard formal-methods libraries.

T.2 Safety-Specific Obligations

T.2.1 Initial-State and Reset-Epoch Correspondence

The induction used to construct a matching Sys_ref execution requires a valid base case before the first observable unit is matched. The post-reset RTL_B state must correspond to the prescribed source initial state, including process control locations, source-visible values, channel state, request state, and the selected abstraction relation.

For the document-aligned theorem instances, A10 requires reset-empty initial occupancy at the selected abstract channel boundaries. There must also be no unaccounted pending endpoint request, stale synchronization state, or hidden accepted message that is absent from the source reference model.

Reset correspondence is not necessarily a one-time property limited to initial system startup. When dynamic reset is supported, each affected process enters a new reset epoch, and the formalization must re-establish the applicable state, channel, anchor, and direct-input correspondence for that epoch. The hold-stable and synchronization-controlled rules for direct inputs are part of ensuring that reset and restart do not introduce a pre-HLS/post-HLS mismatch. In this document, that re-establishment is formalized by the RESET boundary observable unit, the reset well-formedness conditions RW1–RW5, and Lemma J.RS (MatchedResetEpochSplice) in the Common Formal Definitions, which add an explicit reset-splice case to the Theorem J.1 induction.

T.2.2 Source Enablement of the Next Selected Observable Unit

A compatibility proof cannot merely observe that RTL_B performed an action and then assert that Sys_ref can perform the corresponding action. It must prove that, in the currently constructed source-reference state, the specific dynamic operation instance represented by the next selected RTL observable unit is enabled.

The required source-side prerequisites can include the process control position, branch predicates, loop-iteration identity, prior source-ordered operations, synchronization boundaries, signal anchors, channel occupancy, channel availability, and witness-selected non-blocking outcomes. The proof must also establish that the selected unit is the correct dynamic occurrence rather than another static call at the same source location.

This obligation is central because RTL_B may contain added FSM states, overlapped loop iterations, internal pipeline stages, or eager/same-cycle issue of operations that were sequential in the source. Source enablement is the semantic bridge showing that the selected RTL behavior is still a behavior permitted by the prescribed source reference model while preserving the source order required by R4/E3. In this document that obligation is packaged as the named premise GlobalWitnessRealizable (Definition J.GWR); Theorem J.1's Post → Ref clause is stated conditionally on it, and a flow-specific global witness-realizability theorem is the corresponding mechanization target.

T.2.3 Same-Cycle Independence and Atomic Observable Units

A single hardware cycle may contain multiple observable events that are independent and therefore have no meaningful intrinsic order within that cycle. Imposing an arbitrary total order on such events

could introduce false dependencies or make two compatible executions appear different merely because independent actions were listed differently.
Conversely, some groups of events must be treated as one indivisible observable unit. Examples include the paired endpoints of a rendezvous transfer, a synchronization handshake, and a stutter-canonical R2C fixed-point observation unit. The formalization must prevent the internal pieces of such a unit from being split across different matching steps.
The resulting observable model is partial-order and independence based, and is analogous in some respects to pomset or Mazurkiewicz-trace semantics. It is not, however, assumed to be an exact instance of a fixed trace-monoid model, because the relevant independence and ordering relationships may depend on dynamic operation instances, channel relationships, atomic grouping, and the execution-specific RequiredJOrder relation.

T.2.4 Fixed RTL-Time Ideal-Chain Construction

The J.1 construction does not extend the source witness in an arbitrary topological ordering of observable actions. It follows a chain of finite $\prec_J$ ideals selected from the concrete RTL_B execution, ordered by the RTL-time selection rule while respecting every required predecessor relationship.
At each induction step, the next ideal adds one complete observable unit whose required predecessors have already been included. This ensures that the constructed Sys_ref prefix corresponds to the actual RTL_B behavior being analyzed rather than to an unrelated legal reordering of the same actions.
The ideal chain also provides the induction index needed to maintain the proof invariants, including process replay state, source-order completion, witness consistency, channel occupancy, synchronization state, signal anchoring, and compatibility of the matched prefix. The existence of an enumeration with the required witness-realizability property is supplied by the J.GWR premise (Definition J.GWR), not by the chain construction itself.

T.2.5 Witness Compatibility and Faithful Witness Selection (WC)

The source reference model may admit multiple internal ε-step choices, non-blocking outcomes, arbitration outcomes, or replay paths that produce superficially similar observable behavior. A selected witness therefore cannot be treated as an unconstrained proof artifact. WC requires the witness choices used by the proof to be compatible with the actual RTL_B-observed behavior and with the relevant channel and boundary state.
This is especially important for failed or successful non-blocking operations that may be represented partly or wholly through ε-state. WC prevents the proof from selecting a successful poll when the actual boundary state required failure, selecting a failure when the operation was available, or attributing an issue or outcome to the wrong dynamic operation instance.
Snooping, NB-CF, direct witness construction, and mixed per-site reasoning are methods for discharging WC. They are not alternative theorem premises that replace WC. The theorem application must ultimately carry evidence for the actual WC predicate used by Appendices I and J.
WC is load-bearing for both source enablement and deterministic replay. Those arguments determine a source state and next action relative to a chosen witness; WC is what ties that witness to the implementation behavior rather than to a hypothetical source schedule.

T.2.6 Deterministic Replay Under the Selected Witness

Equality of visible labels alone does not necessarily determine the source state reached after a prefix. The source model may contain control-flow choices, failed non-blocking polls, hidden ε-state, arbitration choices, or witness-selected outcomes that affect later behavior without appearing as independent Σ labels.
Deterministic replay states that, once the fixed stimulus, WC-compliant witness, dynamic-operation attribution, and complete process-local observable-unit history are fixed, the replay-derived source control state is fixed. This includes control location, source values that influence later guards or payloads, completed-item status, candidate NextFront, anchored signal values, Pop return values

reconstructed from committed channel history, and normal completion. It does not by itself determine which uncommitted blocking requests are currently asserted; that information is supplied by the residual-offer set O and audited by J.OfferConf.

The formalization must distinguish the canonical immediate post-unit replay state from an explicitly ε-normalized cutpoint. Plain J.1 may use the immediate post-unit form without claiming ε-quiescence after every matched unit. Corollary J.1 and Appendix K may instead use the normalized form when a canonical cutpoint is required.

T.2.7 E4 Occupancy and Explicit Boundary Behavior

E4 defines the legal message-transfer behavior at each explicit abstract channel boundary. For a bounded FIFO, this includes the occupancy update, full and empty conditions, and the circumstances in which Push and Pop are ChannelEnabled or ChannelDisabled. For a rendezvous boundary, it includes the coordinated producer and consumer behavior required for a same-cycle transfer.

The compatibility proof must show more than equality of transferred values. Every committed transfer must be legal under the selected boundary semantics, occupancy must evolve consistently, and the source and RTL_B interpretations of boundary availability must agree at the points used by the theorem. Generic simulation theory does not distinguish an ordinary bounded FIFO from a rendezvous boundary, synchronization gate, staged channel, join, bundle, or another elaborated boundary structure. These semantics are supplied by the scheduling model and must be proved for the selected comparison instance.

T.2.8 B-Faithfulness

The back-annotated capacity B(c) is intended to summarize all storage or credit that can accept, retain, or backpressure messages between the selected explicit producer and consumer boundaries. That storage may be physically distributed across FIFOs, pipeline stages, skid buffers, registers, credit state, or other implementation structures.

B-faithfulness is stronger than a capacity upper-bound claim. The selected RTL boundary must exhibit exactly the Push and Pop behavior permitted by E4 for the declared B(c), including the corresponding ChannelEnabled and ChannelDisabled behavior. Internal movements within the represented realization may be ε-steps, but they may not create, delete, duplicate, reorder, or relabel the Σ-visible boundary transfers.

If hidden implementation structure changes availability in a way that cannot be represented by an ordinary E4 channel with the declared capacity, that structure must instead be represented by explicit Sys_B^elab boundaries. B-faithfulness is therefore an implementation-side compliance obligation, not a consequence of the abstract scheduling theorem itself.

T.2.9 Composition of Process-Local Compatibility into System Compatibility

Process-local compatibility results do not automatically imply a valid system-level result. The individual process witnesses must compose into one common Sys_ref execution whose channel transfers, synchronization actions, values, order constraints, and environment interactions are mutually consistent.

For every point-to-point channel, the producer-side and consumer-side views must describe the same transfer and payload. Rendezvous endpoints must be paired as one atomic unit, occupancy changes must agree globally, and the process-local $\prec_J$ constraints must be compatible with the system-level RequiredJOrder relation.

The composition proof must also ensure that the local witnesses do not encode incompatible environmental choices or contradictory non-blocking outcomes. In elaborated cases, all introduced staged, bundle, join, and synchronization-gate boundaries must connect consistently and project correctly to the selected outer observation boundary.

A generic theorem about composition of simulations normally assumes that the component transition systems and their synchronization operator already satisfy appropriate compatibility conditions. Establishing those conditions for this scheduling model is itself a project-specific obligation.

T.2.10 KObs cutpoint correspondence and direct Appendix K consumption

Trace compatibility alone does not distinguish a process that has not reached a blocking call from one that currently has an attributed uncommitted request. The interface therefore augments the enriched committed/replay history H with the terminal residual-offer set O and uses the compact KObs record ⟨E,w,H,O⟩.

Lemma J.HCanon is the named history-side bridge: compatible annotated source/RTL prefixes under one fixed J.GWR package map to the same canonical replay-history quotient H. J.OfferReach proves that ⟨E,w,H,O⟩ is realized by the selected reachable Sys_ref cutpoint. J.OfferConf proves bidirectional unique correspondence between O and the complete set of KObs-relevant asserted RTL boundary requests. B-faithfulness/E4 interprets the committed history at the explicit boundaries. J.KObsConf packages these facts and yield equality between KObsPost_J and the source KObs.

The process slice, channel contents, frontier, endpoint requests, enablement, Front, WFG, and snapshot drain facts are then reconstructed by typed functions. Mechanization should therefore target a small record plus pure reconstruction functions, rather than a relation containing seven independently supplied state clauses. The initial mechanization targets should include: (i) CanonHist and Lemma J.HCanon; (ii) the typed physical-boundary request set RelevantReqSet_E(ρ_post); (iii) the projection ReqProjection_E(O); and (iv) the equality RelevantReqSet_E(ρ_post) = ReqProjection_E(O), together with unique attribution and endpoint-cardinality invariants.

Historical Issue remains in the transition semantics and in any operational witness needed by Theorem J.1, but it is not part of the cross-model KObs equality. For the RTL request-abstraction boundary, the mechanized checker or certificate must range over the complete selected explicit-boundary request interface, not merely over requests for which attribution records were successfully produced. A missing or unattributed KObs-relevant asserted request is a failed OfferConf proof, not an absent observation. Appendix K now consumes the equal record directly: K.0, K.2, K.2a, and K.1A should be mechanized as reconstruction/extensionality theorems rather than as a broad state-relation transfer.

T.2.10a Certificate and independent-checker boundary

The certificate interface deliberately separates facts that are recorded from facts that are reconstructed. K.CertHdr fixes the theorem instance and trust boundary. CF-0 records H, O, the complete selected-boundary physical request inventory, and the evidence that connects the source and RTL cutpoints. The checker then reconstructs process state, channel state, Front, BoundaryReq, enablement, WFG/WFG+, DrainClosedNow, and the selected deadlock predicate. A certificate that directly supplies those derived objects may include them as caches, but they are non-authoritative and must be checked against the reconstruction functions.

This design concentrates implementation trust in auditable boundary obligations rather than in an opaque full-state relation. In particular, OfferConf completeness is checked against the entire RelevantReqSet at every selected explicit boundary, not only against successfully attributed requests. B-faithfulness and the instance header bind the same physical boundaries and capacities to the channel reconstruction. HCanon and OfferReach bind the replay history and residual offers to a real source witness. Large histories may be content-addressed, but the independent checker must be able to retrieve and replay them.

Historical Issue is intentionally not duplicated in CF-0. That omission is safe only at the cutpoint-observation interface. OfferReach and the A5 route must still validate the operational Issue/Commit, fairness, persistence, and DrainDiscipline facts needed for reachability and continuation reasoning. The certificate schema therefore references those proof artifacts rather than pretending that KObs determines them.

T.2.11 Finite-Local-Trace Continuation

A process may cease producing local observable units while the overall system continues to execute indefinitely. In that case, the proof cannot simply append idle cycles to the process by invoking B6, because local inactivity does not imply that the complete system has reached a global fixed point. Instead, the proof must construct or select an admissible infinite global Sys_ref continuation and project that execution onto the process whose local observable trace is finite. The process-local projection may remain constant while other processes, channels, or the environment continue to evolve.

A separate case applies when the entire system eventually reaches a genuine B5/global fixed point. Only in that case may B6 justify indefinite global idle stutter.

This distinction is required to construct valid infinite reference witnesses without incorrectly treating a process-local terminal projection as a globally terminal execution.

T.2.12 Finite ε-Normalization and Existence of Canonical Cutpoints

An ε-quiescent cutpoint cannot simply be assumed to exist. The proof must establish that the selected internal ε-normalization terminates, or is otherwise bounded so that a finite canonical cutpoint can be reached.

B4 provides the applicable finite ε-stutter or normalization bound for the safety and cutpoint layers. B5 identifies the stronger global fixed-point conditions used where complete system quiescence is required. In the Appendix K layer, Kε additionally requires the relevant ε behavior to be WFG-inert and appropriately bounded so that normalization does not alter the blocking structure being analyzed.

The proof must establish both the existence of the normalized cutpoint and preservation of the certificate facts needed afterward. These include the canonical history H, residual offers O, B-faithful boundary interpretation, and the J.OfferReach/J.OfferConf premises required to construct the common KObs record. Appendix K then reconstructs frontier, enablement, WFG, and snapshot drain facts from that record.

The immediate post-unit state used by plain J.1 and the ε-normalized cutpoint used by Corollary J.1 or Appendix K must therefore remain distinct formal objects unless an explicit theorem connects them.

T.3 Liveness-Specific Global Obligations

The safety obligations above are largely local or compositional. The remaining obligations concern global executions and cannot generally be discharged by proving that each process is locally live. The liveness theorem depends on the interaction of all processes, channels, fairness assumptions, environment behavior, drainable work, and the global wait-for structure.

T.3.1 B2, B3, B5, and Admissible Global Executions

A syntactically legal infinite transition sequence need not be an admissible execution for purposes of liveness. An execution could, for example, postpone an enabled action forever, refuse to drain a full or empty channel despite persistent requests, or rely on hidden environmental changes that invalidate an apparent fixed point.

B2 supplies the applicable fair scheduler or arbiter progress condition: an action that remains continuously ChannelEnabled must eventually be selected and commit. B3 supplies the corresponding channel-progress property for persistent blocking requests at full, empty, or rendezvous boundaries. For a simple explicit channel boundary, B3 may be derived from B2 plus E4, but it remains a useful modular obligation for checking channel realizations and elaborated boundary chains.

B5 governs global ε-quiescent fixed points and environmental stability. It prevents the proof from declaring a state terminal while hidden internal or environmental progress remains possible under the theorem's assumptions.

Together, these premises determine which infinite executions belong to Exec_ref or Exec_post for the liveness theorem. The proof must reason over fair, progress-respecting global executions rather than over arbitrarily stalled transition sequences.

T.3.2 Drain Normalization and Drain Closure

An ε-quiescent state need not be drain-closed. No internal ε-step may be available even though already-issued downstream message transfers can still commit and reduce the amount of in-flight work.
Drain normalization permits such previously issued work to complete fairly, subject to B2/B3, without accepting or issuing new work beyond the blocked source frontier. It therefore requires the applicable drain-only blocked-frontier discipline: no new source-later dynamic operation may be launched past the blocked point, already asserted requests are not withdrawn, and only already-issued downstream work may drain. The resulting cutpoint is drain-closed when no further permitted downstream drain transfer associated with the relevant blocked component remains enabled.
Snapshot/continuation separation. Lemma K.2a transfers only DrainClosedNow at a selected cutpoint by equality of KObs. Appendix K-P reuses the same cutpoint interface inside Appendix K-P: current outstanding requests are O = ResOffers, and the finite non-front subset DrainBag is the drain-normalization measure. Neither step establishes future issue legality. Definition R.16/K.Drain, B2/B3 progress, nonwithdrawal, and the no-replenishment proof remain operational obligations used to show that DrainBag cannot increase and that drain normalization terminates.
This construction separates persistent blocking causes from transient pipeline or channel contents. Without drain closure, a wait-for graph cycle might disappear merely because already-issued work had not yet been allowed to complete.
The drain-normalization proof must also show that the relevant closed WFG component and source frontier survive the normalization, that the witness correspondence remains valid, and that the normalized cutpoint still satisfies the KObs reconstruction premises required by Lemmas R.21b–R.21f. Kε is used to ensure that the ε behavior involved in this process is WFG-inert.
The applicable discipline is not limited to pipelines. Automatic flush supplies it for pipelined or otherwise overlapped control. Ordinary non-overlapped control may supply it through a structural proof that no new source-later dynamic operation can be launched after the blocked point. In either case, finite B-faithful explicit boundary storage or credit makes the already-issued drain set finite, and the no-new-launch rule makes it non-replenishing.

T.3.3 K.2b Dominance and Limit Extraction

[Revision note: this subsection now summarizes the Policy-S-primary proof architecture of Appendix K.4b and Appendix K-P.]
The Sys_ref witness selected from RTL may use liberal/overlapped issue. Policy S withholds source-later blocking issue. The proof does not attempt to replay the entire liberal history or preserve the same deadlock component.
Lemma K.2b instead proves a universal dominance property. Fix any maximal fair Policy-S execution. An identity-aware first-violation argument shows that it cannot overtake a liberal comparison execution whose original closed frontier is frozen. The liberal terminal snapshot is named K_L; the stabilized serial snapshot is K_S. The permanently nonadvancing Policy-S processes stabilize, the finite ResOffers-derived DrainBag drains under the operational no-replenishment discipline, and following actual E-typed point-to-point complementary owners in WFGPlusRec(K_S,I) yields either an internal cycle or an internal chain ending at an EnvStop or DoneStop terminal.
Because the Policy-S execution was arbitrary, any Policy-L internal deadlock would force every maximal fair Policy-S execution to reach the applicable bad predicate. The contrapositive is the practical result: one complete maximal fair Policy-S execution avoiding that predicate suffices. This is not a confluence claim and does not require different Policy-S runs to have the same trace.
In the optional K.EnvOrd / K.Done fragment (may-follow starvation-cone form with completion audit), feeding work cannot bypass a pending cone operation — environment-facing, internal and transitively starving at the environment, or demanding beyond a completed producer — so no starvation terminal is reachable and the Policy-S check targets the ordinary internal K.1 predicate (Corollary K.EnvOrd-1). The

general route retains Dead_KS, with EnvStop and DoneStop terminals, to catch the Example K.CE and K.CE-4 patterns.
The proof still requires K.CV, point-to-point evaluated boundaries, nonwithdrawable requests, Kε, and the Definition R.16/K.Drain discipline. Multi-frontier elaborations require the separate K-O2 composition lemma; the standard K.PSF profile is single-frontier.

T.3.3a A5 certificate interface

The primary CF-S certificate is an execution-and-completeness certificate, not merely a trace log. It identifies one maximal fair Policy-S execution, proves the applicable progress and drain premises for that execution, and supplies sound evidence that the execution is complete in the sense of K.CompleteS. At each canonical cutpoint the checker derives BadSRec from KObs and the fixed instance data. For a nonterminating run, the certificate therefore needs an inductive invariant, recurrence/lasso proof, complete symbolic exploration, or another sound argument covering the unbounded suffix; a long regression trace is insufficient.
CF-3 and CF-4 must keep the snapshot/transition distinction explicit. KObs is the right observation component for the bad-state predicate, but it is not by itself a Markov state for Policy L or Policy S because it omits historical Issue timing and future issue eligibility. An invariant or exploration abstraction may add operational phase, fairness, or drain-control state, and must prove that its abstract Post relation soundly overapproximates the selected operational semantics. This is the mechanization-facing reason the certificate checker should expose separate modules for KObs reconstruction, operational transition validation, fairness/completeness, and route composition.
A5 certificates also bind environment-terminal and waiver reasoning. General-route CF-S evidence uses DeadKSRec and therefore records EnvTerminalAdequate; the EnvOrdDone route records K.EnvOrd/K.Done and uses DeadKRec. Any non-empty waiver set carries KObs observational closure for reconstructed cutpoint predicates and the separate K.2b.Q extraction closure where the Policy-S dominance route uses it.

T.3.4 WFG and Conservative Serial-Source Deadlock Reasoning

The WFG identifies persistent blocking at minimal pending frontiers. The core RTL-to-source theorem transfers a closed drain-closed WFG component through the equal-KObs correspondence packaged by J.KObsConf. The serial route does not replay that state. It names the liberal source cutpoint K_L, uses operational dominance to construct a stabilized serial cutpoint K_S, and checks DeadKSRec or DeadKRec on K_S. Thus A5-L reduces to one complete Policy-S check against the route-selected reconstructed obstruction predicate, without equating the two executions or their histories.
A6 remains essential: point-to-point ownership gives each blocked endpoint one actual complementary owner. Shared resources must be represented through explicit arbiter processes and point-to-point channels before the owner-walk argument applies.
Once one complete maximal fair Policy-S execution avoids Dead_KS^W, Corollary K.2c supplies A5-L and Theorem K.1A rules out the original RTL-matched internal deadlock. If the static Conditions K.EnvOrd and K.Done are discharged, the same run may be checked only for ordinary Dead_K^W cutpoints.
This argument is specific to the scheduling contract. It is substantially stronger than ordinary trace inclusion, weak forward simulation, or per-process progress, because it connects a global implementation-level blocking structure back to a corresponding deadlock in the prescribed serial source semantics.

T.4 Summary

Reusable transition-system and simulation libraries can reduce the amount of generic proof infrastructure needed to mechanize this document. They can provide definitions and standard theorems for finite and infinite executions, ε-closure, simulation, trace lifting, well-founded induction, and temporal reasoning.

The substantive guarantees nevertheless depend on the scheduling-specific obligations summarized above. In the safety layer, those obligations establish witness-relative ordered trace compatibility across source replay, observable-unit ordering, channel semantics, capacity abstraction, reset correspondence, and system composition. In the liveness layer, they establish admissible global progress, canonical drain-closed cutpoints, preservation of the wait-for structure, and reduction from overlapped execution to conservative serial-source deadlock.

Accordingly, standard simulation theory should be treated as reusable proof infrastructure, not as a replacement for the scheduling model or for the named assumptions and implementation-side obligations on which the formal guarantees depend. The certificate architecture makes those obligations independently checkable: K.CertHdr binds one instance, CF-0 validates the source/RTL KObs cutpoint boundary, and K.A5Cert validates the selected deadlock-freedom discharge route. The checker reconstructs K-facing predicates and separately validates operational transition, fairness, completeness, and drain evidence.